Lecture Notes on Artificial Neural Networks and Deep Learning

Lorenzo Vergata

July 12, 2026

Contents

1 Introduction to Artificial Neural Networks and Deep Learning **27**

1.1 What is Machine Learning? . . . 27
1.2 Supervised, Unsupervised, and Reinforcement Learning . . . 27
1.2.1 Supervised Learning . . . 27
1.2.2 Unsupervised Learning . . . 28
1.2.3 Reinforcement Learning . . . 28
1.3 Classification and Regression . . . 28
1.3.1 Classification . . . 28
1.3.2 Regression . . . 28
1.4 Unsupervised Learning Example: Clustering . . . 29
1.5 From Classical Machine Learning to Deep Learning . . . 29
1.6 Deep Learning . . . 29
1.7 Classical Machine Learning . . . 30
1.8 Why Feature Choice Matters . . . 30
1.9 Hand-Crafted Features . . . 30
1.10 Deep Learning . . . 31
1.11 Why Deep Learning Became Important . . . 31
1.12 ImageNet and Convolutional Neural Networks . . . 31
1.13 Tasks Enabled by Deep Learning . . . 31
1.13.1 Semantic Segmentation . . . 32
1.13.2 Instance Segmentation . . . 32
1.13.3 Style Transfer and Super-Resolution . . . 32
1.14 Main Message . . . 32
1.15 Feed Forward Neural Networks and Backprop . . . 32
1.16 AI, Machine Learning, and Deep Learning . . . 32
1.17 Historical Origin of Neural Networks . . . 33
1.18 Why AI Was Studied . . . 33
1.19 Why Neural Networks Were Interesting . . . 33
1.20 The Brain as Inspiration . . . 33
1.21 The Perceptron . . . 34
1.22 Why Start from the Perceptron? . . . 34
1.23 The Perceptron as a Model of Neural Computation . . . 34
1.23.1 From Biology to a Mathematical Model . . . 34
1.23.2 Bias as an Additional Input . . . 35
1.23.3 A Note on Biological Accuracy . . . 35
1.24 History of the Perceptron . . . 35
1.25 Learning the Bias . . . 35
1.26 What Can a Perceptron Compute? . . . 36
1.26.1 Why Is This Useful? . . . 36
1.27 Hebbian Learning and the Perceptron Rule . . . 36
1.27.1 Hebb's Idea . . . 36
1.27.2 A Simple Analogy . . . 36

1.27.3 Hebbian Update Rule . . . 37
1.27.4 Perceptron Learning . . . 37
1.27.5 An Example: Learning the OR Function . . . 37
1.27.6 Why This Matters . . . 37
1.28 Perceptron Learning on a Truth Table . . . 38
1.28.1 First Pass Through the Data . . . 38
1.28.2 Epochs and Iterations . . . 39
1.28.3 Second Pass . . . 39
1.28.4 Does It Always Converge? . . . 39
1.28.5 Does It Converge to the Same Weights? . . . 40
1.28.6 Why the Perceptron Can or Cannot Learn a Function . . . 40
1.28.7 Main Takeaway . . . 40
1.29 Linearly Separable and Nonlinearly Separable Functions . . . 40
1.30 The Limitations of the Perceptron . . . 40
1.31 The First AI Winter . . . 41
1.32 Why Multiple Layers Help . . . 41
1.33 Why Hebbian Learning Fails in Multiple Layers . . . 41
1.34 From Perceptrons to Feedforward Neural Networks . . . 41
1.35 Backpropagation and the Recovery of Neural Networks . . . 41
1.36 Historical Cycles . . . 42
1.37 Summary . . . 42

2 Backpropagation Example **43**
2.1 From the Perceptron to Feedforward Neural Networks . . . 43
2.1.1 The Idea of Multilayer Perceptrons . . . 43
2.1.2 Why Hebbian Learning Fails in Multiple Layers . . . 43
2.1.3 From Perceptrons to Feedforward Neural Networks . . . 43
2.2 Architecture of a Feedforward Neural Network . . . 44
2.2.1 Input Layer . . . 44
2.2.2 Hidden Layers . . . 44
2.2.3 Output Layer . . . 44
2.3 Parameters and Hyperparameters . . . 44
2.4 Dense Connections . . . 44
2.5 Why Dense Layers Are General . . . 45
2.6 Limitations of the Simplified Architecture . . . 45
2.7 Activation Functions in Feedforward Neural Networks . . . 45
2.7.1 Why the Step Function Does Not Work . . . 45
2.7.2 Linear Activation Function . . . 45
2.7.3 Sigmoid Activation Function . . . 45
2.7.4 Hyperbolic Tangent . . . 46
2.7.5 Other Activation Functions . . . 46
2.8 Deep Learning versus Neural Networks . . . 46
2.9 Machine Learning Interpretation . . . 47
2.10 Output Activation for Regression . . . 47
2.11 Choosing Activation Functions . . . 47
2.11.1 Binary Classification . . . 47
2.11.2 Multiclass Classification . . . 47
2.11.3 Summary of the Output Layer . . . 48
2.11.4 Activation Functions in Hidden Layers . . . 48
2.12 Universal Approximation Theorem . . . 48
2.12.1 Important Caveats . . . 49
2.12.2 Classification vs. Regression . . . 49
2.13 Training: Finding the Weights . . . 49
2.13.1 The General Supervised Learning Setup . . . 49

2.13.2 Measuring Goodness of Fit: The Loss Function . . . 50
2.13.3 Non-linear Optimization . . . 50
2.14 Gradient Descent . . . 50
2.14.1 The Error Surface . . . 51
2.14.2 Variants and Improvements . . . 51
2.14.3 Backpropagation = Gradient Descent on Neural Networks . . . 51
2.14.4 Local Minima and Initialization . . . 51
2.15 Worked Example: Computing the Gradient for a Specific Weight . . . 51
2.16 Batch, Stochastic, and Mini-Batch Gradient Descent . . . 53
2.16.1 Stochastic Gradient Descent (SGD) . . . 53
2.16.2 Mini-Batch Gradient Descent . . . 53
2.17 Automatic Differentiation . . . 53

3 Feedforward Neural Networks and Training Loop 55
3.1 Connecting the notebook to Google Drive . . . 55
3.2 Importing the main libraries . . . 55
3.3 Loading the Iris dataset . . . 55
3.4 Inspecting the data . . . 56
3.5 Inspecting the targets . . . 56
3.6 A visual look at the features . . . 56
3.7 Preparing the data . . . 56
3.8 Training, validation, and test . . . 57
3.9 Checking the data shapes . . . 57
3.10 Reprocessing the data . . . 58
3.11 Inspecting a single batch . . . 58
3.12 Defining the neural network . . . 58
3.13 Model structure and parameter count . . . 59
3.14 TensorBoard . . . 59
3.15 Hyperparameters for training . . . 59
3.16 Training and evaluation . . . 60
3.17 What happens during one epoch . . . 60
3.18 Validation . . . 61
3.19 TensorBoard logging . . . 61
3.20 The training loop . . . 61
3.21 Plotting the results . . . 61
3.22 Inspecting TensorBoard . . . 62
3.23 Reloading the model and testing . . . 62
3.24 Next dataset . . . 62

4 Error Function Design 63
4.1 Gradient Descent, GPUs, and Backpropagation . . . 63
4.2 Automatic Differentiation and the Chain Rule . . . 63
4.3 Backpropagation as Repeated Application of the Chain Rule . . . 64
4.4 What Needs to Be Stored . . . 64
4.5 Why This Explains the PyTorch Training Loop . . . 65
4.6 Flexibility of the Loss Function and Arbitrary Differentiable Nodes . . . 65
4.7 Why the Squared Error? Maximum Likelihood Estimation . . . 66
4.7.1 A Simple Gaussian Example . . . 66
4.7.2 Formalizing Maximum Likelihood . . . 66
4.8 Stationary Points and Analytical Derivation of the Gaussian MLE . . . 67
4.9 Connecting Maximum Likelihood to Neural Network Training . . . 68
4.10 The Log-Likelihood for Neural Networks . . . 69
4.11 Why This Perspective Is Useful . . . 69
4.12 When Squared Error Is Inappropriate: Classification . . . 70

4.13 Maximum Likelihood for Classification: The Bernoulli Model . . . 70
4.14 Generalization to Multi-Class: Categorical Cross-Entropy . . . 71
4.15 Why the Choice of Loss Function Matters . . . 71

5 Facing Overfitting **73**
5.1 From training to generalization . . . 73
5.2 Neural networks and loss functions . . . 73
5.3 Universal approximation and its limits . . . 73
5.4 Inductive bias, underfitting, and overfitting . . . 74
5.5 How to detect overfitting . . . 74
5.6 Train/test split and hold-out validation . . . 74
5.7 Stratified sampling and data splitting . . . 75
5.8 Training, validation, and test sets . . . 75
5.9 Why a validation set is needed . . . 75
5.10 Validation error and test error . . . 76
5.11 What if test performance is much worse? . . . 76
5.12 Using all the data at the end . . . 76
5.13 Model complexity and generalization . . . 76
5.14 Noise and the lower bound on error . . . 77
5.15 Bias–variance trade-off . . . 77
5.16 Cross-validation . . . 77
5.17 Limitations of a single hold-out split . . . 77
5.18 Leave-one-out cross-validation . . . 78
5.19 k-fold cross-validation . . . 78
5.20 Which model is used at test time? . . . 78
5.21 What is used in practice for neural networks . . . 79
5.22 Sampling considerations . . . 79
5.23 Hyperparameter search . . . 79
5.24 Early stopping revisited . . . 79
5.25 Regularization . . . 80
5.26 A Bayesian interpretation . . . 80
5.27 Gaussian prior and L_2 regularization . . . 80
5.28 L_1 regularization and sparsity . . . 81
5.29 Summary . . . 81

6 Training Tips and Tricks **83**
6.1 Early stopping, weight decay, and regularization . . . 83
6.2 Weight decay . . . 83
6.3 Early stopping and weight decay together . . . 84
6.4 Dropout . . . 84
6.5 Scaling at test time . . . 85
6.6 Practical aspects of dropout . . . 85
6.7 Relation to ensemble methods . . . 85
6.8 Transition to activation functions . . . 85
6.9 Vanishing gradients and activation functions . . . 85
6.10 Exploding gradients . . . 86
6.11 ReLU . . . 86
6.12 Limitations of ReLU . . . 86
6.13 Leaky ReLU . . . 87
6.14 Modern activation functions and weight initialization . . . 87
6.15 Activation functions in the output layer . . . 87
6.16 Weight initialization . . . 87
6.17 Why initialization matters . . . 88
6.18 Xavier initialization . . . 88

6.19 He initialization . . . 88
6.20 Weight initialization revisited . . . 89
6.21 Xavier initialization . . . 89
6.22 Glorot initialization . . . 89
6.23 He initialization . . . 89
6.24 Practical remarks on initialization . . . 90
6.25 Normalization of the input . . . 90
6.26 Batch normalization . . . 90
6.27 Where batch normalization fits in the network . . . 91
6.28 Batch normalization . . . 91
6.29 Learned scale and shift parameters . . . 91
6.30 Why batch normalization helps . . . 91
6.31 A final remark on optimization . . . 92

7 Overfitting and Regularization 93
7.1 Introduction . . . 93
7.2 Dataset . . . 93
7.3 Class Imbalance . . . 93
7.4 Evaluation Metrics . . . 94
7.5 Multi-Class Classification . . . 94
7.6 Class Weights and Evaluation Metric . . . 94
7.7 Data Preprocessing . . . 94
7.8 Normalization and Data Loaders . . . 95
7.9 Model Definition . . . 95
7.10 Number of Parameters . . . 96
7.11 Training Setup . . . 96
7.12 Training and Validation . . . 96
7.13 Baseline Model and Overfitting . . . 96
7.14 What Is Overfitting? . . . 97
7.15 Model Families, Bias, and Variance . . . 97
7.16 Conclusion . . . 97
7.17 Class Weights and Weighted Metrics . . . 97
7.18 Bias–Variance Trade-off . . . 97
7.19 Error Decomposition . . . 98
7.20 Why Start from a Large Model? . . . 98
7.21 Early Stopping . . . 98
7.22 Implementation of Early Stopping . . . 99
7.23 Effect of Early Stopping . . . 99
7.24 Confusion Matrix and Further Improvements . . . 99
7.25 Practical Remark . . . 99
7.26 Dropout . . . 99
7.26.1 Intuition . . . 100
7.26.2 Implementation in PyTorch . . . 100
7.26.3 Choice of Dropout Rate . . . 100
7.26.4 Effect on Training . . . 100
7.27 L1 and L2 Regularization . . . 100
7.27.1 L2 Regularization . . . 100
7.27.2 L1 Regularization . . . 101
7.27.3 Geometric Intuition . . . 101
7.27.4 Elastic Net . . . 101
7.28 Interpretation . . . 101
7.29 Implementing L1 and L2 Regularization . . . 101
7.30 Effect of L2 Regularization . . . 102
7.31 Why Loss Scaling Matters . . . 102

7.32 Optimization and the Role of the Optimizer . . . 102
7.33 AdamW and Weight Decay . . . 103
7.34 Implementation of AdamW . . . 103
7.35 Final Evaluation on the Test Set . . . 103
7.36 Conclusion . . . 103

8 Cross Validation and Hyperparameter Tuning 105
8.1 How to Use Data . . . 105
8.2 Training Only . . . 105
8.3 Train/Test Holdout . . . 105
8.4 Train/Validation/Test Split . . . 105
8.5 Why a Validation Set Is Necessary . . . 106
8.6 Cross-Validation . . . 106
8.7 Advantages of Cross-Validation . . . 106
8.8 Important Caveat . . . 107
8.9 Model Selection and Hyperparameter Tuning . . . 107
8.10 From Model Selection to Final Deployment . . . 107
8.11 Why Not Simply Average the Epochs? . . . 107
8.12 Why Cross-Validation Matters . . . 108
8.13 Cross-Validation and Stratification . . . 108
8.14 Implementation Strategy . . . 108
8.15 Normalization Must Use Training Data Only . . . 109
8.16 Why Reinitialize the Model at Each Fold? . . . 109
8.17 Choice of Hyperparameters . . . 109
8.18 Practical Advice . . . 109
8.19 Hyperparameter Tuning . . . 109
8.20 Cross-Validation Results . . . 110
8.21 Parameter Tuning . . . 110
8.22 Batch Size . . . 110
8.22.1 Trade-off . . . 110
8.23 Learning Rate . . . 111
8.23.1 Learning Rate Schedulers . . . 111
8.24 Practical Tuning Strategy . . . 111
8.25 Reading the Results . . . 111
8.26 Optimization as a Trade-off . . . 111
8.27 Learning Rate, Model Complexity, and Hyperparameter Search . . . 112
8.28 Model Complexity . . . 112
8.29 Grid Search . . . 112
8.30 Regularization in the Search Space . . . 113
8.31 Why Saving the Best Model Matters . . . 113
8.32 Random Search . . . 113
8.33 Advanced Hyperparameter Optimization . . . 113
8.34 Why Cross-Validation Is Not Always Used . . . 113
8.35 Summary . . . 114

9 Recurrent neural networks + LSTM 115
9.1 Introduction: Static vs. Dynamic Models . . . 115
9.1.1 The Need for Sequence Modelling . . . 115
9.2 Memoryless Approaches: Time-Delay Neural Networks . . . 115
9.2.1 Linear Autoregressive Models . . . 116
9.2.2 Nonlinear Extension with Feed-Forward Networks . . . 116
9.3 Models with Explicit Memory . . . 116
9.3.1 Linear Dynamic Systems (State-Space Models) . . . 116
9.3.2 Hidden Markov Models (HMMs) . . . 116

9.3.3 Recurrent Neural Networks (RNNs) . . . 117
9.4 From State Estimation to Sequence Prediction . . . 117
9.4.1 What Makes a Network Recurrent? . . . 117
9.4.2 Finite vs. Infinite Memory . . . 117
9.5 RNN Architecture . . . 117
9.6 Training Recurrent Networks: Backpropagation Through Time (BPTT) . . . 118
9.6.1 Backpropagation Through Time (BPTT) . . . 118
9.6.2 Truncated Backpropagation Through Time (TBPTT) . . . 118
9.6.3 Weight Sharing and Gradient Averaging . . . 119
9.7 Training via Backpropagation Through Time . . . 119
9.8 The Challenge of Long-Term Dependencies . . . 119
9.8.1 Why Long Memory Matters . . . 119
9.8.2 The Vanishing Gradient in Deep Unrolled Networks . . . 119
9.9 Mathematical Analysis of Gradient Flow in RNNs . . . 119
9.9.1 Model Definition . . . 119
9.9.2 Gradient of the Current State with Respect to the Past . . . 120
9.9.3 The Product Rule: Vanishing and Exploding Gradients . . . 120
9.9.4 The Trivial Solution: A Pure Integrator . . . 120
9.10 Long Short-Term Memory (LSTM) . . . 120
9.10.1 Core Idea: The Constant Error Carousel . . . 121
9.10.2 Gating Mechanisms . . . 121
9.11 The Basic RNN Block (Revisited) . . . 121
9.12 Long Short-Term Memory (LSTM) . . . 121
9.12.1 Notation and Overview . . . 121
9.12.2 Step-by-Step Decomposition . . . 121
9.12.3 Efficiency and Practical Impact . . . 122
9.13 Gated Recurrent Unit (GRU) . . . 122
9.13.1 Key Modifications . . . 122
9.13.2 GRU Equations . . . 122
9.13.3 LSTM vs. GRU in Practice . . . 123
9.14 Architectural Patterns for Recurrent Networks . . . 123
9.14.1 Stacked (Deep) RNNs . . . 123
9.14.2 Online Prediction . . . 123
9.15 The Hidden State as a Temporal Summary . . . 123
9.16 Bidirectional Recurrent Networks . . . 124
9.17 Learning the Initial Hidden State . . . 124
9.18 Choosing the Right Architecture . . . 124
9.19 Sequence-to-Sequence Modelling Paradigms . . . 124
9.19.1 One-to-Many: Fixed Input → Sequence Output . . . 125
9.19.2 Many-to-One: Sequence Input → Fixed Output . . . 125
9.19.3 Many-to-Many (Synced): Sequence → Sequence . . . 125
9.19.4 Many-to-Many (Encoder–Decoder): Sequence → Sequence of Different Length . . . 125
9.20 Word Representations: One-Hot vs. Embeddings . . . 125
9.20.1 Classical Approach: One-Hot Encoding . . . 125
9.20.2 Modern Approach: Learned Embeddings . . . 125

10 Sequence to Sequence Learning **127**
10.1 Motivation: Representing Words for Sequence Models . . . 127
10.1.1 Limitations of One-Hot Encoding . . . 127
10.2 Unsupervised Learning: A Paradigm Shift . . . 127
10.2.1 Recap: Supervised vs. Unsupervised Learning . . . 127
10.2.2 The Unsupervised Criterion . . . 127
10.3 Autoencoders: Learning to Reconstruct . . . 128
10.3.1 Architecture . . . 128

10.3.2 Interpretation and Applications . . . 128
10.4 Key Takeaway: The Loss Function Defines the Task . . . 129
10.5 Introduction: The Need for Better Word Representations . . . 129
10.6 Autoencoders: Unsupervised Compression and Reconstruction . . . 129
10.6.1 Architecture and Training . . . 129
10.6.2 The Bottleneck Principle . . . 129
10.6.3 Applications . . . 130
10.7 Distributed Representations for Words . . . 130
10.7.1 Desired Properties . . . 130
10.8 Language Modeling as a Precursor to Word Embeddings . . . 130
10.8.1 Definition . . . 130
10.8.2 Classical Approaches . . . 130
10.8.3 Neural Language Models and Word Embeddings . . . 131
10.9 The Combinatorial Explosion of Language Modeling . . . 131
10.10 The Neural Network Language Model (Bengio et al., 2001/2003) . . . 131
10.10.1 Architecture . . . 131
10.10.2 Training . . . 131
10.10.3 Complexity Analysis . . . 132
10.10.4 Results and Impact . . . 132
10.11 From Language Modeling to Word Embeddings: Word2Vec . . . 132
10.11.1 Simplification: Removing the Hidden Layer . . . 132
10.11.2 Why Word2Vec Was a Breakthrough . . . 132
10.12 Introduction: Predicting Words from Context . . . 133
10.13 The Two Architectures of Word2Vec . . . 133
10.13.1 Continuous Bag-of-Words (CBOW) . . . 133
10.13.2 Skip-gram . . . 133
10.14 Model Architecture and Computational Efficiency . . . 133
10.14.1 Comparison with NNLM . . . 134
10.14.2 Scalability Gains . . . 134
10.15 Semantic Regularities in the Embedding Space . . . 134
10.15.1 Analogy by Vector Arithmetic . . . 134
10.15.2 Why Does This Happen? . . . 135
10.16 Practical Applications and Legacy . . . 135
10.16.1 From One-Hot to Dense Vectors . . . 135
10.16.2 Simple Semantic Queries . . . 135
10.16.3 Evolution Beyond Word2Vec . . . 135
10.17 Document Similarity and Semantic Search with Word Embeddings . . . 136
10.17.1 Finding Synonyms via Nearest Neighbors . . . 136
10.17.2 k-NN Classification for Text . . . 136
10.18 Why One-Hot Encoding Prevailed for So Long . . . 136
10.18.1 Historical Context . . . 136
10.19 GloVe: Global Vectors for Word Representation . . . 137
10.19.1 Core Idea . . . 137
10.19.2 Properties of GloVe Embeddings . . . 137
10.20 Looking Ahead: A Unified View of Neural Network Layers . . . 137

11 Time Series Classification **139**
11.1 Introduction: Why Time Matters . . . 139
11.2 Dataset: Human Activity Recognition from Wearable Sensors . . . 139
11.3 Initial Data Inspection and Preprocessing . . . 139
11.3.1 Checking Data Types and Missing Values . . . 139
11.3.2 Value Ranges and Anomalies . . . 139
11.3.3 Class Distribution . . . 140
11.3.4 User ID and Activity Relationships . . . 140

11.4 Avoiding Data Leakage: The User-Split Strategy . . . 140
11.4.1 Creating Unique Activity Identifiers . . . 140
11.5 Visualizing Temporal Patterns . . . 140
11.6 From Raw Streams to Neural Network Inputs . . . 141
11.6.1 The Sliding Window Approach . . . 141
11.6.2 Design Choices . . . 141
11.7 From Static to Temporal Data . . . 141
11.8 Dataset Recap and Splitting Strategy . . . 141
11.8.1 User-Based Splitting . . . 141
11.8.2 Normalization . . . 141
11.9 Windowing: Creating Fixed-Length Sequences . . . 142
11.9.1 Window Size (W) . . . 142
11.9.2 Stride (S) . . . 142
11.9.3 Handling the Tail: Padding vs. Truncation . . . 142
11.10 Constructing the Tensor Dataset . . . 142
11.11 Implications for Model Input Shape . . . 143
11.12 From Windowing to Tensor Datasets . . . 143
11.12.1 Impact of Stride on Dataset Size . . . 143
11.12.2 Final Tensor Shapes . . . 143
11.13 Recurrent Classifier Module . . . 143
11.13.1 Architecture Overview . . . 144
11.13.2 Stacking Recurrent Layers . . . 144
11.13.3 Bidirectional Processing . . . 144
11.13.4 Dropout in Recurrent Networks . . . 144
11.14 Training Dynamics and the Vanishing Gradient Problem . . . 144
11.14.1 Observed Training Curves . . . 144
11.14.2 Interpretation: Local Minima and Gradient Instability . . . 144
11.15 Practical Considerations and Extensions . . . 145
11.15.1 Parameter Counting . . . 145
11.15.2 Hyperparameter Tuning . . . 145
11.15.3 Beyond the Basic Model . . . 145
11.16 Introduction: Time as a Dimension . . . 145
11.17 Dataset: Human Activity Recognition . . . 145
11.17.1 Data Inspection . . . 145
11.18 Avoiding Data Leakage: User-Based Splitting . . . 146
11.18.1 Normalization . . . 146
11.19 Windowing: Creating Fixed-Length Sequences . . . 146
11.19.1 Window Size (W) and Stride (S) . . . 146
11.19.2 Padding the Final Window . . . 146
11.19.3 Final Tensor Shape . . . 146
11.20 Recurrent Classifier Module . . . 146
11.20.1 Architecture . . . 147
11.20.2 Bidirectional Processing . . . 147
11.20.3 Dropout in Recurrent Layers . . . 147
11.21 Training Dynamics and Empirical Observations . . . 147
11.21.1 Vanilla RNN: Unstable Training . . . 147
11.21.2 Bidirectional RNN: No Improvement . . . 147
11.21.3 LSTM: Gradual Learning . . . 147
11.21.4 Bidirectional LSTM: Best Performance . . . 147
11.21.5 GRU: Efficient and Effective . . . 147
11.22 Cross-Validation and Hyperparameter Tuning . . . 148
11.22.1 Impact of Window Size . . . 148
11.22.2 The Baseline Principle . . . 148
11.23 Practical Recommendations for the Challenge . . . 148

12 Time Series Classification and Forecasting **149**
12.1 Introduction: From Classification to Forecasting . . . 149
12.2 Dataset: Air Quality and Climate Variables . . . 149
12.2.1 Data Preprocessing . . . 149
12.3 Train/Validation/Test Split for Forecasting . . . 149
12.3.1 Chronological Split . . . 149
12.3.2 Why Not Random Cross-Validation? . . . 150
12.4 From a Single Series to Supervised Samples: Windowing . . . 150
12.4.1 Key Parameters . . . 150
12.4.2 Windowing Procedure . . . 150
12.4.3 Stride and Sample Correlation . . . 150
12.4.4 Multivariate Forecasting . . . 150
12.5 Visualizing the Windowed Data . . . 150
12.6 Modeling Approaches: Classical vs. Neural . . . 151
12.6.1 Classical Time Series Models . . . 151
12.6.2 Neural Networks for Forecasting . . . 151
12.7 Implementation Outline . . . 151
12.8 Introduction: From Classification to Forecasting . . . 151
12.9 Dataset: Air Quality and Climate Variables . . . 151
12.10 Train/Validation/Test Split for Time Series . . . 152
12.10.1 Chronological Split . . . 152
12.10.2 Why Sequential? . . . 152
12.11 From a Single Series to Supervised Samples: Windowing . . . 152
12.11.1 Key Parameters . . . 152
12.11.2 Windowing Procedure . . . 152
12.11.3 Stride and Correlation . . . 152
12.11.4 Tensor Shapes . . . 152
12.12 Direct Forecasting Model: Many-to-One (Encoder-Only) . . . 153
12.12.1 Architecture . . . 153
12.12.2 Why "Many-to-One"? . . . 153
12.12.3 Model Variants . . . 153
12.13 Training the Direct Forecaster . . . 153
12.13.1 Loss Function . . . 153
12.13.2 Learning Rate Scheduling . . . 153
12.13.3 Training Dynamics . . . 153
12.14 Limitations and Next Steps . . . 154
12.15 Introduction: Forecasting Strategies . . . 154
12.16 Learning Rate Scheduling: Balancing Exploration and Exploitation . . . 154
12.16.1 The Principle . . . 154
12.16.2 Common Scheduling Strategies . . . 154
12.16.3 Implementation in PyTorch . . . 155
12.17 Direct Forecaster: Performance Recap . . . 155
12.18 Autoregressive Forecasting . . . 155
12.18.1 Model Design . . . 155
12.18.2 Why Does It Perform Better? . . . 156
12.19 Sequence-to-Sequence Forecasting . . . 156
12.19.1 Analogy with Text Processing . . . 156
12.19.2 Architecture . . . 156
12.19.3 Training and Inference . . . 156
12.19.4 Implementation Considerations . . . 156
12.20 Introduction: The Seq2Seq Paradigm . . . 157
12.20.1 From Words to Numbers . . . 157
12.21 Attention Mechanism: Weighted Context . . . 157
12.21.1 Mathematical Formulation . . . 157

12.21.2 Interpretation . . . 157
12.22 Teacher Forcing: Stabilizing Seq2Seq Training . . . 157
12.22.1 The Teacher Forcing Technique . . . 158
12.22.2 Implementation . . . 158
12.23 Code Complexity and the Role of LLMs . . . 158
12.24 Empirical Performance . . . 158
12.24.1 Visual Inspection . . . 158
12.24.2 Test Set Evaluation . . . 158

13 Image Classification **159**
13.1 Overview: From Vectors to Images . . . 159
13.2 What is Computer Vision? . . . 159
13.2.1 Historical Context . . . 159
13.3 Digital Image Representation . . . 159
13.3.1 Pixels and Color Channels . . . 160
13.3.2 Examples of RGB Values . . . 160
13.3.3 Implications for Neural Networks . . . 160
13.4 The Simplest Visual Recognition Task: Image Classification . . . 160
13.4.1 What Can a Multilayer Perceptron Learn from Images? . . . 160
13.5 Key Challenges in Image Classification . . . 161
13.6 Introduction to Computer Vision . . . 161
13.7 Digital Image Representation . . . 161
13.7.1 Examples of RGB Triplets . . . 161
13.8 The Challenge of Image Size . . . 162
13.9 Basic Image Operations: Linear Filtering . . . 162
13.9.1 Pointwise Operations . . . 162
13.9.2 Neighborhood Operations . . . 162
13.9.3 Moving Average (Box Filter) . . . 162
13.9.4 Linear Filtering with Learnable Weights . . . 162
13.10 From Hand-Crafted to Learned Filters . . . 163
13.11 Recap: Linear Filtering as a Building Block . . . 163
13.11.1 Correlation vs. Convolution . . . 163
13.12 Visual Interpretation: Template Matching via Correlation . . . 163
13.12.1 Extension to Multi-Channel Images . . . 163
13.13 Image Classification: Problem Formulation . . . 164
13.14 The Mismatch: Images vs. Multilayer Perceptrons . . . 164
13.15 Preview: Convolutional Neural Networks . . . 164
13.16 The Mismatch: Images as Vectors . . . 164
13.16.1 Flattening an RGB Image . . . 164
13.16.2 Implications for Network Size . . . 165
13.17 CIFAR-10: A Benchmark for Tiny Images . . . 165
13.17.1 Dimensionality . . . 165
13.18 Interpreting a Trained Neural Network as a Linear Classifier . . . 165
13.18.1 From Scores to Probabilities . . . 165
13.18.2 Interpretation as L Independent Classifiers . . . 165
13.18.3 Number of Parameters . . . 166
13.19 The Role of Nonlinearities . . . 166
13.20 Visualizing the Learned Weights . . . 166
13.21 Limitations of Linear Classifiers on Images . . . 166
13.22 Two Perspectives on a Trained Linear Classifier . . . 166
13.22.1 Geometric Interpretation: Hyperplanes in High Dimensions . . . 166
13.22.2 Template Matching Interpretation: Learned Weights as Images . . . 167
13.23 Visualizing Learned Templates from CIFAR-10 . . . 167
13.23.1 Implications for Data Augmentation . . . 167

13.24 Why Image Classification is Challenging . . . 167
13.24.1 High Dimensionality . . . 167
13.24.2 Semantic Richness and Label Ambiguity . . . 168
13.24.3 Visual Transformations . . . 168
13.24.4 The Failure of Euclidean Distance for Images . . . 168
13.25 Conclusion: The Need for Convolutional Neural Networks . . . 168

14 Convolutional Neural Networks **169**
14.1 From Handcrafted Features to Learned Representations . . . 169
14.1.1 The Classical Image Classification Pipeline . . . 169
14.1.2 Advantages and Limitations . . . 169
14.1.3 The Deep Learning Shift . . . 169
14.2 Recap of the Convolution Operation . . . 170
14.2.1 Definition and Visual Interpretation . . . 170
14.2.2 Filter Effects (Single-Channel Examples) . . . 170
14.3 Convolutional Neural Networks: From Vectors to Volumes . . . 170
14.3.1 Multilayer Perceptrons vs. CNNs . . . 170
14.3.2 Volumetric Representation of Neurons . . . 170
14.4 CNN Architecture: From Volumes to Vectors . . . 171
14.4.1 Spatial Reduction and Channel Expansion . . . 171
14.4.2 Three Core Operations . . . 171
14.5 The Convolutional Layer . . . 171
14.5.1 Local Connectivity and Full Depth . . . 171
14.5.2 Mathematical Definition (Single Output Channel) . . . 171
14.5.3 Filter Dimensions and Hyperparameters . . . 172
14.5.4 Multiple Output Channels . . . 172
14.6 Visualizing Learned Filters . . . 172
14.6.1 Interpreting Activation Maps . . . 172
14.6.2 Multiple Filters: The Output Volume Depth . . . 173
14.6.3 Parameter Count of a Convolutional Layer . . . 173
14.6.4 Important Constraints . . . 173
14.6.5 Padding and Stride . . . 173
14.6.6 Convolution vs. Cross-Correlation in Deep Learning . . . 174
14.6.7 Channel Mixing vs. Per-Channel Convolution . . . 174
14.7 Non-linearity (Activation Functions) . . . 174
14.7.1 Why Non-linearity is Necessary . . . 174
14.7.2 Common Activation Functions . . . 174
14.7.3 Effect of ReLU Visualisation . . . 174
14.8 Pooling Layers . . . 174
14.8.1 Purpose . . . 174
14.8.2 Max Pooling . . . 175
14.9 A Complete CNN Architecture . . . 175
14.9.1 Example Pipeline . . . 175
14.9.2 Role of Each Block . . . 175
14.10 Visualising Internal Representations . . . 175
14.11 Semantic Clustering in Latent Space . . . 176
14.11.1 Motivation . . . 176
14.11.2 Properties of a Good Latent Space . . . 176
14.11.3 Why This Happens . . . 176
14.12 Conclusion and Next Lecture . . . 176
14.13 CIFAR-10 Dataset . . . 176
14.13.1 Overview . . . 176
14.13.2 Loading with TorchVision . . . 177
14.14 Data Preprocessing . . . 177

14.15 Convolutional Neural Networks – Core Operations . . . 177
14.15.1 2D Convolution . . . 177
14.15.2 Hyperparameters of a Convolutional Layer . . . 177
14.15.3 Number of Parameters . . . 178
14.15.4 Output Spatial Dimensions . . . 178
14.16 Pooling Layers . . . 178
14.17 Flattening . . . 178
14.18 Designing a Simple CNN Architecture . . . 178
14.18.1 Backbone and Classification Head . . . 178
14.18.2 Building Blocks . . . 179
14.18.3 Downsampling Strategies . . . 179
14.18.4 Typical Progression . . . 179
14.19 Kernel Size: Trade-offs . . . 179
14.19.1 Small Kernels (e.g., 3×3) . . . 179
14.19.2 Large Kernels (e.g., 7×7, 11×11) . . . 179
14.20 Receptive Field Expansion via Stacking . . . 180
14.21 Downsampling Strategies . . . 180
14.21.1 Max Pooling . . . 180
14.21.2 Strided Convolutions . . . 180
14.22 Example: A Simple CNN for CIFAR-10 . . . 180
14.22.1 Architecture . . . 180
14.22.2 Parameter Count . . . 181
14.23 PyTorch Implementation Details . . . 181
14.23.1 Defining Convolutional and Pooling Layers . . . 181
14.23.2 Vanilla Block Construction . . . 181
14.23.3 Full Network Class . . . 181
14.24 Training Loop Configuration . . . 182
14.24.1 Hyperparameters . . . 182
14.24.2 Training and Validation Steps . . . 182
14.25 Notes on the Input Shape . . . 182
14.26 Experimental Results with Simple CNNs on CIFAR-10 . . . 182
14.26.1 Baseline: 2-Block CNN . . . 182
14.26.2 Deeper Architecture: 3-Block CNN . . . 183
14.26.3 Even Deeper: 4-Block CNN . . . 183
14.27 Visualizing Learned Features . . . 183
14.28 Comparison of Architectures . . . 183
14.29 Regularization and Training Strategies . . . 184
14.29.1 Strided Convolutions vs. Max Pooling . . . 184
14.29.2 Weight Decay . . . 184
14.30 Test Set Evaluation . . . 184
14.31 Outlook . . . 184

15 CNN anatomy and training under data scarcity 185
15.1 Deep Dive into CNN Anatomy . . . 185
15.1.1 Recap of the Convolutional Layer . . . 185
15.1.2 Convolution as a Linear Transformation: The Matrix View . . . 185
15.1.3 Key Insight: Inductive Priors of Convolution . . . 186
15.2 Consequences for Network Design and Training . . . 186
15.3 Looking Ahead . . . 186
15.4 Convolutional Layer as Structured Sparse Matrix Multiplication . . . 187
15.4.1 Parameter Count in the Matrix View (Single-Channel Recap) . . . 187
15.4.2 Extension to Multiple Channels . . . 187
15.4.3 Structure of the Bias Vector . . . 187
15.5 Inductive Biases Encoded by Convolution . . . 187

15.5.1 Locality . . . 188
15.5.2 Translation Equivariance (Parameter Sharing) . . . 188
15.5.3 Why These Biases Matter . . . 188
15.6 When (Not) to Use 2D Convolution . . . 188
15.7 Receptive Field . . . 188
15.8 Receptive Field . . . 189
15.8.1 Definition . . . 189
15.8.2 Computing the Receptive Field . . . 189
15.8.3 Hierarchical Feature Extraction . . . 189
15.9 Backpropagation through Non-Linear Layers . . . 190
15.9.1 Max Pooling Gradient . . . 190
15.9.2 ReLU Gradient . . . 190
15.10 The ImageNet Breakthrough . . . 190
15.10.1 Before 2012: Shallow Models + Handcrafted Features . . . 190
15.10.2 2012: AlexNet — The Deep Learning Revolution . . . 190
15.10.3 Enabling Factors . . . 190
15.11 Practical Deep Learning with Limited Data . . . 191
15.11.1 The Problem . . . 191
15.11.2 Two Key Strategies . . . 191
15.12 Example: Steller Sea Lion Monitoring . . . 191
15.13 Data Augmentation . . . 191
15.13.1 The Steller Sea Lion Counting Problem . . . 191
15.13.2 Geometric and Photometric Transformations . . . 192
15.13.3 MixUp Augmentation . . . 192
15.13.4 Benefits of Augmentation . . . 192
15.13.5 Pitfalls and Best Practices . . . 193
15.13.6 Implementation in Deep Learning Frameworks . . . 193
15.13.7 Limitations . . . 193
15.14 Test-Time Augmentation (TTA) . . . 193
15.15 Designing Augmentations: Label-Specific vs. Label-Agnostic . . . 194
15.16 Transfer Learning . . . 194
15.16.1 Motivation . . . 194
15.16.2 Procedure . . . 194
15.16.3 Why It Works . . . 195
15.17 Fine-Tuning . . . 195
15.17.1 When to Fine-Tune . . . 195
15.17.2 Best Practice: Two-Stage Training . . . 195
15.17.3 Risks . . . 195
15.18 Pretrained Models in PyTorch . . . 195

16 Famous CNN Architectures **197**
16.1 Historical Perspective on CNN Architectures . . . 197
16.1.1 LeNet-5 (1998) – The First Convolutional Neural Network . . . 197
16.1.2 Impact of Padding on Parameter Count . . . 198
16.1.3 CNN vs. MLP for Image Input . . . 198
16.2 AlexNet (2012) – The Deep Learning Breakthrough . . . 198
16.3 Key Takeaways on CNN Network Design . . . 199
16.4 AlexNet (2012) – Details . . . 199
16.4.1 Architecture and Scale . . . 199
16.4.2 Novel Design Choices . . . 199
16.4.3 Two-GPU Implementation . . . 199
16.4.4 Ensembling . . . 200
16.5 VGG Network (2015) – The Power of Small Filters . . . 200
16.5.1 Philosophy: Go Deeper with Smaller Kernels . . . 200

16.5.2 VGG Architecture Pattern . . . 200
16.5.3 Memory Footprint . . . 200
16.6 Understanding CNNs via Filter Visualisation . . . 201
16.6.1 First-Layer Filters as Templates . . . 201
16.6.2 AlexNet First-Layer Filters . . . 201
16.7 Interpreting CNNs Beyond the First Layer . . . 201
16.7.1 Limitations of Filter Visualisation in Deeper Layers . . . 201
16.7.2 Activation Maximisation via Training Set Patches . . . 201
16.7.3 Convolutional Layers as a Cascaded Non-Linear Filter . . . 202
16.7.4 Gradient Ascent for Input Optimisation . . . 202
16.7.5 Network Dreams and Class Visualisation . . . 202
16.7.6 Summary: Hierarchical Feature Learning in CNNs . . . 203
16.8 Network in Networks (2014) – New Layer Types . . . 203
16.8.1 Motivation . . . 203
16.8.2 MLP-Conv Layer . . . 203
16.8.3 Global Average Pooling (GAP) . . . 203
16.8.4 NiN Architecture Summary . . . 204
16.9 Translation Invariance and Global Average Pooling . . . 205
16.9.1 The Problem with Flattening . . . 205
16.9.2 GAP Enables Input-Size Flexibility . . . 205
16.9.3 Translation Invariance: GAP vs. Flattening . . . 205
16.9.4 Illustrative Experiment: Shifted Digits . . . 205
16.10 MobileNets: Lightweight Architectures for Edge Devices . . . 206
16.10.1 Motivation . . . 206
16.10.2 Standard Convolution Cost . . . 206
16.10.3 Depthwise Separable Convolution . . . 206
16.10.4 MobileNet Architecture Features . . . 207
16.11 Landscape of CNN Architectures (2012–2017) . . . 207
16.12 Practical Use: Transfer Learning with MobileNet . . . 207

17 Image Augmentation and Transfer Learning 209
17.1 Evolution of Convolutional Architectures . . . 209
17.1.1 From AlexNet to Advanced Blocks . . . 209
17.1.2 Parameter Explosion in the Classifier Head . . . 209
17.2 Global Average Pooling (GAP) – Network in Network . . . 209
17.2.1 Motivation and Operation . . . 209
17.2.2 Advantages . . . 210
17.2.3 Impact on Architecture . . . 210
17.3 Inception Block – Multi-Scale Feature Processing . . . 210
17.3.1 Concept . . . 210
17.3.2 Dimensionality Reduction with 1×1 Convolutions . . . 210
17.3.3 Building an Inception Block in PyTorch . . . 211
17.3.4 Advantages of Multi-Scale Processing . . . 211
17.4 Summary . . . 211
17.5 Inception Module – Multi-Scale Processing (cont.) . . . 211
17.5.1 Parallel Paths and Concatenation . . . 211
17.6 Batch Normalization . . . 212
17.6.1 Motivation and Operation . . . 212
17.6.2 Placement and Benefits . . . 212
17.7 Residual Networks (ResNet) . . . 212
17.7.1 The Vanishing Gradient Problem . . . 212
17.7.2 Residual Block – Skip Connection . . . 212
17.7.3 Implementation Details . . . 213
17.7.4 Network Architecture . . . 213

17.8 Squeeze-and-Excitation (SE) Block . . . 213
17.8.1 Channel-wise Attention . . . 213
17.8.2 Three Steps . . . 213
17.8.3 Integration . . . 213
17.9 Summary of Architectural Evolution . . . 214
17.10MobileNet: Lightweight Architecture with Inverted Residuals . . . 214
17.10.1 Inverted Residual Bottlenecks . . . 214
17.10.2 Depthwise Separable Convolution (Recap) . . . 214
17.10.3 Squeeze-and-Excitation in MobileNet . . . 214
17.10.4 Activation Functions . . . 215
17.10.5 Architecture Summary . . . 215
17.11Baseline Training without Augmentation: Animal vs. Object Classification . . . 215
17.11.1 Dataset . . . 215
17.11.2 Model: Simple CNN (AlexNet style) . . . 215
17.11.3 Training and Overfitting . . . 215
17.12Image Augmentation . . . 216
17.12.1 Motivation . . . 216
17.12.2 PyTorch Implementation . . . 216
17.12.3 Geometric Augmentations . . . 216
17.13Geometric Augmentations (cont.) . . . 217
17.13.1 Random Affine Transformations . . . 217
17.13.2 Composing Multiple Geometric Transforms . . . 217
17.14Photometric Augmentations . . . 217
17.14.1 Color Jitter . . . 217
17.14.2 Composing Geometric and Photometric Augmentations . . . 217
17.15Important Considerations for Augmentation . . . 218
17.15.1 Label Preservation . . . 218
17.15.2 Advanced Augmentations: Random Erasing and Cutout . . . 218
17.16Implementing Augmented Datasets in PyTorch . . . 218
17.16.1 Custom Dataset with Transforms . . . 218
17.16.2 Training with Augmentation . . . 218
17.17Transfer Learning and Fine-Tuning with MobileNet . . . 219
17.17.1 The Oxford Pets Dataset . . . 219
17.17.2 Preprocessing for Pretrained Models . . . 219
17.17.3 Three Training Strategies . . . 219
17.17.4 Implementation Outline in PyTorch . . . 220
17.18Transfer Learning and Fine-Tuning with MobileNet (cont.) . . . 220
17.18.1 Training Strategies . . . 220
17.18.2 Implementation Details . . . 221
17.18.3 Performance Comparison . . . 221
17.19Image Retrieval in the Latent Space . . . 221
17.19.1 Motivation and Concept . . . 221
17.19.2 Using a Pretrained Network as a Feature Extractor . . . 221
17.19.3 Example: Impact of Feature Quality . . . 221
17.19.4 Workflow . . . 222

18 WSL (CAM), Explainability, Other Architectures 223
18.1 Data Pre-processing for Neural Networks . . . 223
18.1.1 Why Normalise Input Data . . . 223
18.1.2 Common Normalisation Techniques . . . 223
18.1.3 Pre-processing for Image Data . . . 223
18.1.4 Crucial Rules . . . 223
18.2 Batch Normalisation . . . 224
18.2.1 Motivation . . . 224

18.2.2 Operation . . . 224
18.2.3 Parameters . . . 224
18.2.4 Training vs. Inference . . . 225
18.2.5 Fusing Batch Normalisation with a Preceding Convolution . . . 225
18.3 Outlook . . . 225
18.4 Multi-label Classification . . . 225
18.4.1 Limitation of Single-Label Assumption . . . 225
18.4.2 Architecture Modification . . . 225
18.5 Weakly Supervised Localisation with Class Activation Maps . . . 226
18.5.1 Can a Classification Network Localise Objects? . . . 226
18.5.2 Required Architecture . . . 226
18.5.3 Class Activation Map Derivation . . . 226
18.5.4 Visualising and Using the CAM . . . 226
18.5.5 Why This Works . . . 227
18.5.6 Practical Implications . . . 227
18.6 Class Activation Maps (CAM) for Weakly Supervised Localization . . . 227
18.6.1 From Global Average Pooling to the Final Score . . . 227
18.6.2 Intuitive Interpretation . . . 228
18.6.3 Visualising the CAM . . . 228
18.6.4 Two Major Uses . . . 228
18.6.5 Limitation of CAM . . . 228
18.6.6 GAP vs. Global Max Pooling . . . 229
18.7 Weakly Supervised Localization from CAM . . . 229
18.7.1 Problem Setting . . . 229
18.7.2 Procedure . . . 229
18.7.3 Example . . . 229
18.8 Going Deeper: Parallel Convolutional Architectures . . . 229
18.8.1 Motivation for Parallel Paths . . . 229
18.8.2 Inception Module: Multi-Scale Processing in One Block . . . 230
18.8.3 Concatenation and Dimensionality Example . . . 230
18.8.4 Why Parallel Convolutions Help . . . 230
18.9 Inception Networks: Multi-Scale Processing with Bottlenecks . . . 230
18.9.1 The Need for Efficient Multi-Scale Blocks . . . 230
18.9.2 Reducing Computation with 1×1 Bottlenecks . . . 231
18.10 GoogLeNet (Inception v1) Architecture . . . 231
18.10.1 Auxiliary Classifiers for Gradient Flow . . . 231
18.11 The Problem of Very Deep Networks . . . 232
18.12 Residual Networks (ResNet) . . . 232
18.12.1 Skip Connections and Residual Learning . . . 232
18.12.2 Block Design and Practical Considerations . . . 232
18.12.3 Impact and Variants . . . 232
18.13 Concluding Remarks and Outlook . . . 233

19 Semantic Segmentation, Fully Convolutional CNN 235
19.1 Introduction to Image Segmentation . . . 235
19.1.1 From Unsupervised Clustering to Semantic Segmentation . . . 235
19.1.2 Distinction from Instance Segmentation . . . 235
19.2 Supervised Learning Setup for Segmentation . . . 235
19.2.1 Training Data Requirements . . . 235
19.2.2 The COCO Dataset (2014) . . . 236
19.3 Utility of Semantic Segmentation . . . 236
19.4 A Naive Baseline: Patch-Based Classification . . . 236
19.4.1 Principle . . . 236
19.4.2 Why Central-Pixel Labelling? . . . 236

19.4.3 Limitations . . . 236
19.5 Semantic Segmentation: Definition and Motivation . . . 237
19.5.1 From Unsupervised Clustering to Pixel-Wise Classification . . . 237
19.5.2 Distinction from Instance Segmentation and Object Detection . . . 237
19.6 Supervised Learning Setup for Segmentation . . . 237
19.6.1 Training Data and Annotation . . . 237
19.7 A Naive Baseline: Patch-Based Classification . . . 237
19.7.1 Central-Pixel Labelling . . . 237
19.7.2 Severe Limitations . . . 238
19.8 Fully Convolutional Networks (FCNs) for Segmentation . . . 238
19.8.1 Convolution-Only Architecture . . . 238
19.8.2 Limitations . . . 238
19.9 Encoder-Decoder Architecture . . . 238
19.9.1 Conflicting Goals . . . 238
19.9.2 The Spatial Bottleneck . . . 239
19.10 Upsampling Strategies for the Decoder . . . 239
19.10.1 Non-Learnable Upsampling . . . 239
19.10.2 Learnable Upsampling with Convolutions . . . 239
19.10.3 Transposed Convolution (Learnable Upsampling) . . . 239
19.11 Transposed Convolution Details . . . 240
19.11.1 Filter Dimensions . . . 240
19.11.2 Terminology . . . 240
19.11.3 Learnable Upsampling Blocks . . . 240
19.12 Training a Segmentation Network . . . 240
19.12.1 Loss Function . . . 240
19.12.2 Extension to Per-Pixel Regression . . . 240
19.13 U-Net: The Classic Encoder–Decoder Architecture . . . 241
19.13.1 Overview . . . 241
19.13.2 Skip Connections . . . 241
19.13.3 Why Valid Padding? . . . 241
19.13.4 Training Details . . . 241
19.14 Data Augmentation for Segmentation . . . 242
19.14.1 Transformations Used . . . 242
19.15 Transposed Convolution Details . . . 242
19.15.1 Filter Dimensions and Stride . . . 242
19.15.2 Terminology . . . 242
19.15.3 Learnable Upsampling Blocks . . . 242
19.16 Training a Segmentation Network . . . 243
19.16.1 Pixel-Wise Loss Function . . . 243
19.16.2 Extension to Dense Regression . . . 243
19.17 U-Net Architecture . . . 243
19.17.1 Overview . . . 243
19.17.2 Skip Connections . . . 243
19.17.3 Why Valid Padding? . . . 244
19.17.4 Training Details . . . 244
19.18 Data Augmentation for Segmentation . . . 244
19.18.1 Elastic Deformations . . . 244
19.18.2 Augmenting the Segmentation Mask . . . 244
19.18.3 Separation of Touching Objects . . . 244
19.19 Weighted Loss Functions in U-Net . . . 244
19.19.1 Motivation . . . 244
19.19.2 Class-Specific Weights (w_c) . . . 245
19.19.3 Boundary Weight Map . . . 245
19.20 Case Study: Kidney Biopsy Segmentation . . . 245

19.20.1 Problem and Annotation Scarcity . . . 245
19.20.2 Domain Shift and Scribble Annotations . . . 245
19.20.3 Fine-Tuning for Patient Adaptation . . . 245
19.21 From Dense to Convolutional Layers: Revisiting the Connection . . . 245
19.21.1 Feeding Larger Images to a CNN . . . 245
19.21.2 Simplifying to a 1×1 Activation and Dense Layer . . . 246
19.21.3 Linear Combination View . . . 246
19.22 Reinterpreting Dense Layers as Convolutional Layers . . . 246
19.22.1 The Linear Combination View . . . 246
19.22.2 Equivalence to 1×1 Convolution . . . 246
19.22.3 Extending to Multi-Layer Perceptrons . . . 247
19.22.4 Handling Larger Activation Maps (Flattening and GAP) . . . 247
19.23 Fully Convolutional Conversion for Dense Prediction . . . 247
19.23.1 From Classifier to Segmentation Heatmaps . . . 247
19.23.2 Implementation: Weight Transfer . . . 247
19.23.3 Shift-and-Stitch for Higher Resolution . . . 247
19.23.4 When to Use This Approach . . . 248
19.24 Closing the Loop: CNN $\leftrightarrow$ Dense Layers . . . 248
19.24.1 Duality of Interpretations . . . 248
19.24.2 Importance of This Understanding . . . 248
19.25 Outlook . . . 248

20 Famous Architectures, Object Localisation and Class Activation Maps 249
20.1 Advanced Architectures based on ResNet . . . 249
20.1.1 ResNet Recap . . . 249
20.1.2 Wide ResNet . . . 249
20.1.3 ResNeXt . . . 250
20.1.4 Common Structure and Implementation Notes . . . 250
20.2 DenseNet . . . 251
20.2.1 Core Idea . . . 251
20.2.2 Composite Function and Growth Rate . . . 251
20.2.3 Block Structure and Implementation . . . 251
20.2.4 Full Model . . . 251
20.2.5 Visualization (Graph) . . . 251
20.3 EfficientNet . . . 252
20.3.1 Motivation: Balancing Network Dimensions . . . 252
20.3.2 Compound Scaling Formulation . . . 252
20.3.3 Neural Architecture Search (NAS) for Baseline . . . 252
20.3.4 MBConv Block (Mobile Inverted Bottleneck with Squeeze-Excitation) . . . 252
20.3.5 Overall Architecture . . . 253
20.4 Object Localization . . . 253
20.4.1 From Classification to Localization . . . 253
20.4.2 Regression Formulation . . . 253
20.4.3 Coordinate Normalization . . . 254
20.5 Data Preparation for Localization . . . 254
20.5.1 Dataset and Annotations . . . 254
20.5.2 Image Loading and Preprocessing . . . 254
20.5.3 Dataset Class . . . 254
20.6 Combined Classification and Localization . . . 254
20.6.1 Two Independent Networks (Naive Approach) . . . 254
20.6.2 Multi-Task Learning Approach (Preview) . . . 255
20.7 Bounding Boxes and Data Augmentation . . . 255
20.8 Weakly Supervised Localisation via Class Activation Maps . . . 255
20.8.1 From CAM to Bounding Boxes . . . 255

20.8.2 Dataset with Both Cats and Dogs . . . 255
20.9 Squeeze-and-Excitation Block (within EfficientNet's MBConv) . . . 256
20.9.1 Function of the SE Block . . . 256
20.9.2 MBConv Block Structure . . . 256
20.10 Object Localization . . . 256
20.10.1 From Classification to Bounding Box Regression . . . 256
20.10.2 Loss Function and Output Layer . . . 257
20.11 Data Preparation for Localization . . . 257
20.11.1 Dataset Structure and Preprocessing . . . 257
20.11.2 Dataset Class for Both Tasks . . . 257
20.11.3 Bounding Boxes and Data Augmentation . . . 257
20.12 Model Architecture for Localization . . . 258
20.12.1 Transfer Learning with EfficientNet . . . 258
20.12.2 Interpretation of the Final Layer . . . 258
20.13 Training and Evaluation Loop . . . 258
20.13.1 Batch Normalisation and Dropout Modes . . . 258
20.13.2 Training Step . . . 258
20.13.3 Validation and Early Stopping . . . 258
20.14 Combined Classification and Localization . . . 259
20.14.1 Two Independent Networks . . . 259
20.14.2 Multi-Task Learning (Preview) . . . 259
20.15 Weakly Supervised Localisation via Class Activation Maps . . . 259
20.16 Fine-Tuning the Classifier . . . 259
20.16.1 Unfreezing Selected Layers . . . 259
20.16.2 Training Configuration . . . 259
20.16.3 Results . . . 260
20.17 Weakly Supervised Localization via Class Activation Maps (CAM) . . . 260
20.17.1 Motivation . . . 260
20.17.2 CAM Computation . . . 260
20.17.3 Extracting Bounding Boxes from CAM . . . 260
20.18 Grad-CAM: Generalization of CAM . . . 260
20.18.1 Limitations of CAM . . . 260
20.18.2 Grad-CAM Principle . . . 261
20.18.3 Advantages over CAM . . . 261
20.18.4 Implementation Notes . . . 261
20.19 Handling Multiple Objects with CAM . . . 261

21 Fully Convolutional Neural Networks and Semantic Segmentation 263
21.1 Baseline CNN for Sea Lion Patch Classification . . . 263
21.1.1 Dataset and Preprocessing . . . 263
21.1.2 Standard CNN Architecture . . . 263
21.1.3 Training and Performance . . . 263
21.2 Motivation for Fully Convolutional Conversion . . . 264
21.3 Converting Dense Layers to Convolutional Layers . . . 264
21.3.1 The Core Principle . . . 264
21.3.2 Applying to Our Network . . . 264
21.3.3 Parameter Count Remains Identical . . . 264
21.4 From Classification to Dense Prediction . . . 265
21.5 Summary . . . 265
21.6 Fully Convolutional Conversion of the Sea Lion Classifier . . . 265
21.6.1 Reinterpreting Dense Layers as Large-Kernel Convolutions . . . 265
21.6.2 Advantage: Processing Larger Images . . . 266
21.7 Weight Transfer from Trained Dense Layers . . . 266
21.7.1 Reshaping Weights . . . 266

21.7.2 Verification of Exact Equivalence . . . 266
21.8 Dense Prediction on Larger Images . . . 267
21.8.1 Generating Heatmaps . . . 267
21.8.2 Application Context . . . 267
21.9 Dense Prediction on Large Sea Lion Images . . . 267
21.9.1 Generating Class Heatmaps . . . 267
21.9.2 Per-Class Heatmap Visualisation . . . 267
21.10 Semantic Segmentation . . . 268
21.10.1 Definition . . . 268
21.10.2 Cityscapes Dataset (Downsampled) . . . 268
21.10.3 Data Handling and Augmentation . . . 268
21.10.4 Visualisation . . . 268
21.11 UNet Architecture for Semantic Segmentation . . . 268
21.11.1 Motivation . . . 268
21.11.2 Encoder (Contracting Path) . . . 269
21.11.3 Decoder (Expanding Path) . . . 269
21.11.4 Skip Connections . . . 269
21.11.5 Final Layer . . . 269
21.11.6 Implementation Approach . . . 269
21.12 U-Net Implementation for Semantic Segmentation . . . 269
21.12.1 Architecture Construction . . . 269
21.12.2 Network Dimensions . . . 270
21.13 Training the U-Net . . . 270
21.13.1 Loss and Metrics . . . 270
21.13.2 Training Loop and Visualisation . . . 270
21.13.3 Training Behaviour and Overfitting . . . 271
21.14 Test Evaluation and Qualitative Results . . . 271
21.14.1 Performance on Test Set . . . 271
21.14.2 Visualising Predictions . . . 271
21.15 Per-Class Heatmap Visualisation . . . 271
21.15.1 Before the Argmax . . . 271
21.16 Summary . . . 271

22 Object Detection **273**
22.1 From Localization to Object Detection . . . 273
22.1.1 Recap of Single-Object Localization . . . 273
22.1.2 Extending Regression to Multiple Landmarks . . . 273
22.1.3 Multi-Task Learning: Classification + Localization . . . 274
22.1.4 Towards Object Detection: Multiple Instances per Image . . . 274
22.2 From Single-Object Localization to Object Detection . . . 275
22.2.1 Recap: Classification vs. Localization . . . 275
22.2.2 Extending Regression to Multiple Landmarks . . . 275
22.2.3 Multi-Task Learning: Joint Classification and Localization . . . 275
22.2.4 The Object Detection Problem: Multiple Instances per Image . . . 275
22.3 Evaluation Metric: Intersection over Union (IoU) . . . 276
22.4 Early Deep Learning Object Detectors . . . 276
22.4.1 Traditional Region Proposal Algorithms . . . 276
22.4.2 R-CNN: Region-based Convolutional Neural Network . . . 276
22.4.3 Fast R-CNN: Shared Feature Maps and RoI Pooling . . . 276
22.4.4 Faster R-CNN: Learnable Region Proposals . . . 277
22.5 Region Proposal Network (RPN) – Making Proposals Learnable . . . 277
22.5.1 Motivation . . . 277
22.5.2 RPN Operation . . . 277
22.5.3 Anchor Boxes . . . 278

22.5.4 Training the RPN . . . 278
22.6 Faster R-CNN: Unified Object Detection . . . 278
22.6.1 Architecture Overview . . . 278
22.6.2 Training Strategy . . . 278
22.6.3 Speed and Accuracy . . . 279
22.7 Single-Shot Detectors: YOLO . . . 279
22.7.1 Motivation and Philosophy . . . 279
22.7.2 Core Idea . . . 279
22.7.3 Comparison with Anchor-Based Methods . . . 279
22.7.4 Impact . . . 279
22.8 Summary: Evolution of Object Detection . . . 279
22.9 YOLO – Single-Shot Object Detection . . . 280
22.9.1 Core Idea . . . 280
22.9.2 Anchors in YOLO . . . 280
22.9.3 Training and Inference . . . 280
22.10 Instance Segmentation – Mask R-CNN . . . 280
22.10.1 Motivation . . . 280
22.10.2 Mask R-CNN . . . 281
22.10.3 Human Pose Estimation with Mask R-CNN . . . 281
22.11 Feature Pyramid Networks (FPN) . . . 281
22.11.1 Motivation . . . 281
22.11.2 Architecture . . . 281
22.12 Advanced Research Directions . . . 281
22.12.1 3D Object Detection and Autonomous Driving . . . 281
22.12.2 Weakly Supervised and Self-Supervised Detection . . . 282
22.12.3 Other Active Topics . . . 282
22.13 Course Summary and Advanced Deep Learning . . . 282

23 Multi-Task Learning, Object Detection and Instance Segmentation 283
23.1 Multi-Task Learning for Joint Classification and Localization . . . 283
23.1.1 Concept . . . 283
23.1.2 Dataset and Annotations . . . 283
23.1.3 Data Augmentation with Bounding Boxes . . . 283
23.1.4 Data Splitting and Data Loaders . . . 284
23.2 Model Architecture for Multi-Task Localization . . . 284
23.2.1 Backbone and Feature Sharing . . . 284
23.2.2 Task-Specific Heads . . . 284
23.2.3 Forward Pass . . . 284
23.3 Training Considerations . . . 284
23.3.1 Loss Function . . . 284
23.3.2 Evaluation Metric . . . 285
23.4 Training and Evaluation of the Multi-Task Model . . . 285
23.4.1 Loss Combination . . . 285
23.4.2 Training and Validation Loops . . . 285
23.4.3 Training Results . . . 285
23.5 From Localization to Object Detection . . . 286
23.5.1 The Core Difference . . . 286
23.6 The Penn-Fudan Pedestrian Dataset . . . 286
23.6.1 Overview . . . 286
23.6.2 From Masks to Bounding Boxes . . . 286
23.6.3 Dataset Class . . . 286
23.7 Training Considerations for Object Detection . . . 287
23.8 Dataset Preparation for Object Detection . . . 287
23.8.1 On-the-Fly Image Loading . . . 287

23.8.2 Geometric Augmentation via Dataset Concatenation . . . 287
23.8.3 Data Loaders and Batch Size Considerations . . . 288
23.9 Object Detection vs. Semantic and Instance Segmentation . . . 288
23.10 Evaluation Metrics for Object Detection . . . 288
23.10.1 Intersection over Union (IoU) . . . 288
23.10.2 Precision, Recall, and F1 Score . . . 288
23.11 Historical Object Detection Architectures . . . 289
23.11.1 R-CNN (Region-based CNN) . . . 289
23.11.2 Fast R-CNN . . . 289
23.12 Faster R-CNN: Learnable Region Proposals . . . 289
23.12.1 Motivation . . . 289
23.12.2 Region Proposal Network (RPN) . . . 290
23.12.3 Feature Pyramid Network (FPN) Extension . . . 290
23.13 Loading a Pre-trained Faster R-CNN from TorchVision . . . 290
23.13.1 Model Architecture Breakdown . . . 290
23.14 Evaluating the Pre-trained Model on Penn-Fudan . . . 291
23.15 Inspecting the Region Proposal Network . . . 291
23.15.1 Objectness Heatmaps . . . 291
23.15.2 Proposal Generation . . . 291
23.16 Non-Maximum Suppression (NMS) . . . 292
23.16.1 Principle . . . 292
23.16.2 Disabling NMS in Code . . . 292
23.17 Region Proposal Network: From Anchors to Proposals . . . 292
23.17.1 Anchor Generation and Bounding-Box Decoding . . . 292
23.17.2 NMS in the RPN . . . 292
23.17.3 RPN Proposals vs. Final Detections . . . 293
23.18 Fine-Tuning Faster R-CNN on a Custom Dataset . . . 293
23.18.1 Training Loop with Multi-Task Loss . . . 293
23.18.2 Optimizer and Learning Rate Scheduler . . . 293
23.18.3 Training Progress and Metrics . . . 293
23.18.4 Confidence Threshold Tuning . . . 293
23.19 Instance Segmentation with Mask R-CNN . . . 294
23.19.1 Architecture Extension . . . 294
23.19.2 Fine-Tuning and Results . . . 294

Chapter 1

Introduction to Artificial Neural Networks and Deep Learning

1.1 What is Machine Learning?

Machine learning is a category of research in algorithms focused on finding patterns in data and using those patterns to make predictions. It lies under the broader umbrella of artificial intelligence and intersects with many other disciplines, including statistics, data mining, and pattern recognition.

In practice, machine learning is not a single technique but rather a collection of methods that have been discovered and rediscovered over the years by different communities, including AI, statistics, business analytics, and data mining. As a result, some techniques have different names depending on the field in which they were developed.

Many topics fall within machine learning, such as:

- classification,
- regression,
- supervised learning,
- linear discriminant analysis,
- ridge regression,
- model assessment,
- validation and cross-validation.

Even though this course will focus mainly on neural networks and deep learning, many of these concepts will still be relevant.

1.2 Supervised, Unsupervised, and Reinforcement Learning

A classical distinction in machine learning is between supervised learning and unsupervised learning. We also mention reinforcement learning for completeness, although it will not be covered in this course.

1.2.1 Supervised Learning

In supervised learning, we are given a dataset

$$\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\},$$

where each x_i is an input vector and each y_i is the corresponding target.

A very important point is that, in this course, all x_i are vectors, not scalars. Each input corresponds to one record in the dataset, represented as a vector of features.

The goal of supervised learning is to learn a function that can predict the correct target for a new input.

1.2.2 Unsupervised Learning

In unsupervised learning, no target labels are provided. The main difference from supervised learning is precisely the absence of the target given by a human or another external source.

The goal is to exploit regularities in the data in order to build an informative representation. A classical example is clustering.

1.2.3 Reinforcement Learning

In reinforcement learning, the learner receives a reward signal that indicates whether an action was good or bad. However, unlike supervised learning, the reward does not directly specify the correct output.

The reward may also be delayed. For example, in finance, an action taken today may only be judged after some time has passed. Reinforcement learning is therefore concerned with long-term maximization of return. This topic will not be covered in this course.

1.3 Classification and Regression

Within supervised learning, there are two main types of problems: classification and regression.

1.3.1 Classification

In classification, the output is one element from a finite set of possible labels. For example, an image may be classified as either *car* or *motorcycle*. This is a binary classification problem because there are two possible classes. If there are more than two classes, the problem is called multiclass classification.

Suppose we have a feature vector describing an object. A classifier is a machine learning model that maps the feature vector to one of the allowed labels.

Typical classifiers include:

- decision trees,
- logistic regression,
- support vector machines,
- neural networks.

1.3.2 Regression

In regression, the output is a number rather than a class label. The model predicts a real-valued quantity instead of choosing one label from a finite set.

Thus, regression problems differ from classification problems in the nature of the output:

- classification outputs a label from a fixed set;
- regression outputs a numerical value.

These differences affect both the architecture of the model and the choice of loss function.

1.4 Unsupervised Learning Example: Clustering

Unsupervised learning is about finding regularities in data without labels. A classical example is clustering.

Imagine a set of images containing cars, motorcycles, and other objects. Even without labels, we may observe that some images are similar to one another and form groups. Discovering these groups is the goal of clustering.

This is a classic data mining task. Although clustering will not be covered in this course, it is conceptually related to the kinds of representation learning that will appear later, such as word embeddings.

1.5 From Classical Machine Learning to Deep Learning

Classical machine learning often follows the pattern:

$$\text{input features} \longrightarrow \text{hand-crafted representation} \longrightarrow \text{classifier}.$$

For example, in image classification one may use features such as:

- size,
- color,
- number of wheels,
- shape descriptors.

The classifier then learns a decision boundary in feature space.

The Iris dataset is a famous example. It dates back to 1936 and was used by Ronald Fisher. The dataset contains measurements such as:

- sepal length,
- sepal width,
- petal length,
- petal width.

These measurements define a vector space in which the classifier tries to separate the different flower classes.

In many classical classification problems, the main task is to find a decision boundary that separates the classes. If the structure of the data is simple, a linear classifier may be sufficient. If the structure is more complex, one may need a nonlinear classifier.

1.6 Deep Learning

Deep learning is a modern development within machine learning that became especially prominent around 2012. The rise of deep learning was driven by the availability of large datasets, increased computational power, and improved training methods.

The idea of deep learning is to learn representations automatically, rather than relying entirely on hand-crafted features. In this sense, deep learning changes the traditional pipeline:

$$\text{raw data} \longrightarrow \text{learned representation} \longrightarrow \text{prediction}.$$

This is one of the main reasons deep learning has become so influential in recent years.

1.7 Classical Machine Learning

A classic example in machine learning is to classify data points using a suitable decision rule. Sometimes a problem looks difficult at first, but becomes easy once the right classifier is chosen.

For instance, a dataset may be separable by:

- nearest neighbor methods,
- radial basis function machines,
- naive Bayes,

while linear support vector machines may fail on the same data.

In classical machine learning, the main tasks are:

- understanding the data,
- selecting the appropriate classifier,
- designing the right features for the problem.

1.8 Why Feature Choice Matters

A problem may appear impossible to solve simply because the wrong features were chosen. In many cases, the data are distinguishable, but not with the features currently being used.

A useful analogy is the *codice fiscale* or social security number. If the correct digits are examined, some hidden information, such as gender, can be recovered easily. However, if the wrong digit is inspected, no useful separation appears.

This leads to the central idea of **feature engineering**:

$$\text{raw data} \longrightarrow \text{hand-crafted features} \longrightarrow \text{classifier}.$$

Feature engineering uses domain knowledge to extract informative features that make classes easier to separate.

1.9 Hand-Crafted Features

In older systems, one often first extracted low-level features from raw data and then trained a classifier on top of them.

For speech recognition, examples include:

- frequency coefficients,
- MFCC-like descriptors,
- cepstral coefficients.

For images, examples include:

- SIFT features,
- HOG features,
- corner-based descriptors.

These are often called low-level or mid-level features. After extracting them, one may cluster, pool, or classify them.

The problem is that good features are hard to design, and features that work well for one task may fail on another. Features useful for flowers may not work for cars or faces.

1.10 Deep Learning

Deep learning changes this paradigm. Instead of manually designing all features, the model learns the features directly from data.

The key idea is:

$$\text{raw data} \longrightarrow \text{learned representation} \longrightarrow \text{task output}.$$

Deep learning is therefore also called:

- **representation learning**,
- **end-to-end learning**.

The point is not only to learn a classifier, but also to learn a useful representation of the data that makes the task easier.

1.11 Why Deep Learning Became Important

Deep learning became especially successful because of:

- large amounts of data,
- stronger computational resources,
- better training algorithms,
- improved neural network architectures.

A major turning point was the rise of very large datasets and benchmarks such as ImageNet, which contains about 1.5 million images in 1,000 categories. Performance on this task improved dramatically with convolutional neural networks.

This showed that the key enabler was not only computation, but also data.

1.12 ImageNet and Convolutional Neural Networks

The success of deep learning on ImageNet demonstrated that neural networks could learn strong visual representations directly from data.

A convolutional neural network learns a hierarchy of features:

- low-level features such as edges and color patches,
- mid-level features such as textures and parts,
- high-level features such as objects and object components.

These features are not manually coded. They are learned from data.

1.13 Tasks Enabled by Deep Learning

Deep learning has enabled many tasks that were once difficult:

- image classification,
- semantic segmentation,
- instance segmentation,
- face identification,
- visual search,
- super-resolution,
- style transfer.

1.13.1 Semantic Segmentation

In semantic segmentation, each pixel is assigned a class label. This goes beyond simple image classification, since the goal is to label every pixel in the image.

1.13.2 Instance Segmentation

In instance segmentation, the model does not only detect objects, but also separates different instances of the same class. For example, it can distinguish multiple people in the same image.

1.13.3 Style Transfer and Super-Resolution

Style transfer changes the artistic style of an image while preserving its content. Super-resolution increases the resolution of an image and reconstructs fine detail.

1.14 Main Message

The main lesson of this lecture is:

> Deep learning is about learning features and representations from data so that the final task becomes easy.

Instead of manually designing the full processing pipeline, we let neural networks learn both the representation and the solution to the task.

That is why the next topic is neural networks.

1.15 Feed Forward Neural Networks and Backprop

1.16 AI, Machine Learning, and Deep Learning

Before introducing the perceptron, it is useful to briefly distinguish between *artificial intelligence*, *machine learning*, and *deep learning*.

Deep learning can be viewed as a special subset of machine learning in which both the representation and the predictor are learned from data. In other words, instead of following the classical pipeline

$$\text{raw data} \to \text{feature engineering} \to \text{classification/regression},$$

deep learning aims at an end-to-end approach:

$$\text{raw data} \to \text{answer}.$$

Today, when people say *AI*, they often think of deep learning applications such as generative AI, natural language processing, and image recognition. However, artificial intelligence is much broader. It includes:

- logic,
- planning,
- multi-agent systems,
- search and reasoning,
- and many other classical topics.

Artificial intelligence was first developed in the 1950s, and neural networks have been part of that story from the beginning.

1.17 Historical Origin of Neural Networks

Neural networks did not appear only with modern deep learning. Their roots go back to the earliest days of AI. In fact, neural networks were already being studied in the 1950s, and they remained part of machine learning research throughout the 1980s and beyond.

The reason they became much more popular later is that they are a natural tool for end-to-end learning. With the introduction of backpropagation, it became possible to train deep models directly from features to final decisions.

Historically, we can roughly divide the story as follows:

- the 1950s: the beginnings of AI and early neural network models;
- the 1980s: the rise of machine learning and training algorithms for neural nets;
- the last 15 years: the deep learning revolution, especially in image processing.

1.18 Why AI Was Studied

Artificial intelligence was formally founded in 1956 by four researchers: John McCarthy, Marvin Minsky, Nathan Rochester, and Claude Shannon. The famous Dartmouth proposal stated that any aspect of learning or intelligence could in principle be precisely described so that a machine could simulate it.

The original goal was not necessarily to create a human brain, but rather to model aspects of human intelligence and reproduce them computationally.

One important application at the time was automatic computing, including the development of compilers. The idea that one could describe a program and have a machine generate machine code was already present in spirit.

1.19 Why Neural Networks Were Interesting

In the 1940s and 1950s, computers were very different from what we have today. They were:

- brittle,
- serial,
- not adaptive,
- not fault tolerant,
- and poor at dealing with noisy data.

At the time, computers were still useful for numerical computation, but they lacked many properties that people associated with intelligent behavior.

This led researchers in AI to ask whether new computation architectures were needed. Their answer was inspired by the brain.

1.20 The Brain as Inspiration

The human brain contains a huge number of neurons, on the order of 10^{11}, and each neuron is connected to thousands of others. Computation in the brain is:

- distributed,
- massively parallel,
- nonlinear,
- and robust to noise and failure.

This led to the idea of building artificial systems inspired by biological neurons.

1.21 The Perceptron

The first important model in this direction was the perceptron.

A perceptron is a simple computational unit inspired by a biological neuron. At the time, the goal was to design hardware and algorithms that mimicked the way neurons process information.

A biological neuron can be described very simply as follows:

- it receives signals through its dendrites,
- it accumulates electrical or electrochemical charge,
- when the charge exceeds a threshold, it fires,
- the signal is then transmitted through the axon to other neurons.

This is, of course, a simplified description, but it is sufficient for understanding the historical motivation behind perceptrons.

1.22 Why Start from the Perceptron?

We begin with the perceptron because it is simple enough to study on the blackboard. It does not require GPUs or large-scale training machinery. Even so, it already contains the essential ideas that later appear in feedforward neural networks and backpropagation.

In the next step, we will define the perceptron more precisely and study how learning works in this model.

1.23 The Perceptron as a Model of Neural Computation

Neurons communicate by exchanging electrochemical signals. A neuron collects charge through its dendrites, and when the accumulated charge exceeds a threshold, it fires and releases a signal through its axon. The point where one neuron connects to another is called a *synapse*.

At a synapse, the incoming signal can increase or decrease the charge inside the receiving neuron. This depends on the type of neurotransmitter and receptor involved. As a result, synaptic contributions can be:

- **excitatory**, when they increase the charge;
- **inhibitory**, when they decrease the charge.

1.23.1 From Biology to a Mathematical Model

The perceptron models this behavior in two steps:

1. compute a weighted sum of the inputs;
2. compare the result with a threshold.

Let the inputs be $x_1, x_2, \ldots, x_n$, with associated weights $w_1, w_2, \ldots, w_n$. The neuron output is

$$y = h\left(\sum_{i=1}^{n} w_i x_i - b\right),$$

where b is the bias, which plays the role of the threshold, and $h(\cdot)$ is a nonlinear activation function.

Common choices for h are:

- the **sign** function, which outputs -1 or $+1$;
- the **Heaviside step** function, which outputs 0 or 1.

These two versions differ only by a linear transformation, so they are essentially equivalent.

1.23.2 Bias as an Additional Input

The bias can be absorbed into the weighted sum by defining

$$x_0 = 1, \qquad w_0 = -b.$$

Then the output can be written as

$$y = h\left(\sum_{i=0}^{n} w_i x_i\right).$$

This compact notation is often used to simplify formulas.

Equivalently, using vector notation,

$$y = h(\mathbf{w}^\top \mathbf{x}),$$

where $\mathbf{x}$ includes the constant input $x_0 = 1$.

1.23.3 A Note on Biological Accuracy

The perceptron is only a simplified model of a real neuron. Real neurons are much more complex:

- their dynamics are time dependent,
- they have a refractory period after firing,
- signal transmission is not instantaneous,
- their behavior is influenced by many biochemical processes.

The perceptron ignores these details, but it captures the basic idea of thresholded nonlinear computation.

1.24 History of the Perceptron

The first mathematical model of a perceptron was introduced in the 1940s by McCulloch and Pitts. One was a mathematician and the other a biologist, and they proposed a threshold unit as a simplified model of a neuron.

A first physical implementation appeared later, in the form of the *Mark I Perceptron*, developed by Frank Rosenblatt. This was a hardware system built with many cables and potentiometers. In that implementation:

- the input was represented as a voltage,
- the weight was represented by a resistor or potentiometer,
- the weighted sum was obtained electrically,
- the threshold was implemented by a comparator,
- learning was performed by adjusting the potentiometers.

The system was impressive for its time because it could learn from examples by changing its parameters.

1.25 Learning the Bias

In 1960, Bernard Widrow proposed treating the bias as just another input. This led to the Adaline model, which is a variation of the perceptron in which the bias is learned together with the other weights.

1.26 What Can a Perceptron Compute?

A perceptron can implement Boolean logic functions.

For example, with suitable weights, it can represent the logical OR. With other weights, it can represent the logical AND.

This shows that the same computational unit can perform different tasks depending on its parameters. In this sense, the perceptron is like a very early programmable logic device.

1.26.1 Why Is This Useful?

The advantage is that:

- the same hardware can implement different functions,
- the behavior can be changed by adjusting the weights,
- the weights can be learned from examples.

This is very different from classical logic circuits, which must be explicitly designed to realize a desired function.

1.27 Hebbian Learning and the Perceptron Rule

With the perceptron, one shows examples to the model and updates the potentiometers according to a learning rule. The key question is: where does this rule come from?

1.27.1 Hebb's Idea

In the 1940s, researchers were studying biological neurons and neural networks. One of the most influential figures was Donald Hebb, a psychologist who believed that by studying the brain one could better understand human behavior.

Hebb proposed the idea that repeated synchronous activation strengthens the connection between neurons. In other words:

- if two units activate together repeatedly, the synapse between them becomes stronger;
- if they do not agree, the connection weakens.

This is the basic intuition behind *Hebbian learning*.

1.27.2 A Simple Analogy

A useful analogy is learning to drive a manual car. At first, shifting gears requires conscious effort:

- press the clutch,
- disengage the gear,
- engage the next gear,
- release the clutch,
- add gas.

After enough repetition, this sequence becomes automatic. The brain no longer sees each step separately; it sees a single action: *shift gear*.

This is the kind of strengthening Hebb had in mind: repeated activations create a stronger path.

1.27.3 Hebbian Update Rule

The perceptron learning rule reflects this idea. Starting from a weight vector at iteration k, the weights are updated by

$$w^{(k+1)} = w^{(k)} + \Delta w.$$

A simple Hebbian update uses the input and the desired output. If we encode both as $+1$ and -1, then:

- if input and desired output agree, the weight increases;
- if they disagree, the weight decreases.

A common form is

$$\Delta w_i = \eta \, x_i \, y,$$

where:

- x_i is the input,
- y is the desired output,
- η is the learning rate.

The learning rate controls how much the weight is changed at each step.

1.27.4 Perceptron Learning

In the perceptron algorithm, we start from a random initialization. Then we show one sample at a time:

- if the perceptron classifies the sample correctly, nothing is changed;
- if it makes a mistake, the weights are corrected.

This makes the perceptron a very simple model to study by hand.

1.27.5 An Example: Learning the OR Function

Consider a perceptron that should learn the truth table of the OR function. Using the coding $\{-1, +1\}$, the desired output is:

$$(-1,-1) \mapsto -1, \qquad (-1,+1) \mapsto +1, \qquad (+1,-1) \mapsto +1, \qquad (+1,+1) \mapsto +1.$$

We may start with a random initialization, for example

$$w = (0, 0, 0),$$

and choose a learning rate such as

$$\eta = 0.5.$$

Then we apply the Hebbian/perceptron update rule sample by sample.

1.27.6 Why This Matters

The perceptron is simple enough that we can work through the learning process explicitly. This is very different from deep neural networks, where training is typically done by code and the internal process is much less transparent.

With the perceptron, two important questions arise:

- does the algorithm always converge?
- does it always converge to the same weights?

We will return to these questions later.

1.28 Perceptron Learning on a Truth Table

Let us consider the truth table of the OR function in the coding $\{-1, +1\}$. We use:

$$x_0 = 1 \quad \text{(bias)}, \qquad x_1,\ x_2 \quad \text{(inputs)}, \qquad t \quad \text{(target)}.$$

The target table is:

$$(-1,-1) \mapsto -1, \qquad (-1,+1) \mapsto +1, \qquad (+1,-1) \mapsto +1, \qquad (+1,+1) \mapsto +1.$$

We start from the initial weights

$$w^{(0)} = (w_0, w_1, w_2) = (0, 0, 0),$$

and use the perceptron output

$$y = \operatorname{sign}(w_0 x_0 + w_1 x_1 + w_2 x_2)\,.$$

1.28.1 First Pass Through the Data

For the first sample, we have:

$$x_0 = 1, \qquad x_1 = -1, \qquad x_2 = -1.$$

The output is

$$y = \operatorname{sign}(0 \cdot 1 + 0 \cdot (-1) + 0 \cdot (-1)) = \operatorname{sign}(0).$$

Since sign(0) is not defined in the usual convention, we may take it as 0 here, which is incorrect because the target is -1.

Using the Hebbian/perceptron update rule

$$\Delta w_i = \eta\, x_i\, t,$$

with learning rate $\eta = \frac{1}{2}$, the weights become

$$w_0 = 0 + \tfrac{1}{2}(1)(-1) = -\tfrac{1}{2},$$
$$w_1 = 0 + \tfrac{1}{2}(-1)(-1) = \tfrac{1}{2},$$
$$w_2 = 0 + \tfrac{1}{2}(-1)(-1) = \tfrac{1}{2}.$$

Thus after the first update,

$$w^{(1)} = \left(-\tfrac{1}{2},\ \tfrac{1}{2},\ \tfrac{1}{2}\right).$$

Now consider the second sample:

$$x_0 = 1, \qquad x_1 = 1, \qquad x_2 = -1, \qquad t = +1.$$

The output is

$$y = \operatorname{sign}\left(-\tfrac{1}{2}(1) + \tfrac{1}{2}(1) + \tfrac{1}{2}(-1)\right) = \operatorname{sign}\left(-\tfrac{1}{2}\right) = -1,$$

which is wrong, since the target is $+1$.

Applying the update rule again,

$$w_0 \leftarrow -\tfrac{1}{2} + \tfrac{1}{2}(1)(1) = 0,$$
$$w_1 \leftarrow \tfrac{1}{2} + \tfrac{1}{2}(1)(1) = 1,$$
$$w_2 \leftarrow \tfrac{1}{2} + \tfrac{1}{2}(-1)(1) = 0.$$

So after the second update,

$$w^{(2)} = (0, 1, 0).$$

Proceeding with the third sample:

$$x_0 = 1, \qquad x_1 = -1, \qquad x_2 = 1, \qquad t = +1,$$

the output is

$$y = \operatorname{sign}(0 \cdot 1 + 1 \cdot (-1) + 0 \cdot 1) = \operatorname{sign}(-1) = -1,$$

which is again wrong. Updating gives

$$w_0 \leftarrow 0 + \tfrac{1}{2}(1)(1) = \tfrac{1}{2},$$
$$w_1 \leftarrow 1 + \tfrac{1}{2}(-1)(1) = \tfrac{1}{2},$$
$$w_2 \leftarrow 0 + \tfrac{1}{2}(1)(1) = \tfrac{1}{2}.$$

Thus

$$w^{(3)} = \left(\tfrac{1}{2}, \tfrac{1}{2}, \tfrac{1}{2}\right).$$

For the fourth sample,

$$x_0 = 1, \qquad x_1 = 1, \qquad x_2 = 1, \qquad t = +1,$$

we get

$$y = \operatorname{sign}\left(\tfrac{1}{2} + \tfrac{1}{2} + \tfrac{1}{2}\right) = \operatorname{sign}\left(\tfrac{3}{2}\right) = +1,$$

which is correct.

1.28.2 Epochs and Iterations

We have now passed through all four samples once. This is called one *epoch*.

The terminology is:

- **iteration** or **step**: one weight update;
- **epoch**: one full pass through the dataset.

In this example, one epoch consisted of four iterations.

1.28.3 Second Pass

We now check the samples again using

$$w = \left(\tfrac{1}{2}, \tfrac{1}{2}, \tfrac{1}{2}\right).$$

For the first sample:

$$y = \operatorname{sign}\left(\tfrac{1}{2} - \tfrac{1}{2} - \tfrac{1}{2}\right) = \operatorname{sign}\left(-\tfrac{1}{2}\right) = -1,$$

correct.

For the second sample:

$$y = \operatorname{sign}\left(\tfrac{1}{2} + \tfrac{1}{2} - \tfrac{1}{2}\right) = \operatorname{sign}\left(\tfrac{1}{2}\right) = +1,$$

correct.

For the third sample:

$$y = \operatorname{sign}\left(\tfrac{1}{2} - \tfrac{1}{2} + \tfrac{1}{2}\right) = \operatorname{sign}\left(\tfrac{1}{2}\right) = +1,$$

correct.

For the fourth sample:

$$y = \operatorname{sign}\left(\tfrac{1}{2} + \tfrac{1}{2} + \tfrac{1}{2}\right) = \operatorname{sign}\left(\tfrac{3}{2}\right) = +1,$$

correct.

So the algorithm has converged.

1.28.4 Does It Always Converge?

The answer is:

- **yes**, if the function is learnable by a perceptron;
- **no**, if the function is not linearly separable.

If the function can be learned, the perceptron converges. If it cannot be learned, the algorithm never converges.

1.28.5 Does It Converge to the Same Weights?

No. The final solution depends on:

- the initial weights;
- the learning rate;
- the order in which the samples are presented.

Different initializations or different learning rates may lead to different final weights, even if the same classification function is learned.

1.28.6 Why the Perceptron Can or Cannot Learn a Function

The perceptron computes

$$y = \text{sign}(w_0 x_0 + w_1 x_1 + w_2 x_2).$$

Its decision boundary is therefore

$$w_0 x_0 + w_1 x_1 + w_2 x_2 = 0.$$

Since this boundary is linear, the perceptron is a *linear classifier*. In two dimensions, the decision boundary is a line; in higher dimensions, it is a hyperplane.

Therefore, the perceptron can learn only *linearly separable* functions, that is, functions for which the classes can be separated by a line or hyperplane.

1.28.7 Main Takeaway

The perceptron learning rule is simple:

$$\Delta w_i = \eta x_i t,$$

and the algorithm updates the weights only when it makes a mistake.

This is enough to learn linearly separable functions such as OR, but not all possible Boolean functions.

1.29 Linearly Separable and Nonlinearly Separable Functions

Consider the OR function. If we plot the four Boolean inputs

$$(1,1), \quad (-1,1), \quad (-1,-1), \quad (1,-1),$$

then the positive examples and the negative example can be separated by a line. Therefore, OR is a *linearly separable* function, and it can be learned by a perceptron.

The same is true for the AND function. In that case, three points belong to one class and only one point belongs to the other class, so again a separating line exists.

However, not all Boolean functions are linearly separable. The most famous example is XOR. In this case, no line can separate the classes. This limitation was one of the major reasons neural network research slowed down for many years.

1.30 The Limitations of the Perceptron

The perceptron can only represent decision boundaries that are linear. In two dimensions, this means a line; in higher dimensions, a hyperplane.

If the target function is not linearly separable, then a single perceptron cannot learn it. This result was famously emphasized by Minsky and Papert in their 1969 book, where they studied the perceptron from a geometric point of view and showed that its limitations persist in any input dimension.

This had a strong impact on the history of neural networks and contributed to the first *AI winter*.

1.31 The First AI Winter

The first AI winter occurred after the early enthusiasm for artificial intelligence and neural networks faded. Several expectations had not been fulfilled, and the perceptron was shown to be too limited for many important problems.

As a result, research in neural networks slowed down significantly for several years.

1.32 Why Multiple Layers Help

The natural idea is to use more than one perceptron. Even in digital logic, a single gate is not enough for complex computation. By composing several logic gates, one can represent arbitrary Boolean functions.

The same idea applies to perceptrons:

- a single perceptron can only draw a linear boundary;
- two layers can represent more complex regions;
- multiple layers can carve very flexible decision boundaries.

So in principle, multilayer architectures can solve nonlinearly separable problems such as XOR.

1.33 Why Hebbian Learning Fails in Multiple Layers

The problem is training.

Hebbian learning works for a single perceptron because the desired output is directly available. The update rule is simple:

$$w^{(k+1)} = w^{(k)} + \eta x t.$$

For a network with multiple layers, this is no longer possible. For a hidden neuron, the desired output is unknown. In other words, we no longer know what target should be assigned to each internal unit.

Thus, Hebbian learning cannot be directly extended to multilayer perceptrons.

1.34 From Perceptrons to Feedforward Neural Networks

The failure of Hebbian learning for multilayer systems led researchers to explore different architectures. This eventually produced what we now call *feedforward neural networks*.

These are often mistakenly called multilayer perceptrons, but the term is not entirely accurate, because the units are no longer simple perceptrons with threshold activation functions.

The key change is that the activation function is no longer a hard threshold. Instead, smoother nonlinear activation functions are used, which makes training possible.

1.35 Backpropagation and the Recovery of Neural Networks

Between roughly 1969 and 1985, researchers tried to find a way to train multilayer networks. The breakthrough came with the backpropagation algorithm.

Backpropagation made it possible to train feedforward neural networks by propagating the error backward through the layers and updating all weights accordingly.

This development revived interest in neural networks and led to a new wave of research, especially in the 1980s and 1990s.

1.36 Historical Cycles

The history of AI has gone through several cycles of enthusiasm and disappointment. The main phases are often described as follows:

- the first AI boom and the first AI winter;
- the revival of neural networks after backpropagation;
- renewed growth in the 1990s and 2000s;
- the deep learning revolution beginning around 2012.

If history repeats itself, future downturns may occur as well.

1.37 Summary

The main message is:

- a perceptron can learn linearly separable functions;
- OR and AND are linearly separable;
- XOR is not linearly separable;
- multiple layers can represent more complex decision regions;
- Hebbian learning does not work for multilayer architectures;
- backpropagation made modern neural networks trainable.

Tomorrow we will start from feedforward neural networks and see how they are built and trained.

Chapter 2

Backpropagation Example

2.1 From the Perceptron to Feedforward Neural Networks

Let us briefly recap where we left off.

The perceptron is a *linear threshold unit*: it computes a linear combination of the inputs and then applies a threshold at zero. Therefore, it is a linear classifier, and it can only represent a linear decision boundary.

If a dataset is not linearly separable, a single perceptron cannot learn it perfectly, and Hebbian learning does not converge. This was the limitation highlighted by Minsky and Papert in 1969.

2.1.1 The Idea of Multilayer Perceptrons

The conceptual solution was already known at the time: use multiple layers of simple units. A network with several layers can represent much more complex decision boundaries than a single perceptron.

However, there was a major problem: there was no general learning rule for multilayer perceptrons.

2.1.2 Why Hebbian Learning Fails in Multiple Layers

Hebbian learning works for a single perceptron because, for each synapse, we know:

- the input signal,
- the desired output of the neuron.

In a multilayer network, this is only true for the input layer and the output layer. For the hidden layers, we do not know in advance what the desired output of each neuron should be. Therefore, the original Hebbian update rule cannot be applied directly.

This is one of the reasons why training multilayer perceptrons was such a difficult problem.

2.1.3 From Perceptrons to Feedforward Neural Networks

Between 1969 and the mid-1980s, researchers tried many ideas, including:

- the Adaline model,
- the Widrow–Hoff rule,
- the delta rule.

The solution that eventually succeeded led to what we now call *feedforward neural networks*. These are often informally called multilayer perceptrons, but this is not strictly correct, because the units are not perceptrons anymore: they do not use threshold activation functions.

2.2 Architecture of a Feedforward Neural Network

A feedforward neural network has a layered structure.

2.2.1 Input Layer

The input layer simply receives the data and forwards it to the first hidden layer. It does not perform a real computation. One can think of it as an identity layer.

The number of neurons in the input layer is determined by the dimension of the input, plus one if we include the bias as an additional constant input.

2.2.2 Hidden Layers

The layers between the input and output layers are called *hidden layers*. They are called hidden because they are not directly visible from the input-output interface of the network.

The number of hidden layers and the number of neurons in each hidden layer are design choices. These are not learned from data; they are chosen by the designer.

2.2.3 Output Layer

The output layer produces the final answer of the network.

- If the output is one number, the output layer has one neuron.
- If the output is a vector of dimension d, the output layer has d neurons.

Thus, the size of the output layer is determined by the problem.

2.3 Parameters and Hyperparameters

It is important to distinguish between:

- **parameters**: the weights of the network, which are learned from data;
- **hyperparameters**: design choices such as the number of layers, the number of neurons per layer, and the activation function.

Training a neural network means optimizing the parameters. Choosing the architecture means selecting the hyperparameters.

2.4 Dense Connections

In a standard feedforward network, each neuron in one layer is connected to every neuron in the next layer. Such a layer is called *fully connected* or *dense*.

If a layer has n inputs and the next layer has m neurons, then the number of connections is

$$(n+1)m,$$

where the $+1$ accounts for the bias.

If a network has several layers, the total number of parameters is obtained by summing the number of connections across all layers.

Dense layers can become very large very quickly, because the number of parameters grows fast with the width and depth of the network.

2.5 Why Dense Layers Are General

Dense connectivity is very general. If one wants to represent a sparse architecture, one can simply set some weights to zero. For this reason, fully connected networks are a very general starting point.

Later, we will see architectures with sparse or structured connectivity, such as convolutional neural networks.

2.6 Limitations of the Simplified Architecture

For simplicity, we will first consider networks in which:

- each layer is connected only to the previous layer;
- the network has a strictly feedforward structure;
- there are no shortcut connections.

These assumptions make the notation and the analysis much simpler.

In general, however, shortcut connections are allowed and are very important in modern architectures. For example:

- U-Net uses skip connections;
- residual networks use residual blocks.

These ideas will appear later in the course.

2.7 Activation Functions in Feedforward Neural Networks

In a feedforward neural network, the signal propagates only forward. There are no loops and no cross-layer feedback connections. This is why the architecture is called *feedforward*.

A key component of a neural network is the activation function. In order to train the network, the activation function must be differentiable, because learning relies on computing derivatives.

2.7.1 Why the Step Function Does Not Work

The threshold function used in the perceptron is not suitable for training feedforward neural networks.

Although it can be differentiated almost everywhere, its derivative is zero almost everywhere and undefined at the threshold. Therefore, it is a very poor choice for gradient-based learning.

2.7.2 Linear Activation Function

The simplest activation function is the linear function. It is continuous and differentiable everywhere, with derivative equal to 1.

However, using only linear activations in every layer would make the whole network equivalent to a single linear model. In that case, the composition of several linear layers is still linear, so the network would reduce to something essentially equivalent to linear regression.

Thus, linear activations cannot be used in every layer.

2.7.3 Sigmoid Activation Function

A classical nonlinear activation function is the sigmoid:

$$\sigma(a) = \frac{1}{1 + e^{-a}},$$

where

$$a = \sum_i w_i x_i$$

is the activation value.

The sigmoid is smooth, differentiable everywhere, and approximates a step function. Its derivative has a very convenient form:

$$\sigma'(a) = \sigma(a)\big(1 - \sigma(a)\big).$$

For example, at $a = 0$, we have

$$\sigma(0) = \frac{1}{2}, \qquad \sigma'(0) = \frac{1}{2}\left(1 - \frac{1}{2}\right) = \frac{1}{4}.$$

This makes sigmoid both mathematically convenient and historically important.

2.7.4 Hyperbolic Tangent

Another classical activation function is the hyperbolic tangent:

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}.$$

It is continuous, differentiable, and bounded between -1 and $+1$. It is also antisymmetric, which makes it convenient in many settings.

Its derivative can be written in terms of the function itself:

$$\frac{d}{da}\tanh(a) = 1 - \tanh^2(a).$$

Again, this is useful because the derivative can be computed directly from the value of the activation.

2.7.5 Other Activation Functions

The activation functions above are the historical ones, but they are not the only ones. Modern neural networks also use:

- ReLU,
- Leaky ReLU,
- Swish,
- GELU.

We will introduce additional activation functions later.

2.8 Deep Learning versus Neural Networks

It is important not to confuse *neural networks* with *deep learning*.

A neural network becomes part of a deep learning system when it is used end-to-end, from raw data to final output, and when it learns internal representations automatically.

If the input is already a hand-crafted feature vector and the network only predicts a class, then this is machine learning with a neural network, but not necessarily deep learning in the strict sense.

Deep learning is therefore not just a matter of depth. It is about learning representations end-to-end from raw data.

2.9 Machine Learning Interpretation

For now, we interpret the network as a machine learning model:

- the input is a feature vector;
- the output is the prediction;
- the network approximates an input-output function.

Later, when we introduce convolutional neural networks and more structured architectures, the deep learning perspective will become much clearer.

2.10 Output Activation for Regression

If the task is regression, then the output is a real number. Therefore, the output layer must be able to produce values in

$$(-\infty, +\infty).$$

Among the activation functions discussed so far, only the linear activation function has this property.
Therefore, for regression problems, the output neuron usually uses a linear activation function.

2.11 Choosing Activation Functions

For regression, one may normalize the output to lie in an interval such as $[-1, 1]$ and then use a hyperbolic tangent at the output layer. In principle, this is possible. In practice, however, it is often not the best choice.

The reason is that normalization is usually based only on the training set. One typically rescales the output using the minimum and maximum values observed in the data. If these observed values are not the true minimum and maximum of the target variable, then the model may have difficulty extrapolating beyond the training range. In that case, the output is artificially forced to saturate between -1 and 1.

For this reason, in the general regression setting, the standard choice for the output layer is simply a **linear activation function**.

2.11.1 Binary Classification

For binary classification, the network must output one of two classes. A common encoding is:

$$\text{class } 0 \leftrightarrow 0, \qquad \text{class } 1 \leftrightarrow 1,$$

or equivalently,

$$\text{class } 0 \leftrightarrow -1, \qquad \text{class } 1 \leftrightarrow +1.$$

If the output is coded as $\{-1, +1\}$, the hyperbolic tangent is a natural choice because its codomain is exactly $[-1, 1]$.

If the output is coded as $\{0, 1\}$, the sigmoid is the natural choice because its codomain is $[0, 1]$.

In practice, the sigmoid is often preferred for binary classification because its output can be interpreted as a confidence score or as an estimated posterior probability. Of course, this is only an interpretation: a neural network can be very confident and still be wrong.

2.11.2 Multiclass Classification

When there are more than two classes, one usually uses **one-hot encoding**. For example, for three classes:

$$C_1 \mapsto (1, 0, 0), \qquad C_2 \mapsto (0, 1, 0), \qquad C_3 \mapsto (0, 0, 1).$$

This encoding is also widely used for categorical variables such as words in a vocabulary.

If the network has one output neuron per class, then using independent sigmoids is possible, but it does not enforce competition between classes. For multiclass classification, the standard choice is therefore the **softmax** function.

Given the activation values $a_1, \dots, a_K$ of the last layer, softmax is defined as

$$\text{softmax}(a_i) = \frac{e^{a_i}}{\sum_{j=1}^{K} e^{a_j}}.$$

This has two useful properties:

- each output is positive;
- the outputs sum to one:

$$\sum_{i=1}^{K} \text{softmax}(a_i) = 1.$$

Because of this, softmax is the standard output activation for multiclass classification.

2.11.3 Summary of the Output Layer

The usual choices are:

- **regression**: linear output;
- **binary classification**: sigmoid output, or tanh if the labels are coded in $\{-1, +1\}$;
- **multiclass classification**: softmax output.

The input layer is always the identity, so no activation function is needed there.

2.11.4 Activation Functions in Hidden Layers

For hidden layers, one usually chooses a nonlinear activation function. In this lecture, the classical choices are:

- sigmoid,
- hyperbolic tangent.

In modern neural networks, other activation functions such as ReLU are often preferred, but we will introduce them later.

For now, the main idea is simple: choose the output activation according to the task, and use a nonlinear activation in the hidden layers.

2.12 Universal Approximation Theorem

Now that we know how to build a feed-forward neural network, the next natural question is: *how good is it?*

In 1991, Kurt Hornik proved the following fundamental result:

Theorem 2.1 (Universal Approximation Theorem (Hornik, 1991)) *A single-hidden-layer feed-forward neural network with a non-constant, bounded, and monotonically increasing continuous activation function (such as the sigmoid or hyperbolic tangent) can approximate any measurable function to any desired degree of accuracy on a compact set, provided there are sufficiently many neurons in the hidden layer.*

In other words, a neural network with one hidden layer is a *universal function approximator*: it can approximate *any* continuous non-linear function arbitrarily well on a compact domain (i.e., a closed and bounded subset of $\mathbb{R}^n$), as long as we use enough hidden neurons.

We can therefore view a neural network as the non-linear counterpart of linear regression:

- Linear regression approximates any *linear* function.
- A (sufficiently wide) neural network approximates any *non-linear* function.

This result is extremely powerful: in principle, whatever (continuous, non-linear) function we want to learn, a neural network can do it.

2.12.1 Important Caveats

While the theorem guarantees existence, several subtle but crucial points must be kept in mind:

1. The theorem is *non-constructive*: it proves that there *exists* a set of weights capable of approximating the target function to arbitrary accuracy, but it does not tell us *how* to find those weights.
2. To achieve arbitrary accuracy one may need an *exponential* number of hidden neurons. This leads to an exponential number of parameters.
3. With an extremely large number of parameters the model can essentially memorize the training set (perfect fit), but it will fail to generalize to unseen data (overfitting).

Thus the universal approximation theorem tells us that the model class is rich enough, but it does not solve the practical problems of *finding* the weights and *avoiding* overfitting. We will address weight optimization today and regularization/overfitting in the next lecture.

2.12.2 Classification vs. Regression

The universal approximation theorem applies directly to *regression* problems (continuous outputs). Classification problems are not continuous and the decision boundary is typically not defined on a compact set in the same way. For classification it can be shown that *two hidden layers* are in general sufficient.

For the remainder of today's lecture we will therefore work with the simple one-hidden-layer architecture shown below, knowing that it is theoretically powerful enough. In practice deeper networks often make weight optimization easier (with appropriate tricks), but we will introduce those later.

2.13 Training: Finding the Weights

The back-propagation algorithm (introduced in 1985) showed that the weights *can* be learned efficiently. We will not derive the original (rather cumbersome) formulation based on explicit partial derivatives. Instead, we adopt a modern *optimization* perspective that yields exactly the same algorithm but is conceptually cleaner.

2.13.1 The General Supervised Learning Setup

A model is a function $f(\mathbf{x};\boldsymbol{\theta})$ that maps an input vector $\mathbf{x}$ to an output, parameterized by a vector of parameters $\boldsymbol{\theta}$ (the weights of the neural network).

A supervised training set consists of N pairs:

$$\mathcal{D} = \{(\mathbf{x}_1, t_1), (\mathbf{x}_2, t_2), \ldots, (\mathbf{x}_N, t_N)\},$$

where $\mathbf{x}_i$ is the input and t_i is the desired target.

Learning means finding the parameters $\boldsymbol{\theta}$ such that the model prediction $f(\mathbf{x}_i;\boldsymbol{\theta})$ is as close as possible to the target t_i for every training example.

For a neural network with one hidden layer the forward pass is:

$$\mathbf{h} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \qquad \hat{y} = \sigma(\mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}),$$

where σ is a non-linear activation (e.g., sigmoid or tanh) and the matrices $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ together with biases $\mathbf{b}^{(1)}, \mathbf{b}^{(2)}$ constitute the parameter vector $\boldsymbol{\theta}$.

2.13.2 Measuring Goodness of Fit: The Loss Function

We quantify how well the model matches the targets by an *error* (or *loss*) function. The classic choice is the *sum of squared errors* (least-squares loss):

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \big(t_i - f(\mathbf{x}_i; \boldsymbol{\theta})\big)^2.$$

Minimizing the squared error penalizes large deviations more heavily than small ones (because of the quadratic term) and prevents positive and negative errors from cancelling each other out.

Training the neural network therefore reduces to the following optimization problem:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}).$$

This is exactly the same framework used in ordinary linear regression, except that for a neural network the mapping $f(\mathbf{x}; \boldsymbol{\theta})$ is *non-linear* in the parameters. Consequently we must solve a *non-linear* optimization problem.

2.13.3 Non-linear Optimization

For a generic differentiable function $g(\boldsymbol{\theta})$ the necessary condition for a local minimum is that the gradient vanishes:

$$\nabla_{\boldsymbol{\theta}} g(\boldsymbol{\theta}) = \mathbf{0}.$$

In other words, we must solve the system of (generally non-linear) equations obtained by setting all partial derivatives to zero.

In the linear-regression case these equations admit a closed-form solution. For a neural network the equations are highly non-linear and usually cannot be solved analytically; we must resort to iterative numerical methods.

2.14 Gradient Descent

One of the simplest and most widely used iterative methods to find stationary points is *gradient descent*.

The algorithm is extremely intuitive:

1. Start with a random initial guess for the parameters $\boldsymbol{\theta}^{(0)}$.
2. At each iteration k, compute the gradient $\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(k)})$.
3. Update the parameters by taking a step in the *opposite* direction of the gradient:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(k)}),$$

   where $\eta > 0$ is the *learning rate* (step size).

Why does this work? If the gradient is positive, the loss is increasing in that direction, so we move backwards (subtract). If the gradient is negative, the loss is decreasing, so we move forwards. The learning rate η controls how large each step is.

A useful physical analogy is walking down a mountain in the fog: at each point you only need to know the local slope (the derivative) to decide whether to go forward or backward. You do not need to see the whole mountain.

Sigmoid and hyperbolic tangent activations are particularly convenient because their derivatives can be expressed directly in terms of the function value itself, making the computations very efficient.

2.14.1 The Error Surface

When the model is linear, the loss function $\mathcal{L}(\boldsymbol{\theta})$ is quadratic and has a single global minimum (no local minima). Gradient descent (or even direct solution) is trivial.

For a neural network the loss surface is highly non-convex and can have many local minima, saddle points, and plateaus. The picture of a simple quadratic bowl is therefore misleading. Nevertheless, the algorithm is the same: follow the negative gradient iteratively.

2.14.2 Variants and Improvements

Basic gradient descent can be enhanced in several ways:

- *Momentum*: Add a fraction of the previous update to the current one, giving the optimizer "inertia" so it can roll over flat regions or shallow local minima.
- *RProp (Resilient Propagation)*: Use only the *sign* of the gradient; the step size is fixed (or adapted independently per parameter). This removes the influence of gradient magnitude.

Many other variations exist, but they all belong to the family of first-order nonlinear optimization methods.

2.14.3 Backpropagation = Gradient Descent on Neural Networks

Historically, the back-propagation algorithm was derived in 1985 by explicitly propagating the output error backwards through the network using the chain rule (linear approximations at each layer). This made the derivation appear complex and specific to neural networks.

From today's perspective it is much simpler: *back-propagation is nothing more than gradient descent applied to a neural network*. The forward pass computes the loss; the backward pass efficiently computes all partial derivatives $\frac{\partial \mathcal{L}}{\partial w}$ for every weight w using the chain rule. The weight update is exactly the same as the gradient-descent step above.

Because the network is feed-forward and composed of differentiable functions, the gradient with respect to *all* weights (in every layer) can be computed exactly and simultaneously in a single backward pass.

2.14.4 Local Minima and Initialization

Gradient descent is a *greedy* local optimizer: its final solution depends strongly on the starting point. In non-convex landscapes you may end up in a poor local minimum.

Traditional remedies (used when models were small) included multiple random restarts and keeping the best solution. Today this is rarely feasible: training a large model can take weeks or months on thousands of GPUs. Modern practice is therefore to train once with a good random initialization (or sophisticated schemes such as Xavier/He initialization) and accept the result. Theoretical results show that in very high-dimensional parameter spaces (large models) most local minima tend to have similar loss values, so a single run is often sufficient.

2.15 Worked Example: Computing the Gradient for a Specific Weight

Let us make the process concrete with a small example. Consider a feed-forward neural network with one hidden layer:

- Hidden neurons use activation function $h(\cdot)$.
- Output neuron uses activation function $g(\cdot)$.
- We have a single output $\hat{y}$.

The error (loss) for one training sample is the squared difference:

$$E = (t - \hat{y})^2,$$

and the total loss over the whole dataset is the sum over all samples.

We want to compute the partial derivative of the total loss with respect to one particular weight, say $w_{3,5}^{(1)}$ (the weight connecting the 5th input component to the 3rd hidden neuron in the first layer).

Because differentiation is linear,

$$\frac{\partial E_{\text{total}}}{\partial w_{3,5}^{(1)}} = \sum_{n=1}^{N} \frac{\partial}{\partial w_{3,5}^{(1)}} \big(t_n - \hat{y}_n\big)^2.$$

Now focus on a single sample and drop the subscript n for brevity. Let

$$\hat{y} = g(z), \qquad z = \sum_j w_j^{(2)} h_j,$$

and for the third hidden neuron

$$h_3 = h(u_3), \qquad u_3 = \sum_i w_{3i}^{(1)} x_i.$$

Differentiating the squared error:

$$\frac{\partial E}{\partial w_{3,5}^{(1)}} = 2(t - \hat{y}) \cdot \Big(-\frac{\partial \hat{y}}{\partial w_{3,5}^{(1)}}\Big).$$

Apply the chain rule to the output:

$$\frac{\partial \hat{y}}{\partial w_{3,5}^{(1)}} = g'(z) \cdot \frac{\partial z}{\partial w_{3,5}^{(1)}}.$$

Only the term involving the third hidden neuron depends on $w_{3,5}^{(1)}$:

$$\frac{\partial z}{\partial w_{3,5}^{(1)}} = w_3^{(2)} \cdot \frac{\partial h_3}{\partial w_{3,5}^{(1)}}.$$

Now differentiate the hidden activation:

$$\frac{\partial h_3}{\partial w_{3,5}^{(1)}} = h'(u_3) \cdot x_5.$$

Putting everything together (and restoring the sum over the training set), the full gradient component is

$$\frac{\partial \mathcal{L}}{\partial w_{3,5}^{(1)}} = -2 \sum_n (t_n - \hat{y}_n)\, g'(z_n)\, w_3^{(2)}\, h'(u_{3,n})\, x_{5,n}.$$

The same procedure (chain rule applied layer by layer) yields an analytic expression for the gradient with respect to *every* weight in the network. This is exactly what the back-propagation algorithm computes efficiently in one backward pass.

Note that if both g and h are sigmoid functions, i.e.,

$$g(z) = h(z) = \frac{1}{1 + e^{-z}},$$

their derivatives take the simple form $g'(z) = g(z)(1 - g(z))$ (and likewise for h), which can be evaluated very cheaply.

2.16 Batch, Stochastic, and Mini-Batch Gradient Descent

The gradient expression above contains a sum over *all* training samples. This is called *full-batch gradient descent*: one gradient evaluation and one parameter update per epoch. Each iteration therefore processes the entire dataset.

Computing the full gradient in one step is ideal for stability, but it becomes impractical when the dataset is very large (e.g., terabytes). The entire dataset and model must fit in GPU memory, which is often impossible.

Two practical alternatives exist:

2.16.1 Stochastic Gradient Descent (SGD)

Process one sample at a time:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}_n(\boldsymbol{\theta}^{(k)}),$$

where $\mathcal{L}_n$ is the loss on a single randomly chosen example.

This produces a very noisy ("erratic") trajectory, but each step is extremely cheap. The noise can actually help escape shallow local minima. When the batch size is exactly 1, mini-batch gradient descent reduces to classical SGD.

2.16.2 Mini-Batch Gradient Descent

The most common compromise today: split the dataset into small blocks ("mini-batches") of size B (the *batch size*, a hyper-parameter you choose). For each mini-batch compute the gradient on those B samples only, then update.

Number of iterations per epoch = $\lceil N/B \rceil$.

Larger batches give more stable gradients but require more memory and computation per step. Smaller batches (e.g., 32, 64, 128) are faster per iteration and often generalize better thanks to the added noise. In modern deep-learning libraries the batch size is one of the most important hyper-parameters you will tune.

All three variants (full batch, stochastic, mini-batch) are simply different ways of estimating the same true gradient; they all implement the same underlying gradient-descent algorithm.

2.17 Automatic Differentiation

In practice you will never compute these partial derivatives by hand. Modern deep-learning frameworks use *automatic differentiation* (autograd) to obtain the exact gradient for any network architecture you define. This will be the topic of the next lecture, together with further practical details on training.

Chapter 3

Feedforward Neural Networks and Training Loop

3.1 Connecting the notebook to Google Drive

The first cell in this notebook connects to your Google Drive. This allows the notebook to remember all the files we save, because the notebook is executed on a Google server and files are otherwise stored there rather than on your local machine. As a result, if you connect to Google Drive, all the files produced in this notebook will be saved to your Drive.

When you run the cell, it may ask for permissions, since it is connecting to a Google machine. After clicking *Connect*, selecting your account, and accepting the permission prompts, the cell should run without issues. If the output appears as expected, then the notebook is connected to Google Drive, and everything we save will appear inside the `lecture_one` folder.

3.2 Importing the main libraries

We begin with a cell that imports the main libraries we will use in this notebook. In practice, you will often copy and paste this cell, since it contains most of the standard Python libraries needed for machine learning.

We first import system libraries such as `os` and `warnings`, as well as `NumPy`, which is the main data science library for handling vectors and matrices. We also set a seed to make the notebook reproducible.

Next, we import `Torch`, which is the main library we will use to build and train neural networks. Torch also needs a seed in this case. We then import some common Torch shortcuts that are frequently used in neural network code, so that we can write them more conveniently.

We also include code to check whether a GPU is available. At the moment, this notebook is running in a CPU-only environment, which is sufficient because the neural networks we will use today are small. Later in the course, we will work with larger models, and GPUs will become essential to keep training times reasonable. If a GPU is available, the code stores it in the `device` variable, so that later computations can automatically use either the CPU or GPU depending on the machine.

Finally, we import `pandas` for handling tables, `seaborn` and `matplotlib` for plotting graphs, and `scikit-learn`, which is one of the most common libraries for general machine learning tasks.

3.3 Loading the Iris dataset

Let us run the cell. It will take a little time, since all the libraries need to be imported. Once that is done, we can load our first dataset.

The Iris dataset is the classic "hello world" dataset of machine learning. It contains data about three kinds of flowers, which are quite similar to each other. To the untrained eye, they may all look the same, so the goal is to build a model that can correctly identify which flower is which.

The dataset is old; it dates back to 1988. It contains four feature columns and one target column. The four features describe petals and sepals: petal width, petal height, sepal width, and sepal height. In other words, we have four numerical variables describing the flowers.

3.4 Inspecting the data

Using `pandas`, we can display the data in the notebook and inspect how it looks. It is essentially a table, similar to an Excel sheet, with four numerical columns.

We can also check the shape of the data. In general, the first value in the shape indicates the number of rows, and the second value indicates the number of columns. This is also true for tensors used in neural network training. Typically, the first value corresponds to the batch size, and the remaining values describe the structure of the tensor.

In this case, the data forms a matrix with 150 rows and 4 columns.

We can then examine the numerical summary of these columns. The means of the features are quite different from one another. For example, sepal length is usually larger than petal width. This is an important detail that we will need to take into account later when training the neural network. We also obtain other statistics such as the standard deviation, percentiles, and related summary values.

This step may sound simple, but it is one of the most important parts of any machine learning pipeline: you should always check your data, inspect its contents, and understand the variables you are working with. The Iris dataset is unusually clean and stable: it has no missing values, it is perfectly balanced, and it is a very standard example of a machine learning problem. Real-world datasets are rarely this well behaved. They often contain issues that must be handled before training a model, and we will discuss such cases later in the course.

3.5 Inspecting the targets

We have four features, and we want to build a model that takes these four values and outputs a single value: the class of the flower.

The target column is selected using the standard syntax for a single column of data. Its values are the numbers 0, 1, and 2. These labels will eventually need to be encoded properly, because they represent categories rather than ordered values. In reality, there is no meaningful ordering between the flower classes.

We can also observe that each class contains 50 samples, so the dataset is perfectly balanced. We do not need to worry about underrepresented classes here. In more realistic situations, class imbalance can become a serious issue and can make it harder for the model to recognize minority classes.

3.6 A visual look at the features

We now take a more visual look at the dataset. The figure shows a set of pair plots, where two features are plotted at a time and the points are colored according to their class.

A useful observation is that the task can be viewed as finding boundaries that separate the different colors. One immediate conclusion is that the blue class is linearly separable. In other words, even by considering only pairs of features, we can draw a line that separates one class from the other two.

The other two classes are not perfectly linearly separable, but they are still fairly well separated. As a result, we would expect a model trained on this data to achieve at least a good classification of one class. Even a linear model should be able to classify this dataset reasonably well. Since a neural network is more expressive, it should be able to do at least as well. If the final performance is below about 33%, then something probably went wrong during training.

3.7 Preparing the data

We now verify that the dataset has four input features and three output classes.

At this point, the dataset is not yet ready to be passed directly into a neural network. Even though all the features are numerical, in any machine learning pipeline we usually need some preprocessing before feeding raw data into a model.

The first step is to separate features and targets into capital X and lowercase y, which is the standard notation.

Then we split the dataset into training and test sets. For those less familiar with Python notation, this means we are applying a function with several parameters and receiving multiple outputs.

Here, we reserve 20 samples for the test set out of the original 150. We set a seed so that the split is reproducible, and we use stratification.

Stratification means preserving the class proportions when splitting the data. For example, if we divide the 150 samples into 130 training samples and 20 test samples, stratification ensures that the test set still reflects the same relative class distribution. This is important, especially when the test set is small, because it makes it more likely that the split represents the full dataset.

3.8 Training, validation, and test

We then perform a second split to create a validation set.

The purpose of training, validation, and test is as follows: training is used to fit the model, validation is used to tune the model, and test is used to measure final performance. The key difference between training and validation is that training data is used to update the model parameters, while validation data is not.

If we only used the training set, the model would overfit. In machine learning, we care about out-of-sample performance, meaning how well the model performs on unseen data. Since the test set should be reserved only for the final evaluation, we need a separate validation set to compare different model configurations and hyperparameters.

Neural networks have two kinds of parameters. First, there are learned parameters, such as the weights between neurons, which are optimized during training. Second, there are hyperparameters, such as the number of layers, the learning rate, and the number of epochs, which are set before training begins and are not learned from data.

You can think of the process as a nested loop: an outer loop over different model configurations and an inner loop that trains the network on the training data. Validation provides an estimate of out-of-sample performance. However, validation performance is not the final certification of the model, because we have already used it to choose the best configuration. The true out-of-sample estimate comes from the test set, which should contain data the model has never seen before.

During validation, the network parameters are frozen. The model operates in inference mode, meaning that we do not use the fitting procedure; instead, we evaluate the model as it is.

It is true that splitting the data reduces the amount of data available for training. This is unavoidable if we want a reliable estimate of generalization performance. If we used all the data for training, the model might simply memorize the dataset, and we would not know how it would behave in production.

In some cases, after selecting the best hyperparameters, we can retrain the model on the combined training and validation sets. Since the best configuration is already known, we can use all the available labeled data except the test set. Then we evaluate once on the test set and report that final performance.

3.9 Checking the data shapes

From the 150 original samples, we obtain only 110 for training, and even fewer for validation. This means that a single misclassified sample can significantly affect the final accuracy, especially on small datasets. Depending on the application, this tradeoff may need to be adjusted.

We also check the minimum and maximum values of the training features. These values show that the feature scales are very different from one another. This leads to an important question: what happens if we train a network on data with very different scales and no preprocessing?

The issue is that features with larger average values can dominate the forward pass and the gradients during training. If the difference in scale is small, the network may still train successfully, but when the scales are very different, small features can be effectively ignored.

For this reason, we normalize the data so that all features are on a comparable scale. In this case, we transform the values so that the average is around 0.5 and the values lie between 0 and 1.

3.10 Reprocessing the data

The next step is reprocessing. We convert our data, which is currently stored as a NumPy array, into a PyTorch tensor. The function `torch.from_numpy` takes the values from NumPy and moves them into PyTorch, so that all PyTorch functions can operate on them.

An additional advantage of PyTorch tensors is that they keep track of gradients. This means that, if we perform operations on these variables, PyTorch can track the derivatives and later perform backpropagation. With plain NumPy arrays, this is not possible.

We then build a `TensorDataset`, which is simply a wrapper containing all the features and all the labels of the dataset. After that, we define some auxiliary variables, such as the batch size, which determines how many samples are given to the model at each training iteration.

Finally, we use a helper function that takes as input the dataset and returns a `DataLoader`. In PyTorch, a dataset and a data loader are related but not identical objects. A dataset is a fixed collection of data that can be stored in memory or on disk. Usually, a dataset can fit in RAM, but not always. For example, if the dataset is one terabyte large, it is unlikely that a personal computer has enough RAM to hold it entirely. In that case, the data must remain on disk, and the training process must fetch chunks of it as needed.

This is exactly what the data loader does. It takes a pointer to the data and, during training, loads batches of data in parallel and feeds them into the neural network. For large datasets, this is essential, because we cannot assume that everything will fit into memory. For small datasets, we could work directly with the dataset and ignore the data loader, but in general it is best practice to use a data loader, since it is optimized for this task.

We pass to the data loader the dataset, the batch size, whether the data should be shuffled, whether the last batch should be dropped if it contains fewer than the desired number of samples, the number of worker threads, and some memory-related parameters. These are usually left unchanged at first and only tuned later if the model struggles to load data efficiently.

We then create data loaders for the training, validation, and test sets.

The only difference between the three is that the training data loader is shuffled. At each epoch, the training batches are reshuffled, so it is extremely unlikely to see the same batch twice. This helps mix the data and is generally beneficial for backpropagation. We also drop the last batch if it is smaller than the batch size, because this keeps batch sizes consistent and simplifies memory usage.

Validation and test loaders, on the other hand, are not shuffled. This ensures that we always evaluate on the same samples and obtain comparable averages. The last batch is not dropped for validation and test. This is standard practice in deep learning pipelines.

3.11 Inspecting a single batch

We can now extract a single batch from the data loaders. Since data loaders are Python iterators, they can be used inside `for` loops. At each iteration, the loader returns a tuple containing the feature batch and the label batch.

If we check the shapes, the feature batch has 16 rows and 4 columns, corresponding to a batch of 16 samples with four features each. The label batch contains 16 values, one label for each sample in the batch.

3.12 Defining the neural network

Now we reach the interesting part: building a neural network in PyTorch. In general, when defining a model in PyTorch or many other frameworks, we extend the `nn.Module` class.

There are many ways to define a neural network, but this is one of the simplest. We initialize a sequential neural network. I want to emphasize that this is a sequential architecture, meaning that the layers are arranged one after another, without loops or more complex internal structures.

For now, we keep the model simple: layer after layer, like an onion.

What we need to define is a hidden layer and, of course, an output layer. Since we are solving a classification problem with three classes, the output layer must have three neurons. Each of these output neurons will produce a probability, and the probabilities will sum to 1. This set of probabilities is called the logits.

The hidden layer takes as input the same number of features as we have in the dataset. The hidden size is arbitrary; in this case, we use 32, simply because powers of two are convenient. We also need to add a non-linearity, otherwise the model would only be able to solve linear problems.

Our first neural network therefore requires two main steps: the initializer and the `forward` function. The `forward` function defines the order in which data moves through the network. In this case, we have already defined the layers sequentially, but we could also explicitly name them, for example as the hidden layer, activation function, and output layer. Then, inside `forward`, we could write that x goes through the hidden layer, then the activation, and finally the output layer.

After defining the model, we create it and move it to the chosen device. In this notebook, this is not especially important because we are using the CPU, but later we will work with temporal and image data, where GPU acceleration will be very useful.

3.13 Model structure and parameter count

The next recap shows the structure of the model, including the input and output dimensions of each layer.

This may be useful for the exam, so it is worth reading the function carefully. The function shows the number of parameters in the network. In this simple case, the first layer has 160 parameters and the second has 99. Remember that the bias terms are included as well.

3.14 TensorBoard

Now we introduce TensorBoard, which is a very useful tool for inspecting experiments, storing metadata collected during training, and debugging models.

To use it, we first create a dummy input, in this case a single sample with the same shape as the training data. Then we call the corresponding function. Sometimes this step can be tricky, because the notebook may not respond immediately. If that happens, restarting the environment often fixes the issue.

Once it works, all the relevant information is stored in Google Drive and can be loaded into the notebook whenever needed.

When we inspect the model, we can see the feedforward network we defined earlier. In this case, it is just a linear sequential network, so the architecture is not especially surprising. Inside each layer, we can inspect the weights and biases.

As the models become more complex, these tools become much more useful. We will also work later with pretrained models, such as models built by Google or Meta, which are often much more complicated in structure.

3.15 Hyperparameters for training

We now briefly discuss the hyperparameters related to the training loop. We will return to these in more detail in later lectures, where we will study their effects more carefully.

The first important hyperparameter is the batch size, which we already introduced. It is the number of samples the network processes simultaneously during one training step. For each sample, we compute an error with respect to the ground truth, then average those errors, and finally use that average to update the weights.

It is important to understand what it means to have a batch size of 1: the network is updated after every single sample. At the other extreme, the entire dataset can be treated as one single batch, if memory allows it.

There are advantages and disadvantages to both extremes. In practice, we usually prefer a value that is neither too small nor too large. For this dataset, a batch size of 16 is enough.

The next hyperparameters are the learning rate and the number of epochs. The number of epochs is the number of times the model is allowed to see the full dataset. This is necessary because the model usually needs multiple passes over the data to learn useful patterns. Since datasets are rarely infinite, we reuse the same samples multiple times.

The number of epochs is therefore very important, and in challenges it is usually best to keep it sufficiently high. In this case, 500 epochs is high enough.

The learning rate is one of the most important hyperparameters. It determines how strongly the model updates its weights in response to the errors. It is the factor multiplied by the gradient during backpropagation.

If the learning rate is too high, the optimization can become unstable. The model may fail to converge because the updates are too large and the network keeps jumping around the optimum. If the learning rate is too low, learning becomes extremely slow, and the model may take an impractically long time to improve.

For now, we keep the default value and later study how to choose hyperparameters more systematically.

Since this is a classification problem, we use cross-entropy as the loss function. For optimization, we use Adam, which is an improved version of stochastic gradient descent that includes momentum and other mechanisms to find a better path in the loss landscape.

Finally, there is an optional line that enables mixed precision. This is mostly for advanced use cases. It means that some operations are performed using half precision instead of 32-bit precision. This approximation usually does not harm model quality, but it can improve efficiency and reduce computation time in later stages of training.

3.16 Training and evaluation

Now we move on to training. From this point on, we will always work in two phases: a *learning* phase and an *evaluation* phase. The learning phase is performed on the training set, while the evaluation phase is performed on the validation set. This will be the standard workflow throughout the course.

The idea is simple: if someone suggests skipping the evaluation phase, that is generally a bad sign. We will always use an evaluation set in this course. Later on, we will also see some exceptions and more advanced strategies.

3.17 What happens during one epoch

Let us focus on a single epoch. We first take the model and set it in training mode, which means that its weights can be updated. Then we iterate through the training loader batch by batch.

For each batch, we move the input features X and target labels Y to the correct device. Next, we initialize the gradients to zero. We then compute the prediction, that is, the logits produced by the model for each class. After that, we compute the loss using the criterion defined earlier, in this case the cross-entropy loss between the logits and the targets.

Then we perform backpropagation. If mixed precision is enabled, we also apply gradient scaling in half precision before the backward pass. Finally, the optimizer updates the model weights according to the chain rule.

While doing this, we accumulate the loss values so that we can compute the average epoch loss. We also store the predictions. To obtain the predicted class, we apply the `argmax` operation, which returns the index of the largest value in the output vector. This gives us the class predicted by the model. With these predictions, we can compute performance metrics such as accuracy.

So, in summary, during one epoch we: train the model, accumulate the losses, update the weights, and compute metrics such as accuracy.

3.18 Validation

Validation works in almost the same way, but without gradient computation. We only evaluate the model on unseen validation data, because we care about how the model generalizes.

For validation, we set the model in evaluation mode and disable gradient computation explicitly. This is necessary, because validation is not used to update the model. We are only interested in measuring performance.

The quantities we compute at the end of validation are the epoch loss, the validation loss, and the epoch accuracy.

Why do we need a metric in addition to the loss? The reason is that accuracy is easier to interpret. Loss values can be harder to understand directly, because their scale depends on the specific problem and loss function. For example, a cross-entropy loss of 0.001 may mean very different things depending on the task. Accuracy, instead, is bounded between 0 and 1, so it is much easier to interpret in human terms.

3.19 TensorBoard logging

We can also use an optional function to log information inside TensorBoard. This is very useful for visualizing the training process, storing metadata, and debugging the model.

TensorBoard allows us to inspect not only numerical values, but also curves, distributions, weights, and gradients during training. If something goes wrong, it is often helpful to inspect these plots rather than relying only on printed values.

3.20 The training loop

Now we put everything together.

We define a loop over the number of epochs, which is 500 in this case. At each epoch we run one training step and one validation step, then store the resulting metrics. After that, we log the values to TensorBoard and print some progress information.

It is also very important to save the model at the end of training. This may sound obvious, but in practice it is easy to forget. During competitions or experiments, saving the model can prevent losing a good result. Real-world experiments are not always perfectly reproducible, especially when hardware acceleration is involved, so it is best to save the model whenever it performs well.

It is also a good idea to give the saved model a name that includes the date, time, or performance value, so that you do not overwrite a previous version by accident.

The verbosity is set to 10, so the progress is printed every 10 epochs. In the output, the middle column corresponds to training performance, while the right column corresponds to validation performance.

What we observe is that the model improves over time and eventually reaches perfect performance on the validation set. In this case, we set the number of epochs to 500, although we might have achieved convergence earlier. We will later discuss how to choose this number more systematically.

3.21 Plotting the results

We now use Matplotlib to plot the training curves. This is an important step, because visualizing the learning process often reveals more than looking only at the final numbers.

If you are not fully comfortable with the code, modern tools can help explain it. For example, notebook assistants can describe a cell or even rewrite code for you. These tools are increasingly useful, and you should become familiar with them.

Looking at the plots, we see that both the training loss and the validation loss decrease smoothly. This is a very good learning curve. The accuracy also improves over time, although there may still be slightly better stopping points than the one we reached here. We will later discuss how to identify such points in a systematic way.

3.22 Inspecting TensorBoard

We can also inspect the results inside TensorBoard. The same information can be visualized there, but with better options for zooming and interaction.

For example, we can inspect the validation accuracy and other scalar metrics in more detail. TensorBoard also gives access to distributions of weights and gradients.

When we look at the gradients, we see that they are mostly centered around zero at the beginning of training. This suggests that the optimization process is stable, without extreme spikes or unusual behavior.

We can also inspect the biases and weights. At the beginning, the weights are distributed roughly around the center. As training progresses, we start to see clear patterns emerge. This is especially useful when inspecting the output layer, because it can help us understand what the model has learned.

The main point is that TensorBoard helps us inspect the internal behavior of the model. If something is not working, these plots can reveal a lot.

3.23 Reloading the model and testing

At this point, we have saved the model. We reload it to make sure that it is stored correctly. If the model reloads without errors, that means it has been saved properly.

Now we move to the test set. The test loop is essentially the same as the validation loop, except that it is applied to the test loader. We run the model on the test data, concatenate the targets and predictions, and compute the final accuracy.

In this case, we obtain 95% accuracy on the test set, which is consistent with the validation performance. This is a very good sign.

We can also compute a confusion matrix. A confusion matrix is useful because it gives a more detailed view of how the model is performing across classes. From it, we can derive other metrics that become important in more complex settings, such as imbalanced classes.

3.24 Next dataset

For the next lecture, we will work with a dataset that is very similar to Iris: the Penguins dataset. The code to load it is analogous, and you can copy it into the section where we loaded the Iris dataset.

Unlike Iris, this dataset contains missing values. A missing value is a gap in the dataset, and such gaps are common in real-world problems. Depending on the situation, you may choose to discard the missing records, infer the missing values, or even remove an entire column if too many values are missing.

The important thing is to make an informed choice based on the structure of the missing data. This will give you useful practice before working on more realistic problems.

Chapter 4

Error Function Design

4.1 Gradient Descent, GPUs, and Backpropagation

Let us briefly recap the point we reached earlier. We were discussing gradient descent and backpropagation, with a particular focus on making the underlying computations efficient through GPUs, which excel at parallel processing.

To leverage this parallelism, the data must first be transferred to the GPU memory. (You already practiced this in the exercise.) The main practical limitation is memory capacity: if the full dataset does not fit on the GPU, it cannot be loaded all at once. Ideally, we would load the entire dataset onto the GPU and compute the gradient over the complete data in one step. This yields a simple and stable optimization process.

When GPU memory is insufficient, we instead load data incrementally. We can process one sample at a time, which corresponds to *stochastic gradient descent*, or load as many samples as the memory allows, which corresponds to *batch* or *mini-batch gradient descent*. This is why the *batch size*—the number of samples loaded onto the GPU in each forward-backward pass—is one of the most important hyperparameters.

Observe the following relationships:

- If the entire dataset is loaded at once (full-batch gradient descent), then one gradient update corresponds to exactly one *epoch* (a complete pass over the data).
- If a single sample is processed at a time, the number of gradient updates per epoch equals the total number of samples.
- If the batch size is B (e.g., $B = 32$), then the number of iterations required to complete one epoch is $\lceil N/B \rceil$, where N is the total number of samples.

Modern libraries also provide a useful technique called *gradient accumulation*. Instead of applying the weight update after every batch, we can compute gradients for several consecutive batches, sum (accumulate) them, and only then perform a single parameter update. This effectively simulates a larger batch size while still processing the data in memory-friendly chunks.

In practice, the recommended strategy is to choose the largest batch size that comfortably fits in GPU memory and, if a still larger effective batch is desired, to rely on gradient accumulation.

4.2 Automatic Differentiation and the Chain Rule

The next key topic is how the derivatives required for backpropagation can be computed automatically.

As long as all activation functions are differentiable, the required gradient formulas can in principle be derived using standard calculus. The difficulty is not theoretical impossibility but practical tedium: every architectural change (adding a layer, changing an activation function, etc.) would require manually re-deriving the entire set of gradient expressions. This quickly becomes unmanageable for realistic networks.

The solution is *automatic differentiation*. Once the computational graph of the network is defined, the backpropagation formulas are generated automatically by the framework (PyTorch, TensorFlow, etc.) at runtime.

To understand the mechanism, recall the chain rule from calculus. Consider two differentiable functions f and g and a scalar input x. Let

$$z = f(g(x)).$$

Introduce the intermediate variable $y = g(x)$, so that $z = f(y)$. Differentiating z with respect to x yields

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx} = f'(y)\, g'(x) = f'(g(x))\, g'(x).$$

For a deeper composition,

$$z = f(g(q(x))),$$

the chain rule generalizes to

$$\frac{dz}{dx} = \frac{dz}{dg(x)} \cdot \frac{dg(x)}{dq(x)} \cdot \frac{dq(x)}{dx}.$$

This is precisely the principle that will be applied repeatedly in backpropagation.

4.3 Backpropagation as Repeated Application of the Chain Rule

In a neural network, the derivative of the loss with respect to any weight is obtained by multiplying a chain of local derivatives along the path from the loss back to that weight:

- the derivative of the loss with respect to the network output,
- the derivative of the output with respect to its immediate input,
- the derivative of each activation function with respect to its argument,
- and finally the derivative of that argument with respect to the weight.

Thus, backpropagation is simply the systematic, layer-by-layer application of the chain rule through the entire computational graph.

During the *forward pass*, the network evaluates each node and stores the intermediate values (activations and pre-activation inputs) that will later be needed for the derivatives. During the *backward pass*, these stored quantities are retrieved to compute the gradients efficiently, moving from the loss backward to the weights.

Because every operation is recorded in the computational graph, extending the network is straightforward:

- Adding a new node simply requires storing its output and its local derivative.
- Adding a whole layer extends the chain by the corresponding set of operations.
- Introducing a new activation function only requires providing its derivative (which the framework then inserts automatically).

This is the power of automatic differentiation: it allows gradients to be computed for arbitrary feed-forward architectures without any manual derivation.

4.4 What Needs to Be Stored

The quantities that must be retained from the forward pass are:

- the outputs of every node,
- the values of the activation functions,

- the derivatives of the activation functions evaluated at the current inputs.

Certain common activations, such as the sigmoid $\sigma(x)$ and the hyperbolic tangent $\tanh(x)$, are particularly convenient because their derivatives can be expressed directly in terms of the function value itself:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)), \qquad \tanh'(x) = 1 - \tanh^2(x).$$

In these cases, no extra storage beyond the activation values is required. For activations with more complex derivatives, additional intermediate quantities may need to be cached, but the underlying principle remains unchanged.

4.5 Why This Explains the PyTorch Training Loop

The standard PyTorch training loop consists of two distinct phases precisely because of the forward-backward structure of automatic differentiation:

1. **Forward pass**: the network computes its predictions and stores the necessary intermediate values.
2. **Backward pass**: the framework uses those stored values to compute all gradients via the chain rule and then updates the weights.

If we only need inference (i.e., making predictions on new data), the backward pass is unnecessary; we perform only the forward pass. Training, however, always requires both passes.

By handling the derivative bookkeeping internally, modern frameworks free the practitioner from having to derive gradients manually whenever the architecture changes. The user simply defines the model and the loss; the software automatically assembles the complete backpropagation procedure.

4.6 Flexibility of the Loss Function and Arbitrary Differentiable Nodes

What happens if we change the error (loss) function? No problem at all. We only need to replace the very first term in the chain: the derivative of the new loss with respect to the network output. Everything else—the derivatives flowing backward through the layers—remains unchanged.

For example, we could decide to use the square root of the error (even though this would be problematic in practice because the square root is only defined for non-negative arguments, and the raw error could be negative). Or we could use the logarithm of the error. In all cases, once we have computed $\frac{\partial L}{\partial \hat{y}}$ (the derivative of the loss with respect to the network output $\hat{y}$), the rest of the backpropagation chain is identical.

This modularity is extremely convenient. You can implement the derivative for any feed-forward architecture, and you can insert any new node or function as long as you provide two things:

- how to compute its output (the forward pass),
- how to compute its local derivative (the backward pass).

The node could even be an entire computer program. As long as you know how to evaluate the program and how to compute the derivative of its output with respect to its input, you can plug it into the network. (Computing the derivative of a general program is nontrivial and requires specialized techniques such as algorithmic differentiation, but that is beyond today's scope.)

This is exactly why modern libraries make building custom networks so easy. For every activation function, every layer, and every operation you use, the library already knows both the forward and the backward implementations. You simply compose them, and the framework assembles the full computational graph automatically.

Moreover, everything runs in parallel on the GPU: all samples in a batch are processed simultaneously, the local derivatives are computed in parallel, and the gradients are accumulated. The weight update itself is performed in-place directly in GPU memory. The whole forward–backward cycle therefore becomes highly

efficient. If the entire dataset fits in GPU memory, one pass processes everything in parallel. Otherwise, we process the data batch by batch (or even sample by sample) and still obtain the correct gradient through accumulation.

(Note that in PyTorch you must explicitly call the backward pass—if you forget it, the network will only perform inference and no training will occur. In contrast, Keras/TensorFlow 2 often handles the forward and backward steps together behind the scenes. The simple training loop we wrote in the exercise—forward, then backward—is the explicit PyTorch style.)

A useful consequence is that gradient accumulation becomes trivial: you can run several forward passes, accumulate the gradients (without updating the weights), and only then execute a single backward pass and parameter update. This simulates a larger batch size while respecting memory constraints.

4.7 Why the Squared Error? Maximum Likelihood Estimation

So far we have seen that we can train a network by computing a loss (for instance, the sum of squared errors) between the network output and the target, then applying backpropagation and gradient descent to make the network match the data as closely as possible. But why should we choose the sum of squared errors in the first place? Why is this particular function a reasonable measure of "closeness"?

To answer this question we change perspective and return to a basic statistical principle you already know: maximum likelihood estimation (MLE). We will see that the squared-error loss is not an arbitrary choice—it naturally arises when we assume Gaussian noise.

4.7.1 A Simple Gaussian Example

Suppose we have a set of measurements that we know come from a Gaussian (normal) distribution with unknown mean μ but known variance σ^2. The probability density function for a single observation x is

$$p(x \mid \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Imagine these measurements are produced by a laser range-finder pointed at a wall. Each reading differs from the true distance because of Gaussian sensor noise (e.g., standard deviation of a few centimeters). We collect many such readings and want to estimate the true distance μ.

Which value of μ would you choose as the estimate? Most people intuitively pick the value near the center of the data cloud—the one that makes the observed points most probable under the model.

To see why, consider the opposite: if we picked a mean far to the left (the "red" line in a sketch), the rightmost data point would be extremely unlikely. If we picked a mean far to the right (the "green" line), the leftmost point would be extremely unlikely. The "purple" line near the middle assigns high probability density to all observed points simultaneously. In other words, it makes the entire dataset as likely as possible.

This is precisely maximum likelihood estimation: we choose the parameters of the model that maximize the probability of the observed data.

Important clarification: we are *not* maximizing the probability of the parameters themselves. We are maximizing the probability of the *data* given the parameters. The data are the only thing we actually observe; the parameters are what we infer.

4.7.2 Formalizing Maximum Likelihood

Let the distribution be parameterized by a vector $\boldsymbol{\theta} = (\theta_1, \theta_2, \ldots, \theta_p)$. For a dataset $\{x_1, x_2, \ldots, x_N\}$ assumed to be independent and identically distributed, the likelihood function is the joint probability of all observations:

$$\mathcal{L}(\boldsymbol{\theta}) = p(x_1, x_2, \ldots, x_N \mid \boldsymbol{\theta}) = \prod_{i=1}^{N} p(x_i \mid \boldsymbol{\theta}).$$

We seek the $\boldsymbol{\theta}$ that maximizes $\mathcal{L}(\boldsymbol{\theta})$.

In practice we almost always work with the *log-likelihood*

$$\ell(\boldsymbol{\theta}) = \log \mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \log p(x_i \mid \boldsymbol{\theta}).$$

There are three excellent reasons for taking the logarithm:

1. The logarithm is a strictly increasing (monotonic) function, so the value of $\boldsymbol{\theta}$ that maximizes $\mathcal{L}$ also maximizes ℓ.
2. Logarithms convert products into sums, turning the product of probabilities into a simple summation.
3. Numerical stability: each $p(x_i \mid \boldsymbol{\theta})$ is between 0 and 1. Multiplying thousands of such numbers quickly underflows to zero in floating-point arithmetic. The log-likelihood avoids this completely because we sum logarithms instead of multiplying tiny probabilities.

To find the maximum we take the derivatives of $\ell(\boldsymbol{\theta})$ with respect to each parameter and set them to zero:

$$\frac{\partial \ell}{\partial \theta_j} = 0 \quad \text{for all } j.$$

Solving this system of (possibly nonlinear) equations yields the maximum-likelihood estimates of the parameters.

In the Gaussian case with known variance it is easy to show that the MLE for μ is exactly the sample mean $\hat{\mu} = \frac{1}{N}\sum_{i=1}^{N} x_i$. The sum-of-squared-errors loss therefore emerges directly from the assumption of Gaussian noise: minimizing squared error is equivalent to maximizing the likelihood under that model.

This statistical perspective gives a principled justification for the squared-error loss we have been using, and it also opens the door to other loss functions that correspond to different noise assumptions (e.g., cross-entropy for classification under a categorical distribution). But that is a topic for another day.

4.8 Stationary Points and Analytical Derivation of the Gaussian MLE

We know that stationary points (where the derivative is zero) include both maxima and minima (and saddle points). Therefore, after solving the system of equations, we must verify that the critical point we found is indeed a maximum and not a minimum. In most practical cases the nature of the problem makes this clear, but formally one should check the second derivative (or the Hessian) is negative definite.

How do we actually perform the maximization? One possibility is to solve the equations analytically. When that is difficult or impossible, we resort to numerical optimization techniques (gradient descent, Newton's method, etc.). For the simple case of estimating the mean of a Gaussian with known variance, an analytical solution is still possible.

Let each observation x_n be independently drawn from

$$p(x_n \mid \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x_n - \mu)^2}{2\sigma^2}\right),$$

where σ^2 is known and μ is the unknown parameter we wish to estimate.

Because the samples are independent and identically distributed (IID), the joint probability (likelihood) of the entire dataset $\{x_1, \ldots, x_N\}$ is simply the product of the individual probabilities:

$$\mathcal{L}(\mu) = \prod_{n=1}^{N} p(x_n \mid \mu, \sigma^2).$$

It is convenient to work with the log-likelihood instead:

$$\ell(\mu) = \log \mathcal{L}(\mu) = \sum_{n=1}^{N} \log p(x_n \mid \mu, \sigma^2).$$

Substituting the Gaussian density and simplifying (the constant term $\frac{1}{\sigma\sqrt{2\pi}}$ does not depend on μ) yields

$$\ell(\mu) = -N\log(\sigma\sqrt{2\pi}) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}(x_n-\mu)^2.$$

To find the maximum we take the derivative with respect to μ and set it to zero:

$$\frac{\partial \ell}{\partial \mu} = -\frac{1}{2\sigma^2}\sum_{n=1}^{N}2(x_n-\mu)(-1) = \frac{1}{\sigma^2}\sum_{n=1}^{N}(x_n-\mu).$$

Setting the derivative equal to zero gives

$$\sum_{n=1}^{N}(x_n-\mu)=0 \quad\Longrightarrow\quad \sum_{n=1}^{N}x_n = N\mu \quad\Longrightarrow\quad \hat{\mu} = \frac{1}{N}\sum_{n=1}^{N}x_n.$$

This is exactly the sample mean—the well-known maximum-likelihood estimator for the mean of a Gaussian when the variance is known.

(The second derivative $\frac{\partial^2 \ell}{\partial \mu^2} = -\frac{N}{\sigma^2} < 0$ confirms that the critical point is indeed a maximum.)

4.9 Connecting Maximum Likelihood to Neural Network Training

We now apply exactly the same statistical principle to neural networks. Thanks to the universal approximation theorem, a sufficiently large neural network can represent (almost) any function. We therefore view the network as a flexible parametric function $G(\mathbf{x};\mathbf{w})$ that we hope can recover the unknown deterministic mapping that generated the data.

In practice, however, the observed targets y_n are not exactly equal to the true function; they are corrupted by noise. We make the same modeling assumption as before: the noise is additive Gaussian with zero mean and known variance σ^2:

$$y_n = G(\mathbf{x}_n;\mathbf{w}) + \varepsilon_n, \qquad \varepsilon_n \sim \mathcal{N}(0,\sigma^2).$$

Consequently, each target is itself Gaussian distributed with mean given by the network output:

$$y_n \sim \mathcal{N}\big(G(\mathbf{x}_n;\mathbf{w}),\sigma^2\big).$$

Training the network is now equivalent to estimating the mean of this Gaussian distribution, where the mean itself is produced by the neural network. In other words, we want to find the weights $\mathbf{w}$ that maximize the probability of the observed data under the model

$$p(y_n \mid \mathbf{x}_n,\mathbf{w}) = \frac{1}{\sigma\sqrt{2\pi}}\exp\left(-\frac{(y_n-G(\mathbf{x}_n;\mathbf{w}))^2}{2\sigma^2}\right).$$

Because the observations are IID, the likelihood of the whole dataset is

$$\mathcal{L}(\mathbf{w}) = \prod_{n=1}^{N} p(y_n \mid \mathbf{x}_n,\mathbf{w}).$$

Taking the logarithm yields the log-likelihood

$$\ell(\mathbf{w}) = \sum_{n=1}^{N}\log p(y_n \mid \mathbf{x}_n,\mathbf{w}) = \text{const.} - \frac{1}{2\sigma^2}\sum_{n=1}^{N}\big(y_n - G(\mathbf{x}_n;\mathbf{w})\big)^2.$$

Maximizing $\ell(\mathbf{w})$ with respect to the weights $\mathbf{w}$ is therefore mathematically identical to minimizing the sum of squared errors between the network predictions and the targets. The squared-error loss we have been using all along is not an arbitrary choice—it is precisely the negative log-likelihood under the assumption of Gaussian noise.

This perspective unifies classical statistics and modern deep learning: training a neural network by gradient descent on the mean-squared-error loss is equivalent to performing maximum-likelihood estimation of the network parameters when the observation noise is Gaussian. The same recipe we just applied analytically to the simple Gaussian mean can now be applied (numerically, via backpropagation) to arbitrarily complex networks.

4.10 The Log-Likelihood for Neural Networks

We now compute the log-likelihood explicitly. Under the assumption that each target t_n is generated by the network output plus additive Gaussian noise,

$$t_n \sim \mathcal{N}\big(G(\mathbf{x}_n; \mathbf{w}), \sigma^2\big),$$

the likelihood of the whole dataset is

$$\mathcal{L}(\mathbf{w}) = \prod_{n=1}^{N} \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(t_n - G(\mathbf{x}_n; \mathbf{w}))^2}{2\sigma^2}\right).$$

Taking the natural logarithm (and using the fact that $\log(ab) = \log a + \log b$) gives the log-likelihood

$$\ell(\mathbf{w}) = \sum_{n=1}^{N} \log p(t_n \mid \mathbf{x}_n, \mathbf{w}) = N \log\left(\frac{1}{\sigma\sqrt{2\pi}}\right) - \frac{1}{2\sigma^2} \sum_{n=1}^{N} \big(t_n - G(\mathbf{x}_n; \mathbf{w})\big)^2.$$

The first term is a constant that does not depend on the weights $\mathbf{w}$. Therefore, maximizing $\ell(\mathbf{w})$ with respect to $\mathbf{w}$ is exactly equivalent to minimizing the sum-of-squared-errors term

$$\frac{1}{2\sigma^2} \sum_{n=1}^{N} \big(t_n - G(\mathbf{x}_n; \mathbf{w})\big)^2.$$

(The factor $1/(2\sigma^2)$ is also a positive constant and does not change the location of the minimum.) In other words, the mean-squared-error loss that we have been using all along is (up to an irrelevant scaling factor) precisely the negative log-likelihood under the Gaussian noise model.

4.11 Why This Perspective Is Useful

This derivation gives us three important insights:

1. We now understand exactly where the squared-error loss comes from. It is not an arbitrary choice; it is the direct consequence of assuming that the observed targets are generated by a deterministic function $G(\mathbf{x}; \mathbf{w})$ corrupted by additive Gaussian noise with constant variance σ^2.

2. If we have prior knowledge about the noise, we can adapt the loss function accordingly. For example, when the variance appears to grow proportionally to the value of the function (heteroscedasticity), a common remedy is to predict $\log y$ instead of y. This turns the original multiplicative noise model

   $$y = G(\mathbf{x}; \mathbf{w}) \cdot \varepsilon$$

   into an additive model on the log scale:

   $$\log y = \log G(\mathbf{x}; \mathbf{w}) + \log \varepsilon.$$

   After this transformation the noise becomes additive again and the squared-error loss becomes appropriate once more. Such transformations are called *variance-stabilizing transformations*.

3. If the assumption "function plus constant-variance Gaussian noise" is violated, the squared-error loss can lead to suboptimal results. In real data the noise variance often changes with the predicted value (heteroscedasticity). The squared error then gives disproportionate weight to points where the variance is large, pulling the fitted function away from the true underlying relationship.

The assumption of constant variance is known in statistics as *homoscedasticity*. When the variance changes with the mean the data are said to be *heteroscedastic*. In practice you can diagnose this by plotting the residuals (target minus prediction) against the predicted values; any clear pattern indicates heteroscedasticity. If the problem is mild, variance-stabilizing transformations (log, square-root, etc.) often help. If the distortion is severe, you may need more advanced models that also predict the variance (e.g., heteroscedastic regression networks).

4.12 When Squared Error Is Inappropriate: Classification

The Gaussian-plus-noise model makes perfect sense for regression tasks, but it breaks down completely for classification. Consider binary classification where the target $t_n \in \{0, 1\}$. The possible "errors" are only two discrete outcomes:

- predict 0 when the true label is 1,
- predict 1 when the true label is 0.

There is no continuous Gaussian noise; the output is strictly Bernoulli.

Therefore we must model the data differently. We interpret the network output $G(\mathbf{x}_n; \mathbf{w})$ as the posterior probability that the example belongs to class 1:

$$G(\mathbf{x}_n; \mathbf{w}) = p(t_n = 1 \mid \mathbf{x}_n).$$

Under this interpretation the targets follow a Bernoulli distribution:

$$p(t_n \mid \mathbf{x}_n, \mathbf{w}) = \big[G(\mathbf{x}_n; \mathbf{w})\big]^{t_n} \big[1 - G(\mathbf{x}_n; \mathbf{w})\big]^{1-t_n}.$$

If the network predicts a high probability (say 0.99) for class 1, then an observed label of 1 is very likely; if it predicts a low probability (say 0.1), then an observed label of 1 is unlikely. The appropriate loss function that arises from maximizing the likelihood under this Bernoulli model is the binary cross-entropy loss (which we will derive in the next part of the lecture). The squared-error loss is simply the wrong statistical model for classification and would lead to poor training behavior.

4.13 Maximum Likelihood for Classification: The Bernoulli Model

In a binary classification problem the targets $t_n \in \{0, 1\}$ are generated from a Bernoulli distribution whose single parameter is the probability that the example belongs to class 1. We let the neural network output $G(\mathbf{x}_n; \mathbf{w})$ represent exactly this probability:

$$p(t_n = 1 \mid \mathbf{x}_n, \mathbf{w}) = G(\mathbf{x}_n; \mathbf{w}), \qquad p(t_n = 0 \mid \mathbf{x}_n, \mathbf{w}) = 1 - G(\mathbf{x}_n; \mathbf{w}).$$

The probability mass function for a single observation can therefore be written compactly as

$$p(t_n \mid \mathbf{x}_n, \mathbf{w}) = \big[G(\mathbf{x}_n; \mathbf{w})\big]^{t_n} \big[1 - G(\mathbf{x}_n; \mathbf{w})\big]^{1-t_n}.$$

(This expression automatically selects the correct term: when $t_n = 1$ it reduces to G, and when $t_n = 0$ it reduces to $1 - G$.)

Because the samples are independent and identically distributed (IID), the likelihood of the whole dataset is the product

$$\mathcal{L}(\mathbf{w}) = \prod_{n=1}^{N} \big[G(\mathbf{x}_n; \mathbf{w})\big]^{t_n} \big[1 - G(\mathbf{x}_n; \mathbf{w})\big]^{1-t_n}.$$

Taking the logarithm yields the log-likelihood

$$\ell(\mathbf{w}) = \sum_{n=1}^{N} \Big[t_n \log G(\mathbf{x}_n; \mathbf{w}) + (1 - t_n) \log\big(1 - G(\mathbf{x}_n; \mathbf{w})\big) \Big].$$

Maximizing $\ell(\mathbf{w})$ with respect to the weights is equivalent to minimizing its negative:

$$-\ell(\mathbf{w}) = -\sum_{n=1}^{N} \Big[t_n \log G(\mathbf{x}_n; \mathbf{w}) + (1 - t_n) \log\big(1 - G(\mathbf{x}_n; \mathbf{w})\big) \Big].$$

This is the *binary cross-entropy* loss (sometimes called log-loss). Notice that it looks very different from the sum-of-squared-errors loss used in regression; the statistical model (Bernoulli instead of Gaussian) forces us to change the loss function.

For the network to output a valid probability we also replace the final linear activation with a sigmoid:

$$G(\mathbf{x}; \mathbf{w}) = \sigma\big(\mathbf{w}^\top \mathbf{h}(\mathbf{x})\big) \in (0, 1).$$

Gradient descent is then performed on the cross-entropy loss (not on squared error). Historically, in the 1990s many practitioners still used squared error even for classification; it produced numbers but performed noticeably worse. Switching to cross-entropy gave immediate improvements on essentially every classification benchmark, which is why it is now the default choice.

4.14 Generalization to Multi-Class: Categorical Cross-Entropy

The same idea extends naturally to K classes. We represent the target of the n-th example by a one-hot vector $\mathbf{t}_n \in \{0, 1\}^K$ (exactly one entry is 1, all others are 0). The network now outputs a vector of K probabilities $\mathbf{G}(\mathbf{x}_n; \mathbf{w})$ (obtained by applying the softmax function to the final layer). The probability of the observed class is

$$p(\mathbf{t}_n \mid \mathbf{x}_n, \mathbf{w}) = \prod_{k=1}^{K} \big[G_k(\mathbf{x}_n; \mathbf{w})\big]^{t_{n,k}}.$$

The log-likelihood for the dataset becomes

$$\ell(\mathbf{w}) = \sum_{n=1}^{N} \sum_{k=1}^{K} t_{n,k} \log G_k(\mathbf{x}_n; \mathbf{w}).$$

Equivalently, the loss to minimize (negative log-likelihood) is

$$-\ell(\mathbf{w}) = -\sum_{n=1}^{N} \mathbf{t}_n^\top \log \mathbf{G}(\mathbf{x}_n; \mathbf{w}).$$

This expression is called *categorical cross-entropy*. It is identical to the binary case when $K = 2$ and we use one-hot encoding, but it is written in a vectorial form that works for any number of classes.

An important computational advantage appears when we differentiate the loss with respect to the network output: for each sample the gradient is zero everywhere except at the correct class, where it equals $1/G_k$ (the reciprocal of the predicted probability for the true class). This makes the backward pass extremely efficient.

4.15 Why the Choice of Loss Function Matters

The loss function is how we tell the network "what the task is."

- For regression we assume additive Gaussian noise with constant variance (homoscedasticity) $\Rightarrow$ squared-error loss.
- For classification we assume Bernoulli (or categorical) observations $\Rightarrow$ cross-entropy loss.

Using the wrong loss is like giving the network the wrong statistical model of the data, which leads to suboptimal (sometimes dramatically worse) results.

In modern practice dozens of specialized loss functions exist:

- Dice loss or Intersection-over-Union (IoU) for image segmentation,
- Focal loss for class-imbalanced detection,
- Triplet loss or contrastive loss for metric learning,

- Perceptual losses for image generation,
- and many more.

Some are derived from explicit probabilistic assumptions; others exploit domain knowledge about the task (e.g., overlap of bounding boxes). During the exam you will often be able to identify the intended task simply by looking at the output layer (linear, sigmoid, softmax) together with the loss function.

The general rule is: choose the output activation and the loss function that together correctly describe the data-generating process and the goal of the task. If the assumption "function plus constant-variance Gaussian noise" does not hold, or if the targets are discrete labels rather than continuous values, squared error is the wrong tool. Cross-entropy (or its multi-class generalization) is the statistically principled choice for classification.

Chapter 5

Facing Overfitting

5.1 From training to generalization

We have now concluded the discussion of the design of the error function, or *loss function*, which is the quantity minimized during training. This closes the part of the course devoted to neural-network training. The first block of slides, concerning the architecture and training of neural networks, has therefore been completed.

In the next lectures, we will address two key issues:

- how to make training more effective;
- how to deal with one of the most common problems in machine learning, namely *overfitting*.

5.2 Neural networks and loss functions

As a recap, neural networks are defined as collections of layers of neurons. So far, we have considered only *dense layers*, and we will continue to focus on this architecture for the moment. Next week, we will also discuss *dropout* and *batch normalization*. With Professor Boracchi, you will see *convolutional layers* as well.

A neural network is trained on a task defined by a loss function. We have seen two important examples:

- the *sum of squared errors*, which corresponds to the residual sum of squares in classical regression;
- the *cross-entropy*, or *categorical cross-entropy*, used for classification.

Of course, many other loss functions exist, and we will mention one of them later in this lecture. You will encounter others in Professor Boracchi's course.

5.3 Universal approximation and its limits

At this point, we have all the ingredients needed to train a neural network. This leads to the so-called *universal approximation* property: with sufficiently many layers and neurons, a neural network can approximate any nonlinear function arbitrarily well.

What does this mean precisely? It means that there exists a set of weights such that the neural network approximates the target function. However, this theorem does *not* say that we will necessarily find those weights in practice, nor does it guarantee that backpropagation will lead to a global optimum.

There is also another important caveat: the theorem says nothing about how large the network must be. In principle, it may require a very large number of neurons to achieve arbitrary accuracy.

Most importantly, the universal approximation property does *not* imply that the resulting model is useful. Being able to fit any dataset does not mean being able to predict unseen data. In other words, a model may fit the training set perfectly and still perform poorly on new samples.

This ability to perform well on unseen data is called *generalization*. Generalization is a central issue in every machine learning method. The more powerful the model, the more likely it is to suffer from generalization problems. Neural networks, being very flexible nonlinear approximators, are therefore particularly exposed to this risk.

This is closely related to *Occam's razor*: when two models perform similarly, one should prefer the simpler one. Complex models may fit the training data very well while failing on new data. This is the phenomenon of *overfitting*.

5.4 Inductive bias, underfitting, and overfitting

Let us recall the idea of *inductive bias*. Machine learning relies on the assumption that patterns learned from a finite dataset will also hold for new data. For example:

- if we assume that the data come from a linear model, we fit a line and expect it to work also on unseen data;
- if we assume that the classes are linearly separable, we may use logistic regression and expect it to generalize beyond the training set.

In general, the simpler the assumption about the model, the more likely it is to generalize well. Conversely, the more complex the model class, the easier it becomes to overfit.

Consider a simple example. Suppose the true data-generating process is a second-degree polynomial, and we fit the data using a linear model. In this case, the model is too simple to capture the structure of the data. The training error remains high, and we say that the model is *underfitting*.

The opposite situation occurs when the model is too flexible. For instance, if we fit a polynomial of very high degree, we may reduce the training error almost to zero. However, such a model may behave very poorly on new data, sometimes even worse than a much simpler linear model. In this case, the model is *overfitting*: it has learned the training data too well, including noise and irregularities that do not reflect the true underlying pattern.

Thus, the two extremes are:

- a model that is too simple and cannot fit the training data adequately;
- a model that is too complex and fits the training data perfectly but generalizes poorly.

5.5 How to detect overfitting

Before solving the problem of overfitting, we first need a way to detect it. The standard approach is to evaluate the model on data that have *not* been used for training.

This is essential: if we assess a model using the training data, we obtain an overly optimistic estimate of its performance. Instead, we must use new data to obtain a more realistic measure of how the model will behave in practice. This is analogous to an exam: exercises done in class are useful for learning, but the exam must contain new problems in order to test understanding rather than memorization.

Therefore, one of the worst mistakes in machine learning is to evaluate or select a model by looking only at the training data. A model should always be tested on an independent test set.

5.6 Train/test split and hold-out validation

Where does the test set come from? In some cases, it is provided externally. For example, in a challenge or competition, one dataset is used for training, while the test set remains hidden and is used only for final evaluation.

In other cases, the dataset must be split manually. A common strategy is to set aside part of the data, train the model on the remaining portion, and then evaluate its performance on the held-out data. This is known as the *hold-out* approach.

Training and testing must be independent, meaning that the two sets should not overlap.

When the dataset is very small, one may consider resampling methods such as the *bootstrap* or the *jackknife*. However, these are not typically used with neural networks. In practice, for neural-network applications, one usually either:

- receives a predefined train/test split, as in many challenges; or
- creates the split manually.

A common choice is to reserve about 10%–20% of the data for testing. In classification problems, the split should usually be *stratified*, meaning that the class proportions should be preserved in both the training and test sets.

5.7 Stratified sampling and data splitting

When dealing with a classification problem, the dataset may be highly unbalanced: one class can be much larger than the others. In this case, a purely random split may produce a test set that contains only a subset of the classes, or even miss some classes entirely. This would make the training and test distributions significantly different.

To avoid this issue, the split should preserve the class proportions as much as possible. This procedure is called *stratified sampling*. In stratified sampling, when extracting, for instance, 10% of the data for the test set, we ensure that the class frequencies in the test set remain approximately the same as in the original dataset.

5.8 Training, validation, and test sets

The full available dataset is typically divided into three disjoint parts:

- a *training set*, used to fit the parameters (weights) of the model;
- a *validation set*, used to choose the model and tune hyperparameters;
- a *test set*, used only at the very end to estimate the final performance.

This distinction is essential. If we train different models, for example networks with 10, 20, 50, 100, or 1000 neurons, we cannot choose among them by looking at the test set, because the test set must remain unseen until the end of the process. Instead, we train the models on the training set and compare their performance on the validation set. The model with the best validation performance is then selected.

The test set is used only once, after the model has been chosen, to provide an unbiased estimate of its true performance.

5.9 Why a validation set is needed

If we used only a training set and a test set, we would have no data available for model selection. In particular, we would not know whether to increase or decrease the number of layers, the number of neurons, or other hyperparameters.

The validation set solves this problem. It allows us to:

- train the model on one part of the data;
- evaluate alternative choices on unseen data;
- select the best configuration before touching the test set.

A simple analogy is the following: exercises done in class correspond to training, while the exam corresponds to testing. We do not want the exam to be identical to the exercises, because the goal is to assess whether the method has been learned, not whether the answers have been memorized.

5.10 Validation error and test error

The validation set is already an evaluation on unseen data, so why do we still need a test set? The reason is that the validation set influences model selection. Since we choose the model that performs best on the validation data, the validation error is slightly optimistic.

The test set provides a more neutral estimate of the final performance. In a well-designed experiment, validation and test errors should be similar. A large discrepancy usually indicates a problem.

5.11 What if test performance is much worse?

If a model performs very well on the training set and validation set, but very poorly on the test set, then there is likely a mismatch between the training and test distributions. This violates the basic assumption of machine learning: the data used for training and the data encountered at test time should come from the same distribution.

This problem may occur when:

- the dataset is very small;
- the dataset is highly unbalanced;
- the test set contains outliers or atypical samples;
- the training set does not adequately represent the variability of the problem.

In such cases, one may need to revisit the dataset, clean it, or redesign the training split. In practice, this is a warning sign that the training data do not adequately cover the phenomenon represented in the test data.

5.12 Using all the data at the end

A useful trick is the following. Suppose the best model has been selected using the training and validation sets. Once the architecture and hyperparameters have been fixed, one may retrain the model using all available labeled data, including the validation set. This may provide a small improvement in performance, especially when the dataset is not very large.

This is not a form of overfitting, because the validation set has already served its purpose: it was used to limit the complexity of the model. Once the best configuration has been chosen, using all the data helps the final model learn from more examples.

5.13 Model complexity and generalization

A common way to visualize model behavior is to plot the error as a function of model complexity. Here, complexity can be thought of as the number of degrees of freedom of the model, or more informally, its flexibility.

For very simple models, the training error is high, because the model cannot capture the structure of the data. This is the underfitting regime.

As complexity increases, the training error typically decreases. In the limit of very large complexity, the model may even fit the training data almost perfectly.

However, the behavior on unseen data is different. The validation error usually decreases at first, but after a certain point it starts increasing again because the model begins to overfit. Thus, there is an intermediate region where the model is sufficiently flexible to fit the main structure of the data, but not so flexible that it memorizes noise.

This is the ideal operating point: low error, but still good generalization.

5.14 Noise and the lower bound on error

Even with a very good model, the error cannot be reduced below a certain limit. This limit is determined by the noise in the data. If the data contain random noise, that noise cannot be predicted perfectly, precisely because it is random.

Therefore, even the best possible model has a residual error. Increasing the complexity of the model beyond a certain point will not improve performance; it will only increase the risk of overfitting.

5.15 Bias–variance trade-off

The variance of the noise is usually unknown. In practice, the noise is never exactly zero, so there is always a lower bound on the achievable prediction error. In other words, even the best model cannot predict the random component of the data perfectly.

One important subtlety is that, on the training set, a sufficiently flexible model may fit not only the underlying signal but also the noise specific to that sample. In this sense, the training error can even go below the level imposed by the noise. This is precisely what happens when the model memorizes the training data.

This phenomenon is summarized by the *bias–variance trade-off*. Here:

- *bias* is the error due to using a model that is too simple;
- *variance* is the error due to the sensitivity of the model to the particular training sample;
- their sum gives the *expected prediction error*.

As model complexity increases, the bias typically decreases, while the variance increases. Therefore, the expected prediction error usually has a minimum at an intermediate level of complexity. This is the model complexity we would like to select.

However, this optimal point cannot be identified by looking at the training data only. It must be estimated using validation data.

5.16 Cross-validation

The general idea of splitting the available data into subsets for training and evaluation is often referred to as *cross-validation*. In this setting, part of the dataset is used to train the model and another part is used to assess its quality.

There are also other, more qualitative forms of validation. For example, in clustering one may rely on a human expert to judge whether the clusters are meaningful. This is sometimes called *human validation*. In this course, however, we are mainly concerned with data-driven validation.

For neural networks, the standard approach is usually a simple hold-out split: one part of the data is used for training and another part for validation. This is practical because neural networks are typically trained on large datasets and model fitting is expensive.

5.17 Limitations of a single hold-out split

A drawback of a single train/validation split is that the result depends on how the data were partitioned. If the validation subset happens to be unusually easy or unusually hard, the estimated performance may be misleading.

For example, with a small dataset containing both easy and difficult samples, one split may place most difficult examples in the training set and most easy ones in the validation set, leading to an overly optimistic estimate. Another split may do the opposite and give a pessimistic estimate.

We would like our estimate of model performance to be less dependent on this random choice.

5.18 Leave-one-out cross-validation

One way to reduce this dependence is *leave-one-out cross-validation*. The idea is simple:

1. remove one sample from the dataset;

2. train the model on all the remaining samples;

3. test on the held-out sample;

4. repeat this procedure for every sample;

5. average the resulting errors.

This gives an estimate of the expected error obtained when each observation is left out in turn. In principle, this is a very accurate method.

However, it is computationally expensive. If the dataset contains n samples, then n training procedures are required. For neural networks, this is usually impractical except for very small datasets.

A useful variant is to leave out entire groups instead of single samples. For example, in medical applications one may leave out one hospital at a time, or one subject at a time, in order to test whether the model generalizes across groups. This is appropriate only when the number of groups is small.

5.19 k-fold cross-validation

A compromise between a single hold-out split and leave-one-out is *k-fold cross-validation*. The dataset is divided into k folds. The model is then trained k times:

- each time, one fold is used for validation;

- the remaining $k - 1$ folds are used for training.

The final estimate is the average of the k validation errors.

This approach reduces the dependence on a single arbitrary split while remaining much cheaper than leave-one-out. For example, with five-fold cross-validation, the model is trained five times instead of n times.

In classification problems, the folds should be *stratified*, so that class proportions are preserved in each fold.

5.20 Which model is used at test time?

Suppose that k-fold cross-validation has been used to compare several hyperparameter settings. Each fold produces a different trained model. Which one should be used for the final test?

There are two common strategies:

- combine the models, for instance by majority vote in classification or by averaging in regression;

- select the hyperparameter setting that performs best on average, and then retrain a final model on all available training data.

In practice, the second option is often preferred. Once the best hyperparameters have been chosen, one may retrain the final model using all the available training data, including the validation portion. This often improves performance slightly because the final model is trained on more data.

5.21 What is used in practice for neural networks

For neural networks, the most common strategy is still the simple hold-out procedure:

- split the available data into training and validation sets;
- tune the model using the validation set;
- keep the test set untouched until the very end.

Leave-one-out cross-validation is generally too expensive for neural networks. It is mainly useful for very small datasets or for special grouped data, such as multiple samples collected from different hospitals or different subjects.

5.22 Sampling considerations

As a general rule, random sampling should be used with care. In classification problems, the split should preserve the class distribution, especially when the dataset is unbalanced. This is why stratified sampling is important.

In regression problems, a useful trick is to sort the data by the target value and then sample across the full range, so that the validation set covers the whole domain more uniformly.

In summary, the key point is that the training, validation, and test data should be chosen so as to provide a reliable estimate of the model's true generalization performance.

5.23 Hyperparameter search

If the model has only one hyperparameter, selecting the best configuration is relatively easy. For instance, if we need to choose the number of neurons in a single hidden layer, we can train a few candidate networks and compare their validation errors.

The situation becomes much more difficult when the model has several hyperparameters. For example, with two hidden layers we must choose:

$$n_1 = \text{number of neurons in the first hidden layer}, \qquad n_2 = \text{number of neurons in the second hidden layer}.$$

In general, the number of possible configurations grows very quickly, and it is often impossible to test all combinations exhaustively.

A common strategy is *grid search*: we define a discrete set of candidate values for each hyperparameter and evaluate the model on the resulting grid. If the grid is too large, one may use a hierarchical or coarse-to-fine strategy, first exploring a rough region of the hyperparameter space and then refining the search around the most promising configurations.

5.24 Early stopping revisited

Early stopping uses the validation set to prevent overfitting. During training, we monitor the validation error and stop when it no longer improves. In this way, the validation set serves both for model selection and for regularizing the training process.

This is effective, but it requires sacrificing part of the available data for validation. This raises a natural question: is it possible to limit overfitting without relying too much on a validation split? The answer is yes, and one of the most important tools is *regularization*.

5.25 Regularization

Regularization is a general technique used to reduce overfitting by constraining the freedom of the optimization problem. The idea is to discourage overly complex models, typically by penalizing large weights.

This idea is not new. In linear models, ridge regression and lasso are classical examples of regularization. In neural networks, the same principle is used under different names:

- gradient descent is implemented through *backpropagation*;
- L_2 regularization is often called *weight decay*.

The intuition is simple: the more free parameters a model has, the more flexible it becomes. Regularization limits that flexibility.

5.26 A Bayesian interpretation

To understand regularization, it is useful to recall the Bayesian point of view. In a frequentist setting, parameters are usually estimated by maximizing the likelihood:

$$\hat{w} = \arg\max_w p(\mathcal{D} \mid w).$$

In Bayesian statistics, instead, we combine the likelihood with a prior distribution over the parameters:

$$\hat{w}_{\text{MAP}} = \arg\max_w p(w \mid \mathcal{D}) = \arg\max_w p(\mathcal{D} \mid w)\, p(w).$$

This is called *maximum a posteriori* (MAP) estimation.

The role of the prior is to express a belief about what values the parameters should plausibly take before seeing the data. When the dataset is small, the prior has a strong influence. When the dataset becomes very large, the likelihood dominates and the MAP estimate approaches the maximum-likelihood estimate.

5.27 Gaussian prior and L_2 regularization

For neural networks, a common and convenient assumption is to place a zero-mean Gaussian prior on the weights:

$$p(w_j) \propto \exp\left(-\frac{w_j^2}{2\sigma_w^2}\right).$$

Assuming independent weights, the full prior becomes

$$p(w) = \prod_j p(w_j).$$

If we also assume Gaussian noise on the outputs, then maximizing the posterior is equivalent to minimizing a loss of the form

$$\mathcal{L}(w) = \frac{1}{2\sigma^2} \sum_{i=1}^{N} \big(y_i - f(x_i; w)\big)^2 + \frac{1}{2\sigma_w^2} \sum_j w_j^2 \; + \; \text{constant}.$$

After rescaling, this can be written as

$$\mathcal{L}(w) = \sum_{i=1}^{N} \big(y_i - f(x_i; w)\big)^2 + \lambda \sum_j w_j^2, \qquad \lambda = \frac{\sigma^2}{\sigma_w^2}.$$

This is precisely the standard L_2 penalty. In neural networks, this is called *weight decay*. The term λ controls the strength of regularization:

- if λ is too small, the model may overfit;
- if λ is too large, the weights are forced toward zero and the model underfits.

Thus, regularization introduces a trade-off between data fitting and model complexity.

5.28 L_1 regularization and sparsity

If instead of a Gaussian prior one uses a Laplace prior, the resulting penalty is proportional to the absolute value of the weights:

$$\mathcal{L}(w) = \sum_{i=1}^{N} \big(y_i - f(x_i; w)\big)^2 + \lambda \sum_j |w_j|.$$

This is the L_1 regularization term, analogous to lasso in linear regression.

The effect of L_1 regularization is different from L_2:

- L_2 regularization makes weights small, but typically not exactly zero;
- L_1 regularization promotes sparsity, often driving many weights exactly to zero.

For this reason, L_1 regularization can produce more compact and interpretable models.

5.29 Summary

Regularization is a way of preventing overfitting without relying only on early stopping. Its Bayesian interpretation is especially useful:

- the likelihood measures how well the model fits the data;
- the prior encodes a preference for simpler or smaller-weight models;
- the posterior combines both pieces of information.

With a Gaussian prior, the resulting penalty is L_2 regularization, or weight decay. With a Laplace prior, the resulting penalty is L_1 regularization, which encourages sparsity.

Chapter 6

Training Tips and Tricks

6.1 Early stopping, weight decay, and regularization

In the previous lecture, we introduced the problem of overfitting. The key idea was that a model should ultimately be evaluated on data it has never seen during training. For this reason, the test set must be kept aside and used only at the very end, to assess the final quality of the model.

During model development, however, we still need a way to make decisions about the architecture and the complexity of the model. This is why we introduced the validation set. In short:

$$\text{training set} \;\to\; \text{fit the model}, \qquad \text{validation set} \;\to\; \text{select the model}, \qquad \text{test set} \;\to\; \text{final evaluation}.$$

One simple way to reduce overfitting is *early stopping*. The idea is to stop training before the network starts fitting the noise in the training data. In practice, one monitors the validation error during training and stops when it stops improving. A common implementation is to interrupt training if the validation error has not decreased for a fixed number of epochs, often called the *patience*.

Early stopping is effective, but it requires setting aside some data for validation. For this reason, other techniques have been introduced to limit overfitting without removing data from training. One of the most important is *weight decay*.

6.2 Weight decay

Weight decay can be interpreted as a Bayesian maximum a posteriori approach, and it acts as a regularization term added to the loss function. Instead of minimizing only the data-fitting term, we minimize a penalized objective:

$$\mathcal{L}(w) = \sum_{i=1}^{N} \big(y_i - f(x_i; w)\big)^2 + \gamma \sum_j w_j^2,$$

where $\gamma \geq 0$ controls the strength of the regularization.

The first term measures how well the model fits the training data. The second term penalizes large weights, encouraging simpler models. The intuition is that networks with smaller weights tend to overfit less, because their predictions are less sensitive to small changes in the input and the model has less freedom to memorize noise.

The parameter γ controls the trade-off:

- if $\gamma = 0$, we recover the standard sum-of-squares objective and overfitting becomes more likely;
- if γ is very large, the weights are forced toward zero and the model underfits;
- an intermediate value of γ often gives the best generalization.

The value of γ is a hyperparameter, so it must be chosen by model selection, typically through cross-validation. In practice, one trains the model for several candidate values of γ, evaluates each one on the validation set, and selects the value that gives the lowest validation error. Since γ may vary over several orders of magnitude, it is common to perform a coarse-to-fine grid search.

Once the best $\gamma^\star$ has been selected, the final model can be retrained using the chosen regularization strength, possibly on the full training data.

In practice, weight decay can be applied to all layers of the network or only to selected layers. In principle, different values of γ could be chosen for different layers, but this quickly becomes expensive and cumbersome, so in most cases a single global value is used.

6.3 Early stopping and weight decay together

Early stopping and weight decay can be used together. In fact, this is often done in practice.

Early stopping prevents the network from training for too long, while weight decay keeps the weights small during optimization. Both methods reduce overfitting, but they act in different ways. Early stopping is especially convenient because it is simple to implement and often shortens training time. Weight decay is useful as an additional form of regularization, especially when the dataset is small or the model is highly flexible.

6.4 Dropout

A third important technique for reducing overfitting is *dropout*. Unlike early stopping and weight decay, dropout introduces stochasticity during training.

The basic idea is to randomly deactivate some neurons during each training step. Suppose the network is very large and has more capacity than needed. During training, some neurons may become co-adapted: instead of learning robust features individually, they may rely too much on one another. Dropout prevents this by forcing each unit to learn a more independent representation.

More precisely, at each iteration we sample a random binary mask for each layer. The mask contains zeros and ones:

$$m^{(\ell)} \in \{0,1\}^{d_\ell}.$$

The activations in layer ℓ are then multiplied elementwise by this mask:

$$\tilde{h}^{(\ell)} = h^{(\ell)} \odot m^{(\ell)},$$

where $\odot$ denotes the Hadamard product.

Units with mask value 0 are temporarily switched off, while units with value 1 remain active. Since the mask changes at every iteration, the network is trained under many different random sub-networks. This stochastic regularization makes the final model less likely to overfit.

Dropout is a stochastic regularization technique used to reduce overfitting. The basic idea is to temporarily remove, during training, a random subset of the neurons in a layer together with all the connections passing through them. In this way, the masked units do not participate in the forward pass and therefore do not contribute to the backward pass either.

As a consequence, at each iteration gradient descent updates only a subset of the weights. At the next iteration, a different random mask is generated, so a different subset of neurons is active. Repeating this process many times forces the network to learn representations that do not rely too heavily on specific neurons or paths through the model.

This can be interpreted as training an ensemble of many different sub-networks sampled from the original architecture. The final network behaves, in a loose sense, like an average of these many smaller models. This is one of the reasons why dropout often improves generalization: it reduces the tendency of the network to co-adapt its hidden units.

6.5 Scaling at test time

During training, some units are randomly switched off, so the following layers receive only a fraction of their usual input. For example, if one third of the units in a layer are dropped, the next layer receives only about two thirds of the input signal.

At test time, however, dropout is no longer applied. Since all neurons are now active, the network would otherwise produce activations that are systematically larger than those seen during training. To compensate for this, the weights or activations are rescaled so that the expected output remains approximately unchanged.

In this way, the network at test time approximates the average behavior of the many sub-networks seen during training.

6.6 Practical aspects of dropout

Dropout has one main hyperparameter: the probability of dropping a unit. This value must be chosen by the user, typically through validation or grid search. Larger dropout rates correspond to stronger regularization, while smaller rates have a weaker effect.

As with the other techniques discussed so far, dropout can be combined with early stopping and weight decay. These methods act at different levels:

- early stopping acts on the training procedure;
- weight decay modifies the loss function;
- dropout modifies the network architecture during training.

Because of this, the three methods are compatible and are often used together in practice.

6.7 Relation to ensemble methods

Dropout is sometimes informally compared with ensemble learning because it trains many slightly different subnetworks and averages their effect. This analogy is useful to understand why dropout works, but dropout is not exactly the same as bagging or boosting.

The key point is that each iteration sees a different masked version of the network, so the model becomes less sensitive to the specific configuration of weights and less likely to overfit. In practice, dropout often improves robustness, although it does not guarantee optimal performance in every setting.

6.8 Transition to activation functions

At this point, there are still a few topics left to clarify. One of them is the choice of activation function. Up to now, we have mentioned sigmoid and hyperbolic tangent activations, but these are not always the best choice in modern deep networks.

In particular, both sigmoid and tanh are saturating functions. When the input becomes too large in magnitude, their derivatives become very small. As a result, the gradient can vanish during backpropagation, which makes training difficult in deep architectures.

This is one of the reasons why more modern activation functions are preferred in many deep learning models.

6.9 Vanishing gradients and activation functions

Recall that the sigmoid activation has its largest derivative at the origin, where it equals 0.25. Outside a small neighborhood around zero, its derivative quickly approaches zero. The hyperbolic tangent behaves similarly: its derivative is maximal at zero, where it equals 1, and it vanishes as the input moves away from the origin.

This becomes a serious problem in backpropagation. For a network with several hidden layers, the gradient of the loss with respect to the parameters of an early layer is obtained by multiplying many derivatives, one for each layer traversed by the signal. If each derivative is smaller than one, the product shrinks rapidly.

For example, if we multiply ten factors equal to 0.1, the result is 10^{-10}. With one hundred such factors, the result is essentially zero. This phenomenon is called *vanishing gradient*. It makes the training of very deep networks difficult, because the gradient reaching the first layers becomes too small to produce meaningful updates.

This problem was already known in early neural-network research and became especially clear in the study of recurrent neural networks. It also affects deep feed-forward networks when sigmoid or tanh activations are used.

6.10 Exploding gradients

The opposite situation may also occur. If the derivatives are larger than one, then the product of many such terms grows exponentially. This leads to the *exploding gradient* problem, which is numerically unstable.

Therefore, from the point of view of gradient propagation, it is desirable to use activation functions whose derivative is close to one over a large region of the input space. A linear activation would solve this issue, but a purely linear network would still be a linear model. We therefore need a nonlinear activation that preserves a stable gradient.

6.11 ReLU

A very successful choice is the *rectified linear unit* (ReLU), defined by

$$\phi(x) = \max(0, x).$$

Thus,

$$\phi(x) = \begin{cases} x, & x > 0, \\ 0, & x \leq 0. \end{cases}$$

On the positive side, the derivative is equal to 1, so gradients do not vanish there. On the negative side, the output is exactly zero. This simple piecewise-linear form is easy to implement and computationally efficient. In practice, ReLU often leads to faster convergence than sigmoid or tanh.

ReLU also induces sparse activations: only a subset of the neurons are active at a given time. This is often beneficial both computationally and statistically.

6.12 Limitations of ReLU

Despite its advantages, ReLU is not perfect.

First, it is not differentiable at zero. In practice, this is not a major issue, because the set of inputs that land exactly at zero has measure zero, and one can simply define the derivative at zero by convention.

A more relevant issue is that ReLU outputs are unbounded and not centered around zero. This may sometimes lead to numerical instability, although in practice this is usually manageable with a proper initialization and learning rate.

The most important drawback is the *dying ReLU* problem. If a neuron receives a sufficiently negative input, its output becomes zero. Since the derivative on the negative side is also zero, the neuron stops receiving useful gradient information. Once this happens, the neuron may remain inactive forever and never recover.

This is especially problematic early in training or when the learning rate is too large. A large update may push a neuron into the negative region, after which it becomes effectively dead.

6.13 Leaky ReLU

A common remedy is the *leaky ReLU*. Instead of setting the negative part exactly to zero, we give it a small slope:

$$\phi(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \end{cases} \qquad \text{with } 0 < \alpha \ll 1.$$

The parameter α is usually small, for example 0.01.

This modification preserves a nonzero gradient even when the input is negative. As a result, a neuron that has moved to the wrong side can still receive gradient information and can potentially recover. In this sense, leaky ReLU alleviates the dying-neuron problem while keeping many of the benefits of standard ReLU.

6.14 Modern activation functions and weight initialization

If changing from ReLU to leaky ReLU does not noticeably improve performance, this is not necessarily a problem. These methods are most useful when the network is close to the right size and dying neurons become an actual limitation. If the network is already very large, losing a few units may not matter much. In practice, one should try these activations and check whether they improve performance on the validation set.

Over the years, many variants of ReLU have been proposed. One example is *ELU* (exponential linear unit), which replaces the negative linear part with a smooth exponential curve. This makes the function continuous and differentiable everywhere. The negative side still has a small gradient, so the vanishing of gradients is not completely eliminated, but the method often works well in practice.

Another modern activation is *Swish*, defined by

$$\phi(x) = x\,\sigma(x),$$

where $\sigma(x)$ is the sigmoid function. As $x \to -\infty$, the output tends to 0, while as $x \to +\infty$, the function behaves approximately like the identity. Empirically, smoother activation functions such as Swish sometimes lead to slightly better and faster convergence than piecewise-linear activations.

These improvements are usually more visible when moving from sigmoid or tanh to ReLU-like functions than when moving from ReLU to one of its smoother variants. In other words, the largest gain is typically obtained by replacing saturating activations with non-saturating ones in deep networks.

6.15 Activation functions in the output layer

The discussion above concerns the hidden layers. The output layer is chosen according to the task:

- sigmoid for binary classification;
- softmax for multi-class classification;
- linear activation for regression.

In some special cases, one may choose an output activation that matches known constraints on the target. For instance, if the output must be positive, one may use a suitable transformation of the target or a constrained output activation. In most standard settings, however, the usual choices above are sufficient.

6.16 Weight initialization

Another important topic is *weight initialization*. So far, we have said that the weights are initialized randomly, but the precise choice matters a lot.

Initializing all weights to zero is a bad idea. If all weights are zero, then the gradients flowing through the network become zero as well, so the network cannot learn properly. Likewise, initializing the weights

with very large values is also problematic: it may cause exploding gradients, and for saturating activations it pushes the units into regions where the derivative is very small.

In general, we want the initial activations to remain in a reasonable range, so that:

- the forward signals are not too large or too small;
- the gradients remain stable during backpropagation;
- training starts from a numerically well-behaved configuration.

A good initialization should produce activations with variance that does not grow or shrink excessively as the signal propagates through the network.

6.17 Why initialization matters

Consider a neuron with input

$$h = \sum_{i=1}^{n} w_i x_i.$$

If the inputs and weights are independent random variables with zero mean, then the variance of h depends on both the number of inputs and the variance of the weights. In particular, as the signal passes through many layers, the variance may either explode or vanish if the weights are not initialized carefully.

This is why weight initialization is especially important in deep networks. The choice of initialization affects not only the forward pass, but also the backward pass, because the gradient is propagated through the same layers.

6.18 Xavier initialization

One classical solution is *Xavier initialization*. The idea is to choose the variance of the weights so that the variance of the activations remains approximately constant across layers.

This reasoning is often derived under the simplifying assumption that the neuron is approximately linear. In practice, this is a reasonable approximation near the origin, and it is especially useful for activations whose behavior around zero is close to linear.

The goal is to keep the signal in a stable range, roughly between -1 and 1, or more generally within a moderate interval where gradient descent can work efficiently.

6.19 He initialization

For ReLU-type activations, a different initialization is often more appropriate: *He initialization*. Since ReLU suppresses negative values, the variance propagates differently than in the linear case. He initialization takes this into account and chooses a larger variance than Xavier initialization in order to compensate for the half-rectified nature of ReLU.

In practice:

- Xavier initialization is commonly associated with tanh and similar activations;
- He initialization is commonly associated with ReLU and leaky ReLU.

Both methods aim to prevent vanishing or exploding activations and gradients at the start of training.

6.20 Weight initialization revisited

Let h denote the output of a neuron. For a dense layer, the pre-activation is a linear combination of the previous layer:

$$h = \sum_{i=1}^{n_{\text{in}}} w_i x_i,$$

where n_{in} is the number of inputs of the layer.

If the weights have zero mean and are independent from the inputs, then the variance of the output depends on both the variance of the inputs and the variance of the weights. Under the usual simplifying assumptions,

$$\text{Var}(h) \approx n_{\text{in}} \, \text{Var}(w) \, \text{Var}(x).$$

This relation shows why initialization matters. If the weights are initialized too large, the signal may grow from layer to layer and lead to exploding activations. If they are initialized too small, the signal may shrink and vanish. In both cases, optimization becomes unstable.

The goal of a good initialization is therefore to keep the scale of the signal roughly constant at the beginning of training. In other words, we would like each layer to preserve the variance of the activations as much as possible.

6.21 Xavier initialization

A classical solution is *Xavier initialization*. The idea is to choose the variance of the weights so that the variance of the signal remains approximately unchanged as it passes through each layer.

A common choice is

$$\text{Var}(w) = \frac{1}{n_{\text{in}}},$$

with weights drawn from a zero-mean distribution, typically Gaussian or uniform.

For example, if a dense layer has 10 inputs, the weights may be initialized from a Gaussian distribution with mean 0 and variance 0.1. If the next layer has 100 inputs, the corresponding variance would be 0.01. This keeps the scale of the activations under control and improves numerical stability during the first stages of training.

Xavier initialization is a good default choice when the activation function is approximately symmetric around zero.

6.22 Glorot initialization

A more balanced version is *Glorot initialization*, which takes into account both the number of input and output units:

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

This is often interpreted as the inverse of the average fan-in and fan-out.

The motivation is that one may want to preserve not only the forward variance, but also the backward variance during backpropagation. Glorot initialization is therefore a compromise between the two directions.

6.23 He initialization

For ReLU-type activations, a slightly different scaling is usually preferred. Since ReLU keeps only the positive part of the signal, roughly half of the values are set to zero on average. For this reason, the variance should be larger than in the symmetric case.

A common rule is *He initialization*:

$$\text{Var}(w) = \frac{2}{n_{\text{in}}}.$$

This is particularly well suited to ReLU and related activation functions.

In practice, Xavier, Glorot, and He initialization all aim at the same goal: keeping the signal and the gradients in a stable range so that training starts from a well-conditioned configuration.

6.24 Practical remarks on initialization

If the initialization is too naive, training may become unstable, especially in deep networks. For this reason, one should avoid arbitrary or overly creative initializations. A good initialization should:

- keep the activations within a reasonable range;
- avoid vanishing or exploding gradients;
- make the beginning of training numerically stable.

In most modern frameworks, suitable initializers are already available as defaults, so in practice one usually uses Xavier/Glorot or He initialization depending on the activation function.

6.25 Normalization of the input

A related idea is to normalize the input data before training. If the input variables have very different scales, gradient descent may become inefficient. A common preprocessing step is to center each feature and scale it to unit variance:

$$x' = \frac{x - \mu}{\sigma}.$$

This transforms the data into a more numerically convenient range and often improves optimization.

This preprocessing also helps reduce *covariate shift*, meaning changes in the distribution of the data seen by the model during training.

6.26 Batch normalization

Even if the input is normalized, the distribution of the activations may change from layer to layer during training. To address this, one can normalize the activations inside the network as well. This is the idea behind *batch normalization*.

For each mini-batch, batch normalization computes the mean and variance of the activations of a given layer, then normalizes them:

$$\hat{h} = \frac{h - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}},$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the batch mean and variance, and ε is a small constant for numerical stability.

After normalization, the layer applies a learnable affine transformation:

$$y = \gamma \hat{h} + \beta,$$

where γ and β are trainable parameters.

Batch normalization has several advantages:

- it stabilizes training;
- it helps gradients propagate more easily;
- it reduces sensitivity to initialization;
- it can sometimes improve generalization.

6.27 Where batch normalization fits in the network

Batch normalization is inserted inside the network, typically after a linear layer and before the activation function:

$$\text{linear layer} \rightarrow \text{batch normalization} \rightarrow \text{activation}.$$

Thus, it acts as a form of preconditioning of the optimization process at each layer. Since the operation is differentiable, it can be fully integrated into forward and backward propagation.

6.28 Batch normalization

Batch normalization is another layer that can be added to a neural network to ease training. It is now a standard component in many well-known architectures.

The idea is simple. During training, for each mini-batch, the layer computes the mean and standard deviation of its inputs, and then normalizes them:

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}},$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the mean and variance of the current batch, and ε is a small constant for numerical stability.

Backpropagation works through this normalization without any special difficulty, since the normalization step is part of the computational graph.

6.29 Learned scale and shift parameters

At test time, the network may receive one sample at a time, so batch statistics are no longer available. For this reason, batch normalization includes two trainable parameters: a scale parameter γ and a shift parameter β. After normalization, the output is transformed as

$$y = \gamma \hat{x} + \beta.$$

These parameters allow the network to recover the appropriate scale and offset at inference time.

Thus, batch normalization does not only normalize the data during training: it also learns how to adapt the normalized values so that the layer remains useful at test time.

6.30 Why batch normalization helps

Batch normalization often improves training in several ways:

- it makes optimization more stable;
- it improves convergence;
- it often allows the use of a larger learning rate;
- it reduces sensitivity to weight initialization;
- it can sometimes reduce the need for dropout.

Its main practical effect is to keep the activations in a more controlled range, which helps gradient-based optimization.

6.31 A final remark on optimization

Before closing this part of the course, it is useful to recall that backpropagation is only the mechanism used to compute gradients. The actual training algorithm is gradient descent or one of its variants.

We have already seen that classical gradient descent can be improved by adding momentum. A related method is *Nesterov accelerated gradient*. In standard momentum, one moves in the direction of the previous velocity and then applies the gradient. In Nesterov's method, one first takes a tentative step in the direction of the momentum and then evaluates the gradient at that updated position. This often gives a slightly more accurate update and reduces oscillations.

More generally, many optimization algorithms have been developed for neural networks. The most common ones include:

- stochastic gradient descent;
- momentum;
- Nesterov accelerated gradient;
- AdaGrad;
- Adadelta;
- Adam;
- AdamW.

These methods differ in how they adapt the learning rate and how they use the history of past gradients. Some of them are especially effective when different parameters of the model evolve at very different speeds.

In practice, Adam and AdamW are among the most widely used optimizers today, because they often work well with little tuning. A typical choice is to start from a small learning rate, such as 10^{-4} or 10^{-5}, and let the optimizer adapt the updates during training.

Chapter 7

Overfitting and Regularization

7.1 Introduction

In the previous lecture, we saw how to train a very simple neural network. The usual machine learning pipeline begins with data loading, followed by data analysis and exploration, preprocessing, model design, training, and finally validation, to verify that the model has learned correctly.

Today we will discuss the first major difficulty you often encounter when training neural networks. Neural networks are extremely powerful function approximators, and this can be a double-edged sword in machine learning applications. Since this course is focused on neural networks, we will continue using them throughout, but in real-world problems other types of models may sometimes be more appropriate.

The goal of this lecture is to give you a broader view of the advantages and limitations of neural networks, and to clarify when it makes sense to use them and when it may not.

7.2 Dataset

As a new example, we move from flowers to glass. The *Glass* dataset comes from the UCI Machine Learning Repository, a well-known source of benchmark datasets for machine learning research. This dataset concerns *glass identification*.

The original motivation for this task came from criminology: by identifying the type of glass found at a crime scene, investigators may obtain useful forensic evidence.

The dataset contains:

- 214 samples,
- 9 input features,
- 1 target variable, corresponding to the glass class.

All features are numerical, so no categorical encoding is required. However, the features should be scaled, since they are measured on very different ranges. For instance, the refractive index and the concentrations of chemical elements have very different magnitudes.

The class labels correspond to different types of glass, such as windows, containers, and tableware. Although the original dataset defines seven classes, one of them is absent here, so we effectively work with six classes.

7.3 Class Imbalance

A key difference with respect to the previous dataset is that this one is highly imbalanced. Some classes are much more frequent than others: for example, one class has only a handful of samples, while others are much better represented.

This is a serious issue because a classifier may obtain very high accuracy simply by predicting the majority class most of the time. In such cases, accuracy is not a reliable measure of performance. We therefore need more informative metrics.

7.4 Evaluation Metrics

For classification problems, performance is often analyzed through the *confusion matrix*, which contains:

$$\text{TP} = \text{true positives}, \qquad \text{TN} = \text{true negatives}, \qquad \text{FP} = \text{false positives}, \qquad \text{FN} = \text{false negatives}.$$

Accuracy is defined as

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}.$$

Although accuracy is easy to interpret, it can be misleading on imbalanced datasets. Two much more useful metrics in this setting are *precision* and *recall*:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Intuitively, precision measures how many of the samples predicted as positive are actually positive, while recall measures how many of the actual positive samples are recovered by the model.

These two quantities are often summarized by the *F1-score*, which is their harmonic mean:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

7.5 Multi-Class Classification

Since the Glass dataset is a multi-class problem, we extend these ideas using a one-vs-all strategy. For each class, we treat that class as the positive class and all the others as negative. In this way, we can compute precision and recall for each class separately.

Finally, these class-wise values are averaged over all classes, yielding a global metric for the multi-class setting. This gives a much more meaningful evaluation than accuracy alone, especially when the dataset is imbalanced.

7.6 Class Weights and Evaluation Metric

For imbalanced classification problems, we can also introduce *class weights* when computing aggregate metrics. In this way, the precision and recall of each class contribute according to its representation in the dataset. Classes with more samples receive a larger weight, while minority classes contribute proportionally less to the weighted average.

In this notebook, we will therefore use the weighted F1-score as our main validation metric. This allows us to account for class imbalance and to evaluate performance more fairly across both majority and minority classes.

7.7 Data Preprocessing

We now move to data preprocessing, following the same general workflow as in the previous lecture.

First, we define:

$$X \in \mathbb{R}^{N \times d}$$

as the matrix containing all feature values, and

$$y \in \{0, 1, \ldots, C-1\}^N$$

as the vector of class labels. Here, X is a floating-point tensor, while y contains integer class indices.

Next, we split the dataset into training, validation, and test sets. As before, the data are partitioned into three subsets, but this time it is even more important to use *stratified splitting*, so that the class distribution is preserved across all three splits.

Because the dataset is relatively small, this step is crucial for obtaining a meaningful evaluation.

7.8 Normalization and Data Loaders

We then normalize the input features using min–max scaling. For each feature, we store the minimum and maximum values and rescale the data so that all features lie in the interval $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

After normalization, we convert the data from NumPy arrays to PyTorch tensors.

Finally, we create the data loaders exactly as before. During training, the data are shuffled and loaded in batches while backpropagation is running. For validation and test, we do not shuffle the data and we do not drop the last incomplete batch.

For instance, with a batch size of 32, the resulting tensors have shape

$$(32, 9)$$

for the input features and

$$(32,)$$

for the labels.

7.9 Model Definition

We now define the neural network architecture. Compared with the previous lecture, the model is larger and deeper. In particular, we can specify:

- the input dimension,
- the number of hidden layers,
- the size of each hidden layer,
- the number of output classes,
- whether dropout should be used.

The model is implemented as a single class that automatically builds the full architecture from these parameters.

The construction follows this pattern:

1. start with an input layer,
2. add a first hidden layer,
3. append additional hidden layers,
4. optionally insert dropout,
5. apply the activation function after each linear layer,
6. finish with the output layer, which maps the hidden representation to the number of classes.

In this lecture, the hidden layers are quite large, with 512 neurons each. This is much larger than the number of input features, and it will later help us discuss overfitting.

7.10 Number of Parameters

When counting the number of trainable parameters in a linear layer, we must include both weights and bias terms. For a layer with input dimension d_{in} and output dimension d_{out}, the total number of parameters is

$$(d_{\text{in}} + 1)\, d_{\text{out}}.$$

For example, a layer mapping from 512 units to 512 units has

$$(512 + 1) \cdot 512$$

parameters, since we also account for the bias term.

The same rule applies to every linear layer in the network.

7.11 Training Setup

We now define the training configuration. The model is trained for 500 epochs using:

- the Adam optimizer,
- cross-entropy loss,
- a learning rate of 10^{-3},
- GPU acceleration, when available.

These are standard hyperparameters, but they are not guaranteed to produce good performance automatically. We therefore need to check the validation curves carefully.

7.12 Training and Validation

The training loop is organized in the usual way. We define:

- a function to train one epoch,
- a function to evaluate the model on the validation set,
- a function to log metrics to TensorBoard,
- a global `fit` function that combines all the previous steps.

By modifying the arguments of the `fit` function, we can later add regularization techniques to the model.

7.13 Baseline Model and Overfitting

We first train a baseline model, that is, a neural network with default initialization and no regularization.

The results show a clear case of overfitting:

- the training F1-score rises from about 0.2 to nearly 1.0,
- the training loss becomes very small,
- but the validation F1-score remains much lower and does not improve in the same way.

This means that the model has learned the training set very well, but it does not generalize properly to unseen data.

The confusion matrix confirms this behavior: the classifier learns something meaningful, but there is still substantial confusion between the most represented classes. Therefore, even though the result is not completely random, the model can clearly be improved.

7.14 What Is Overfitting?

Overfitting occurs when a model learns the training data too well, including noise and sample-specific patterns, instead of capturing the underlying structure of the problem.

In other words, the model memorizes the training set rather than generalizing to new examples. In machine learning, however, we are interested in *out-of-sample performance*, namely how well the model performs on unseen data.

7.15 Model Families, Bias, and Variance

This is a good point to recall a general principle of machine learning: when solving a problem, we first choose a *model family*. Examples include linear models, decision trees, kernel methods, and neural networks.

Different model families behave differently, and their error can often be understood through the bias–variance trade-off.

Bias. Bias measures the systematic error of a model family. A model with high bias is too rigid and tends to underfit the data. Its predictions are consistently far from the true target.

Variance. Variance measures how sensitive a model is to small changes in the training set. A model with high variance may perform very differently depending on the particular sample realization used for training.

In the geometric intuition shown in the lecture:

- low bias and low variance correspond to a good model family;
- high bias corresponds to a model that is too simple or too constrained;
- high variance corresponds to a model that is too flexible and unstable.

Neural networks are typically very expressive, so in this course they often fall into the high-variance regime unless regularization is applied.

7.16 Conclusion

At this stage, we have completed the preprocessing pipeline and trained a first baseline neural network. The results clearly show that the model overfits the training set. This motivates the next part of the lecture, where we will study regularization methods and learn how to improve generalization.

7.17 Class Weights and Weighted Metrics

For imbalanced classification problems, we can also introduce *class weights* when computing aggregate metrics. In this way, the precision and recall of each class contribute according to its representation in the dataset. Classes with more samples receive a larger weight, while minority classes contribute proportionally less to the weighted average.

In this notebook, we therefore use the weighted F1-score as our main validation metric. This allows us to account for class imbalance and evaluate performance more fairly across both majority and minority classes.

7.18 Bias–Variance Trade-off

As in life, everything is a trade-off. In practice, it is very unlikely that a model will lie exactly on one of the two extreme diagonals of the bias–variance diagram. Instead, we typically move along the trade-off curve between:

- **high bias, low variance**, as in simple models such as linear regressors;

- **low bias, high variance**, as in flexible models such as neural networks.

The goal of model selection is to find the right balance between these two extremes and choose a model complexity that gives the best possible generalization performance.

7.19 Error Decomposition

A common way to understand model performance is through the decomposition of the total error into three terms:

$$\text{Total error} = \text{Bias} + \text{Variance} + \text{Irreducible error}.$$

The irreducible error is intrinsic to the data and cannot be controlled by the model. It represents the part of the prediction problem that no learning algorithm can eliminate.

If we order models by increasing complexity, we usually observe the following behavior:

- bias decreases as complexity increases;
- variance increases as complexity increases.

Therefore, the optimal model is often found in the middle, where the combination of bias and variance is minimized.

7.20 Why Start from a Large Model?

A natural question is whether it is better to start from a simple model and increase complexity, or to start from a large model and simplify it later.

A useful intuition is that regularization works by *reducing* model complexity. For example, it can shrink weights, drop units, or otherwise constrain the network. This means that if we start from a large model, we can use regularization to move automatically toward a simpler and better-behaved solution during training.

By contrast, starting from a very small model and trying to increase its capacity is usually less convenient, because it often requires redesigning the architecture and retraining from scratch.

A good analogy is sculpting: it is easier to begin with a large block of stone and remove material until the statue emerges, rather than trying to glue together many small pieces and expecting the same result.

7.21 Early Stopping

The first strategy to prevent overfitting is *early stopping*. This is one of the most effective and practical techniques for training neural networks, and it should be used whenever possible.

During training, we monitor both:

- the training loss or training metric,
- the validation loss or validation metric.

Overfitting usually appears when the training curve keeps improving while the validation curve stops improving or starts worsening. Early stopping detects this situation and interrupts training when no improvement has been observed for a given number of epochs.

This number is called *patience*. If the metric does not improve for more than `patience` epochs, the training loop is stopped.

7.22 Implementation of Early Stopping

In our implementation, early stopping is handled inside the `fit` function. The general logic is the following:

1. initialize a counter for the number of epochs without improvement;
2. compare the current validation metric with the best value seen so far;
3. if the current metric improves, save the model weights and reset the counter;
4. otherwise, increment the counter;
5. if the counter exceeds the patience value, stop training;
6. restore the best model weights at the end.

Restoring the best weights is essential. It is a common mistake to stop training early but forget to reload the best checkpoint, which would make early stopping ineffective.

7.23 Effect of Early Stopping

When early stopping is applied, the training curve typically ends before the maximum number of epochs. In our example, training stops around epoch 300 instead of epoch 500.

Since we use the F1-score as the monitored metric, the best model is selected according to the epoch with the highest validation F1. If instead we monitor the loss, training may stop earlier, but the final F1-score may be slightly lower.

There is no universal rule that works best in every situation. The choice depends on the dataset, the task, and the objective of the experiment.

7.24 Confusion Matrix and Further Improvements

The confusion matrix after early stopping is slightly better than before. In particular, class 0 is classified more accurately, although some confusion between classes 0 and 1 still remains.

This confirms that early stopping helps, but it does not necessarily solve every problem. In practice, hyperparameters and regularization strategies must always be tuned experimentally.

7.25 Practical Remark

All of these settings are guidelines, not guarantees. In real-world datasets, it is common to still observe overfitting even after applying early stopping or other regularization methods. This is normal, because training neural networks is a nonlinear optimization problem and there is no universally optimal solution.

7.26 Dropout

Another standard technique to reduce overfitting is *dropout*. The idea is to randomly deactivate some neurons during training so that the network cannot rely too heavily on any single path through the model.

More precisely, at each training iteration, dropout temporarily sets a fraction of activations to zero. This means that the effective architecture changes from batch to batch, and the network is forced to learn redundant and more robust internal representations.

Dropout is applied only during training. During validation and test time, all neurons remain active, and the full network is used.

7.26.1 Intuition

Why does dropout help? By randomly removing parts of the network during training, we encourage the model to learn features that are not too specific to one particular configuration of weights. In this sense, dropout acts as an automatic way of reducing model complexity and pushing the network toward more stable solutions.

This also explains why dropout can be seen as a bridge between a very flexible model and a simpler one: it effectively regularizes the network by preventing co-adaptation between neurons.

7.26.2 Implementation in PyTorch

In PyTorch, dropout is implemented as a layer and can be inserted directly inside the model construction loop. If the dropout probability is greater than zero, we append a dropout module to the list of layers.

A crucial detail is the use of:

`model.train()` and `model.eval()`.

When the model is in training mode, gradients are tracked and dropout is active. When the model is in evaluation mode, gradients are disabled and dropout is turned off. This behavior must always be handled correctly in a custom training loop.

7.26.3 Choice of Dropout Rate

In this lecture we use a dropout rate of 0.5, which is relatively large. This means that at each forward pass, roughly half of the activations are switched off.

If a model still overfits with such a strong dropout rate, then the architecture itself is probably too large for the dataset. In that case, it is often better to reduce the number of layers or the number of hidden units instead of increasing dropout even further.

7.26.4 Effect on Training

With dropout enabled, training usually becomes slower and noisier, since the network is effectively changing at every iteration. This is expected. The training curve may look less smooth, but the model often generalizes better.

Because the validation set is small, even a difference of a single sample can noticeably change the validation metric. For this reason, small differences between checkpoints should not be overinterpreted.

7.27 L1 and L2 Regularization

A second family of regularization techniques is based on adding a penalty term to the loss function. Historically, models that overfit often learn weights with very large magnitude. Regularization discourages this behavior.

7.27.1 L2 Regularization

L2 regularization adds the squared norm of the weights to the loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \sum_i w_i^2.$$

This is also known as *weight decay*. The main effect of L2 regularization is to penalize large weights more strongly than small ones, while not forcing weights exactly to zero.

7.27.2 L1 Regularization

L1 regularization adds the absolute value of the weights:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \sum_i |w_i|.$$

Unlike L2, L1 tends to push many weights exactly to zero. For this reason, it can be interpreted as a form of automatic feature selection or parameter sparsification.

7.27.3 Geometric Intuition

A useful way to think about these penalties is to imagine the loss surface as a function of the weights.

- L2 regularization smooths the optimization landscape and discourages very large parameters.
- L1 regularization creates sharper constraints and promotes sparse solutions.

In practice, L2 is often the most common choice, while L1 can be especially useful when sparsity or feature selection is desirable.

7.27.4 Elastic Net

The two penalties can also be combined:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda_1 \sum_i |w_i| + \lambda_2 \sum_i w_i^2.$$

This combination is known as *Elastic Net*. It interpolates between L1 and L2 regularization and can be useful when both sparsity and smooth shrinkage are desired.

7.28 Interpretation

Regularization techniques such as dropout, L1, and L2 all have the same general goal: reduce the effective complexity of the model and improve generalization.

Dropout can be thought of as a stochastic way of simplifying the network during training, while L1 and L2 modify the loss directly. In particular, L1 has the advantage of making some weights exactly zero, which can improve interpretability because certain features may effectively disappear from the model.

The right regularization strategy depends on the dataset and the task. In practice, it is often necessary to try several options and compare their validation performance.

7.29 Implementing L1 and L2 Regularization

Let us briefly see how these regularization terms are implemented in practice.

In the `train_one_epoch` function, the loss is first computed in the usual way by applying the criterion, namely the cross-entropy loss, to the model logits and the target labels:

$$\mathcal{L}_{\text{CE}} = \texttt{criterion}(\texttt{logits}, \texttt{targets}).$$

Since PyTorch allows us to manipulate tensors directly, we can also compute additional penalty terms from the model parameters. In particular:

- **L1 regularization** is obtained by summing the absolute values of all parameters;
- **L2 regularization** is obtained by summing the squared values of all parameters.

In code, `model.parameters()` is an iterator over all parameter tensors in the network, so we can apply any differentiable operation to them and include the result in the loss. This is one of the main strengths of PyTorch: any differentiable quantity can be inserted into the training objective and backpropagated through.

Of course, nothing prevents us from applying L1 or L2 only to specific layers, or from designing more complex combinations of penalties. The standard choices are the ones most commonly used in the literature, but in principle many other regularization schemes can be implemented as well.

7.30 Effect of L2 Regularization

In our experiment, we use L2 regularization. This is implemented by adding a penalty term weighted by a coefficient λ to the training loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{L2}}.$$

The parameter λ controls the relative importance of the regularization term. If λ is too large, then the loss becomes dominated by the penalty term and the network is pushed toward weights that are nearly zero. In that case, the model may become overly constrained and essentially useless.

This illustrates an important point: when multiple loss terms are combined, their relative scale matters a great deal. The cross-entropy loss should remain the main term driving optimization, while any additional penalty should stay in a reasonable range.

After adding L2 regularization, the training curves improve somewhat, although overfitting is still present. The best epoch is reached earlier than before, and the validation performance becomes slightly better. However, the model is still far from perfect.

7.31 Why Loss Scaling Matters

Any additional regularization term must be carefully balanced with the main loss. This is especially important because different penalties may have very different numerical scales.

For example, L1 and L2 regularization do not behave in the same way:

$$\mathcal{L}_{\text{L1}} = \sum_i |w_i|, \qquad \mathcal{L}_{\text{L2}} = \sum_i w_i^2.$$

Since the L2 term squares the weights, its magnitude can grow much faster than the L1 term. For this reason, it is often useful to inspect the individual components separately before combining them.

7.32 Optimization and the Role of the Optimizer

Training a neural network is a nonlinear optimization problem, so it often requires experimentation and some degree of trial and error. The optimizer is the mechanism that searches for a minimum of the loss function.

Different optimizers behave differently. A useful visual intuition is to think of the loss surface as a landscape:

- stochastic gradient descent follows the gradient directly;
- momentum adds inertia to the updates;
- RMSprop adapts the step size using gradient statistics;
- Adam combines several heuristic corrections and is usually a strong default choice.

In practice, Adam often performs better than plain stochastic gradient descent, and it is therefore widely used for neural networks.

7.33 AdamW and Weight Decay

A particularly important variant is `AdamW`, which is now commonly preferred over standard Adam when using L2 regularization.

The key idea is that standard Adam does not handle weight decay in the most effective way when the L2 penalty is directly added to the loss. The reason is that Adam adapts the learning rate for each parameter, and this adaptation also affects the regularization term in an undesirable way.

AdamW decouples weight decay from the adaptive gradient update. In practice, this means:

1. Adam is applied to the main loss, such as cross-entropy;
2. the weights are then shrunk separately by the decay term.

This decoupling has been shown to work better in practice, which is why AdamW is often considered the default optimizer for modern neural network training.

7.34 Implementation of AdamW

To use AdamW, we simply replace the optimizer while keeping the rest of the training loop unchanged. The regularization coefficient is no longer passed to the training function as a manual penalty term; instead, it is handled directly by the optimizer through the weight decay parameter.

With this change, the model still overfits to some extent, but the overall performance is slightly better than before. This confirms that the regularization strategy is helping, even if it does not solve the problem completely.

7.35 Final Evaluation on the Test Set

After selecting the best hyperparameters and the best validation checkpoint, we can finally evaluate the model on the test set.

This is the last step of the pipeline. The test set must be used only once, after all model choices have already been made. In particular, we must not tune hyperparameters based on test performance, because that would amount to optimizing on the test set and would make the reported result unreliable.

To compute the final score, we:

- run inference on the test set,
- compute the weighted F1-score,
- inspect the confusion matrix.

The resulting confusion matrix is reasonable: most majority-class samples are classified correctly, while the remaining errors are consistent with the difficulty of the task. The final test performance is slightly lower than validation performance, which is expected and desirable.

7.36 Conclusion

The main message of this lecture is that training neural networks requires careful tuning. Regularization, early stopping, dropout, and optimizer choice all interact with one another, and there is rarely a single best configuration.

The correct workflow is therefore:

1. train several models,
2. compare validation performance,
3. select the best configuration,

4. evaluate once on the test set.

This is the standard way to obtain a reliable estimate of generalization performance.

Chapter 8

Cross Validation and Hyperparameter Tuning

8.1 How to Use Data

A central topic in machine learning is how to use data properly. In practice, the data you receive may be incomplete, noisy, inconsistent, or collected from heterogeneous sources that must first be merged and cleaned. Once this preprocessing step is done, we obtain a dataset, which we may represent as a matrix or a table.

The main question is how to split this dataset for training and evaluation.

8.2 Training Only

The simplest strategy is to train a model using the entire dataset. We call this *training only*.

This approach has an obvious drawback: since the model sees all available data during training, there is no independent estimate of how well it will perform on unseen examples. In other words, we cannot properly evaluate generalization.

The main issue is *overfitting*: the model may fit the training data very well, but fail to generalize to new data. A model that cannot learn the training set at all is clearly not useful, but a model that memorizes the training set without generalizing is also problematic.

In some settings, however, training only can still be useful. For example, it may help us check whether the preprocessing pipeline is correct, or whether the model is at least capable of fitting the data. In this course, though, this approach will never be sufficient as a final evaluation strategy.

8.3 Train/Test Holdout

A better strategy is to split the data into a training set and a test set. This is the classical *holdout* approach.

The advantage is that we can train the model on one subset and then evaluate it on unseen data. This gives us an estimate of how the model may behave in practice.

However, there is still a limitation. If we use only a training set and a test set, we cannot perform proper model selection or hyperparameter tuning. The test set should remain untouched until the very end. If we use it too early, we are indirectly tuning the model on the test data, which defeats its purpose.

8.4 Train/Validation/Test Split

The standard solution is to divide the data into three subsets:

- a **training set**, used to fit the model;

- a **validation set**, used to tune hyperparameters and select the best model;
- a **test set**, used only once at the end to estimate final generalization performance.

This is the minimal fair protocol for model evaluation.
The roles of the three splits are different:

- the training score tells us whether the model is learning;
- the validation score guides model selection;
- the test score provides the final unbiased estimate of performance.

The validation score is therefore an estimate of an estimate: it helps us predict what performance we may expect on the test set, which itself is only an estimate of performance on truly unseen data.

8.5 Why a Validation Set Is Necessary

A validation set is needed because model selection requires an independent criterion. If we train a model and then compare multiple configurations, we need a separate dataset to decide which model is best.

This is especially important when the dataset is small. In that case, an unlucky split may distort the evaluation, because the training and validation sets may not have the same distribution. This can make the selected model unreliable.

8.6 Cross-Validation

To reduce the dependence on a single split, we can use *cross-validation*. This is a more robust strategy in which the dataset is repeatedly split into multiple parts and the model is trained and evaluated several times.

In K-fold cross-validation, the dataset is divided into K subsets, called folds. The procedure is repeated K times:

1. one fold is used for validation;
2. the remaining $K-1$ folds are used for training;
3. the process is repeated so that each fold is used once as validation.

At the end, the performance is averaged across all folds:

$$\text{score}_{\text{CV}} = \frac{1}{K} \sum_{i=1}^{K} \text{score}_i.$$

This reduces variance in the evaluation and makes the result less dependent on a lucky or unlucky split.

8.7 Advantages of Cross-Validation

Cross-validation has several advantages:

- it gives a more stable estimate of performance;
- it reduces the impact of random splitting;
- it is especially useful for small datasets;
- it provides a better basis for model selection.

For this reason, cross-validation is often considered a robust evaluation strategy.

8.8 Important Caveat

Cross-validation must still be used carefully. The test set should remain separate and untouched until the very end. If the test set is examined during model selection, then the evaluation becomes biased and no longer reliable.

In other words, the test set is reserved for the final assessment only. All model comparisons and hyperparameter tuning should be performed using the training and validation data.

8.9 Model Selection and Hyperparameter Tuning

Cross-validation is used for *model selection*. This means choosing among multiple candidate models or configurations. Even changing a single parameter can be considered a form of model selection.

Hyperparameter tuning is part of this process. Examples include:

- the number of layers,
- the number of hidden units,
- the learning rate,
- regularization strength,
- dropout rate.

The goal is to choose the configuration that performs best on validation data, and then evaluate it once on the test set.

8.10 From Model Selection to Final Deployment

Suppose we have identified a good model during validation. In a deep learning setting, the next question is: *how should we use it in practice?*

In classical machine learning, after choosing the best hyperparameters, one typically retrains a final model on the entire dataset and then deploys that model. This is a natural strategy when the training procedure is stable and the selected hyperparameters are sufficiently robust.

However, in neural networks there is an additional complication: *early stopping*. The optimal stopping epoch is often different for different data splits, even when all the other hyperparameters are fixed. For instance, one fold may reach its best validation score at epoch 20, while another may peak around epoch 100.

This creates an important practical question: should we average the best epochs and train one final model, or should we keep all the trained models?

A reasonable answer is to keep all the models and combine them into an *ensemble*. Each model has been trained on a different split and has learned slightly different aspects of the data. At inference time, the models can then vote or average their predictions. For classification, this is usually done by averaging the class probabilities. For regression, one can average the predicted values.

In this sense, the final system is not necessarily a single model, but rather a collection of models that each capture a different view of the problem.

8.11 Why Not Simply Average the Epochs?

A tempting idea is to compute the average of the best epochs found across folds and then retrain one final model using that average stopping point. In practice, this is often not ideal.

The main reason is that the optimal epoch may vary significantly across splits, especially when the dataset is small or heterogeneous. If one fold peaks at epoch 101 and another at epoch 20, the average does not necessarily correspond to a meaningful training duration for the full dataset.

Moreover, the optimal epoch depends on the interaction between the data split, the initialization, and the optimization dynamics. There is no guarantee that the average of several best epochs will produce the best final model.

For this reason, keeping the fold-specific models and using them as an ensemble is often a better strategy.

8.12 Why Cross-Validation Matters

When the dataset is small, a single train/validation split may be too unstable. Some splits may be easier than others, and the validation score may vary significantly depending on which samples happen to fall into each subset.

Cross-validation reduces this problem by repeating the experiment over multiple splits. In this way, we obtain:

- a more reliable estimate of performance;
- less sensitivity to a lucky or unlucky split;
- a more robust basis for model selection.

This is especially important when the dataset is heterogeneous or when class balance is uneven.

8.13 Cross-Validation and Stratification

In classification problems, it is often useful to use *stratified* splitting, especially when classes are imbalanced. Stratification ensures that the class proportions remain approximately the same in each split.

This is important because otherwise some folds may contain too few samples from minority classes, making evaluation unstable and misleading.

In practice, stratified K-fold cross-validation is a very strong default choice when the data are limited and class imbalance is present.

8.14 Implementation Strategy

In our implementation, the data preprocessing and splitting are moved inside the cross-validation loop. This is necessary because each fold must be treated as a separate experiment.

The logic is as follows:

1. start from the original dataset;
2. define the number of folds K;
3. for each fold:
   (a) split the data into training and validation/test subsets;
   (b) fit the preprocessing statistics using only the training data;
   (c) normalize the data using those training statistics;
   (d) initialize a fresh model;
   (e) train the model with early stopping;
   (f) store the fold performance;
4. compute the mean and standard deviation of the performance across folds.

This gives a much more robust estimate of how the model behaves.

8.15 Normalization Must Use Training Data Only

A very important detail is that normalization must be computed using only the training subset.

For example, if we use min–max scaling, then the minimum and maximum values must be extracted only from the training data:

$$x' = \frac{x - x_{\min}^{(\text{train})}}{x_{\max}^{(\text{train})} - x_{\min}^{(\text{train})}}.$$

Using validation or test data to compute these statistics would leak information from unseen data into the training process. This would artificially improve the reported performance and invalidate the evaluation.

8.16 Why Reinitialize the Model at Each Fold?

At the beginning of each fold, the model must be reinitialized from scratch.

This is necessary because each fold must represent an independent training run. If we reused the weights from the previous fold, then the folds would no longer be independent, and the cross-validation estimate would become biased.

In practice, we store the initial state of the model and reload it for each new fold.

8.17 Choice of Hyperparameters

At this stage, we are no longer focused on the architecture itself, but on the tuning procedure.

The main hyperparameters include:

- the number of folds;
- the size of the validation or test split;
- the number of epochs;
- the patience used for early stopping;
- the learning rate;
- the dropout rate;
- the regularization strength.

These values must be chosen in a way that balances computational cost and evaluation reliability.

8.18 Practical Advice

In practice, the number of epochs should be large enough that the model has time to converge. If early stopping is used, the training will usually stop much earlier anyway, so setting a large maximum number of epochs is not a problem.

Similarly, patience should be chosen according to the typical behavior of the model on the dataset. If the validation score usually stabilizes after only a few epochs, there is no need to use a very large patience. On the other hand, if the metric fluctuates a lot, a larger patience may be necessary.

The important point is to become familiar with the dataset before attempting expensive hyperparameter searches.

8.19 Hyperparameter Tuning

Once a baseline model has been identified, the next step is to tune its hyperparameters. In deep learning, this is usually done by running several experiments and comparing their validation performance.

The general idea is simple: keep the model family fixed, change one hyperparameter at a time, and observe how the performance varies. This is the core of model selection.

8.20 Cross-Validation Results

Using cross-validation, we may obtain quite different results across folds. For example, the same model can achieve an F1-score of 71% on one fold, 68% on another, and 59% on another.

This variability is not surprising. It shows that the split of the data has a strong impact on the final result, especially when the dataset is small or heterogeneous. For this reason, a single train/validation split may be misleading, while cross-validation provides a more reliable estimate.

The average performance is what matters most, not a single lucky or unlucky fold.

8.21 Parameter Tuning

Parameter tuning means searching for the best hyperparameter values for a fixed model architecture. In our case, this includes the feedforward neural network we have used so far.

The tuning procedure is implemented as a wrapper around the cross-validation loop. For each configuration, the model is trained from scratch, evaluated on all folds, and the resulting scores are collected.

The main hyperparameters we consider here are:

- the batch size;
- the learning rate;
- the number of epochs;
- the early stopping patience;
- the hidden layer size;
- the number of layers;
- the dropout rate.

8.22 Batch Size

Batch size is one of the most important hyperparameters.

At one extreme, we have batch size equal to 1. In this case, we perform one update for each sample, which makes training slow and noisy.

At the other extreme, we have batch size equal to the full dataset. In this case, training becomes efficient, since each epoch requires only one update, but the optimization becomes less flexible and may be overly influenced by the average behavior of the data.

The optimal batch size lies between these two extremes. In practice, we usually explore values such as 4, 8, 16, 32, 64, or 128.

A useful practical rule is to prefer powers of two, since this often leads to better computational efficiency on GPUs.

8.22.1 Trade-off

Small batches:

- are noisier;
- may improve exploration;
- require more updates per epoch;
- are slower.

Large batches:

- are more stable;
- are computationally efficient;
- may reduce the effect of noise;
- can become less flexible.

8.23 Learning Rate

The learning rate controls the size of each parameter update. It is one of the most sensitive hyperparameters in neural network training.

A useful geometric intuition is to imagine optimization as a ball moving in a loss landscape. The learning rate determines how large each step is.

- If the learning rate is too large, the optimizer may overshoot the minimum or bounce around the valley.
- If the learning rate is too small, training becomes extremely slow and the model may get stuck near its initial region.

For this reason, the learning rate must be chosen carefully.

8.23.1 Learning Rate Schedulers

Sometimes the best strategy is to start with a relatively large learning rate and then reduce it during training. This is the role of a *learning rate scheduler*.

The idea is that the optimizer can explore the landscape more freely at the beginning, and then refine the solution more carefully later.

8.24 Practical Tuning Strategy

In practice, it is better to reason in terms of neighborhoods rather than exact values. For example, instead of testing a hidden layer size of 16, 17, and 18 units, it is often more useful to compare values around 100 or around 500.

The same principle applies to batch size and learning rate. We do not search blindly over all possible values. Instead, we test a few reasonable regions and observe where the model performs best.

This is especially important because training is noisy: changing the split, the initialization, or the batch order can already modify the final result.

8.25 Reading the Results

To compare tuning results, we often use box plots. These show the distribution of the validation scores obtained across folds or across repeated experiments.

A box plot is useful because it gives not only the average performance, but also the variability and the presence of outliers.

When the number of evaluated folds is small, the plot should be interpreted carefully. Increasing the number of folds usually gives a more reliable statistical picture, but also increases the computational cost.

8.26 Optimization as a Trade-off

A recurring theme in machine learning is that there is no perfect choice in every situation. Training requires balancing several trade-offs:

- stability versus speed;

- exploration versus precision;
- efficiency versus flexibility;
- simplicity versus expressive power.

The goal is not to find the absolute best hyperparameter in theory, but the most effective compromise for the given dataset and task.

8.27 Learning Rate, Model Complexity, and Hyperparameter Search

The key message is that hyperparameter tuning requires reasoning. In practice, one should test several learning rates and compare the resulting validation performance.

For example, two nearby learning rates may lead to noticeably different results, even when the model architecture remains unchanged. This is a reminder that optimization is sensitive to the choice of hyperparameters.

8.28 Model Complexity

Another important quantity to tune is the model complexity. So far, we have discussed how to feed the data and how to control the training procedure. Now we must understand whether the model is powerful enough to solve the task.

In principle, model complexity is not a good standalone tuning parameter, because its effect is usually visible only through the training set. If a model can overfit the training data, then it has enough capacity to represent the task. Once a reasonable range of complexity has been identified, we can refine the architecture further.

For instance, if we know that the number of parameters is somewhere between tens of thousands and hundreds of thousands, we can focus on that region instead of testing every possible configuration individually.

8.29 Grid Search

A standard strategy for hyperparameter tuning is *grid search*. In grid search, we define a discrete set of values for each hyperparameter and try all possible combinations.

Suppose we want to tune:

- batch size,
- learning rate,
- model complexity.

If each parameter has several candidate values, the total number of experiments can grow very quickly. This is why grid search can become extremely expensive.

One possible strategy is to tune the parameters one at a time:

1. choose the best batch size;
2. fix it;
3. tune the learning rate;
4. fix it;
5. tune the model complexity.

This incremental strategy is simpler, but it may discard useful combinations of parameters. A fully combined grid search explores more possibilities, but at a much higher computational cost.

8.30 Regularization in the Search Space

Once a strong baseline configuration has been found, regularization can be added on top of it. In our case, we consider:

- dropout,
- L1 regularization,
- L2 regularization.

The model can first be tuned for batch size, learning rate, and size, and only afterwards regularization can be introduced. This often gives better results than trying to tune everything at once from the beginning.

In the example discussed in class, adding regularization after finding a good baseline slightly improves the final performance.

8.31 Why Saving the Best Model Matters

During tuning, it is essential to save the best configuration. If the experiment is repeated later, the result may change because neural network training is stochastic.

This means that even with the same architecture and hyperparameters, repeated runs can produce different outcomes. For this reason, when a good configuration is found, it should be saved immediately.

If time allows, the best experiment can be repeated several times to check whether the result was stable or simply lucky.

8.32 Random Search

When the search space becomes too large, grid search may become impractical. In that case, a good alternative is *random search*.

Random search simply samples hyperparameter configurations at random from the search space and keeps the best one. Surprisingly, this strategy is often very competitive and is a strong baseline.

Its main advantage is efficiency: instead of wasting time on a complete exhaustive grid, we explore the search space more freely and often discover good configurations faster.

8.33 Advanced Hyperparameter Optimization

More advanced methods also exist. For example:

- Bayesian optimization;
- genetic algorithms;
- other automatic hyperparameter search strategies.

These methods use an optimization procedure to choose the next configurations to test, often by balancing exploration and exploitation.

Tools such as Optuna or Ray can be used to automate this process. However, these methods are not always better than a simple random search, especially if they are not used carefully.

8.34 Why Cross-Validation Is Not Always Used

Cross-validation is useful, but it is not always the best choice.

The main reason is computational cost. If a single training run is already expensive, repeating it many times can become prohibitive. For large models or large datasets, cross-validation may require too much time and energy.

In practice, cross-validation is especially useful when:

- the dataset is small;
- the data are heterogeneous;
- model selection must be robust;
- the class distribution is uneven.

When the dataset is large and well distributed, a simpler train/validation/test split may be sufficient.

8.35 Summary

The main lessons are:

- test different learning rates and batch sizes;
- reason in terms of model complexity rather than isolated values;
- save the best model whenever a good configuration is found;
- use grid search when the search space is small enough;
- use random search or more advanced methods when the search space is large;
- use cross-validation when robustness matters and computation allows it.

In short, hyperparameter tuning is not about finding a magic setting, but about exploring the search space intelligently and selecting the configuration that performs best in practice.

Chapter 9

Recurrent neural networks + LSTM

9.1 Introduction: Static vs. Dynamic Models

We have so far considered classical feed-forward neural networks. Given an input vector $\mathbf{x}$, the network produces an output $\mathbf{y}$. The mapping $\mathbf{x} \mapsto \mathbf{y}$ is *static*:

- Input $\mathbf{x}^{(A)}$ always yields $\mathbf{y}^{(A)}$.
- Input $\mathbf{x}^{(B)}$ always yields $\mathbf{y}^{(B)}$.
- Changing the order of presentation does not affect the outputs.

There is no notion of *sequence*, *history*, or *memory*. Every forward pass is independent; the network effectively restarts from scratch for each input (though its trained weights are fixed).

Determinism. It is a common misconception that neural networks are stochastic. Given identical numerical input, the forward pass is entirely deterministic. (We disregard hardware faults such as bit flips from cosmic rays.)

9.1.1 The Need for Sequence Modelling

Many real-world data sources are sequential:

- Words are sequences of letters; sentences are sequences of words.
- Time series: stock prices, sensor readings, control actions.

In these settings, the *order* of inputs carries essential information. A permuted sequence of letters may lose all meaning. When the output at step t depends not only on the current input $\mathbf{x}_t$ but also on past inputs $\mathbf{x}_{t-1}, \mathbf{x}_{t-2}, \ldots$, the model must incorporate some form of *history* or *dynamics*.

We will examine two broad families of approaches:

1. **Memoryless models** that explicitly concatenate a window of past inputs.
2. **Models with memory** that maintain an internal state.

9.2 Memoryless Approaches: Time-Delay Neural Networks

A simple way to handle a sequence $\{\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \ldots\}$ without explicit memory is to reformat the problem as a static supervised learning task.

9.2.1 Linear Autoregressive Models

Classical linear models, such as Moving Average (MA) or Autoregressive Moving Average (ARMA) processes, predict a future value using a weighted sum of past observations. For a window of size 2:

$$\hat{\mathbf{x}}_2 = w_0\mathbf{x}_0 + w_1\mathbf{x}_1.$$

In signal processing and telecommunications, these delayed inputs are often called *delay taps*.

9.2.2 Nonlinear Extension with Feed-Forward Networks

We can replace the linear combination with a feed-forward neural network:

$$\hat{\mathbf{x}}_t = f_\theta(\mathbf{x}_{t-k}, \ldots, \mathbf{x}_{t-1}).$$

To train such a model, we transform the original sequence into a dataset of input–output pairs:

Input Features	Target
$(\mathbf{x}_0, \mathbf{x}_1)$	$\mathbf{x}_2$
$(\mathbf{x}_1, \mathbf{x}_2)$	$\mathbf{x}_3$
$(\mathbf{x}_2, \mathbf{x}_3)$	$\mathbf{x}_4$
$\vdots$	$\vdots$

This sliding-window technique yields a standard supervised learning problem, which can be solved with any feed-forward architecture. No online training is required; the dataset is constructed offline.

9.3 Models with Explicit Memory

For many tasks, an explicit internal state $\mathbf{h}_t$ (a *memory*) is desirable. The general principle is:

1. A new input $\mathbf{x}_t$ arrives.
2. It *updates* the hidden state $\mathbf{h}_t$.
3. The output $\mathbf{y}_t$ is produced based on the current state $\mathbf{h}_t$ (and possibly a direct shortcut from $\mathbf{x}_t$).

9.3.1 Linear Dynamic Systems (State-Space Models)

A classical example from control theory and system identification is the discrete-time linear state-space model:

$$\mathbf{h}_{t+1} = \mathbf{A}\mathbf{h}_t + \mathbf{B}\mathbf{x}_t \quad \text{(State equation)} \tag{9.1}$$

$$\mathbf{y}_t = \mathbf{C}\mathbf{h}_t \quad \text{(Output equation)} \tag{9.2}$$

Here $\mathbf{h}_t$ is the memory (state) of the system. The matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ parameterise the dynamics. If the output also depends directly on the input, we add a term $+\mathbf{D}\mathbf{x}_t$, which corresponds to a *shortcut connection* in neural network terminology.

9.3.2 Hidden Markov Models (HMMs)

A Hidden Markov Model also fits this framework:

- The state s_t is a discrete random variable.
- The state evolves according to a *transition matrix* $P(s_{t+1} \mid s_t)$.
- The output $\mathbf{y}_t$ is drawn from an emission distribution $P(\mathbf{y}_t \mid s_t)$.

In both linear systems and HMMs, inference about the hidden state can be performed by well-known algorithms (Kalman filter for linear Gaussian systems; Viterbi algorithm for HMMs).

9.3.3 Recurrent Neural Networks (RNNs)

To obtain a nonlinear, learnable version of the state-space model, we replace the linear transformations with neural network layers:

$$\mathbf{h}_{t+1} = \sigma(\mathbf{W}_h \mathbf{h}_t + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}_h),$$
$$\mathbf{y}_t = \phi(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y).$$

The feedback loop $\mathbf{h}_t \to \mathbf{h}_{t+1}$ grants the network a persistent memory across time steps. This architecture is the foundation of *Recurrent Neural Networks*, which will be examined in detail shortly.

Note on Shortcut Connections. Adding a direct path from $\mathbf{x}_t$ to $\mathbf{y}_t$ (i.e., $\mathbf{y}_t = \phi(\mathbf{W}_y \mathbf{h}_t + \mathbf{W}_s \mathbf{x}_t)$) can mitigate issues such as the vanishing gradient problem. Structured use of such connections appears in modern architectures (e.g., residual networks), which will be covered in later lectures.

9.4 From State Estimation to Sequence Prediction

In classical state-space models (linear dynamic systems, hidden Markov models), the goal is often to *estimate* the hidden state. With neural networks, our objective shifts: we seek a model that produces accurate predictions by incorporating past inputs, without necessarily recovering an interpretable state.

This leads to the **Recurrent Neural Network** (RNN).

9.4.1 What Makes a Network Recurrent?

An RNN contains neurons whose output is fed back as input in subsequent time steps. These recurrent connections form a *context* or *memory* layer. If we denote the memory at time t as $\mathbf{h}_t$, its update depends on both the current input $\mathbf{x}_t$ and its own previous value $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = \phi\big(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{b}_h\big),$$

where ϕ is a nonlinear activation function.

9.4.2 Finite vs. Infinite Memory

A feed-forward network with a sliding window (delay taps) has **finite memory**: the prediction depends only on the k most recent inputs. In contrast, the recurrent loop creates the *potential* for **infinite memory**. Because $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, which depends on $\mathbf{h}_{t-2}$, and so on, information from arbitrarily distant past inputs can, in principle, be retained.

This feedback mechanism is analogous to the state equation in linear systems or to the feedback loop in a digital flip-flop—a fundamental building block of electronic memory.

9.5 RNN Architecture

Figure **??** (conceptual) illustrates the recurrent structure. The network comprises:

- **Context neurons** (blue): the recurrent part, storing the memory $\mathbf{h}_{t-1}$.
- **Hidden layer** (yellowish): optionally, a feed-forward pathway from the input.
- **Output layer**: a nonlinear combination of the hidden state and possibly a direct shortcut from the input.

The most general form of the equations is:

$$\mathbf{c}_t = \phi_c\big(\mathbf{W}_{xc} \mathbf{x}_t + \mathbf{W}_{cc} \mathbf{c}_{t-1} + \mathbf{b}_c\big) \quad \text{(Context update)} \tag{9.3}$$
$$\mathbf{h}_t = \phi_h\big(\mathbf{W}_{xh} \mathbf{x}_t + \mathbf{W}_{ch} \mathbf{c}_{t-1} + \mathbf{b}_h\big) \quad \text{(Hidden state)} \tag{9.4}$$
$$\mathbf{y}_t = \phi_y\big(\mathbf{W}_{hy} \mathbf{h}_t + \mathbf{W}_{cy} \mathbf{c}_t + \mathbf{b}_y\big) \quad \text{(Output)} \tag{9.5}$$

Simpler variants exist; the only strict requirement for recurrence is the presence of the feedback loop in Equation (9.3). The architecture described here is often termed the **Elman network** (1995). It was proven to be computationally universal—equivalent in power to a Turing machine.

Determinism. Despite the dynamic nature of the recurrent loop, the RNN remains a fully deterministic system. Given the same initial state and the same input sequence, it will always produce the identical output sequence. Noise is not inherent to the model; any stochasticity arises from the data or from training regularization.

9.6 Training Recurrent Networks: Backpropagation Through Time (BPTT)

The primary obstacle in training RNNs is the recurrent connection: standard backpropagation assumes a *feed-forward* computation graph.

The solution is to **unroll** (or unfold) the network over time. Consider a sequence of length T. By replicating the recurrent neurons at each time step and connecting them forward, we obtain a *feed-forward* network with T layers (Figure **??**). This unrolled network can then be trained with ordinary backpropagation.

9.6.1 Backpropagation Through Time (BPTT)

1. **Unroll** the RNN for the length of the input sequence.

2. The weights **W** are **shared** across all time steps in the unrolled graph (i.e., the same weight matrix appears at every layer).

3. Compute the gradient of the loss with respect to each shared weight by summing (or averaging) the gradients obtained from each time step where the weight appears.

4. Update the single copy of the weight using the averaged gradient.

Mathematically, for a shared weight w, the total gradient is:

$$\frac{\partial \mathcal{L}}{\partial w} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial w},$$

where $\mathcal{L}_t$ is the loss at time t.

9.6.2 Truncated Backpropagation Through Time (TBPTT)

Unrolling the entire sequence can be computationally expensive for long sequences and hinders batch parallelism when sequences have different lengths.

Truncated BPTT fixes the unrolling length to a constant K (e.g., $K = 20$ steps). The network is unrolled only K steps backward. This:

- Creates a uniform computation graph across all sequences in a batch.

- Enables efficient parallelization.

- Approximates the full gradient by ignoring dependencies beyond the truncation horizon.

9.6.3 Weight Sharing and Gradient Averaging

A crucial implementation detail: during backpropagation, the unrolled copies of a weight receive different gradient contributions from different time steps. To maintain the shared-weight constraint, we *average* the gradients from all copies and apply a single update to the master weight.

The procedure is equivalent to initializing all copies identically, backpropagating, and then averaging the updated values. In practice, it is more efficient to:

1. Compute gradients for each unrolled time step.
2. Sum (or average) the gradients for each shared weight.
3. Update the single weight instance using the aggregated gradient.

9.7 Training via Backpropagation Through Time

With the unrolling mechanism and gradient averaging in place, we can now train Recurrent Neural Networks (RNNs) using **Truncated Backpropagation Through Time** (TBPTT). The unrolled network is treated as a feed-forward model with shared weights across time steps, and updates are performed using the averaged gradients.

9.8 The Challenge of Long-Term Dependencies

9.8.1 Why Long Memory Matters

Many sequence modelling tasks require the network to retain information over long intervals. Consider the following sentence:

"Jane walked into the room. John walked into it too. It was late in the day. Jane said hi to ____."

To predict the missing word (e.g., `John` or `him`), the model must bridge a gap of over ten words—a long-term dependency.

9.8.2 The Vanishing Gradient in Deep Unrolled Networks

When an RNN is unrolled for T time steps, the recurrent layer becomes a feed-forward network of depth T. Training such a deep network via backpropagation inevitably encounters the **vanishing gradient** problem. In the late 1990s, a classic benchmark task demonstrated this limitation: an RNN was asked to remember whether a `1` had appeared anywhere in the preceding 1000 time steps. Despite varying network size, training consistently failed for horizons beyond roughly 100 steps.

9.9 Mathematical Analysis of Gradient Flow in RNNs

To understand the failure, we examine the simplest possible RNN: a single recurrent neuron with state h_t.

9.9.1 Model Definition

$$h_t = \sigma\big(v\, h_{t-1} + w\, x_t\big) \tag{9.6}$$

$$y_t = \phi\big(u\, h_t\big) \tag{9.7}$$

where σ is a nonlinear activation function (e.g., sigmoid or tanh), and v, w, u are scalar weights.

9.9.2 Gradient of the Current State with Respect to the Past

Let $\mathcal{L}_t$ be the error at time t. To update the weights, we need gradients of the loss with respect to parameters from earlier time steps. The derivative of the current hidden state h_t with respect to a past state h_k ($k < t$) is given by the chain rule:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}.$$

From Equation (9.6), the local derivative is:

$$\frac{\partial h_j}{\partial h_{j-1}} = v \cdot \sigma'\big(v\, h_{j-1} + w\, x_j\big).$$

Taking the norm, we obtain an upper bound:

$$\left\| \frac{\partial h_t}{\partial h_k} \right\| \leq \left(\gamma_v \cdot \gamma_{\sigma'}\right)^{t-k},$$

where:

- $\gamma_v = \|v\|$ (the norm of the recurrent weight).
- $\gamma_{\sigma'} \leq 1$ is the maximum value of the derivative of the activation function σ.

9.9.3 The Product Rule: Vanishing and Exploding Gradients

The behaviour of the gradient over long horizons ($t - k \gg 1$) is dominated by the base $\gamma_v \cdot \gamma_{\sigma'}$:

a. **Vanishing Gradient:** If $\gamma_v \cdot \gamma_{\sigma'} < 1$, the gradient decays exponentially to zero as the time gap increases. The network cannot learn dependencies spanning many steps.

b. **Exploding Gradient:** If $\gamma_v \cdot \gamma_{\sigma'} > 1$, the gradient grows exponentially, causing instability and preventing convergence.

The Role of Activation Functions. Classic RNNs used sigmoid or tanh activation, for which $\sigma'(x) < 1$ almost everywhere. This inherently pushes the product below 1, contributing heavily to the vanishing gradient.

The Recurrent Weight Constraint. Even if we replace the activation with a Rectified Linear Unit (ReLU) where $\sigma'(x) = 1$ for $x > 0$, we still must contend with γ_v. The only stable choice for long-term memory is $\gamma_v \approx 1$. If $\gamma_v < 1$, gradients vanish; if $\gamma_v > 1$, they explode.

9.9.4 The Trivial Solution: A Pure Integrator

If we set $\sigma(x) = x$ (linear activation) and $v = 1$, the state update becomes a simple integrator:

$$h_t = h_{t-1} + w\, x_t.$$

While this solves the gradient problem (the derivative is exactly 1), the model loses expressive power: it can only accumulate a weighted sum of inputs and cannot learn complex, nonlinear dynamics to selectively retain or discard information.

9.10 Long Short-Term Memory (LSTM)

In 1997, Hochreiter and Schmidhuber proposed the **Long Short-Term Memory** (LSTM) architecture, which addresses the vanishing gradient problem while maintaining the ability to learn complex temporal dependencies.

9.10.1 Core Idea: The Constant Error Carousel

The LSTM block is built around a linear memory cell with a self-loop weight fixed at 1. This unit is called the **Constant Error Carousel** (CEC) because the gradient of the cell state c_t with respect to c_{t-1} is exactly 1. This allows error signals to flow backwards across hundreds or thousands of time steps without vanishing or exploding.

9.10.2 Gating Mechanisms

To control what is stored, forgotten, or read from the CEC, the LSTM employs trainable **gates** (nonlinear neurons with sigmoid activation). The original LSTM block contained:

- **Input Gate**: decides whether to write new information into the memory cell.
- **Forget Gate** (introduced slightly later, now standard): decides what information to discard from the cell.
- **Output Gate**: decides whether the current memory content should influence the output (the hidden state).

By keeping the core memory loop linear (gradient = 1) and delegating the *decisions* about writing and reading to separate, gated pathways, the LSTM enables robust learning of long-range dependencies.

9.11 The Basic RNN Block (Revisited)

The simplest recurrent neural network block can be described by the following computation:

$$\mathbf{h}_t = \tanh\big(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h\big),$$

where:

- $\mathbf{x}_t \in \mathbb{R}^I$ is the input at time t (I features).
- $\mathbf{h}_t \in \mathbb{R}^H$ is the hidden state (memory) with H recurrent neurons.

The output $\mathbf{y}_t$ is typically a nonlinear function of the hidden state $\mathbf{h}_t$.

9.12 Long Short-Term Memory (LSTM)

The LSTM block was designed to overcome the vanishing gradient problem by introducing a **Constant Error Carousel** (CEC)—a linear memory cell with a self-loop weight of 1—and three gating mechanisms that control information flow.

9.12.1 Notation and Overview

The memory cell state at time t is denoted $\mathbf{c}_t$. The hidden state (output) is $\mathbf{h}_t$. The input is $\mathbf{x}_t$. All gates are sigmoidal ($\sigma(\cdot)$) neurons whose outputs lie in $(0, 1)$. Candidate values are computed with a tanh nonlinearity.

9.12.2 Step-by-Step Decomposition

1. Candidate Memory Update (What to write?)

$$\tilde{\mathbf{c}}_t = \tanh\big(\mathbf{W}_{xc}\mathbf{x}_t + \mathbf{W}_{hc}\mathbf{h}_{t-1} + \mathbf{b}_c\big).$$

This computes the new information that *could* be added to the memory cell.

2. Input Gate (When to write?)

$$\mathbf{i}_t = \sigma\big(\mathbf{W}_{xi}\mathbf{x}_t + \mathbf{W}_{hi}\mathbf{h}_{t-1} + \mathbf{b}_i\big).$$

The input gate modulates how much of the candidate $\tilde{\mathbf{c}}_t$ is actually added. A value of 1 means "write fully"; 0 means "ignore".

3. Forget Gate (What to erase?)

$$\mathbf{f}_t = \sigma\big(\mathbf{W}_{xf}\mathbf{x}_t + \mathbf{W}_{hf}\mathbf{h}_{t-1} + \mathbf{b}_f\big).$$

The forget gate decides which parts of the previous cell state $\mathbf{c}_{t-1}$ should be retained. A value of 1 means "keep"; 0 means "erase".

4. Cell State Update (The Memory Line)

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,$$

where $\odot$ denotes element-wise multiplication.

- If $\mathbf{f}_t = \mathbf{1}$ and $\mathbf{i}_t = \mathbf{0}$, the memory is preserved perfectly (gradient flows unchanged).
- If $\mathbf{f}_t = \mathbf{0}$ and $\mathbf{i}_t = \mathbf{1}$, the memory is overwritten.
- Any combination in between is possible, learned via backpropagation.

5. Output Gate (What to read?)

$$\mathbf{o}_t = \sigma\big(\mathbf{W}_{xo}\mathbf{x}_t + \mathbf{W}_{ho}\mathbf{h}_{t-1} + \mathbf{b}_o\big).$$

The output gate determines how much of the (nonlinearly transformed) cell state should be exposed as the hidden state:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$

9.12.3 Efficiency and Practical Impact

The LSTM is remarkably parameter-efficient. Inference involves only a few matrix multiplications, element-wise products, and nonlinearities. A model with a few thousand to one million parameters can learn complex long-range dependencies (e.g., the classic 1000-step memory benchmark). This efficiency makes LSTMs well-suited for deployment on resource-constrained devices (e.g., microcontrollers) where Transformers may be impractical.

9.13 Gated Recurrent Unit (GRU)

The GRU is a popular variant of the LSTM that streamlines the gating mechanism to reduce the parameter count.

9.13.1 Key Modifications

- Merges the **forget** and **input** gates into a single **update gate** $\mathbf{z}_t$.
- Introduces a **reset gate** $\mathbf{r}_t$ to control how much past information influences the candidate state.
- Eliminates the separate memory cell $\mathbf{c}_t$; the hidden state $\mathbf{h}_t$ serves as the memory.

9.13.2 GRU Equations

$$\mathbf{z}_t = \sigma\big(\mathbf{W}_{xz}\mathbf{x}_t + \mathbf{W}_{hz}\mathbf{h}_{t-1} + \mathbf{b}_z\big) \quad \text{(Update gate)} \tag{9.8}$$

$$\mathbf{r}_t = \sigma\big(\mathbf{W}_{xr}\mathbf{x}_t + \mathbf{W}_{hr}\mathbf{h}_{t-1} + \mathbf{b}_r\big) \quad \text{(Reset gate)} \tag{9.9}$$

$$\tilde{\mathbf{h}}_t = \tanh\big(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h\big) \quad \text{(Candidate state)} \tag{9.10}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad \text{(State update)} \tag{9.11}$$

Intuition. When the update gate $\mathbf{z}_t \approx 0$, the previous state is copied forward (long-term memory retention). When $\mathbf{z}_t \approx 1$, the state is overwritten by the new candidate. The reset gate $\mathbf{r}_t$ allows the model to discard irrelevant history when computing the candidate.

9.13.3 LSTM vs. GRU in Practice

- GRUs have slightly fewer parameters than LSTMs (due to the merged gates), which can reduce overfitting on small datasets.
- Historically, GRUs were favored in some NLP tasks, but modern empirical evidence shows that neither architecture consistently outperforms the other across all domains.
- In frameworks like PyTorch, switching between `nn.LSTM` and `nn.GRU` is trivial. The choice is often guided by hyperparameter tuning rather than a strict theoretical advantage.

9.14 Architectural Patterns for Recurrent Networks

9.14.1 Stacked (Deep) RNNs

Recurrent layers can be stacked to capture multiple time scales:

$$\mathbf{h}_t^{(1)} = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}^{(1)}), \quad \mathbf{h}_t^{(2)} = \text{LSTM}(\mathbf{h}_t^{(1)}, \mathbf{h}_{t-1}^{(2)}), \quad \dots$$

In this configuration:

- The first layer processes the input sequence at the original sampling rate.
- The second layer processes the *outputs* of the first layer, effectively modelling slower, higher-level dynamics.
- This forms a hierarchical temporal representation.

The final hidden states can be combined (e.g., concatenated) before being passed to a feed-forward output layer or a softmax classifier.

9.14.2 Online Prediction

For tasks requiring step-by-step prediction (e.g., forecasting the next value in a time series), the network is simply unrolled over the sequence:

$$\text{Input: } \mathbf{x}_1 \to \text{Output: } \hat{\mathbf{y}}_1, \quad \mathbf{x}_2 \to \hat{\mathbf{y}}_2, \quad \dots$$

The hidden state is carried forward from one step to the next, enabling the model to make predictions based on the entire observed history up to that point.

9.15 The Hidden State as a Temporal Summary

When a recurrent network processes a sequence $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$:

- At time $t = 1$, the hidden state $\mathbf{h}_1$ summarizes the history $\mathbf{x}_1$.
- At time $t = 2$, $\mathbf{h}_2$ summarizes $\mathbf{x}_1, \mathbf{x}_2$.
- In general, $\mathbf{h}_t$ is a learned feature vector that represents the entire past up to time t.

This is powerful when only the past is available, but in many tasks (e.g., sentence classification) the *future* context is also known and informative.

9.16 Bidirectional Recurrent Networks

To incorporate both past and future context, we can use a **bidirectional** architecture.

- Process the sequence in the forward direction: $\overrightarrow{\mathbf{h}}_t = \mathrm{RNN}(\mathbf{x}_t, \overrightarrow{\mathbf{h}}_{t-1})$.
- Process the sequence in the backward direction: $\overleftarrow{\mathbf{h}}_t = \mathrm{RNN}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1})$.
- Concatenate the two hidden states at each time step: $\mathbf{h}_t^{\mathrm{bi}} = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$.

This yields a representation at time t that is informed by the entire input sequence.

Limitations. Bidirectional models require the *entire* sequence to be available *before* processing. They are not suitable for online (streaming) predictions where the future is unknown. However, for offline tasks like sentiment analysis of a full sentence, they often improve performance.

9.17 Learning the Initial Hidden State

The recurrent update depends on an initial state $\mathbf{h}_0$ (and $\mathbf{c}_0$ for LSTM):

$$\mathbf{h}_1 = f(\mathbf{x}_1, \mathbf{h}_0).$$

Common practice is to set $\mathbf{h}_0 = \mathbf{0}$. However, we can treat $\mathbf{h}_0$ as a **learnable parameter**:

- Initialize $\mathbf{h}_0$ randomly (or to zero).
- During backpropagation through time, gradients flow all the way back to the initial state.
- Update $\mathbf{h}_0$ along with the other parameters.

Learning the initial state can improve performance, especially on short sequences where the initial transient matters. In bidirectional models, there are typically two learned initial states: one for the forward pass and one for the backward pass.

9.18 Choosing the Right Architecture

The choice between a feed-forward network with a time window (TDNN), a vanilla RNN, an LSTM, or a GRU depends on the required memory length:

Model	**Memory Length**	**Trade-offs**
TDNN (sliding window)	Fixed, short (k steps)	Parameter count grows with k. Simple to train.
Vanilla RNN	Short to medium	Parameter-efficient; suffers from vanishing gradients.
LSTM / GRU	Long (hundreds of steps)	More parameters per cell; robust to long dependencies.

Rule of Thumb.

- If the dependency horizon is ≤ 5 steps, a TDNN may suffice.
- For medium horizons, a vanilla RNN can be effective and fast.
- For long-range dependencies (e.g., $>$ 10–20 steps), use an LSTM or GRU.

Ultimately, the choice is a hyperparameter to be validated on the specific dataset.

9.19 Sequence-to-Sequence Modelling Paradigms

Recurrent networks can handle various input–output configurations:

9.19.1 One-to-Many: Fixed Input → Sequence Output

Example: Image captioning. A static input (image features) is fed into an RNN that generates a sequence of words. The hidden state maintains coherence across generated words (e.g., "a group of young people playing a game of frisbee").

9.19.2 Many-to-One: Sequence Input → Fixed Output

Example: Sentiment classification. A sentence (sequence of words) is processed, and the final hidden state (or a pooled representation) is used to predict a single label (positive/negative). This is a natural fit for bidirectional LSTMs because the entire sentence is available before classification. **Note:** The loss is computed only on the final output; gradients must be backpropagated through the entire unrolled sequence.

9.19.3 Many-to-Many (Synced): Sequence → Sequence

Example: Part-of-speech tagging. For each input token, an output label is produced. This can be done with a bidirectional RNN where each time step's concatenated hidden state is classified.

9.19.4 Many-to-Many (Encoder–Decoder): Sequence → Sequence of Different Length

Example: Machine translation.

1. **Encoder:** Reads the entire source sentence and compresses it into a context vector (the final hidden state).
2. **Decoder:** Initialized with the context vector, generates the target sentence one token at a time, using its own previous output as the next input.

This architecture (left side of typical diagrams) is preferred over word-by-word aligned translation because it captures the full meaning before generating output, accommodating languages with different word orders (e.g., Chinese → English). A special `<START>` token initiates decoding, and an `<END>` token signals completion.

9.20 Word Representations: One-Hot vs. Embeddings

9.20.1 Classical Approach: One-Hot Encoding

Given a vocabulary of size V (e.g., $V = 400{,}000$), each word is represented as a binary vector of length V with a single 1 at the index of the word.

$$\mathbf{w}_i = [0, \ldots, 0, 1, 0, \ldots, 0]^\top.$$

This representation is sparse, high-dimensional, and treats all words as equally distant (no notion of semantic similarity).

9.20.2 Modern Approach: Learned Embeddings

Instead of one-hot vectors, words are mapped to dense, low-dimensional vectors $\mathbf{e}_i \in \mathbb{R}^d$ (e.g., $d = 300$). These embeddings are learned jointly with the task, capturing semantic and syntactic relationships (e.g., king − man + woman ≈ queen). We will discuss embedding methods in more detail in the next lecture.

Chapter 10

Sequence to Sequence Learning

10.1 Motivation: Representing Words for Sequence Models

In sequence-to-sequence tasks involving text—such as machine translation, sentence classification, or image captioning—words must be converted into numerical vectors suitable for neural networks.

10.1.1 Limitations of One-Hot Encoding

The most straightforward representation is **one-hot encoding**:

- Define a vocabulary of size V (e.g., $V = 400{,}000$).
- Represent the i-th word as a vector $\mathbf{w}_i \in \{0, 1\}^V$ with a single 1 at position i and 0 elsewhere.

While simple, one-hot vectors suffer from two major drawbacks:

1. **High dimensionality:** V is typically very large, leading to inefficient computation and memory usage.

2. **Semantic ignorance:** All words are equidistant; the vectors encode no information about similarity or relationships (e.g., "cat" and "dog" are as different as "cat" and "philosophy").

These shortcomings motivate the use of **word embeddings**—dense, low-dimensional vectors that capture semantic meaning.

10.2 Unsupervised Learning: A Paradigm Shift

10.2.1 Recap: Supervised vs. Unsupervised Learning

In **supervised learning**, we have a dataset of input–target pairs $\{(\mathbf{x}_i, \mathbf{t}_i)\}_{i=1}^N$. The goal is to learn a function f such that $f(\mathbf{x}) \approx \mathbf{t}$ for new inputs.

In **unsupervised learning**, only the inputs $\{\mathbf{x}_i\}_{i=1}^N$ are available. The objective is to discover and exploit *regularities* in the data to build a useful representation, without requiring an external labeler to provide targets.

- **Clustering** groups similar inputs; the cluster ID becomes a new feature.
- More generally, unsupervised methods aim to transform the data into a form that aids subsequent tasks (prediction, classification, visualization).

10.2.2 The Unsupervised Criterion

A key question: *Does the learning algorithm require a human-provided label for each example?* If not—if the task can be defined solely from the input data—it is unsupervised.

10.3 Autoencoders: Learning to Reconstruct

An **autoencoder** is a neural network trained to output a reconstruction of its own input. Formally, for an input $\mathbf{x} \in \mathbb{R}^D$, the network learns a function $\hat{\mathbf{x}} = f_\theta(\mathbf{x})$ such that $\hat{\mathbf{x}} \approx \mathbf{x}$.

At first glance, this seems trivial (learning the identity function). The key is to constrain the network, typically by forcing the data through a **bottleneck**—a hidden layer with significantly fewer neurons than the input dimension.

10.3.1 Architecture

- **Encoder:** Maps $\mathbf{x} \in \mathbb{R}^D$ to a latent representation $\mathbf{z} \in \mathbb{R}^K$, where $K \ll D$.

$$\mathbf{z} = \phi_e(\mathbf{W}_e\mathbf{x} + \mathbf{b}_e).$$

- **Decoder:** Maps $\mathbf{z}$ back to the original space $\hat{\mathbf{x}} \in \mathbb{R}^D$.

$$\hat{\mathbf{x}} = \phi_d(\mathbf{W}_d\mathbf{z} + \mathbf{b}_d).$$

The network is trained to minimize the **reconstruction loss**, typically the Mean Squared Error (MSE):

$$\mathcal{L}_{\text{recon}} = \frac{1}{N}\sum_{i=1}^{N} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2.$$

10.3.2 Interpretation and Applications

The bottleneck forces the network to learn a *compressed* representation that captures the essential structure of the data.

1. Data Compression. The encoder acts as a nonlinear compressor, reducing a 124-dimensional vector to, say, 64 dimensions. The decoder decompresses it. This has been explored as an alternative to classical compression algorithms.

2. Denoising. If we train an autoencoder on clean data and then feed it noisy inputs $\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\epsilon}$, the latent space tends to ignore the noise and reconstruct the clean signal $\hat{\mathbf{x}} \approx \mathbf{x}$. A more direct approach is the **Denoising Autoencoder**, trained explicitly to map a corrupted input $\tilde{\mathbf{x}}$ to the original clean version $\mathbf{x}$.

3. Anomaly Detection. After training on normal data, the autoencoder will reconstruct normal patterns accurately. An anomalous input will produce a large reconstruction error $\|\mathbf{x}-\hat{\mathbf{x}}\|^2$, which can be thresholded to flag outliers.

4. Nonlinear Dimensionality Reduction. The latent code $\mathbf{z}$ provides a low-dimensional representation of the input, analogous to a nonlinear version of Principal Component Analysis (PCA). This can be used for visualization or as pre-trained features for supervised tasks (by discarding the decoder and using the encoder as a feature extractor).

5. Clustering via Sparsity Constraints. By adding a **sparsity penalty** to the loss function, we encourage the latent neurons to be mostly inactive (output near zero). The modified loss is:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda \sum_{j=1}^{K} |z_j| \quad \text{(or a similar sparsity term)}.$$

If the sparsity is extreme (e.g., only one latent neuron active per input), similar inputs collapse to the same active neuron. The decoder then reconstructs them from that single point, effectively performing a nonlinear form of K-means clustering.

10.4 Key Takeaway: The Loss Function Defines the Task

A recurring principle: to understand what a network is learning, examine its **output** and its **loss function**. The architecture provides capacity, but the loss function specifies the objective.

- **MSE reconstruction** → learn to copy input through a bottleneck (compression/denoising).
- **MSE + sparsity penalty** → learn sparse representations useful for clustering or feature selection.

10.5 Introduction: The Need for Better Word Representations

In sequence modelling tasks involving text—such as translation, classification, or captioning—words must be mapped to numerical vectors. The naive **one-hot encoding** uses a vocabulary of size V and assigns a binary vector $\mathbf{w} \in \{0,1\}^V$ with a single 1. For realistic vocabularies ($V \approx 400{,}000$ to 10^6), this yields extremely sparse, high-dimensional vectors. Such sparsity leads to:

- Inefficient use of vector space.
- The **curse of dimensionality**: learning distributions in high-dimensional spaces requires exponentially many samples.
- Meaningless distances: all words are equidistant in Hamming or Euclidean metrics, obscuring semantic similarity.

Consider two semantically equivalent sentences:

> *"Obama speaks to the media in Illinois."*
> *"The president addresses the press in Chicago."*

In one-hot representation, the words `Obama` and `president` are orthogonal vectors, despite their contextual synonymy. We seek a **dense, low-dimensional embedding** where semantically similar words map to nearby points.

10.6 Autoencoders: Unsupervised Compression and Reconstruction

Before addressing word embeddings, we introduce the **autoencoder**—a neural network that learns to reconstruct its own input. It serves as a foundational example of unsupervised representation learning.

10.6.1 Architecture and Training

An autoencoder consists of two parts:

- **Encoder** $e_\phi : \mathbb{R}^D \to \mathbb{R}^K$ (with $K \ll D$), mapping input $\mathbf{x}$ to latent code $\mathbf{z} = e_\phi(\mathbf{x})$.
- **Decoder** $d_\psi : \mathbb{R}^K \to \mathbb{R}^D$, mapping $\mathbf{z}$ back to reconstruction $\hat{\mathbf{x}} = d_\psi(\mathbf{z})$.

Training minimizes the reconstruction loss (e.g., mean squared error):

$$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2.$$

No external labels are required; the target is the input itself, making this an **unsupervised** method.

10.6.2 The Bottleneck Principle

Forcing the data through a narrow hidden layer ($K \ll D$) prevents the network from learning the trivial identity function. Instead, it must learn to compress the essential information of the input distribution into the low-dimensional code $\mathbf{z}$.

10.6.3 Applications

- **Data Compression**: Nonlinear alternative to classical codecs.
- **Denoising**: Train on clean data; at inference, noisy inputs are projected onto the clean data manifold.
- **Anomaly Detection**: Large reconstruction error $\|\mathbf{x} - \hat{\mathbf{x}}\|$ signals outliers.
- **Nonlinear Dimensionality Reduction**: The encoder provides a low-dimensional embedding (nonlinear PCA).
- **Feature Extraction**: The encoder can be used as a pre-trained feature extractor for supervised tasks.
- **Clustering (Sparse Autoencoders)**: Adding a sparsity penalty $\lambda \sum_j |z_j|$ to the loss encourages most latent neurons to be near zero; extreme sparsity yields a nonlinear K-means.

Note on Variational Autoencoders (VAEs). By replacing the deterministic encoder with a probabilistic one (outputting mean and variance of a Gaussian), we obtain a generative model capable of sampling new data. This is outside the scope of the present discussion.

10.7 Distributed Representations for Words

The goal is to map words to vectors $\mathbf{e} \in \mathbb{R}^d$ (with $d \approx 100$–500) that are:

- **Dense**: Most dimensions carry non-zero values, efficiently utilizing the space.
- **Low-dimensional**: Mitigating the curse of dimensionality.
- **Semantically Smooth**: Small changes in the vector correspond to small changes in meaning; similar words cluster together.

10.7.1 Desired Properties

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$
$$\text{walking} - \text{walk} + \text{swim} \approx \text{swimming}$$
$$\text{Paris} - \text{France} + \text{Italy} \approx \text{Rome}$$

Such analogies emerge naturally when words are embedded in a continuous vector space.

10.8 Language Modeling as a Precursor to Word Embeddings

The breakthrough in learning such embeddings arose from attempts to solve a different problem: **language modeling**.

10.8.1 Definition

A language model assigns a probability to any sequence of words $w_1, w_2, \ldots, w_T$. Using the chain rule of probability:

$$\begin{aligned} P(w_1, w_2, \ldots, w_T) &= P(w_T \mid w_1, \ldots, w_{T-1}) \cdot P(w_{T-1} \mid w_1, \ldots, w_{T-2}) \cdots P(w_2 \mid w_1) \cdot P(w_1) \\ &= \prod_{t=1}^{T} P(w_t \mid w_{<t}). \end{aligned}$$

This decomposes the joint probability into a product of conditional probabilities, each predicting the next word given the preceding context.

10.8.2 Classical Approaches

- **n-gram Models**: Approximate $P(w_t \mid w_{<t}) \approx P(w_t \mid w_{t-n+1}, \ldots, w_{t-1})$ using frequency counts.
- **Bag-of-Words / TF-IDF**: Sparse, count-based representations for documents, ignoring word order.

These methods suffer from sparsity and limited ability to capture semantic similarity.

10.8.3 Neural Language Models and Word Embeddings

The key innovation was to represent each word as a dense vector $\mathbf{v}_w \in \mathbb{R}^d$ and to learn these vectors jointly with the task of predicting the next word. This **distributed representation** approach—training a neural network to model $P(w_t \mid \text{context})$—yields word embeddings as a byproduct. The resulting vectors capture syntactic and semantic regularities.

10.9 The Combinatorial Explosion of Language Modeling

A language model assigns a probability $P(w_1, w_2, \ldots, w_T)$ to any sequence of words. Using the chain rule, we decompose this as a product of conditional probabilities:

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1}).$$

If the vocabulary size is V and the sentence length is T, the number of possible sequences is V^T. For a realistic vocabulary of $V = 400{,}000$ and sentences of length $T = 20$, we face $400{,}000^{20}$ possible combinations—an astronomically large space that cannot be represented, let alone estimated from data.

To make language modeling tractable, we introduce the **n-gram approximation**:

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \ldots, w_{t-1}).$$

Even with this simplification, the number of parameters grows as V^n. For a vocabulary of 10,000 and $n = 10$, we have $10{,}000^{10}$ parameters—far exceeding modern large models. In practice, classical n-gram models (Katz, 1987) are limited to $n \leq 3$ due to data sparsity and computational constraints.

10.10 The Neural Network Language Model (Bengio et al., 2001/2003)

To overcome the limitations of discrete n-gram models, Yoshua Bengio and collaborators proposed using a **neural network** to learn a continuous representation of words. The key insight: instead of storing probabilities in a sparse hypercube, distribute them across the continuous parameters of a neural network.

10.10.1 Architecture

The model takes as input the $(n-1)$ previous words, each represented as a **one-hot vector** of size V. These are processed as follows (Figure **??**):

1. **Projection Layer**: Each one-hot vector $\mathbf{w}_{t-i}$ is multiplied by a shared projection matrix $\mathbf{C} \in \mathbb{R}^{V \times m}$, yielding a dense embedding $\mathbf{e}_{t-i} \in \mathbb{R}^m$.

$$\mathbf{e}_{t-i} = \mathbf{C}^\top \mathbf{w}_{t-i} \quad \text{(i.e., selects the corresponding row of } \mathbf{C}\text{)}.$$

2. **Concatenation**: The embeddings of the $(n-1)$ context words are concatenated into a single vector of size $(n-1)m$.

3. **Hidden Layer**: A feed-forward hidden layer with a nonlinear activation (e.g., tanh) and typically 500–1000 units.

4. **Output Layer**: A softmax layer of size V that produces a probability distribution over the next word.

10.10.2 Training

The model is trained as an **unsupervised classification** task: given the context words, predict the one-hot encoding of the next word. The loss is the cross-entropy between the predicted distribution and the true next word. No manual labels are required—any text corpus can be used.

10.10.3 Complexity Analysis

The computational bottleneck lies in the output layer, which involves a matrix multiplication of size $V \times H$ (where H is the hidden size). The projection layer cost is $O(Vm)$, and the hidden layer is $O((n-1)m \times H)$. Since V can be 60,000 to 150,000 or more, the output softmax dominates training time.

10.10.4 Results and Impact

Bengio's Neural Network Language Model (NNLM) achieved significant improvements over state-of-the-art n-gram models:

- 24% improvement on one benchmark (Brown corpus), 8% on another.
- Used context sizes of 5 and 6 grams, with embedding dimensions $m = 30$ or 100, and hidden layers of 100 or 60 units.
- Training on 40 CPU cores took weeks—GPUs were not yet prevalent.
- Vocabulary sizes ranged from 60,000 to 150,000 words, trained on corpora of up to 40 million words.

Despite its success, the model was too computationally expensive to scale to very long contexts or massive vocabularies.

10.11 From Language Modeling to Word Embeddings: Word2Vec

The NNLM inadvertently produced a valuable byproduct: the projection matrix $\mathbf{C}$, which maps sparse one-hot vectors to dense, low-dimensional embeddings. These embeddings captured semantic similarities (e.g., similar words had nearby vectors).

In 2013, Tomas Mikolov and colleagues at Google proposed to **discard the language modeling objective** and focus solely on learning high-quality word embeddings. This led to **Word2Vec**, a family of computationally efficient models.

10.11.1 Simplification: Removing the Hidden Layer

The core idea is to train only the projection matrix $\mathbf{C}$ and its (approximate) inverse, using a simple log-linear classifier. Two main architectures were introduced:

Continuous Bag-of-Words (CBOW): Predict the current word from the average of its surrounding context words.

Skip-gram: Predict the surrounding context words given the current word.

Both models are shallow (no hidden layer) and can be trained on billions of words efficiently using **negative sampling** or **hierarchical softmax** to avoid the expensive V-way softmax.

10.11.2 Why Word2Vec Was a Breakthrough

- **Efficiency**: Training on the entire Google News corpus (100 billion words) became feasible on a single machine.
- **Quality**: The learned embeddings exhibit remarkable linear algebraic properties, e.g., king $-$ man $+$ woman $\approx$ queen.
- **Reusability**: Pre-trained embeddings could be plugged into downstream NLP tasks as fixed features or fine-tuned.

10.12 Introduction: Predicting Words from Context

Consider the task of identifying a missing word based on its surrounding context. For instance, given the sentence:

> *"Pelé has called ____ an excellent player. At the age of just 22 years, ____ had scored 40 goals in 58 internationals. Occasionally playing as an attacking midfielder, ____ was called a true phenomenon."*

A human reader easily infers that the missing word is **Neymar** (or a similar football prodigy). This ability stems from a fundamental linguistic principle articulated by J.R. Firth in the 1950s: *"You shall know a word by the company it keeps."* The meaning of a word is largely determined by the words that typically surround it.

This insight is the cornerstone of **Word2Vec**, a family of models developed by Tomas Mikolov and colleagues at Google (2013). Instead of building a full language model, Word2Vec focuses solely on learning high-quality **word embeddings**—dense vector representations that capture semantic and syntactic regularities.

10.13 The Two Architectures of Word2Vec

Word2Vec offers two complementary approaches to learn embeddings by predicting words from their context or vice versa.

10.13.1 Continuous Bag-of-Words (CBOW)

The CBOW model predicts a target word w_t given the average of its surrounding context words within a symmetric window of size C:

$$\text{Input: } w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C} \quad \longrightarrow \quad \text{Output: } w_t.$$

Each context word is mapped to its embedding vector $\mathbf{v}_{w_i} \in \mathbb{R}^d$ via a projection matrix $\mathbf{W} \in \mathbb{R}^{V \times d}$ (where V is vocabulary size). These embeddings are averaged element-wise:

$$\mathbf{h} = \frac{1}{2C} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \mathbf{v}_{w_{t+j}}.$$

The averaged vector $\mathbf{h}$ is then used to predict the target word via a softmax over the vocabulary.

10.13.2 Skip-gram

The Skip-gram model inverts the prediction task: given a target word w_t, it predicts each of the surrounding context words independently.

$$\text{Input: } w_t \quad \longrightarrow \quad \text{Output: } w_{t-C}, \ldots, w_{t+C} \ (j \neq 0).$$

The target word's embedding $\mathbf{v}_{w_t}$ is used to compute a softmax distribution over the vocabulary for each context position.

10.14 Model Architecture and Computational Efficiency

Both CBOW and Skip-gram are **shallow** networks: they consist only of an embedding layer and an output projection. Crucially, they avoid the hidden layer present in Bengio's Neural Network Language Model (NNLM).

10.14.1 Comparison with NNLM

Recall the complexity of the NNLM:

- **Projection layer:** $O(V \cdot d)$
- **Hidden layer:** $O(C \cdot d \cdot H)$
- **Output softmax:** $O(H \cdot V)$

The dominant term is $H \cdot V$, which becomes prohibitively expensive for large vocabularies.

Word2Vec eliminates the hidden layer entirely ($H = 0$) and employs two critical optimizations to reduce the output cost:

1. **Hierarchical Softmax**: Uses a binary tree (often a Huffman tree) over the vocabulary, reducing softmax computation from $O(V)$ to $O(\log V)$.
2. **Negative Sampling**: Approximates the softmax by contrasting the true target word with a small number (k) of randomly sampled "negative" words, reducing complexity to $O(k \cdot d)$.

With these optimizations, the training complexity becomes approximately:

$$O(C \cdot d + \log V) \quad \text{or} \quad O(k \cdot d).$$

This is orders of magnitude faster than NNLM.

10.14.2 Scalability Gains

The efficiency leap enabled training on massive corpora:

- **NNLM (Bengio)**: 14 million words, vocabulary 150k, embedding size 100, training time $\approx$ weeks on 40 CPU cores.
- **Word2Vec (CBOW/Skip-gram)**: 6 billion words, vocabulary 1 million, embedding size 1000, training time $\approx$ 2–2.5 days on a single multi-core machine.

By removing the hidden layer and using negative sampling, Word2Vec made it feasible to learn embeddings from web-scale text.

10.15 Semantic Regularities in the Embedding Space

A remarkable empirical discovery was that the learned vector space exhibits **linear algebraic regularities** that mirror semantic and syntactic relationships.

10.15.1 Analogy by Vector Arithmetic

Given embeddings trained on a large corpus, one can perform analogies by simple vector operations:

$$\begin{aligned}
\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} &\approx \mathbf{v}_{\text{queen}} \\
\mathbf{v}_{\text{Paris}} - \mathbf{v}_{\text{France}} + \mathbf{v}_{\text{Italy}} &\approx \mathbf{v}_{\text{Rome}} \\
\mathbf{v}_{\text{Windows}} - \mathbf{v}_{\text{Microsoft}} + \mathbf{v}_{\text{Google}} &\approx \mathbf{v}_{\text{Android}} \\
\mathbf{v}_{\text{Einstein}} - \mathbf{v}_{\text{scientist}} + \mathbf{v}_{\text{painter}} &\approx \mathbf{v}_{\text{Picasso}}
\end{aligned}$$

These relationships were never explicitly programmed; they emerged naturally from the training objective.

10.15.2 Why Does This Happen?

The embeddings capture **distributional semantics**: a word's vector is essentially a compressed representation of the words that typically co-occur with it.

- The vector for `king` is the average of context words like *throne*, *crown*, *royal*, *majesty*, and *he*.
- Subtracting the `man` component removes context words associated with masculinity (*he*, *his*).
- Adding `woman` injects context words associated with femininity (*she*, *her*).
- The resulting vector approximates the context of `queen`.

Thus, the vector space encodes not the word itself, but the **company it keeps**.

10.16 Practical Applications and Legacy

Word embeddings became a foundational building block for natural language processing.

10.16.1 From One-Hot to Dense Vectors

Modern NLP pipelines almost universally replace sparse one-hot representations with pre-trained embeddings. Given an input text, each word token is mapped to its corresponding vector $\mathbf{v}_w$ from a pre-trained embedding matrix (e.g., Word2Vec, GloVe, or FastText). These vectors are then fed into downstream models such as LSTMs, GRUs, or Transformers.

10.16.2 Simple Semantic Queries

Even without a complex neural architecture, word embeddings enable interesting semantic operations:

- **Information Retrieval**: Find restaurants matching a query vector $\mathbf{v}_{\text{restaurant}} + \mathbf{v}_{\text{Mountain View}} + \mathbf{v}_{\text{not}} + \mathbf{v}_{\text{good}}$.
- **Analogy Completion**: "Czech + currency" yields vectors near `koruna`, `euro`, etc.
- **Capital Identification**: "Vietnam + capital" produces vectors close to `Hanoi`.

These capabilities work well for short phrases (2–4 words) but degrade for longer, complex prompts.

10.16.3 Evolution Beyond Word2Vec

While Word2Vec revolutionized word representation, subsequent advances include:

- **FastText**: Represents words as bags of character n-grams, handling out-of-vocabulary words and morphology.
- **Contextual Embeddings (ELMo, BERT, GPT)**: Generate word vectors that depend on the entire sentence context, capturing polysemy (e.g., "bank" as river vs. financial institution).

Nevertheless, Word2Vec remains a fundamental concept and a practical starting point for many text-based tasks.

10.17 Document Similarity and Semantic Search with Word Embeddings

Word embeddings provide a powerful way to compare documents based on their semantic content. Given two documents, we can:

1. Map each word to its dense embedding vector $\mathbf{v}_w \in \mathbb{R}^d$.
2. Aggregate the word vectors (e.g., by averaging or summing) to obtain a document-level representation:
$$\mathbf{d} = \frac{1}{|\text{doc}|} \sum_{w \in \text{doc}} \mathbf{v}_w.$$
3. Compare documents using cosine similarity or Euclidean distance in the embedding space.

Even with a simple one-dimensional projection for illustration, documents with similar meaning tend to cluster together.

10.17.1 Finding Synonyms via Nearest Neighbors

The embedding space naturally clusters semantically related words. Given a query word $\mathbf{v}_q$, we can retrieve its k-nearest neighbors:
$$\text{NN}_k(\mathbf{v}_q) = \{w \in V \mid \|\mathbf{v}_w - \mathbf{v}_q\| \text{ is small}\}.$$
For example, a query for "sad" may return neighbors such as *saddening, saddened, heartbreaking, disheartening, tragic*. This property enables synonym detection and word sense disambiguation.

10.17.2 k-NN Classification for Text

For text classification tasks, we can represent each class by a prototypical vector (e.g., the average embedding of all training documents in that class). A new document is then assigned to the class whose prototype is closest in the embedding space. This approach works surprisingly well for short texts and provides a simple, interpretable baseline.

10.18 Why One-Hot Encoding Prevailed for So Long

A natural question arises: given the limitations of one-hot vectors, why did they remain the dominant word representation for decades?

10.18.1 Historical Context

- **Simplicity**: One-hot encoding directly corresponds to the notion of categorical classes, which is natural for softmax-based classification.
- **Alternative Encodings Attempted**: Researchers in the 1990s experimented with binary codes (e.g., using $\lceil \log_2 V \rceil$ bits) and even error-correcting codes to reduce dimensionality. These discrete representations, however, did not outperform one-hot and were harder to train due to output correlations.
- **Focus Shift**: Between the early 2000s and 2010, the NLP community prioritized document-level tasks like text categorization using bag-of-words and TF-IDF representations, rather than word-level language modeling.
- **Neural Network Winter**: After an initial surge, neural networks fell out of favor in the mid-2000s. The dominant machine learning tools were Support Vector Machines and probabilistic graphical models (e.g., Latent Dirichlet Allocation).

The revival came in 2012–2013, catalyzed by the success of deep convolutional networks on ImageNet and the release of Word2Vec. The key enabler was the ability to learn *continuous*, *distributed* representations efficiently from massive text corpora.

10.19 GloVe: Global Vectors for Word Representation

Word2Vec's skip-gram and CBOW models rely on local context windows. An alternative approach, **GloVe** (Pennington et al., 2014), explicitly incorporates global corpus statistics.

10.19.1 Core Idea

GloVe constructs a word-word co-occurrence matrix $\mathbf{X}$, where X_{ij} denotes how often word j appears in the context of word i. The model then learns word vectors $\mathbf{w}_i$ and context vectors $\tilde{\mathbf{w}}_j$ such that their dot product approximates the log co-occurrence:

$$\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j \approx \log X_{ij}.$$

The loss function is a weighted least squares regression that emphasizes frequent co-occurrences while avoiding domination by rare pairs.

10.19.2 Properties of GloVe Embeddings

Like Word2Vec, GloVe embeddings exhibit linear substructures that capture semantic relationships:

- Comparative/superlative forms: *strong, stronger, strongest* form a consistent vector displacement.
- Nearest neighbor queries: For "frog", the closest vectors include *frogs, toad, littoria, leptodactylid, rana* (Spanish for frog), and interestingly *lizard*—which, while taxonomically incorrect, reflects contextual similarity (small, green, four-legged creatures).

These regularities emerge because the embedding space encodes *distributional similarity*—the patterns of word co-occurrence in large corpora.

10.20 Looking Ahead: A Unified View of Neural Network Layers

As we transition from text to image processing, it is helpful to recognize that all neural architectures we have studied are built from a common set of **layer types**:

Layer Type	**Function**
Dense (Fully Connected)	Linear transformation + nonlinearity
Batch Normalization	Stabilize training, accelerate convergence
Dropout	Regularization by random deactivation
Softmax	Probability distribution over classes
Recurrent (RNN, LSTM, GRU)	Sequence modeling with memory
Bidirectional RNN	Context from both past and future
Convolutional	Local feature extraction (to be introduced)

These layers can be combined in various topologies:

- **Sequential stacking**: layer $L_1 \to L_2 \to \cdots \to L_K$.
- **Shortcut connections**: skip connections that add the input of a layer to its output (e.g., ResNet).
- **U-Net architectures**: encoder-decoder with lateral connections for image segmentation.

The upcoming lectures by Professor Boracchi will introduce **Convolutional Neural Networks (CNNs)**, extending our "dictionary" of layers to handle spatial data efficiently. Just as recurrent layers capture temporal dependencies, convolutional layers capture spatial hierarchies—making them the foundation of modern computer vision.

Chapter 11

Time Series Classification

11.1 Introduction: Why Time Matters

"Everything flows." Time is a fundamental dimension that orders events into *before* and *after*. While static data (e.g., a single image) captures an instantaneous snapshot, many real-world phenomena are inherently sequential. A single measurement—such as an electrocardiogram sample or an accelerometer reading—conveys little meaning in isolation. It is the *evolution* of these values over time that encodes the underlying pattern.

In this exercise session, we shift from standard tabular classification to **time series classification**. The core task remains classification, but the data source introduces new challenges: we must handle temporal dependencies, variable-length sequences, and the risk of data leakage across time.

11.2 Dataset: Human Activity Recognition from Wearable Sensors

We will work with a dataset collected from a wearable IoT device equipped with a tri-axial accelerometer. For each time stamp t, the device records:

- `X`, `Y`, `Z`: acceleration along the three spatial axes.
- `timestamp`: the time at which the measurement was taken.
- `user_id`: an identifier for the subject wearing the device.
- `activity`: the label of the activity being performed (constant over a recording session).

The dataset comprises multiple subjects performing various activities such as `walking`, `jogging`, `sitting`, `standing`, and `upstairs`. The goal is to classify the activity based on a window of accelerometer readings.

11.3 Initial Data Inspection and Preprocessing

Before building any model, we must thoroughly inspect the data. This step is often more critical than the choice of algorithm—a poorly set up problem will defeat even the most sophisticated architecture.

11.3.1 Checking Data Types and Missing Values

- The dataset contains no missing values.
- Features are stored as `float64`; we downcast to `float32` to reduce memory usage and ensure compatibility with certain models.

11.3.2 Value Ranges and Anomalies

Descriptive statistics reveal that the three acceleration axes (`X`, `Y`, `Z`) have similar value ranges. However, one must always verify the absence of extreme outliers (e.g., sensor malfunctions producing values like ± 1000).

Such outliers, if not handled, would distort any subsequent normalization (e.g., min-max scaling or z-score standardization), compressing the meaningful signal into a tiny interval.

11.3.3 Class Distribution

The activity labels are **imbalanced**. Some activities (e.g., `walking`) appear far more frequently than others. Consequently, **accuracy is not a suitable evaluation metric**. We should consider metrics robust to class imbalance, such as:

- Balanced accuracy
- Macro F1-score
- Confusion matrix analysis

11.3.4 User ID and Activity Relationships

A crucial observation: each `user_id` may have performed multiple activities, and each activity may have been recorded for multiple users. This structure has profound implications for data splitting.

11.4 Avoiding Data Leakage: The User-Split Strategy

Consider a naive random split of the raw samples into training, validation, and test sets. Since a single user's recording session spans many consecutive time steps, such a split would scatter segments of the *same user's activity* across all three sets. The model could then learn to recognize the *user* rather than the *activity*—a classic case of **data leakage**.

To prevent this, we adopt a **user-based split**:

1. Identify all unique `user_id` values (36 in this dataset).
2. Randomly partition the *users* into training, validation, and test groups (e.g., 26 for training, 5 for validation, 5 for test).
3. Assign all samples belonging to a user to the corresponding split.

This ensures that the model is evaluated on *unseen subjects*, providing a realistic measure of generalization.

11.4.1 Creating Unique Activity Identifiers

To facilitate this split, we construct a new column `unique_id` by concatenating `user_id` and `activity`:

$$\text{unique_id} = \text{user_id} + \text{“_”} + \text{activity}.$$

This column uniquely identifies each recording session and will be used to verify that no session is split across sets.

11.5 Visualizing Temporal Patterns

Plotting the raw accelerometer signals for different activities reveals distinct temporal signatures:

- **Standing**: Low-amplitude, noisy fluctuations around a constant offset (gravity).
- **Walking / Upstairs**: Periodic patterns with moderate amplitude.
- **Jogging**: High-frequency, high-amplitude oscillations.

While differentiating `jogging` from `standing` is trivial, distinguishing `walking` from `upstairs` or `jogging` from `running` requires the model to capture subtle differences in periodicity and magnitude.

11.6 From Raw Streams to Neural Network Inputs

A neural network expects fixed-size input tensors. Our raw data, however, consists of variable-length sequences (each `unique_id` has a different duration). We must transform the continuous stream into **fixed-length windows** that the network can digest.

11.6.1 The Sliding Window Approach

For each recording session, we extract overlapping (or non-overlapping) windows of length W:

$$\mathbf{X}_i = [\mathbf{x}_i, \mathbf{x}_{i+1}, \ldots, \mathbf{x}_{i+W-1}], \quad \mathbf{x}_t = [X_t, Y_t, Z_t]^\top.$$

Each window $\mathbf{X}_i \in \mathbb{R}^{W \times 3}$ is assigned the activity label of the session.

11.6.2 Design Choices

- **Window length** W: Must be long enough to capture at least one full cycle of the periodic activity (e.g., one stride) but short enough to avoid mixing activities.
- **Stride**: Overlap between consecutive windows can augment the dataset but increases computational cost.

After windowing, we obtain a standard supervised learning dataset: input tensors of shape $(N_{\text{windows}}, W, 3)$ and corresponding labels of shape $(N_{\text{windows}},)$.

11.7 From Static to Temporal Data

In standard tabular classification, the dataset is described by two dimensions: the number of samples N and the number of features D. A feed-forward neural network expects input tensors of shape $(\text{batch_size}, D)$.

Time series classification introduces a third dimension: **time**. Each sample is now a sequence of length T, with D features observed at each time step. The input tensor becomes:

$$\mathbf{X} \in \mathbb{R}^{\text{batch_size} \times T \times D}.$$

This additional dimension captures the temporal evolution of the signal, which is essential for tasks like human activity recognition from accelerometer data.

11.8 Dataset Recap and Splitting Strategy

We work with a dataset of tri-axial accelerometer readings (X, Y, Z) collected from 36 subjects performing various activities (`walking`, `jogging`, `standing`, etc.).

11.8.1 User-Based Splitting

To avoid data leakage, we split the dataset by **user_id** rather than by individual samples. This ensures that no subject appears in more than one split, providing a realistic measure of generalization to unseen individuals.

After splitting (e.g., 26 users for training, 5 for validation, 5 for test), we verify that the class distribution remains reasonably balanced across splits. We then encode the string labels as integers using a mapping dictionary.

11.8.2 Normalization

We apply min-max normalization (or alternatively z-score standardization) to the features, fitting the scaler *only on the training set* and transforming all splits accordingly. Care must be taken to handle potential outliers that could skew the scaling.

11.9 Windowing: Creating Fixed-Length Sequences

Neural networks require fixed-size inputs. Our raw recordings have variable lengths (each `unique_id` corresponds to a session of arbitrary duration). We therefore partition each session into **windows** of fixed length W.

11.9.1 Window Size (W)

The window size determines how much temporal context the model receives. It must be:

- Large enough to capture at least one full cycle of the periodic activity (e.g., one stride).
- Small enough to avoid mixing multiple activities (if activities change rapidly) and to keep computational cost manageable.

There is no universal "correct" value; W is a hyperparameter to be tuned. In our example, we choose $W = 200$ time steps.

11.9.2 Stride (S)

The stride controls the overlap between consecutive windows extracted from the same session. Given a session of length L, the number of windows generated is approximately:

$$N_{\text{windows}} = \left\lfloor \frac{L - W}{S} \right\rfloor + 1.$$

- **Small stride** (e.g., $S = 1$): High overlap, yields many nearly identical windows. Increases dataset size but introduces redundancy.
- **Large stride** (e.g., $S = W$): No overlap, each time step belongs to exactly one window. Maximizes diversity but may produce too few samples.

The choice of S involves a trade-off between data quantity and sample independence. In practice, moderate overlap (e.g., $S = 50$ with $W = 200$, giving 75% overlap) is often used.

11.9.3 Handling the Tail: Padding vs. Truncation

When $L - W$ is not an exact multiple of S, the final segment of the session has length $L' < W$. We have two options:

1. **Truncation**: Discard the incomplete window.
2. **Padding**: Append a constant value (typically zero) to the sequence to reach length W.

Padding introduces artificial data points, but if used sparingly (most windows are unpadded), it acts as a mild regularizer and ensures no data is wasted. The choice is often a matter of convenience and dataset size.

11.10 Constructing the Tensor Dataset

The windowing procedure transforms the raw DataFrame into a tensor $\mathbf{X} \in \mathbb{R}^{N_{\text{windows}} \times W \times 3}$ and corresponding labels $\mathbf{y} \in \{0, \ldots, C-1\}^{N_{\text{windows}}}$.

For each `unique_id`:

1. Extract the feature matrix $\mathbf{S} \in \mathbb{R}^{L \times 3}$.
2. For $i = 0, S, 2S, \ldots$ while $i + W \leq L$:
   - Take window $\mathbf{X}_i = \mathbf{S}[i : i + W, :]$.
3. If padding is enabled and a remainder exists, pad the final segment to length W and append it.

The label for each window is the activity label of the session (taken from the first row, as the label is constant per session).

11.11 Implications for Model Input Shape

When training a recurrent neural network (RNN, LSTM, GRU) on this data, the expected input shape is:

$$(\text{batch_size}, W, 3).$$

The model will process the sequence step by step, learning temporal dependencies across the W time steps. The batch size is an independent hyperparameter governing gradient aggregation; it does not alter the per-sample shape.

11.12 From Windowing to Tensor Datasets

After defining the window size W and stride S, we construct the dataset by iterating over each `unique_id`. For each recording session:

1. Extract the feature matrix of accelerations $\mathbf{S} \in \mathbb{R}^{L \times 3}$.
2. If padding is enabled, ensure the length L is a multiple of the desired windowing parameters (e.g., by appending zeros).
3. Slide a window of length W with step S, appending each window $\mathbf{X}_i \in \mathbb{R}^{W \times 3}$ to the dataset.

This transforms the raw DataFrame into a tensor $\mathbf{X} \in \mathbb{R}^{N_{\text{windows}} \times W \times 3}$ and corresponding labels $\mathbf{y} \in \mathbb{R}^{N_{\text{windows}}}$.

11.12.1 Impact of Stride on Dataset Size

The stride S directly controls the number of windows extracted:

$$N_{\text{windows}} \approx \frac{L - W}{S} + 1.$$

A smaller stride increases the number of samples (useful when data is scarce) but introduces redundancy. We choose $S = 50$ with $W = 200$, yielding a substantial dataset while maintaining moderate overlap.

11.12.2 Final Tensor Shapes

For training, validation, and test sets, the data is wrapped into a TensorDataset. Each batch drawn from the DataLoader has shape:

$$\mathbf{X}_{\text{batch}} \in \mathbb{R}^{\text{batch_size} \times W \times 3}, \quad \mathbf{y}_{\text{batch}} \in \mathbb{R}^{\text{batch_size}}.$$

A batch size of 64 (a power of two) is typical, though larger sizes may not yield benefits and can slow convergence.

11.13 Recurrent Classifier Module

We define a flexible `RecurrentClassifier` class that can instantiate any of the three primary recurrent architectures:

- **Vanilla RNN**: Simple recurrent cells, prone to vanishing gradients.
- **LSTM**: Long Short-Term Memory, with explicit gating mechanisms to control information flow.
- **GRU**: Gated Recurrent Unit, a more computationally efficient variant of LSTM.

11.13.1 Architecture Overview

The model consists of two main components:

1. **Recurrent Feature Extractor**: One or more stacked recurrent layers that process the input sequence $\mathbf{X} \in \mathbb{R}^{W \times 3}$ and produce a sequence of hidden states.

2. **Classifier Head**: A fully connected layer that maps the final hidden state to class logits.

Mathematically, given an input sequence $\mathbf{x}_1, \ldots, \mathbf{x}_W$, the recurrent layer computes:

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}; \boldsymbol{\theta}),$$

where f depends on the cell type. After processing the entire sequence, we take the last hidden state $\mathbf{h}_W$ (or the concatenation of forward and backward states if bidirectional) and pass it through a linear layer:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_c \mathbf{h}_W + \mathbf{b}_c).$$

11.13.2 Stacking Recurrent Layers

Multiple recurrent layers can be stacked by passing the sequence of hidden states from one layer as input to the next. This allows the model to learn hierarchical temporal representations.

11.13.3 Bidirectional Processing

Setting `bidirectional=True` instantiates two independent recurrent networks: one processes the sequence forward ($1 \to W$), the other backward ($W \to 1$). Their final hidden states are concatenated, providing context from both past and future within the window.

11.13.4 Dropout in Recurrent Networks

When dropout is specified within a recurrent layer, it is applied *across time steps* rather than across features. This temporal dropout is more effective at preventing overfitting in sequence models. Additional dropout layers can be placed after the recurrent stack to regularize the feature dimension.

11.14 Training Dynamics and the Vanishing Gradient Problem

With the model defined, training proceeds using the same loop as for feed-forward networks, optimizing the cross-entropy loss. However, the learning curves reveal a characteristic behavior of vanilla RNNs.

11.14.1 Observed Training Curves

During training, the validation accuracy initially rises, reaches a peak, and then abruptly collapses. Subsequent epochs may show partial recovery, but the model rarely regains its previous performance.

11.14.2 Interpretation: Local Minima and Gradient Instability

This phenomenon stems from the inherent difficulty of training vanilla RNNs. The network discovers a solution that partially separates the most frequent classes, but the gradients required to refine the representation of rarer classes vanish or explode. The optimizer becomes trapped in a suboptimal region of the loss landscape, where further updates fail to improve generalization.

This behavior underscores the practical superiority of LSTM and GRU architectures, whose gating mechanisms mitigate the vanishing gradient problem and enable learning of longer temporal dependencies.

11.15 Practical Considerations and Extensions

11.15.1 Parameter Counting

A custom `recurrent_summary` function is provided to display the number of trainable parameters, as the standard `summary()` does not handle recurrent layers correctly.

11.15.2 Hyperparameter Tuning

Key hyperparameters to explore:

- **Window size** W: Must be long enough to capture relevant activity cycles.
- **Stride** S: Balances dataset size and sample diversity.
- **Number of recurrent layers** and **hidden size**: Control model capacity.
- **Dropout rate**: Regularization strength.
- **Learning rate** and **patience** for early stopping.

11.15.3 Beyond the Basic Model

The modular design allows experimentation with hybrid architectures: stacking different cell types, adding residual connections, or combining recurrent layers with 1D convolutional front-ends for feature extraction.

11.16 Introduction: Time as a Dimension

In standard tabular classification, data is represented by a matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ (samples $\times$ features). Time series classification introduces a third dimension: **time**. Each sample becomes a sequence of length T, yielding tensors of shape:

$$\mathbf{X} \in \mathbb{R}^{N \times T \times D}.$$

This temporal dimension encodes the evolution of the signal, which is essential for tasks like human activity recognition from accelerometer data.

11.17 Dataset: Human Activity Recognition

We work with a dataset collected from a wearable device equipped with a tri-axial accelerometer. For each time stamp t, we record:

- `X`, `Y`, `Z`: acceleration along the three spatial axes.
- `timestamp`: Unix time (used only for ordering).
- `user_id`: subject identifier (36 unique users).
- `activity`: label (e.g., *walking*, *jogging*, *standing*).

11.17.1 Data Inspection

Before modeling, we inspect the data:

- No missing values.
- Features are downcast from `float64` to `float32` for efficiency.
- Value ranges are similar across axes; we verify the absence of extreme outliers that could distort normalization.
- Class distribution is **imbalanced**: accuracy is not a suitable metric; we use **F1-score** (macro-averaged).

11.18 Avoiding Data Leakage: User-Based Splitting

A naive random split of individual time steps would scatter segments of the *same user's activity* across training, validation, and test sets. The model could then learn to recognize users rather than activities—a clear data leakage.

We adopt a **user-based split**:

1. Identify all 36 unique `user_id`s.
2. Randomly partition *users* into train (26), validation (5), and test (5) sets.
3. Assign all samples of a user to the corresponding split.

We verify that class distributions remain reasonably balanced across splits, then encode string labels as integers.

11.18.1 Normalization

We apply min-max normalization (or z-score standardization), fitting the scaler *only on the training set* to avoid information leakage.

11.19 Windowing: Creating Fixed-Length Sequences

Neural networks require fixed-size inputs. Each recording session has a variable length L, so we partition it into **windows** of fixed length W.

11.19.1 Window Size (W) and Stride (S)

Given a session of length L, the number of windows extracted is approximately:

$$N_{\text{windows}} = \left\lfloor \frac{L - W}{S} \right\rfloor + 1.$$

- **Window size** W: Must be long enough to capture at least one full cycle of the activity (e.g., one stride). We initially choose $W = 200$.
- **Stride** S: Controls overlap between consecutive windows. A smaller S yields more (redundant) samples; a larger S reduces dataset size but increases diversity. We set $S = 50$ (75% overlap).

11.19.2 Padding the Final Window

If $L - W$ is not a multiple of S, the last segment has length $L' < W$. We may either **discard** it or **pad** it with zeros to length W. Padding introduces artificial data but can act as a mild regularizer when used sparingly.

11.19.3 Final Tensor Shape

After windowing, the dataset is reshaped to:

$$\mathbf{X} \in \mathbb{R}^{N_{\text{windows}} \times W \times 3}, \quad \mathbf{y} \in \mathbb{R}^{N_{\text{windows}}}.$$

Each batch drawn from the DataLoader has shape $(\text{batch_size}, W, 3)$.

11.20 Recurrent Classifier Module

We define a flexible `RecurrentClassifier` that supports three cell types:

- **Vanilla RNN**: Simple recurrent cell, prone to vanishing gradients.
- **LSTM**: Long Short-Term Memory, with gating mechanisms to control information flow.
- **GRU**: Gated Recurrent Unit, a computationally efficient variant of LSTM.

11.20.1 Architecture

1. **Recurrent stack**: One or more recurrent layers process the input sequence and produce a sequence of hidden states.

2. **Classifier head**: A fully connected layer maps the final hidden state (or concatenated forward/backward states if bidirectional) to class logits.

11.20.2 Bidirectional Processing

Setting `bidirectional=True` instantiates two independent recurrent networks: one forward, one backward. Their final hidden states are concatenated, providing context from both past and future within the window.

11.20.3 Dropout in Recurrent Layers

Dropout within a recurrent layer is applied **across time steps** (temporal dropout), which is more effective for regularization in sequence models.

11.21 Training Dynamics and Empirical Observations

11.21.1 Vanilla RNN: Unstable Training

The basic RNN exhibits characteristic instability:

- Validation accuracy initially rises, reaches a peak, then abruptly collapses.
- The model tends to predict only the most frequent classes, failing to generalize.

This behavior stems from the **vanishing/exploding gradient** problem, which prevents the network from learning long-range dependencies.

11.21.2 Bidirectional RNN: No Improvement

Surprisingly, making the vanilla RNN bidirectional *worsens* performance. The additional capacity does not help because the underlying cell is still unable to propagate gradients effectively; the model becomes even more confused.

11.21.3 LSTM: Gradual Learning

The LSTM, despite its higher parameter count, eventually escapes the initial plateau and achieves reasonable performance. Its gating mechanisms allow it to retain useful information and discard noise, leading to an F1-score around 82%.

11.21.4 Bidirectional LSTM: Best Performance

Combining LSTM cells with bidirectional processing yields the best results. The model can now exploit patterns that are visible from both temporal directions, pushing the F1-score to approximately 83–85%.

11.21.5 GRU: Efficient and Effective

The GRU matches or slightly exceeds LSTM performance while converging faster (fewer epochs). Its simplified gating makes it a strong practical choice for many sequence tasks.

11.22 Cross-Validation and Hyperparameter Tuning

To assess model stability and perform hyperparameter search, we employ **user-based cross-validation**:

- For each fold, a new user-based split is generated with a different random seed.
- The entire data preparation pipeline (windowing, normalization) is re-run inside the fold loop to ensure consistency.

11.22.1 Impact of Window Size

A grid search over window size W and stride S reveals substantial performance variation. Increasing W from 200 to 400 (while adjusting S proportionally) can boost the F1-score by over 8 percentage points (e.g., from 82% to 90.5%). This underscores the importance of tuning these preprocessing hyperparameters.

11.22.2 The Baseline Principle

A recurring theme in data science: **always establish a simple baseline**. A cautionary tale from time series forecasting research illustrates this:

> A complex transformer-based model (Informer) won a Best Paper Award. Later work showed that a trivial baseline—predicting the last observed value forward ("no change")—outperformed it on standard benchmarks.

Before adding complexity, ensure that your model improves upon a naive baseline. In our activity recognition task, a simple LSTM or GRU already provides a strong foundation.

11.23 Practical Recommendations for the Challenge

- **Save models**: Due to random initialization and data splits, results may not be exactly reproducible. Store checkpoints.
- **Start simple**: Begin with a single recurrent layer, moderate hidden size, and reasonable window parameters. Add complexity incrementally.
- **Tune systematically**: First identify promising regions of hyperparameter space (e.g., $W \in [200, 400]$), then refine.
- **Monitor F1-score**: Accuracy is misleading on imbalanced data.
- **Use TensorBoard**: Log all experiments for comparison.

Chapter 12

Time Series Classification and Forecasting

12.1 Introduction: From Classification to Forecasting

In the previous session, we addressed **time series classification**: given a sequence of observations, assign a single label. Today, we tackle a more challenging problem: **time series forecasting**. The goal is to predict future values of a time series based on its past behavior.

Applications of forecasting span numerous domains:

- Financial markets (stock prices, though notoriously difficult).
- Risk assessment and anomaly detection.
- Climate and environmental monitoring.
- Demand forecasting in supply chains.

The core idea is to model the temporal dynamics of the data and extrapolate them into the future.

12.2 Dataset: Air Quality and Climate Variables

We will work with an air quality dataset containing three time series:

- `dew`: Dew point (a measure of humidity).
- `temperature`: Ambient temperature.
- `pressure`: Atmospheric pressure.

All variables are numerical and recorded over time. A quick inspection reveals no missing values. The data exhibit clear seasonal patterns—periodic fluctuations that a well-designed model should capture.

12.2.1 Data Preprocessing

- Convert all columns to `float32` for neural network compatibility.
- The time series are plotted to visualize trends, seasonality, and noise.

From visual inspection, `temperature` appears the most regular and predictable, while `dew` and `pressure` contain more high-frequency noise.

12.3 Train/Validation/Test Split for Forecasting

Unlike random shuffling in standard supervised learning, time series data possesses a natural temporal order. We must preserve this order when splitting to avoid **future leakage**.

12.3.1 Chronological Split

We partition the dataset sequentially:

1. **Test set**: The final portion of the time series (e.g., last 20%).
2. **Validation set**: The segment immediately preceding the test set.
3. **Training set**: All remaining earlier data.

This mimics a realistic scenario where we train on historical data and evaluate on the most recent observations.

12.3.2 Why Not Random Cross-Validation?

While cross-validation is possible with time series (e.g., rolling-window or expanding-window validation), the sequential split is a common and interpretable baseline. In practice, one may evaluate on multiple historical windows (backtesting) to assess robustness.

12.4 From a Single Series to Supervised Samples: Windowing

A neural network requires a dataset of input–output pairs. We must transform the long, continuous time series into fixed-length windows.

12.4.1 Key Parameters

- **Window size** W: Number of past time steps used as input.
- **Telescope size** H: Number of future time steps to predict (forecast horizon).
- **Stride** S: Step size between consecutive windows.

12.4.2 Windowing Procedure

Given a time series $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$ (each $\mathbf{x}_t \in \mathbb{R}^D$ with D features), we create samples:

$$\text{Input: } [\mathbf{x}_t, \mathbf{x}_{t+1}, \ldots, \mathbf{x}_{t+W-1}], \quad \text{Label: } [\mathbf{x}_{t+W}, \mathbf{x}_{t+W+1}, \ldots, \mathbf{x}_{t+W+H-1}].$$

The starting index t moves with step S:

$$t = 0, S, 2S, \ldots, T - W - H.$$

This yields a dataset of $N_{\text{samples}} = \lfloor (T - W - H)/S \rfloor + 1$ pairs.

12.4.3 Stride and Sample Correlation

- **Small S (e.g., $S = 1$)**: Generates many highly correlated samples. Useful for validation/testing to maximize evaluation coverage, but may cause overfitting in training.
- **Large S (e.g., $S > 1$)**: Reduces correlation between training samples, promoting better generalization. We typically use a moderate stride (e.g., $S = 5$) for training and $S = 1$ for validation/test.

12.4.4 Multivariate Forecasting

In our dataset, $D = 3$ (dew, temperature, pressure). The input tensor has shape $(N_{\text{samples}}, W, D)$ and the output tensor has shape $(N_{\text{samples}}, H, D)$. The model must learn to predict all three variables simultaneously.

12.5 Visualizing the Windowed Data

Plotting a few random input–label pairs reveals:

- The seasonal, long-range pattern is lost within a short window ($W = 200$ in our example).
- The model must learn local dynamics and extrapolate them H steps ahead.
- Among the three variables, `temperature` exhibits the clearest periodic structure and is likely the easiest to forecast.

12.6 Modeling Approaches: Classical vs. Neural

12.6.1 Classical Time Series Models

Traditional methods like **ARIMA** (Autoregressive Integrated Moving Average) and its seasonal variants explicitly model trend, seasonality, and noise components. They are highly effective when the underlying process is linear and the patterns are well-defined (e.g., strong annual cycles).

12.6.2 Neural Networks for Forecasting

When the dynamics are complex, nonlinear, or the data lacks clear seasonal decomposition, neural networks (RNNs, LSTMs, GRUs, or even Transformers) can learn more flexible representations. They are particularly useful when multiple correlated series must be predicted jointly.

A Cautionary Tale: The Baseline Principle. In recent years, complex Transformer-based forecasters (e.g., Informer) gained popularity. However, subsequent studies revealed that a trivial **persistence baseline**—predicting the last observed value forward indefinitely—often outperforms them on standard benchmarks. *Always benchmark against a simple, interpretable baseline before adding complexity.*

12.7 Implementation Outline

The core logic for creating supervised samples is encapsulated in a function:

1. Accepts a DataFrame and parameters W, S, H.
2. Initializes empty lists for inputs and labels.
3. Iterates t from 0 to $T - W - H$ with step S:
   - Input slice: `data[t : t+W]`.
   - Label slice: `data[t+W : t+W+H]`.
4. Returns arrays $\mathbf{X} \in \mathbb{R}^{N \times W \times D}$ and $\mathbf{Y} \in \mathbb{R}^{N \times H \times D}$.

After constructing $\mathbf{X}_{\text{train}}$, $\mathbf{Y}_{\text{train}}$, etc., we can train any sequence-to-sequence model (e.g., an LSTM encoder-decoder) to minimize the mean squared error between predictions and true future values.

12.8 Introduction: From Classification to Forecasting

In previous sessions, we addressed **time series classification**: assigning a single label to an entire sequence. Today, we tackle the more challenging task of **time series forecasting**: predicting future values of a multivariate signal based on its past evolution.

Forecasting finds applications in finance, climate modeling, demand prediction, and anomaly detection. The core difficulty lies in the accumulation of errors over the prediction horizon, especially when the underlying dynamics are nonlinear or noisy.

12.9 Dataset: Air Quality and Climate Variables

We work with a dataset containing three time series recorded over time:

- `dew`: Dew point (a humidity measure).
- `temperature`: Ambient temperature.
- `pressure`: Atmospheric pressure.

All variables are numerical, with no missing values. Visual inspection reveals clear seasonal patterns—periodic fluctuations that a well-designed model should capture. Among the three, `temperature` exhibits the strongest regular periodicity and is likely the easiest to forecast.

12.10 Train/Validation/Test Split for Time Series

Unlike standard supervised learning, time series data has a natural temporal order. We must preserve this order to avoid **future leakage**.

12.10.1 Chronological Split

We partition the dataset sequentially:

1. **Test set**: The final portion (e.g., last 20%).
2. **Validation set**: The segment immediately preceding the test set.
3. **Training set**: All remaining earlier data.

This mimics real-world deployment, where models are trained on historical data and evaluated on the most recent observations.

12.10.2 Why Sequential?

If the forecast horizon H is small relative to the dominant seasonal period, the model may not need to learn long-range seasonal patterns. The choice of window size W and horizon H should reflect the intended application.

12.11 From a Single Series to Supervised Samples: Windowing

A neural network requires a dataset of input–output pairs. We transform the continuous time series into fixed-length windows.

12.11.1 Key Parameters

- **Window size** W: Number of past time steps used as input.
- **Telescope size** H: Number of future time steps to predict.
- **Stride** S: Step size between consecutive windows.

12.11.2 Windowing Procedure

Given a multivariate time series $\mathbf{x}_1, \ldots, \mathbf{x}_T$ (each $\mathbf{x}_t \in \mathbb{R}^D$, here $D = 3$), we extract samples:

$$\text{Input: } [\mathbf{x}_t, \ldots, \mathbf{x}_{t+W-1}], \quad \text{Label: } [\mathbf{x}_{t+W}, \ldots, \mathbf{x}_{t+W+H-1}],$$

for $t = 0, S, 2S, \ldots, T - W - H$. The number of samples is $N = \lfloor (T - W - H)/S \rfloor + 1$.

12.11.3 Stride and Correlation

- **Training**: Use a moderate stride (e.g., $S > 1$) to reduce correlation between samples and promote generalization.
- **Validation/Test**: Use $S = 1$ to maximize the number of evaluation samples, even if they are correlated.

12.11.4 Tensor Shapes

After windowing, we obtain:

$$\mathbf{X} \in \mathbb{R}^{N \times W \times D}, \quad \mathbf{Y} \in \mathbb{R}^{N \times H \times D}.$$

Each batch drawn from the DataLoader has shape $(\text{batch_size}, W, D)$ for inputs and $(\text{batch_size}, H, D)$ for targets.

12.12 Direct Forecasting Model: Many-to-One (Encoder-Only)

We begin with the simplest recurrent forecaster: an encoder RNN followed by a linear layer that maps the final hidden state to the entire forecast horizon.

12.12.1 Architecture

1. **Recurrent Encoder**: An RNN, LSTM, or GRU processes the input window $\mathbf{X}$ step by step, producing a final hidden state $\mathbf{h}_W \in \mathbb{R}^d$ (where d is the hidden size).

2. **Linear Forecaster**: A fully connected layer maps $\mathbf{h}_W$ to a flattened vector of size $H \cdot D$, which is then reshaped to (H, D).

Formally:

$$\mathbf{h}_W = \text{RNN}(\mathbf{X}), \quad \hat{\mathbf{Y}}_{\text{flat}} = \mathbf{W}\mathbf{h}_W + \mathbf{b}, \quad \hat{\mathbf{Y}} = \text{reshape}(\hat{\mathbf{Y}}_{\text{flat}}, (H, D)).$$

12.12.2 Why "Many-to-One"?

Although the output contains $H \times D$ values, the recurrent network only produces *one* final state. All future time steps are predicted simultaneously via a single linear transformation. This contrasts with **autoregressive** or **sequence-to-sequence** models, which generate outputs step by step, conditioning each prediction on previous outputs.

12.12.3 Model Variants

- **Bidirectional**: Processes the input forward and backward, concatenating the two final states. This doubles the hidden size fed into the linear layer.
- **Stacked RNNs**: Multiple recurrent layers can be stacked to increase capacity.

12.13 Training the Direct Forecaster

12.13.1 Loss Function

We minimize the **Mean Squared Error** (MSE) between the predicted and true future values, summed over all features and time steps:

$$\mathcal{L} = \frac{1}{NHD} \sum_{i=1}^{N} \sum_{h=1}^{H} \sum_{d=1}^{D} \left(\hat{y}_{i,h,d} - y_{i,h,d}\right)^2 .$$

12.13.2 Learning Rate Scheduling

Time series forecasting, especially with multivariate outputs, presents a challenging optimization landscape. The loss surface may contain narrow valleys that are missed if the learning rate is too large.

Learning rate scheduling dynamically reduces the learning rate when the validation loss plateaus. A common strategy is to reduce the learning rate by a factor (e.g., 0.1) when no improvement is observed for a certain number of epochs. This allows the optimizer to take smaller, more precise steps and potentially settle into better minima.

In our implementation, we reduce the learning rate twice: from 10^{-3} to 10^{-4}, then to 10^{-5}, each time after validation performance stalls.

12.13.3 Training Dynamics

The training and validation MSE curves show a steady decrease, with no overfitting. The learning rate reductions help refine the solution.

12.14 Limitations and Next Steps

The direct forecaster, while simple, has drawbacks:

- It predicts all future steps simultaneously, which may be suboptimal for long horizons.
- It does not explicitly model the temporal dependencies within the forecast window.

Subsequent models will address these issues:

- **Autoregressive models**: Predict one step, then feed it back as input for the next.
- **Sequence-to-sequence (Seq2Seq)**: Use an encoder–decoder architecture to generate outputs step by step.

12.15 Introduction: Forecasting Strategies

In the previous section, we introduced **direct forecasting**: an encoder RNN whose final hidden state is mapped to the entire forecast horizon via a linear layer. While simple, this approach struggles with long horizons because all future steps must be predicted simultaneously. Today, we explore two more sophisticated paradigms:

1. **Autoregressive forecasting**: Predict one step (or a short block), then feed the prediction back as input for the next step.
2. **Sequence-to-sequence (Seq2Seq) forecasting**: Use an encoder–decoder architecture to generate the forecast step by step, conditioning each output on previously generated ones.

We also delve into **learning rate scheduling**, a crucial technique for navigating the challenging optimization landscape of multivariate regression.

12.16 Learning Rate Scheduling: Balancing Exploration and Exploitation

Training deep networks on high-dimensional regression tasks (like forecasting multiple time series over many steps) requires careful control of the learning rate η. The loss surface is often non-convex and contains narrow valleys that are easily missed if η is too large.

12.16.1 The Principle

Learning rate scheduling mimics the **exploration–exploitation** trade-off:

- **Exploration**: Start with a relatively large η to quickly traverse the loss landscape and locate promising regions.
- **Exploitation**: Gradually reduce η to take smaller, more precise steps and settle into deep, narrow minima.

12.16.2 Common Scheduling Strategies

Step Decay / Reduce on Plateau Reduce η by a factor $\gamma \in (0, 1)$ (e.g., $\gamma = 0.1$) when the validation metric fails to improve for a specified number of epochs (patience). This is the most widely used strategy.

Cosine Annealing Vary η according to a cosine function:

$$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \left(1 + \cos\left(\pi \frac{t}{T}\right)\right),$$

where T is the cycle length. Inspired by annealing in metallurgy, this periodic cooling–heating schedule often yields better final solutions. It can be repeated over multiple cycles.

Exponential Decay $\eta_t = \eta_0 \cdot \gamma^t$. This is more aggressive and less common due to rapid decay.

Warm-up For problems with highly noisy gradients at initialization, start with a very small η and linearly increase it over a few epochs (e.g., 5–10) to the target initial value. This stabilizes early training before aggressive exploration begins.

12.16.3 Implementation in PyTorch

The `ReduceLROnPlateau` scheduler automatically reduces η when a tracked metric stalls:

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.1, patience=5, min_lr=1e-6
)
```

At each epoch, call `scheduler.step(val_loss)`. Ensure that the scheduler's patience is smaller than the early-stopping patience; otherwise, training will stop before η is reduced.

12.17 Direct Forecaster: Performance Recap

The direct forecaster (many-to-one RNN + linear layer) produced reasonable forecasts for `temperature`, which exhibits clear periodicity. However, `dew` and `pressure` were poorly predicted, with errors growing rapidly as the forecast horizon extended. This motivates the need for iterative, step-by-step generation.

12.18 Autoregressive Forecasting

An autoregressive model predicts a **short block** of future values and then feeds its own predictions back as input to generate the next block. This decomposes the long-horizon problem into a sequence of easier subproblems.

12.18.1 Model Design

We retain the same architecture as the direct forecaster (RNN encoder + linear output layer), but with a smaller **autoregressive horizon** $H_{\text{ar}} \ll H$. For example, with $H = 50$ and $H_{\text{ar}} = 10$, the model is invoked 5 times.

Training. The training dataset is constructed using the shorter horizon H_{ar}:

$$\mathbf{X} \in \mathbb{R}^{N \times W \times D}, \quad \mathbf{Y} \in \mathbb{R}^{N \times H_{\text{ar}} \times D}.$$

The model learns to predict the next H_{ar} steps given the preceding W steps.

Inference (Rollout). To generate a forecast of length H:

1. Start with the input window $\mathbf{X}_0$ of length W.
2. For $i = 0, 1, \ldots, \lceil H/H_{\text{ar}} \rceil - 1$:
   - Predict $\hat{\mathbf{Y}}_i \in \mathbb{R}^{H_{\text{ar}} \times D}$ using the current input.
   - Append $\hat{\mathbf{Y}}_i$ to the output sequence.
   - Update the input window by discarding the oldest H_{ar} steps and appending $\hat{\mathbf{Y}}_i$.

This autoregressive loop is implemented with `torch.no_grad()` for efficiency.

12.18.2 Why Does It Perform Better?

- The model is trained to predict a much shorter horizon (H_{ar} instead of H), which is an inherently easier task.
- During inference, each step benefits from the model's own "simulated" future, allowing it to adapt to the evolving dynamics.
- The error accumulates over the rollouts, but the per-step accuracy remains higher than a single-shot prediction over the full horizon.

Empirically, autoregressive models yield significantly better forecasts, especially for noisy variables like `dew` and `pressure`.

Note on Loss Comparability. The training loss (MSE over H_{ar} steps) is not directly comparable to the direct forecaster's loss (MSE over H steps). Always use a common evaluation metric (e.g., MSE over the full horizon H) on the validation/test set for fair model selection.

12.19 Sequence-to-Sequence Forecasting

The most flexible approach is the **encoder–decoder** (Seq2Seq) architecture, originally developed for machine translation. Here, we adapt it to multivariate numerical forecasting.

12.19.1 Analogy with Text Processing

In NLP, words are first mapped to dense embeddings. For numerical time series, the raw measurements $\mathbf{x}_t \in \mathbb{R}^D$ already constitute a meaningful continuous representation. Thus, we can directly feed them into the encoder without an embedding layer.

12.19.2 Architecture

- **Encoder**: A (optionally bidirectional) RNN processes the input window $\mathbf{X} \in \mathbb{R}^{W \times D}$ and produces a final hidden state $\mathbf{h}_{\text{enc}}$, which summarizes the entire input sequence.
- **Decoder**: Another RNN, initialized with $\mathbf{h}_{\text{enc}}$, generates the forecast step by step. At each time step t, the decoder receives the previous true target (during training, *teacher forcing*) or its own previous prediction (during inference). It outputs a vector $\hat{\mathbf{y}}_t \in \mathbb{R}^D$.

The decoder thus produces the forecast sequence $\hat{\mathbf{Y}} = [\hat{\mathbf{y}}_1, \ldots, \hat{\mathbf{y}}_H]$ autoregressively, but with a dedicated recurrent module that conditions on the encoded context.

12.19.3 Training and Inference

- **Teacher Forcing**: During training, the decoder receives the ground-truth previous value $\mathbf{y}_{t-1}$ as input. This stabilizes learning.
- **Inference**: The decoder receives its own previous prediction $\hat{\mathbf{y}}_{t-1}$ (or the last input value for $t = 1$).

Seq2Seq models are more expressive than the simple autoregressive encoder-only model because the decoder has its own recurrent dynamics tailored to generation.

12.19.4 Implementation Considerations

The code becomes more involved due to the separate encoder and decoder modules and the logic for teacher forcing. However, the core components remain standard PyTorch RNN layers.

12.20 Introduction: The Seq2Seq Paradigm

The most expressive architecture for time series forecasting is the **Sequence-to-Sequence** (Seq2Seq) model, originally developed for machine translation. It consists of two recurrent components:

- **Encoder**: Processes the entire input window and summarizes it into a context vector (or sequence of hidden states).
- **Decoder**: Generates the forecast step by step, conditioned on the encoder's output.

In our implementation, the encoder is **bidirectional** (capturing context from both past and future within the window), while the decoder is **unidirectional** (generating forward in time).

12.20.1 From Words to Numbers

In NLP, words are first mapped to dense embeddings. For numerical time series, the raw measurements $\mathbf{x}_t \in \mathbb{R}^D$ already constitute a meaningful continuous representation. We can therefore feed them directly into the encoder without an explicit embedding layer.

12.21 Attention Mechanism: Weighted Context

A key enhancement to Seq2Seq models is the **attention mechanism**. At each decoding step, attention computes a weighted sum of the encoder outputs, emphasizing the parts of the input that are most relevant for the current prediction.

12.21.1 Mathematical Formulation

Let $\mathbf{h}_1^{\text{enc}}, \ldots, \mathbf{h}_W^{\text{enc}}$ be the hidden states produced by the encoder. At decoder step t, we compute attention scores:

$$e_{t,i} = \text{score}(\mathbf{h}_{t-1}^{\text{dec}}, \mathbf{h}_i^{\text{enc}}), \quad i = 1, \ldots, W.$$

In our implementation, the scoring function is a simple linear layer:

$$e_{t,i} = \mathbf{v}^\top \tanh(\mathbf{W}_1 \mathbf{h}_{t-1}^{\text{dec}} + \mathbf{W}_2 \mathbf{h}_i^{\text{enc}}).$$

The scores are normalized via softmax to obtain attention weights $\alpha_{t,i} \in (0, 1)$ with $\sum_i \alpha_{t,i} = 1$:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{W} \exp(e_{t,j})}.$$

The **context vector** $\mathbf{c}_t$ is the weighted sum of encoder states:

$$\mathbf{c}_t = \sum_{i=1}^{W} \alpha_{t,i} \mathbf{h}_i^{\text{enc}}.$$

This context is concatenated with the decoder's input and fed into the decoder RNN.

12.21.2 Interpretation

The attention weights $\alpha_{t,i}$ indicate which time steps in the input window the model "attends to" when generating the t-th forecast step. In practice, these weights often align with intuitive temporal dependencies (e.g., recent inputs receive higher weights for near-term predictions), but they should not be over-interpreted as human-understandable explanations—they are optimized for the task, not for interpretability.

12.22 Teacher Forcing: Stabilizing Seq2Seq Training

Training a Seq2Seq model from scratch is notoriously unstable because the decoder's inputs are its own (initially random) predictions. Errors accumulate rapidly, making convergence difficult.

12.22.1 The Teacher Forcing Technique

During training, with probability p_{tf} (e.g., 0.2), we replace the decoder's own previous prediction with the **ground-truth** value from the training data. This "teacher" signal guides the decoder, reducing noise and accelerating convergence.

12.22.2 Implementation

In the forward pass, for each decoder step:

- Generate a random number $r \sim \text{Uniform}(0, 1)$.
- If $r < p_{\text{tf}}$ (and we are in training mode), set the decoder input to the true target value $\mathbf{y}_{t-1}$.
- Otherwise, use the model's own prediction $\hat{\mathbf{y}}_{t-1}$.

Teacher forcing is gradually reduced or turned off in later epochs to allow the model to learn to generate autonomously.

12.23 Code Complexity and the Role of LLMs

The Seq2Seq implementation involves multiple interacting components (encoder, decoder, attention, output layer) and careful handling of bidirectionality and tensor reshaping. The code is therefore more intricate than earlier models.

In modern development, **Large Language Models (LLMs)** serve as valuable assistants for understanding and extending such code. They can:

- Explain line-by-line what each function does.
- Provide references to original papers and architectural variants.
- Suggest modifications or debug common errors.

For this course, the use of LLMs is encouraged as a learning and productivity tool. The exam is designed to be "LLM-compliant," focusing on conceptual understanding and the ability to adapt existing architectures rather than memorizing syntax.

12.24 Empirical Performance

The Seq2Seq model with attention generally produces forecasts that are smoother and less noisy than the direct or simple autoregressive models, particularly for variables like `dew` and `pressure`. However, the improvements are incremental, and the increased complexity may not always justify the gains over a well-tuned autoregressive forecaster.

12.24.1 Visual Inspection

- `temperature` is consistently well-predicted across all models. - `pressure` and `dew` benefit from the stepwise generation and attention mechanism, showing reduced erratic fluctuations.

12.24.2 Test Set Evaluation

Quantitative metrics (MSE over the full horizon) confirm that Seq2Seq matches or slightly exceeds autoregressive performance, but the difference is often small. The choice of model should be guided by the specific forecasting horizon and the available computational budget.

Chapter 13

Image Classification

13.1 Overview: From Vectors to Images

So far in this course, we have addressed classification problems where each sample is described by a vector of attributes (e.g., sepal length, petal width in the Iris dataset). We now shift our focus to a fundamentally different type of input: **digital images**. This transition introduces new challenges that require specialized neural network architectures, which we will explore in the coming lectures.

13.2 What is Computer Vision?

Computer vision aims to enable machines to interpret and understand the content of digital images and videos. By "understanding," we refer to tasks such as:

- **Object Detection**: Identifying and localizing multiple objects within an image (e.g., cars, persons, traffic lights) by drawing bounding boxes and assigning class labels.
- **Human Pose Estimation**: Detecting the positions of human joints (e.g., elbows, knees) in images or video frames, even under varied poses and activities.
- **Image Classification**: Assigning a single label to an entire image (e.g., "cat," "dog," "automobile").
- **Image Captioning**: Generating a textual description of the visual content.
- **Industrial Inspection**: Detecting defects or anomalies in manufacturing processes (e.g., wafer defect maps).

These capabilities are not limited to natural photographs; they extend to medical imaging, satellite imagery, and industrial quality control.

13.2.1 Historical Context

Prior to approximately 2012, computer vision relied heavily on hand-crafted mathematical, statistical, and geometric models. Algorithms were designed based on explicit assumptions about image formation and structure. While such methods remain valuable for specific low-level tasks (e.g., image restoration, geometric calibration), the field has been revolutionized by **deep neural networks**, which now dominate semantic understanding tasks. Modern flagship conferences (e.g., CVPR) are overwhelmingly focused on learning-based approaches.

13.3 Digital Image Representation

To understand why specialized networks are needed, we must first understand how a computer represents an image.

13.3.1 Pixels and Color Channels

A digital image is not a continuous scene but a discrete grid of **pixels** (picture elements). For a standard color image, each pixel is represented by three integer values, corresponding to the intensities of the **Red**, **Green**, and **Blue** (RGB) channels.

Formally, a color image of height H and width W is stored as a three-dimensional array (tensor):

$$\mathbf{I} \in \mathbb{N}^{H \times W \times 3}.$$

Each channel is a $H \times W$ matrix of intensity values, typically in the range $[0, 255]$. This range corresponds to 8 bits (1 byte) per channel, allowing $2^8 = 256$ distinct intensity levels. Consequently, the total number of representable colors is:

$$2^{8 \times 3} = 2^{24} \approx 16.7 \text{ million}.$$

13.3.2 Examples of RGB Values

- A **pure red** pixel: $(R, G, B) = (255, 0, 0)$.
- A **white** pixel: $(255, 255, 255)$.
- A **black** pixel: $(0, 0, 0)$.

When you zoom into a digital image, the individual pixels become visible as small colored squares, each encoding its specific RGB triplet.

13.3.3 Implications for Neural Networks

For a modest image size of 224×224 pixels, the flattened input vector would contain:

$$224 \times 224 \times 3 = 150{,}528 \text{ values}.$$

A fully connected neural network (multilayer perceptron) processing such inputs would require an enormous number of parameters, leading to:

- Severe overfitting on typical datasets.
- Computational inefficiency.
- Inability to exploit spatial structure (e.g., translation invariance).

These limitations motivate the development of **Convolutional Neural Networks (CNNs)**, which we will introduce in subsequent lectures.

13.4 The Simplest Visual Recognition Task: Image Classification

Image classification is the task of assigning a single label $\hat{y} \in \{1, \ldots, C\}$ to an input image $\mathbf{I}$. While the problem formulation is identical to tabular classification (we still optimize a cross-entropy loss), the nature of the input data fundamentally changes the design requirements.

13.4.1 What Can a Multilayer Perceptron Learn from Images?

If we naïvely flatten the image and feed it into a standard fully connected network, the model can, in principle, learn some patterns. However, it will:

- Treat every pixel as an independent feature, ignoring spatial locality.
- Fail to recognize that a shifted version of an object is essentially the same object (lack of translation equivariance).
- Require an impractical number of parameters to model even simple visual hierarchies.

Empirically, such models perform poorly on image classification benchmarks and do not scale to real-world image sizes.

13.5 Key Challenges in Image Classification

When designing neural networks for visual data, we must address several inherent challenges:

1. **High Dimensionality**: Images contain tens of thousands to millions of pixels, each contributing to the input dimension.
2. **Spatial Structure**: Pixels are not independent; meaningful patterns arise from local correlations (edges, textures, object parts).
3. **Translation Invariance/Equivariance**: The identity of an object should not depend on its exact position in the image.
4. **Hierarchical Composition**: Visual concepts are built hierarchically: edges combine into textures, textures into parts, parts into objects.
5. **Variability**: Illumination, viewpoint, scale, occlusion, and background clutter introduce enormous intra-class variation.

Convolutional Neural Networks (CNNs) are specifically architected to address these challenges through mechanisms such as local receptive fields, weight sharing, and pooling. These will be the focus of the next lecture.

13.6 Introduction to Computer Vision

Computer vision aims to enable machines to interpret and understand the content of digital images and videos. Tasks include:

- **Object Detection**: Localizing and classifying multiple objects.
- **Human Pose Estimation**: Identifying joint positions.
- **Image Classification**: Assigning a single label to an image.
- **Industrial Inspection**: Detecting defects or anomalies.

Prior to the deep learning revolution (circa 2012), computer vision relied on hand-crafted mathematical and geometric models. Today, deep neural networks dominate semantic understanding tasks, though classical methods remain valuable for low-level processing.

13.7 Digital Image Representation

A digital image is a discrete grid of **pixels**. For a color image, each pixel is represented by three integer values corresponding to the Red, Green, and Blue (RGB) channels.

Formally, a color image of height H and width W is a three-dimensional array:

$$\mathbf{I} \in \mathbb{N}^{H \times W \times 3}.$$

Each channel is a $H \times W$ matrix with intensity values typically in the range $[0, 255]$ (8 bits per channel). The total number of representable colors is:

$$2^{8 \times 3} = 2^{24} \approx 16.7 \text{ million}.$$

13.7.1 Examples of RGB Triplets

- Pure red: $(255, 0, 0)$
- White: $(255, 255, 255)$
- Black: $(0, 0, 0)$

13.8 The Challenge of Image Size

A 48-megapixel camera captures 48×10^6 pixels. With 3 bytes per pixel (RGB), the raw uncompressed size is approximately:

$$48 \times 10^6 \times 3 \approx 144 \text{ MB}.$$

When saved as JPEG or PNG, compression reduces file size to a few megabytes, but when loaded into memory for processing, the image expands back to its full uncompressed dimensions. Moreover, neural networks typically operate on floating-point numbers (4 bytes per value), increasing memory usage fourfold.

For video, a Full HD frame (1920×1080) contains about 2 million pixels, yielding ≈ 6 MB per frame. At 24 frames per second, a single second of uncompressed video requires ≈ 150 MB. This scale necessitates specialized architectures and careful memory management.

13.9 Basic Image Operations: Linear Filtering

13.9.1 Pointwise Operations

The simplest image operations process each pixel independently, e.g., gamma correction or histogram equalization. These are not true "image processing" operations because they ignore spatial context.

13.9.2 Neighborhood Operations

A genuine image operation considers a local neighborhood around each pixel. For a pixel at position (r, c), we define a neighborhood as a set of relative displacements:

$$\mathcal{N} = \{(u, v)\} \quad \text{e.g., a } 3 \times 3 \text{ window: } u, v \in \{-1, 0, 1\}.$$

The output $G(r, c)$ is computed from the intensities in this neighborhood:

$$G(r, c) = f\big(\{I(r + u, c + v) \mid (u, v) \in \mathcal{N}\}\big).$$

13.9.3 Moving Average (Box Filter)

The simplest neighborhood operation is the arithmetic mean:

$$G(r, c) = \frac{1}{|\mathcal{N}|} \sum_{(u,v)\in\mathcal{N}} I(r + u, c + v).$$

For a 3×3 window, $|\mathcal{N}| = 9$. This operation blurs the image: sharp transitions (e.g., a white line on a black background) become wider and less intense because the averaging spreads the intensity across neighboring pixels.

13.9.4 Linear Filtering with Learnable Weights

To introduce learnable parameters, we replace the fixed uniform weights with a set of coefficients $W(u, v)$:

$$G(r, c) = \sum_{(u,v)\in\mathcal{N}} W(u, v)\, I(r + u, c + v).$$

The matrix $\mathbf{W}$ (size 3×3 in this example) is called the **filter** or **kernel**. The operation of sliding this filter across the entire image and computing the weighted sum at each position is known as **correlation** (in signal processing terms) or **convolution** (with appropriate index flipping).

This operation possesses two crucial properties:

1. **Locality**: Each output pixel depends only on a small spatial neighborhood.
2. **Translation Equivariance**: The same filter is applied identically at every spatial position.

These properties are the foundation of Convolutional Neural Networks (CNNs), which we will explore in subsequent lectures.

13.10 From Hand-Crafted to Learned Filters

In classical image processing, filters are designed manually to detect specific patterns (e.g., edges, corners, textures). In deep learning, the filter weights $W(u, v)$ are treated as parameters to be learned from data via backpropagation. This allows the network to discover hierarchical feature representations optimized for the task at hand.

13.11 Recap: Linear Filtering as a Building Block

We previously introduced the concept of a neighborhood operation on an image. For each pixel location (r, c), we consider a set of relative displacements $\mathcal{N}$ (e.g., a 3×3 window) and compute an output value based on the intensities in that neighborhood. The simplest parameterized version is a linear combination:

$$G(r, c) = \sum_{(u,v)\in\mathcal{N}} W(u, v)\, I(r + u, c + v),$$

where $\mathbf{W}$ is a matrix of learnable weights called the **filter** or **kernel**. This operation is applied identically at every spatial position, a property known as **translation equivariance**.

13.11.1 Correlation vs. Convolution

In signal processing, the term **convolution** typically involves flipping the filter:

$$(I * W)(r, c) = \sum_{(u,v)} I(r - u, c - v)\, W(u, v).$$

This definition yields convenient algebraic properties (e.g., convolution theorem in Fourier analysis). However, in deep learning, filters are learned from data, and there is no predefined meaning to the weights. Therefore, the operation computed in **Convolutional Neural Networks** is technically **cross-correlation**, not convolution. We will use the term "correlation" to reflect this implementation detail, though the literature overwhelmingly uses "convolution."

13.12 Visual Interpretation: Template Matching via Correlation

Consider a binary image containing text (white pixels on a black background) and a filter that is a small patch extracted from that text (e.g., the letters "NO"). Computing the correlation between the image and the filter yields a new image where the intensity at each location indicates the degree of overlap between the filter and the local image patch.

- In regions where the image patch is entirely black, the output is 0.
- Where there is partial overlap, the output is proportional to the number of aligned white pixels.
- The maximum output occurs exactly at the position where the filter perfectly matches the image content.

This is the simplest form of **template matching** and serves as a precursor to modern object detection networks (e.g., YOLO). The core operation—sliding a filter over an image and computing a similarity score—remains a fundamental building block.

13.12.1 Extension to Multi-Channel Images

For a color image with C channels (e.g., $C = 3$ for RGB), a filter can either operate independently on each channel or be defined as a three-dimensional kernel. In the latter case, the output is obtained by summing correlations over all channels:

$$G(r, c) = \sum_{k=1}^{C} \sum_{(u,v)\in\mathcal{N}} W_k(u, v)\, I_k(r + u, c + v).$$

This is precisely the operation performed by the first layer of a Convolutional Neural Network.

13.13 Image Classification: Problem Formulation

The **image classification** task is to assign a label $\hat{y} \in \{1, \ldots, L\}$ to an input image $\mathbf{I}$. A neural network classifier outputs a probability distribution over the L classes:

$$f_\theta : \mathbb{R}^{H \times W \times C} \to [0,1]^L, \quad \sum_{\ell=1}^{L} f_\theta(\mathbf{I})_\ell = 1.$$

The parameters θ are learned by minimizing a cross-entropy loss on a labeled training set.

13.14 The Mismatch: Images vs. Multilayer Perceptrons

A standard multilayer perceptron (MLP) expects a one-dimensional vector as input. An image, however, is a three-dimensional tensor (height, width, channels). To feed an image into an MLP, one must **flatten** it into a long vector:

$$\mathbf{x} = \text{flatten}(\mathbf{I}) \in \mathbb{R}^{H \cdot W \cdot C}.$$

For a modest 224×224 RGB image, this yields 150,528 input features. The subsequent fully connected layers would require an enormous number of parameters, leading to:

- Severe overfitting on all but the largest datasets.
- Computational inefficiency.
- Complete disregard for the spatial structure of the image (pixels that are adjacent in the image become distant in the flattened vector).

Furthermore, the MLP architecture is not **translation equivariant**: shifting an object by a few pixels would produce a completely different input vector, forcing the network to relearn the same pattern at every possible position. This is highly inefficient and contrary to how visual perception works.

13.15 Preview: Convolutional Neural Networks

The limitations of MLPs for image data motivate the need for specialized architectures. Convolutional Neural Networks (CNNs) address these issues by:

1. **Preserving spatial structure**: Operating on tensors rather than vectors.
2. **Local connectivity**: Each neuron depends only on a small spatial neighborhood.
3. **Weight sharing**: The same filter is applied across all spatial locations, drastically reducing parameter count and enforcing translation equivariance.
4. **Hierarchical feature learning**: Stacking multiple convolutional layers allows the network to learn increasingly abstract representations.

The linear filtering operation we have studied is the core computational primitive of a CNN. In the next lecture, we will see how these filters are organized into layers, combined with nonlinearities and pooling, and trained end-to-end via backpropagation.

13.16 The Mismatch: Images as Vectors

A standard multilayer perceptron (MLP) expects a one-dimensional vector as input. An image, however, is a three-dimensional tensor of shape $H \times W \times C$ (height, width, channels). To feed an image into an MLP, we must **flatten** it into a long vector.

13.16.1 Flattening an RGB Image

For an RGB image, flattening is typically performed by concatenating the pixel values channel by channel. For a $32 \times 32 \times 3$ image (as in CIFAR-10), the resulting vector has dimension:

$$D = 32 \times 32 \times 3 = 3072.$$

The first 1024 entries correspond to the red channel, the next 1024 to green, and the final 1024 to blue.

13.16.2 Implications for Network Size

A fully connected layer from D input neurons to H hidden neurons requires $D \times H$ weights. For $H \approx D = 3072$, the weight matrix contains approximately 9.4×10^6 parameters. Adding a second hidden layer would multiply this count further, making training infeasible on typical datasets. For this reason, when using MLPs for image classification, we typically restrict ourselves to shallow architectures (e.g., a single hidden layer or even directly from input to output).

13.17 CIFAR-10: A Benchmark for Tiny Images

CIFAR-10 is a widely used dataset for image classification, consisting of 60,000 color images of size 32×32 pixels across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.

13.17.1 Dimensionality

Each image, when flattened, yields a 3072-dimensional vector. This is still a high-dimensional input, but manageable for a simple linear classifier or a shallow MLP.

13.18 Interpreting a Trained Neural Network as a Linear Classifier

Consider a neural network with:

- An input layer of size D (flattened image).
- An output layer of size L (number of classes), with softmax activation.
- No hidden layers.

After training, the network computes a score for each class. Let $\mathbf{x} \in \mathbb{R}^D$ be the input vector. For class j, the pre-softmax score s_j is:

$$s_j = \mathbf{w}_j^\top \mathbf{x} + b_j,$$

where $\mathbf{w}_j \in \mathbb{R}^D$ is the weight vector connecting all inputs to the j-th output neuron, and b_j is the bias.

Stacking all L score equations yields a matrix formulation:

$$\mathbf{s} = \mathbf{W}\mathbf{x} + \mathbf{b},$$

where $\mathbf{W} \in \mathbb{R}^{L \times D}$ has rows $\mathbf{w}_j^\top$, and $\mathbf{b} \in \mathbb{R}^L$.

13.18.1 From Scores to Probabilities

The softmax function converts scores to probabilities without changing their relative order:

$$P(y = j \mid \mathbf{x}) = \frac{\exp(s_j)}{\sum_{k=1}^{L} \exp(s_k)}.$$

Therefore, the class with the largest score s_j will receive the largest probability. At inference time, we can omit the softmax and simply take $\arg\max_j s_j$.

13.18.2 Interpretation as L Independent Classifiers

The score for class j depends only on $\mathbf{w}_j$ and $\mathbf{x}$. The L classifiers are trained jointly via backpropagation, but once trained, each $\mathbf{w}_j$ can be viewed as a template for class j. The inner product $\mathbf{w}_j^\top \mathbf{x}$ measures the similarity between the input image and the class template.

13.18.3 Number of Parameters

For CIFAR-10 with $D = 3072$ and $L = 10$, the linear classifier has:

$$\text{Weights: } L \times D = 10 \times 3072 = 30{,}720, \quad \text{Biases: } L = 10.$$

Total parameters: 30,730.

13.19 The Role of Nonlinearities

Why must we include nonlinear activation functions between layers? Consider stacking two linear layers:

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1,$$
$$\mathbf{s} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2 = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2).$$

The composition of linear transformations is itself a linear transformation. Thus, a multi-layer network without nonlinearities collapses to a single linear classifier, regardless of depth. Nonlinear activation functions (e.g., ReLU, tanh) break this linearity, allowing the network to represent complex, hierarchical functions.

13.20 Visualizing the Learned Weights

Since each row $\mathbf{w}_j$ corresponds to a class template, we can reshape it back to the original image dimensions ($32 \times 32 \times 3$) and visualize it. The resulting image highlights which pixel patterns contribute positively (bright regions) or negatively (dark regions) to the class score. For a well-trained linear classifier, these templates often resemble blurry, averaged versions of the class instances.

13.21 Limitations of Linear Classifiers on Images

Linear classifiers suffer from fundamental limitations when applied directly to raw pixel vectors:

- **No translation invariance**: Shifting an object by one pixel produces a completely different input vector, and the template $\mathbf{w}_j$ must learn to detect the object at every possible position independently.
- **No spatial hierarchy**: The model cannot exploit local correlations or compose simple features (edges) into complex ones (object parts).
- **High parameter count for larger images**: Scaling to higher resolutions (e.g., 224×224) would require millions of parameters, leading to severe overfitting.

These shortcomings motivate the development of Convolutional Neural Networks (CNNs), which preserve spatial structure, share parameters across locations, and learn hierarchical representations.

13.22 Two Perspectives on a Trained Linear Classifier

After training a neural network consisting of a flattened input layer and a softmax output layer (i.e., a linear classifier), we can interpret what the network has learned from two complementary viewpoints.

13.22.1 Geometric Interpretation: Hyperplanes in High Dimensions

The score for class j is given by:

$$s_j = \mathbf{w}_j^\top \mathbf{x} + b_j,$$

where $\mathbf{x} \in \mathbb{R}^D$ is the flattened image and $\mathbf{w}_j \in \mathbb{R}^D$ is the weight vector for class j.

For a two-dimensional input ($D = 2$), this equation describes a straight line in the plane (x_1, x_2). Points on one side of the line yield positive scores; points on the other yield negative scores. In D dimensions, each class score defines a **hyperplane**. The linear classifier partitions the D-dimensional space into L regions, each associated with one class. An input image is classified according to which hyperplane yields the largest score.

This geometric view reveals a fundamental challenge: the classifier attempts to separate clusters of high-dimensional points with linear decision boundaries. For complex data like natural images, such linear separation is rarely sufficient.

13.22.2 Template Matching Interpretation: Learned Weights as Images

A more intuitive perspective arises from reshaping each weight vector $\mathbf{w}_j \in \mathbb{R}^D$ back into an image of the original dimensions $H \times W \times C$. This is achieved by reversing the flattening operation.

When we compute the score $s_j = \mathbf{w}_j^\top \mathbf{x}$, we are effectively computing the **correlation** (inner product) between the input image and the learned template $\mathbf{w}_j$:

$$s_j = \sum_{c=1}^{C} \sum_{r=1}^{H} \sum_{c'=1}^{W} W_j(r, c', c) \cdot I(r, c', c).$$

The score is large when the input image aligns well with the template, and small (or negative) otherwise. The class with the largest score corresponds to the template that most closely matches the input.

13.23 Visualizing Learned Templates from CIFAR-10

When we train a linear classifier on CIFAR-10 and visualize the reshaped weight vectors, we obtain images such as those shown in Figure **??**. Several patterns emerge:

- **Plane, ship, boat**: Dominated by blue backgrounds (sky or sea).
- **Deer, bird**: Dominated by green backgrounds (grass or trees).
- **Car**: A reddish blob, reflecting the prevalence of red cars in the training set.
- **Horse**: A peculiar "two-headed" shape, because horses in the dataset appear facing both left and right.

These templates are learned *jointly* to maximize the correlation with images of the correct class while minimizing correlation with images of other classes. The weights can be positive (encouraging certain colors or shapes) or negative (penalizing features associated with other classes).

13.23.1 Implications for Data Augmentation

The two-headed horse template illustrates a crucial principle: if we want the classifier to become **invariant** to a transformation (e.g., left/right orientation), we must provide training examples exhibiting that variation. Otherwise, the network will learn a template that averages over the available poses, potentially losing discriminative detail. This insight motivates the use of **data augmentation**—artificially expanding the training set with transformed versions of images—to encourage desired invariances.

13.24 Why Image Classification is Challenging

Despite the simplicity of the linear classifier, applying it to real-world images reveals several fundamental difficulties.

13.24.1 High Dimensionality

CIFAR-10 images, though tiny ($32 \times 32 \times 3$), yield 3072-dimensional input vectors. At the time of its release, fewer than 8% of datasets in the UCI Machine Learning Repository had such high dimensionality. Larger images (e.g., 224×224) would be completely intractable for a linear classifier due to the quadratic growth in parameters.

13.24.2 Semantic Richness and Label Ambiguity

Assigning a single label to an image often ignores other objects present. For instance, an image labeled "man" may also contain a glass of beer. The classifier is penalized for producing any evidence of the "beer" class, creating conflicting signals during training.

13.24.3 Visual Transformations

Images of the same semantic content can vary drastically due to:

- Illumination changes (brightness, contrast, shadows).
- Viewpoint and pose variations.
- Deformations and occlusions.
- Background clutter.

From the geometric perspective, these variations map semantically identical content to widely separated points in the pixel space. A single hyperplane cannot effectively group such dispersed points.

13.24.4 The Failure of Euclidean Distance for Images

Many classical classifiers (e.g., K-nearest neighbors) rely on a notion of distance between data points, typically Euclidean distance:

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{\sum_{i=1}^{D} (x_i - y_i)^2}.$$

When applied to flattened images, this distance correlates poorly with semantic similarity. As illustrated in Figure **??**, an image can be darkened, shifted, or partially obscured. The Euclidean distance to the original may be identical in all three cases, yet the semantic content is preserved in the first two and severely degraded in the third.

A t-SNE visualization of CIFAR-10 using Euclidean distance confirms this: images of the same class do not form coherent clusters; a frog may be closer to a truck than to another frog. A classifier based on such a distance metric is doomed to fail.

13.25 Conclusion: The Need for Convolutional Neural Networks

The linear classifier on flattened pixels is:

- Computationally expensive for all but the tiniest images.
- Incapable of capturing translation invariance.
- Reliant on a distance metric (Euclidean) that does not reflect image semantics.

These limitations motivate the development of **Convolutional Neural Networks (CNNs)**. CNNs preserve the spatial structure of images, exploit local connectivity and weight sharing to drastically reduce parameters, and learn hierarchical features that are robust to translations and other transformations. In the next lecture, we will introduce the convolutional layer and see how it addresses these challenges.

Chapter 14

Convolutional Neural Networks

14.1 From Handcrafted Features to Learned Representations

14.1.1 The Classical Image Classification Pipeline

- Images are first mapped to a **feature vector** via a handcrafted feature extraction step.
- Example: a 2D vector containing height and area of the object.
- This approach circumvents challenges of working directly in high-dimensional pixel space:
  - Invariance to translation, scale, etc.
  - Difficulties in measuring distances between raw images.
- A **data-driven classifier** (e.g., a neural network) then operates on this low-dimensional feature vector.
- The pipeline: `Image` → `Handcrafted Features` → `Classifier` → `Class label`.

14.1.2 Advantages and Limitations

Advantages:

- Leverages domain expertise; features are interpretable and controllable.
- Works in a low-dimensional feature space → requires fewer training samples.
- Direct ability to improve the system by refining feature extraction code.

Limitations:

- Heavy design and programming effort; not scalable.
- Some distinctions (e.g., "car vs. bus") may be impossible to capture with handcrafted rules.
- The problem shifts to: can you write a program that extracts discriminative feature numbers for arbitrary classes?

14.1.3 The Deep Learning Shift

- Deep learning brings **learning into the feature extraction step** itself.
- Convolutional Neural Networks (CNNs) are best understood as making the feature extractor **data-driven**.
- The whole pipeline becomes a single neural network:

`Image` → `Learned Feature Extractor` → `Classifier`

- No handcrafted expert knowledge needed; the system learns to extract optimal features for the task.
- The two blocks (feature extraction and classification) are **trained simultaneously** via backpropagation using the classification loss.

14.2 Recap of the Convolution Operation

14.2.1 Definition and Visual Interpretation

- Convolution (correlation) at a pixel: multiply each pixel in a local neighbourhood by a corresponding filter weight, then sum.
- For a single-channel image, the operation is defined over a sliding window.

14.2.2 Filter Effects (Single-Channel Examples)

- **Identity filter**: centre weight = 1, all others = 0. Output equals input (no change).
- **Averaging (box) filter**: uniform weights. Replaces each pixel by the mean of its neighbours → blurring. Larger filter → stronger blur.
- **Diagonal motion blur**: filter with non-zero weights along a diagonal. Result mixes pixel values along that diagonal, simulating motion blur.
- **Template matching**: the operation can be viewed as a similarity measure. A high response occurs where the image patch matches the filter pattern.

14.3 Convolutional Neural Networks: From Vectors to Volumes

14.3.1 Multilayer Perceptrons vs. CNNs

- **MLP**: input is a vector $\mathbf{x}$. Hidden layers compute $\mathbf{W}\mathbf{x} + \mathbf{b}$, producing vector activations. The underlying mathematical model is entirely vector-based.
- **CNN**: input is a **3D volume** (height × width × channels, e.g., RGB). Activations throughout the network remain **3D tensors** that preserve spatial structure.

14.3.2 Volumetric Representation of Neurons

- In a CNN layer, neurons are organized in a **3D grid**: height, width, and depth (channels).
- Each neuron holds a single scalar value (its activation).
- Input layer: the raw pixel intensities form the first 3D activation volume.
- Output of a convolutional layer: a new 3D volume, with spatial dimensions determined by the filter size, stride, etc.
- Thus, CNNs inherently maintain the spatial organisation of the input throughout the feature extraction hierarchy.

14.4 CNN Architecture: From Volumes to Vectors

14.4.1 Spatial Reduction and Channel Expansion

- Input: an image volume of size $H \times W \times C$ (e.g., $28 \times 28 \times 3$).
- As the volume progresses through the network:
  - Spatial dimensions (H, W) gradually shrink.
  - Number of channels (C) increases.
- Example progression: $28 \times 28 \times 3 \;\rightarrow\; 28 \times 28 \times 8 \;\rightarrow\; 14 \times 14 \times 16 \;\rightarrow\; \ldots$
- Eventually, the spatial extent is reduced to a single vector (e.g., via flattening), which is passed to a standard MLP classifier.
- Intuition: the volume is *squeezed* spatially while being *stretched* in depth (channels), transforming raw pixels into a compact feature vector.

14.4.2 Three Core Operations

A CNN is built from only three types of layers:

1. **Convolutional layer** (new)
2. **Non-linearity** (already known, e.g., ReLU)
3. **Pooling layer** (new)

The convolutional layer is the heart of the feature extraction process.

14.5 The Convolutional Layer

14.5.1 Local Connectivity and Full Depth

- Operates between an input volume and an output volume.
- For a single output neuron at spatial location (r, c) of an output channel:
  - It looks at a **small spatial neighbourhood** in the input, centred at the same (r, c).
  - It spans **all channels** of the input (full depth).
- The neighbourhood is defined by a spatial displacement set (u, v), e.g., $u, v \in \{-1, 0, 1\}$ for a 3×3 filter.
- The operation is a **linear combination** of the input values in that local 3D patch, plus a bias.

14.5.2 Mathematical Definition (Single Output Channel)

Let the input volume be I (size $H \times W \times C_{\text{in}}$). For the first output channel, the activation at position (r, c) is:

$$A_1(r, c) = \sum_{(u,v) \in \text{neighborhood}} \sum_{k=1}^{C_{\text{in}}} W_1(u, v, k)\; I(r + u,\; c + v,\; k) \;+\; b_1$$

- W_1: the **filter** (or kernel) for output channel 1.
- b_1: scalar bias for channel 1.
- The same W_1 and b_1 are applied at every spatial location (r, c) – **weight sharing**.
- This is exactly a 2D correlation (cross-correlation) with a filter that extends through the full depth of the input.

14.5.3 Filter Dimensions and Hyperparameters

- **Spatial extent** (e.g., 3×3, 5×5, 11×11): a hyperparameter of the layer.
- **Depth**: **must equal the number of input channels** C_{in}. This is not a free parameter; it is determined by the previous layer.
- Thus, the filter W_1 has dimensions $\mathrm{K}_h \times \mathrm{K}_w \times C_{\text{in}}$, containing $K_h \cdot K_w \cdot C_{\text{in}}$ learnable weights.
- One bias per output channel.

14.5.4 Multiple Output Channels

- To produce a full output volume with C_{out} channels, we need **one independent filter** per output channel.
- Output channel j is computed as:

$$A_j(r,c) = \sum_{u,v} \sum_{k=1}^{C_{\text{in}}} W_j(u,v,k)\, I(r+u, c+v, k) \;+\; b_j$$

- The complete set of filters forms a 4D tensor: $C_{\text{out}} \times K_h \times K_w \times C_{\text{in}}$.
- Each output channel is a 2D slice; stacking them gives the output volume of size $H' \times W' \times C_{\text{out}}$.

14.6 Visualizing Learned Filters

- Filters from the first layer (applied to RGB input) can be interpreted as small colour images.
- Example: AlexNet's first layer filters of size $11 \times 11 \times 3$; each filter can be displayed.
- Visualisation reveals patterns the network learns to detect (e.g., oriented edges, colour blobs).
- The output activation map for a given filter shows where in the image that pattern produces a high response (template matching behaviour).
- Note: visualisation often requires scaling/rescaling because filter weights can be both positive and negative, while image display expects non-negative values.

14.6.1 Interpreting Activation Maps

- The output of a convolutional layer is a volume of real-valued activations, not limited to positive values.
- In visualisations:
  - Zero response $\rightarrow$ grey.
  - Positive response $\rightarrow$ white.
  - Negative response $\rightarrow$ black.
- Example (salamander tail): a filter with blue/orange patterns produces strong positive activations where the image matches that pattern (yellow/black), and negative activations where the opposite contrast occurs.
- This reinforces the **template matching** interpretation: neurons activate most where the local image patch correlates strongly (positively or negatively) with the learned filter.

14.6.2 Multiple Filters: The Output Volume Depth

- A convolutional layer can have more than one filter.
- The number of filters, denoted C_{out}, is a hyperparameter.
- Each filter is independent and produces its own 2D activation map (one channel of the output volume).
- All filters in a layer share the same spatial dimensions (kernel size), but their weights are distinct and learnable.
- Thus, with C_{out} filters, the output volume has depth C_{out}.
- Different filters learn to detect different patterns (e.g., edges of various orientations, colour transitions).

14.6.3 Parameter Count of a Convolutional Layer

- Let $K_h \times K_w$ be the spatial size of each filter, C_{in} the number of input channels, and C_{out} the number of filters.
- Each filter contains $K_h \cdot K_w \cdot C_{\text{in}}$ weights.
- One scalar bias per filter.
- Total learnable parameters:
$$C_{\text{out}} \times (K_h \cdot K_w \cdot C_{\text{in}} + 1)$$
- Example: 5×5 filter, 3 input channels, 2 filters $\rightarrow 2 \times (25 \times 3 + 1) = 152$ parameters.
- The number of parameters is **independent of the input spatial size** — a key efficiency property.

14.6.4 Important Constraints

- The depth of each filter is **not a hyperparameter**; it is forced to match the number of input channels C_{in}.
- Consequently, the same layer definition (e.g., `Conv2d(3, 8, 3)`) placed at different depths in a network will have a different number of parameters because C_{in} changes.

14.6.5 Padding and Stride

Padding

- Zero-padding is added around the spatial borders of the input volume.
- "Same" padding: add enough zeros to keep the output spatial size equal to the input size.
- Without padding, the output shrinks: a $K \times K$ filter on a $H \times W$ input yields $(H-K+1) \times (W-K+1)$ output.

Stride

- The filter normally shifts by one pixel at a time (stride 1).
- A stride $S > 1$ means the filter is applied only every S pixels (both horizontally and vertically).
- Effect: spatial dimensions are reduced by a factor of approximately S, reducing computational load and the volume size.
- Example: input 5×5, 3×3 filter, stride 2 with appropriate padding yields 3×3 output.

14.6.6 Convolution vs. Cross-Correlation in Deep Learning

- In signal processing, **convolution** involves flipping the kernel: $I * W = \sum_{u,v} I(r-u, c-v)\, W(u,v)$.
- In neural network frameworks, what is implemented is actually **cross-correlation**: $\sum_{u,v} I(r+u, c+v)\, W(u,v)$ (no flip).
- The distinction is irrelevant for learning because the filters are not fixed — if a flipped kernel were needed, the network would simply learn the flipped weights.
- As long as the operation is used consistently, calling it "convolution" is an accepted abuse of terminology.

14.6.7 Channel Mixing vs. Per-Channel Convolution

- In classic image processing (e.g., blurring), convolution is applied to each channel independently, producing an output with the same number of channels.
- In CNNs, **each filter mixes all input channels** via summation: the dot product is computed over the full depth, yielding a single output channel per filter.
- Therefore, a convolutional layer with C_{out} filters transforms a $H \times W \times C_{\text{in}}$ volume into a $H' \times W' \times C_{\text{out}}$ volume.

14.7 Non-linearity (Activation Functions)

14.7.1 Why Non-linearity is Necessary

- The convolution operation is linear; cascading multiple linear layers would collapse into a single equivalent linear transformation.
- To build a deep, expressive model, a non-linear function must be applied after each convolutional layer.

14.7.2 Common Activation Functions

- **ReLU (Rectified Linear Unit)**: $\sigma(x) = \max(0, x)$.
  - Thresholds negative values to zero, keeps positives unchanged.
  - Most popular in CNNs due to simplicity and effectiveness.
- Other options: hyperbolic tangent (tanh), sigmoid, etc.
- The activation is applied **element-wise** to every neuron in the volume (each pixel, each channel).

14.7.3 Effect of ReLU Visualisation

- Negative activations are discarded (become zero); only positive responses propagate forward.
- This means only regions where the filter had a positive correlation with the input continue to influence later layers.

14.8 Pooling Layers

14.8.1 Purpose

- Systematically reduce spatial dimensions (H, W) while preserving channel depth.
- Achieve further data condensation and computational savings.

14.8.2 Max Pooling

- Operates **channel-wise** (independently on each feature map).
- Typical configuration: 2×2 window, stride 2 (non-overlapping blocks).
- Operation: partition the input into blocks; for each block, output only the **maximum** value.
- Retains the strongest activations while discarding exact spatial location within the block.
- Generalises to larger windows and strides (e.g., size 4 stride 4 reduces dimensions by factor 4).
- Can be seen as a form of downsampling.

14.9 A Complete CNN Architecture

14.9.1 Example Pipeline

- Input: $28 \times 28 \times 3$ image.
- Conv layer with 8 filters (output: $28 \times 28 \times 8$) + ReLU.
- Max pooling size 2 (output: $14 \times 14 \times 8$).
- Conv layer with 24 filters (output: $14 \times 14 \times 24$) + ReLU.
- Max pooling size 4 (output: $4 \times 4 \times 24$).
- **Flatten**: reshape volume into a vector of length $4 \times 4 \times 24 = 384$.
- **Dense (Fully Connected) layers**:
  - Hidden layer: 256 neurons.
  - Output layer: 128 neurons (corresponding to 128 classes).

14.9.2 Role of Each Block

- Convolution + ReLU + Pooling: **data-driven feature extraction**, mapping raw pixels to a compact, informative feature vector.
- The output vector (e.g., 384-dimensional) is the **latent representation** of the input image.
- Flattening: the transition from 3D feature volumes to a 1D vector, after spatial dimensions have been sufficiently reduced.
- Dense layers: a standard MLP classifier operating on the learned features.

14.10 Visualising Internal Representations

- In early layers, feature maps retain visible structure (e.g., car shape, edges) because each channel is a 2D correlation with a filter.
- As we progress deeper:
  - Spatial resolution decreases (pooling).
  - Number of channels increases (more filters).
  - Each pixel in a deep layer arises from mixing all previous channels over a local spatial window.
- Visualisation: deep layers can be displayed, but "pixels" correspond to larger regions of the original input due to receptive field growth.
- Example: a 6-layer ConvNet with 10 filters per layer, final spatial size 4×4, yields a 160-D vector.

14.11 Semantic Clustering in Latent Space

14.11.1 Motivation

- Directly comparing images in pixel space (e.g., Euclidean distance of flattened pixels) fails: semantically different images can be close, and similar objects can be far due to pose, lighting, etc.
- Instead, compare images using their **latent representations** (the output vector from the convolutional block).

14.11.2 Properties of a Good Latent Space

- After training for classification, the network organises the latent space so that images of the same class cluster together, regardless of low-level variations.
- Visual example:
  - Landscapes, red cars, coffee machines, TV screens, yellow cars each form distinct clusters.
  - Intra-class distances are small, inter-class distances are large.
- This makes the final classification task easy for the MLP — a linear decision boundary can separate the clusters.

14.11.3 Why This Happens

- The latent features are not accidental; they are **optimised** end-to-end to minimise classification error.
- The attached dense layers impose a training signal (categorical cross-entropy loss) that forces the feature extractor to produce linearly separable representations.
- If the network classifies correctly, the latent space *must* be structured in this semantically meaningful way.

14.12 Conclusion and Next Lecture

- CNNs combine learnable convolution, non-linearity, pooling, and finally dense layers to directly map images to class predictions.
- The convolutional block is a trainable feature extractor; the dense block is a trainable classifier.
- In the next lecture: history of the first CNNs and further properties of training these models.

14.13 CIFAR-10 Dataset

14.13.1 Overview

- Classic image classification dataset with **10 classes**: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.
- Images are **32×32 pixels, RGB** (3 channels).
- 50,000 training images, 10,000 test images.
- Balanced: 5,000 images per class in the training set.

14.13.2 Loading with TorchVision

- Use `torchvision.datasets.CIFAR10` with `train=True` (training) or `train=False` (test).
- Returns an array of shape `(N, H, W, C)`: `N` samples, height 32, width 32, channels 3.
- Labels are integers from 0 to 9.

14.14 Data Preprocessing

- Pixel values are integers in $[0, 255]$. Normalize to $[0, 1]$ by dividing by 255 (applied to both training and test sets).
- Further split training set to obtain a validation set: 10% held out → **45,000 training, 5,000 validation**.
- Wrap data in `TensorDataset`, then create `DataLoader` objects.
- Batch size: 256.
  - Not too small (inefficient), not too large (too few batches per epoch with only 45k samples).
  - Larger batch size trains faster but may reduce gradient noise.
- DataLoader parameters: `shuffle=True` for training, `drop_last=True` to discard incomplete final batch if needed.
- Example batch shape: `(256, 3, 32, 32)` for images, `(256,)` for labels.

14.15 Convolutional Neural Networks – Core Operations

14.15.1 2D Convolution

- A convolution applies a learnable **kernel** (filter) to local regions of the input.
- Output pixel = dot product between kernel and input patch → high value when the patch matches the kernel (template matching).
- Example: Laplacian filter extracts edges (high response where central pixel differs from neighbours).
- The kernel is a 3D tensor: spatial dimensions $K_h \times K_w$ and depth equal to input channels C_{in}.
- One filter produces one channel of output; a layer has C_{out} filters, so output volume depth = C_{out}.

14.15.2 Hyperparameters of a Convolutional Layer

- **Kernel size** ($F_h \times F_w$): spatial extent of the filter (typically odd, e.g., 3×3, 5×5).
- **Stride** (S): step size when sliding the filter. $S > 1$ reduces spatial dimensions.
- **Padding** (P): number of zeros added to borders. "Same" padding ensures output spatial size equals input size (for $S = 1$).
- **Number of filters** (C_{out}): determines output depth.

14.15.3 Number of Parameters

- One filter has $F_h \times F_w \times C_{\text{in}}$ weights plus 1 bias.
- For C_{out} filters, total learnable parameters:

$$\text{Params} = C_{\text{out}} \times (F_h \cdot F_w \cdot C_{\text{in}} + 1)$$

- Note: The input channels C_{in} are determined by the previous layer; this is *not* a hyperparameter.

14.15.4 Output Spatial Dimensions

Given input height H_{in}, width W_{in}, padding P, kernel size F, stride S:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - F}{S} \right\rfloor + 1, \qquad W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - F}{S} \right\rfloor + 1$$

- Both formulas are fundamental for exam exercises.
- Output depth is simply C_{out}.

14.16 Pooling Layers

- Downsampling operation that reduces spatial dimensions while preserving depth.
- Operates **independently on each channel**.
- **Max Pooling**: output the maximum value in each window.
  - Non-linear, retains strongest activations.
  - Typical configuration: pool size 2×2, stride 2 (halves spatial size).
- **Average Pooling**: output the mean. Linear operation, less common.
- Stride is usually set equal to the pooling window size.

14.17 Flattening

- After several convolutional + pooling blocks, the volume is reshaped into a 1D vector.
- This bridges the convolutional backbone to the final **Multi-Layer Perceptron (MLP)** classifier.
- Example: volume $(H, W, C) \rightarrow$ vector of length $H \cdot W \cdot C$.

14.18 Designing a Simple CNN Architecture

14.18.1 Backbone and Classification Head

- Standard structure: **Convolutional Backbone** (feature extractor) followed by a **Classification Head** (MLP).
- Backbone: sequence of convolutional blocks + downsampling + flatten.
- Head: one or two fully-connected layers outputting class probabilities.

14.18.2 Building Blocks

A typical **convolutional block** consists of:

1. `Conv2d` with 3×3 kernels (preserves spatial size if padding=1).
2. ReLU activation.
3. Downsampling step.

14.18.3 Downsampling Strategies

Two common ways to reduce spatial dimensions:

1. **Max Pooling** (appended after the convolution + ReLU).
2. **Strided Convolutions**: use a convolution with stride > 1 instead of a separate pooling layer.
   - More efficient: combines convolution and downsampling into one layer.
   - Input size: $C_{\text{in}} \times H \times W$.
   - Output size: $C_{\text{out}} \times \frac{H}{d} \times \frac{W}{d}$, where d is the downsampling factor (e.g., 2).
   - Usually C_{out} is increased (e.g., doubled) to compensate for spatial reduction, maintaining representational capacity.

14.18.4 Typical Progression

- Each block halves spatial resolution and doubles channels (if using stride 2 and $C_{\text{out}} = 2 \times C_{\text{in}}$).
- This mirrors the pattern seen in the earlier theoretical lecture: *spatial shrinkage, channel expansion*.

14.19 Kernel Size: Trade-offs

14.19.1 Small Kernels (e.g., 3×3)

- **Fewer parameters** – efficient and less prone to overfitting.
- Excellent at capturing **low-level features** (edges, corners, textures) because they examine small local regions.
- Require **deeper architectures** to aggregate low-level patterns into high-level concepts.
- Stacking many small-kernel layers builds a hierarchy of features.

14.19.2 Large Kernels (e.g., 7×7, 11×11)

- Can detect **broader, more complex templates** (e.g., a wheel) in a single layer.
- Drastically more parameters $\rightarrow$ risk of overfitting.
- May overlook fine-grained details (corners).
- Allow **shallower networks** – computationally lighter, but with less representational depth.
- In modern practice, **small kernels dominate**; large kernels are occasionally used in the first layer.

14.20 Receptive Field Expansion via Stacking

- Two consecutive 3×3 convolutions (with stride 1, no pooling) give an effective receptive field of 5×5.
- Three such layers cover 7×7, and so on.
- This enables the network to learn the same spatial coverage as a large kernel, but with **fewer parameters** and **more non-linearities** (ReLUs between layers).
- Hence, many architectures simply repeat 3×3 convolutions to build complex feature extractors.

14.21 Downsampling Strategies

14.21.1 Max Pooling

- Fixed, non-trainable operation: takes the maximum over a window (typically 2×2, stride 2).
- Non-linear, simple, reduces spatial dimensions by a factor equal to the stride.
- Loses exact location information, but retains strong activations.

14.21.2 Strided Convolutions

- Convolution with `stride > 1` replaces the combination "conv + pooling" in a **single learnable layer**.
- More flexible (the downsampling kernel is trained), but introduces additional parameters.
- Often more computationally efficient because it merges two operations.
- Typical pattern: in a block, the last convolution uses stride 2 to halve spatial size while increasing channels.

Choice: depends on the task; both are widely used, with modern networks favouring strided convolutions for compactness.

14.22 Example: A Simple CNN for CIFAR-10

14.22.1 Architecture

- Input: $3 \times 32 \times 32$ (RGB, CIFAR-10 images).
- **Two convolutional blocks**:
  - Block 1: `Conv2d(3, 32, kernel=3, padding=1)` $\rightarrow$ ReLU $\rightarrow$ `MaxPool2d(2, 2)` $\rightarrow$ output $32 \times 16 \times 16$.
  - Block 2: `Conv2d(32, 64, kernel=3, padding=1)` $\rightarrow$ ReLU $\rightarrow$ `MaxPool2d(2, 2)` $\rightarrow$ output $64 \times 8 \times 8$.
- **Flatten** the $64 \times 8 \times 8$ volume into a vector of length $64 \cdot 8 \cdot 8 = 4096$.
- **Classification head**: a single fully-connected layer `Linear(4096, 10)`.
- Total output: logits for 10 classes.

14.22.2 Parameter Count

- **Block 1**: $3 \times 3 \times 3 \times 32 + 32 = 896$ parameters.
- **Block 2**: $3 \times 3 \times 32 \times 64 + 64 = 18{,}496$ parameters.
- **Fully-connected**: $4096 \times 10 + 10 = 40{,}970$ parameters.
- Total: $\approx 60{,}000$ parameters.
- Note: the majority of parameters reside in the final dense layer – a common trait of small CNNs.
- Convolutional layers are **parameter-efficient** because the same filters are reused across all spatial positions (weight sharing).

14.23 PyTorch Implementation Details

14.23.1 Defining Convolutional and Pooling Layers

- `nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`
  - To preserve spatial size (same output shape): set `padding = (kernel_size - 1) // 2` (valid for odd kernel).
  - `dilation` (optional) controls spacing between kernel elements; used in segmentation, not here.
- `nn.MaxPool2d(kernel_size, stride=None)` – if stride is `None`, it defaults to `kernel_size`.

14.23.2 Vanilla Block Construction

A `VanillaCNNBlock` encapsulates:

1. A sequence of 3×3 convolutions (default 1 per block). If multiple, they are stacked to simulate a larger kernel, with ReLUs after each.
2. Downsampling method:
   - **Strided convolution**: the last convolution in the block has stride > 1 (e.g., 2). This replaces pooling.
   - **Max pooling**: a final `MaxPool2d` is appended after the convolutions (stride 1 convs kept).
3. Output: a volume with reduced spatial size and (optionally) doubled channels.

14.23.3 Full Network Class

- `CNN` class receives `input_shape`, `num_classes`, `dropout`, `num_blocks`, `num_convs_per_block`, `use_stride`, etc.
- **Feature extractor**: `nn.Sequential` of `VanillaCNNBlock`s.
- **Classifier**: `nn.Flatten()` → `nn.Dropout(p=dropout)` → `nn.Linear(features_dim, num_classes)`.
- `forward(x)`: extract features, then classify.
- Softmax is **not** included in the forward pass; it is applied internally by `CrossEntropyLoss`.

14.24 Training Loop Configuration

14.24.1 Hyperparameters

- **Loss**: Cross-entropy (suitable for multi-class classification).
- **Optimizer**: `AdamW` with weight decay set to 0 → effectively `Adam` (L2 regularisation disabled initially; can be enabled).
- **Learning rate**: 10^{-3} (standard starting point).
- **Batch size**: 256 (balances speed and gradient stability on 45k training samples).
- **Maximum epochs**: 1000, with **early stopping** patience 50 (example uses 20 for faster experimentation).
- **Dropout**: applied before the final linear layer; rate can be tuned.
- **Metrics**: F1 score (macro) tracked for validation; early stopping monitors this metric.

14.24.2 Training and Validation Steps

- `train_epoch`: model in `train()` mode, gradients enabled. For each batch:
  1. Forward pass (logits computed).
  2. Cross-entropy loss.
  3. (Optional) L2 penalty added to loss, if weight decay not used in optimizer.
  4. Backward pass and optimizer step.
- `val_epoch`: model in `eval()` mode, wrapped in `torch.no_grad()`. Same forward pass, loss and metrics calculated without weight updates.
- `fit` function: runs epochs, logs to TensorBoard, checks early stopping condition, restores best model weights at the end.

14.25 Notes on the Input Shape

- Unlike previous tabular or time-series tasks, image batches are 4-dimensional tensors: `(batch_size, channels, height, width)`.
- All other training logic (loop, logging, early stopping) remains identical.
- This modularity allows reusing the same training infrastructure across different data modalities.

14.26 Experimental Results with Simple CNNs on CIFAR-10

14.26.1 Baseline: 2-Block CNN

- Architecture: 2 blocks, each with one 3×3 convolution + max pooling; classifier on 4096-dim vector.
- Parameters: $\approx$ 60k.
- Training: 1000 epochs, patience 20, Adam ($lr = 10^{-3}$), no explicit regularization.
- **Validation**: accuracy $\approx$ 74%, F1 score similar.
- Learning curve: after epoch $\approx$ 40, validation loss flattens while training loss continues to fall → **overfitting**.
- The model is too simple; struggles to learn complex patterns and still overfits the training set.

14.26.2 Deeper Architecture: 3-Block CNN

- Increased depth: 3 blocks, each with *two* 3×3 convolutions (effective 5×5 receptive field), max pooling.
- Channel progression: 32, 64, 128; final feature vector size $128 \times 4 \times 4 = 2048$.
- Parameters: $\approx$ 300k (large increase from the last conv block: $9\times64\times128+128 \approx 73$k per filter group).
- Training: same hyperparameters.
- **Validation**: accuracy $\approx 80\%$, F1 score $\approx 80\%$.
- Overfitting occurs *much earlier* – around epoch 10.
- A deeper network learns more powerful features but with more parameters comes a higher tendency to overfit, especially without strong regularization.

14.26.3 Even Deeper: 4-Block CNN

- Added a fourth block, still two 3×3 convolutions per block, max pooling.
- Final spatial size: 2×2 with 256 channels $\rightarrow$ feature vector length 1024.
- Parameters: over **1 million** (large last conv layers: 295k and 590k parameters).
- **Validation**: similar to 3-block network, $\approx 80\%$ F1 score.
- Overfitting starts even earlier (within first 10 epochs).
- Increasing model capacity without corresponding increase in data or regularization yields diminishing returns and severe overfitting.
- Mentioned future remedies: **data augmentation**, **transfer learning** (to be covered later).

14.27 Visualizing Learned Features

- **First layer** (closest to input): filters act as edge and corner detectors. Feature maps are interpretable and highlight object contours.
- **Deeper layers**: activations become progressively less interpretable – spatial resolution shrinks (down to 8×8 or 4×4), and each pixel is a complex, nonlinear combination of many lower-level features.
- Despite the loss of visual interpretability, deeper representations capture more abstract, class-discriminative patterns.
- Central pixels often show higher activations, reflecting that objects tend to be centered in CIFAR-10 images.

14.28 Comparison of Architectures

- **Training F1**: 2-block CNN learns slowly; 3-block and 4-block learn much faster initially, but plateau early due to overfitting.
- **Validation loss**: simplest network has higher final loss but overfits later; deeper networks reach lower loss values before overfitting quickly.
- Conclusion: deeper models are more expressive but need careful regularization to prevent memorisation of the training set.

14.29 Regularization and Training Strategies

14.29.1 Strided Convolutions vs. Max Pooling

- Replaced max pooling with strided convolutions (stride 2) in a 3-block network.
- Result: performance *worsened* (validation F1 dropped).
- Reason: max pooling is a **non-linear** downsampling; a strided convolution is a linear operation (followed by ReLU, but still less effective in this setting). Learnable downsampling is more flexible but removes the strong inductive bias of taking the maximum activation.
- Despite theoretical compactness, max pooling often remains preferable in small-scale settings.

14.29.2 Weight Decay

- Added L2 regularization (weight decay = 10^{-3}) to the 3-block network.
- Observed a slight improvement ($\approx +1\%$ F1).
- Indicates that overfitting can be mitigated by penalising large weights.
- Scope for further tuning: stronger weight decay, more blocks with heavy regularisation, or data augmentation.

14.30 Test Set Evaluation

- Selected best model (3-block CNN) based on validation F1.
- Test accuracy: 78% – consistent with validation performance, confirming no significant overfitting to the validation split.
- Confusion matrix shows typical confusion between visually similar classes (e.g., cat vs. dog), partly due to the very low resolution (32×32).

14.31 Outlook

- The techniques explored are fundamental; the next challenge will be on **image classification**, directly applying these concepts.
- Convolutions are also applicable to **1D sequences** (time series) using `Conv1d`, and **3D volumes** (`Conv3d`).
- Future topics will cover advanced regularisation (data augmentation) and transfer learning to tackle overfitting in deeper architectures.

Chapter 15

CNN anatomy and training under data scarcity

15.1 Deep Dive into CNN Anatomy

15.1.1 Recap of the Convolutional Layer

- A convolutional layer has learnable **filters** $W_j(u, v, k)$ (small spatial extent, full depth) and a **bias** b_j per filter.
- Each filter produces one output channel; the operation is a linear combination (dot product over a local 3D patch).
- Output volume dimensions: $H' \times W' \times C_{\text{out}}$ where C_{out} equals the number of filters.

15.1.2 Convolution as a Linear Transformation: The Matrix View

To understand the inductive biases of convolution, we express the operation as a matrix multiplication.

Flattening the Volumes

- Input volume X: size $H \times W \times C_{\text{in}}$.
- Flatten into a vector $\mathbf{x}$ of length $H \cdot W \cdot C_{\text{in}}$.
- Output volume A: size $H' \times W' \times C_{\text{out}}$.
- Flatten into a vector $\mathbf{a}$ of length $H' \cdot W' \cdot C_{\text{out}}$.

Dense (Fully-Connected) Layer Interpretation

If we treated the transformation as a standard dense layer:

$$\mathbf{a} = \mathbf{W}\mathbf{x} + \mathbf{b},$$

where $\mathbf{W}$ is a matrix of size $(H'W'C_{\text{out}}) \times (HWC_{\text{in}})$ and $\mathbf{b}$ has the same size as $\mathbf{a}$.

- Every output neuron is connected to **all** input neurons.
- For even a small image (e.g., 32×32 with 100 channels):
  - Input size: $32 \cdot 32 \cdot 100 \approx 10^5$.
  - Output size: similar order.
  - $\mathbf{W}$ would have $\approx 10^{10}$ parameters – completely unaffordable.
- Bias vector would also have one parameter per output neuron.

Sparse Connectivity in a Convolutional Layer

Now view a single-channel convolution (no depth for simplicity) as a matrix product:

- Input image X $(H \times W)$ flattened to $\mathbf{x}$; output A $(H \times W)$ flattened to $\mathbf{a}$.
- The matrix $\mathbf{W}$ is square of size $(HW) \times (HW)$.
- For a 3×3 filter, each output pixel depends on only **9 input pixels** (those in its local neighbourhood).
- Therefore, each row of $\mathbf{W}$ contains only **9 non-zero entries**; the rest are zero.
- The matrix is **extremely sparse** – most connections are absent by design.

Weight Sharing (Parameter Tying)

- The same filter is applied at every spatial location.
- This means the non-zero entries in different rows of $\mathbf{W}$ are not independent; they are **repeated** (shifted according to the receptive field).
- The effective number of learnable parameters reduces dramatically:
  - Only $F_h \cdot F_w \cdot C_{\text{in}}$ weights per filter (plus one bias).
  - Total parameters: $C_{\text{out}} \times (F_h \cdot F_w \cdot C_{\text{in}} + 1)$, independent of the spatial dimensions.
- In matrix language: $\mathbf{W}$ has a block-Toeplitz structure with repeated entries.

15.1.3 Key Insight: Inductive Priors of Convolution

- Compared to a dense layer, a convolutional layer encodes two strong assumptions:
  1. **Locality**: features depend only on a small spatial neighbourhood.
  2. **Translation equivariance**: the same feature detector is reused everywhere (weight sharing).
- These priors drastically reduce the number of parameters, enabling deep processing of high-dimensional inputs (images) without overfitting.
- They also make the network **equivariant to translation**: shifting the input shifts the output activation map identically.

15.2 Consequences for Network Design and Training

- Even a modest input $(224 \times 224 \times 3)$ yields a huge vector when flattened; a fully-connected first layer would be impossible.
- Convolutional layers are what allow us to process images directly and stack many layers to build hierarchical representations.
- The sparsity and weight sharing are hard-coded architectural choices – they are **not learned**, which is why CNNs work so well with limited data compared to a hypothetical fully-connected vision model.

15.3 Looking Ahead

- This view will later help understand advanced topics: e.g., how to train with limited annotated data (data augmentation, transfer learning), and how to modify CNNs for different tasks.
- The matrix perspective also illuminates the computational efficiency: the sparse, structured matrix multiplication can be implemented as a highly optimised correlation operation (convolution).

15.4 Convolutional Layer as Structured Sparse Matrix Multiplication

15.4.1 Parameter Count in the Matrix View (Single-Channel Recap)

- For a single input and output channel, a convolution with a 3×3 filter can be expressed as a matrix multiplication $\mathbf{a} = \mathbf{W}\mathbf{x}$.
- Each row of $\mathbf{W}$ contains only 9 non-zero entries (the filter weights), resulting in extreme **sparsity**.
- Moreover, these 9 weights are **shared** across all rows – they are the same filter shifted spatially.
- Hence, the total number of learnable parameters (ignoring bias) is just 9, irrespective of the image size.
- Adding the bias: a convolutional bias is a **single scalar** per filter, not one per output neuron.

15.4.2 Extension to Multiple Channels

When the input has C_{in} channels and the layer has C_{out} filters:

- Each output pixel in channel j depends on a local spatial patch **across all** C_{in} input channels.
- In the flattened matrix $\mathbf{W}$ (size $(H'W'C_{\text{out}}) \times (HWC_{\text{in}})$), this translates to:
  - The row corresponding to an output pixel at channel j has non-zero entries only at positions linked to the spatial neighbourhood, replicated for each input channel.
  - The same filter weights $W_j(u, v, k)$ are reused for every spatial location, giving a **block-Toeplitz** structure with shared values.
- Different output channels (different filters) produce distinct blocks in $\mathbf{W}$.
  - Each block corresponds to one filter's weights and has the same sparse, repeated pattern, but with different numerical values.
- Overall, the matrix is **extremely sparse** and exhibits **weight sharing** both within each filter's spatial application and across channels.

15.4.3 Structure of the Bias Vector

- In a convolutional layer, each filter has **one bias** b_j.
- This bias is added to **all** spatial positions of the corresponding output channel.
- Consequently, the bias vector $\mathbf{b}$ (of length $H'W'C_{\text{out}}$) is **block-constant**:
  - The first $H'W'$ entries equal b_1,
  - the next $H'W'$ entries equal b_2,
  - and so on.
- This is in stark contrast to a dense layer, where each output neuron has its own independent bias.

15.5 Inductive Biases Encoded by Convolution

The structural constraints of a convolutional layer are not merely a way to reduce parameters; they impose strong **inductive biases** that are especially suited for image data.

15.5.1 Locality

- Only a small spatial neighbourhood of the input influences each output neuron.
- This reflects the natural property of images: pixels that are close together are more likely to be related.

15.5.2 Translation Equivariance (Parameter Sharing)

- The *same* filter is applied at every spatial position.
- If a pattern appears anywhere in the input, the same operation detects it, producing a corresponding activation at that location.
- Mathematically: shifting the input shifts the output feature map identically.
- The network does not need to learn separate detectors for the same pattern at different positions – a very useful prior when object positions are arbitrary.

15.5.3 Why These Biases Matter

- They drastically reduce the number of parameters, making learning feasible with limited data and computation.
- They guide the network towards solutions that *generalise* well on image tasks because they assume spatial structure that holds in real images.
- The same architecture would be **inappropriate** for data where spatial equivariance does not make sense.

15.6 When (Not) to Use 2D Convolution

- **Good fit**: natural images, spectrograms, or any data where local spatial patterns are meaningful and translation invariance is desired.
- **Bad fit**: generic tabular data arranged as a 2D matrix without inherent spatial correlation.
  - Example: stacking different feature vectors from multiple subjects as an "image". If features (weight, height, grade, postal code) have no spatial order, applying the same filter to weight *and* postal code is meaningless.
  - In such cases, a dense network or a graph-based model is more appropriate.
- **Multivariate time series**: each time step yields a feature vector. If we treat the matrix (time steps $\times$ features) as a 2D image and apply 2D convolution, the filter will mix features arbitrarily. This lacks a sensible inductive bias. Instead, use **1D convolution along the time axis**, keeping the feature dimension as channels.
- **Key principle**: choose a model architecture that aligns with the structure of your data. Not just the best-performing one – what *makes sense* will generalise better.

15.7 Receptive Field

- The **receptive field** of an activation (neuron) is the region in the input image that influences its value.
- Because convolution is local, a neuron in the first layer sees only a small patch (e.g., 3×3).
- Stacking convolutional layers (and pooling) causes the receptive field to grow; a neuron in a deeper layer can "see" a much larger portion of the original input.

- This concept is critical for designing architectures and understanding what information is available to the final classifier.
- Often, we refer specifically to the receptive field of the output neurons (e.g., the vector before the classifier) with respect to the input image.

15.8 Receptive Field

15.8.1 Definition

- The **receptive field** of an activation (neuron) is the spatial region in the input image that influences its value.
- Typically refers to the final feature vector (before the classifier), ignoring channels — only spatial extent matters.
- It tells *how large a patch* each feature "sees"; larger fields enable detection of more global patterns.

15.8.2 Computing the Receptive Field

- Receptive field grows as we stack convolutional and pooling layers.
- **Rule**: work *backward* from the output neuron to the input.
- **Simple example**: successive 3×3 convolutions (stride 1, no pooling):
  - Layer L: 3×3
  - Layer $L-1$: 5×5
  - Layer $L-2$: 7×7
  - ... every new 3×3 conv adds 2 to the spatial size.
- **Effect of pooling / stride**:
  - Max pooling with 2×2 window and stride 2 also expands the receptive field because it downsamples.
  - Example: 2×2 pooling $\rightarrow 4 \times 4$ after a 3×3 conv $\rightarrow 6 \times 6$ after another 3×3 conv, etc.
- Overall, any operation that reduces spatial resolution (pooling, strided convolution) increases the receptive field.

15.8.3 Hierarchical Feature Extraction

- Early layers: small receptive field (3×3 or 5×5) — neurons detect very local patterns (edges, corners).
- Deeper layers: large receptive field — neurons respond to more complex, larger structures (eyes, wheels, etc.).
- Final feature vector integrates information from the whole image (or a significant part of it), enabling high-level reasoning.
- This hierarchical assembly is what makes CNNs powerful: local to global representation learned entirely from data.

15.9 Backpropagation through Non-Linear Layers

15.9.1 Max Pooling Gradient

- Max pooling is not differentiable everywhere, but we can define a subgradient for backpropagation.
- During the forward pass, we remember the spatial position of the maximum within each pooling window.
- In the backward pass, the gradient is passed only to that winning neuron; all others receive zero.
- This yields a **binary mask** pattern: 1 at the argmax location, 0 elsewhere.

15.9.2 ReLU Gradient

- ReLU derivative is a step function: 1 for positive input, 0 for negative input.
- Easy to compute and backpropagate.
- Combined with convolutional and pooling layers, all gradients can be computed efficiently, enabling end-to-end training.

15.10 The ImageNet Breakthrough

15.10.1 Before 2012: Shallow Models + Handcrafted Features

- Dominant approaches: expert-designed feature extractors + shallow classifiers (SVMs, etc.).
- ImageNet Large Scale Visual Recognition Challenge (ILSVRC) provided 1 million training images over 1000 categories.
- Error rates decreased slowly year over year — marginal gains from more complex handcrafted pipelines.

15.10.2 2012: AlexNet — The Deep Learning Revolution

- First CNN to win ILSVRC: end-to-end trainable, learned features directly from pixels.
- Dramatic drop in classification error (from $\sim 25\%$ to $\sim 16\%$ top-5 error).
- From then on, all top entries used deep convolutional networks, eventually surpassing human-level performance.

15.10.3 Enabling Factors

- **Large annotated datasets**: ImageNet (14M+ images, 20k categories) made it possible to train huge models without overfitting.
- **GPUs**: parallel computing power essential for training deep networks in reasonable time.
- **Improved software**: early pioneers had to code backprop manually; modern frameworks (PyTorch, TensorFlow) dramatically lower the barrier.

15.11 Practical Deep Learning with Limited Data

15.11.1 The Problem

- State-of-the-art models often have **over 100 million parameters**.
- In real-world projects, large annotated datasets are rare — companies struggle to provide even a few thousand labeled examples.
- Training a huge model from scratch on small data leads to severe overfitting.

15.11.2 Two Key Strategies

1. **Data Augmentation**: artificially expand the training set by applying label-preserving transformations (flips, rotations, crops, colour changes).
2. **Transfer Learning**: use a network pre-trained on a large dataset (e.g., ImageNet) and fine-tune it for the target task with limited data.

(These will be detailed next.)

15.12 Example: Steller Sea Lion Monitoring

- Real-world problem: drone surveys of remote Aleutian islands to count Steller sea lions (population declined by 90%).
- Very high-resolution images (5K $\times$ 20K pixels, or similar) cover entire islands.
- Kaggle competition: count and locate sea lions in these giant images.
- Why this example? It illustrates the challenges of limited annotated data and the need for strategies like transfer learning (and later segmentation).
- The same drone-based monitoring setup is common in many conservation and industrial inspection tasks.

15.13 Data Augmentation

15.13.1 The Steller Sea Lion Counting Problem

- Task: count Steller sea lions in very high-resolution drone images (5K $\times$ 20K pixels) and classify them into five categories (adult males, sub-adult males, adult females, juveniles, pups).
- Direct end-to-end regression with a CNN on full images is infeasible due to extreme image size.
- Approach: decompose the problem into **patch classification**.
  - Extract small patches (90×90) from the original images.
  - Classify each patch into one of the five sea lion classes or a **background** class.
  - Subsequent post-processing to aggregate patch predictions into counts.
- Initial classifier performed poorly; for example, an adult male was misclassified as a female, and a pup was confused with other classes.
- Key challenge: very limited annotated data, especially for rare classes (e.g., only a few adult males), leading to severe overfitting.

15.13.2 Geometric and Photometric Transformations

- Data augmentation artificially expands the training set by applying **label-preserving** transformations to existing images.
- **Geometric transformations**: rotation, flipping, shifting, shearing, scaling, perspective distortion.
  - Simulate realistic variations (e.g., drone flying in a different direction, sea lion facing another way).
  - Interpolation at borders may create artifacts (e.g., filled-in pixels after rotation).
- **Photometric transformations**: changes in brightness, contrast, addition of noise, superimposing patterns.
- Each transformed image retains the *same label* as the original; therefore the transformation must not alter the class.
- **Counterexample**: predicting the time from an analog clock without numbers.
  - Rotating the clock changes the time, so rotation is *not* label-preserving.
  - This illustrates the need to carefully choose augmentations based on domain knowledge.

15.13.3 MixUp Augmentation

- A domain-agnostic augmentation technique that does not require expert knowledge.
- Given two training images I_i, I_j and their labels, create a new sample by convex combination:

$$\tilde{I} = \lambda I_i + (1-\lambda) I_j, \qquad \tilde{y} = \lambda y_i + (1-\lambda) y_j$$

- $\lambda \in [0,1]$ is typically sampled from a Beta distribution; here $\lambda = 0.7$ produces an image that is 70% cat and 30% dog, with the same mixing proportions for the label vector.
- In feature space, this creates new points along the line segment between original samples, encouraging the classifier to behave linearly between classes and smooth decision boundaries.
- The target becomes a **soft label** (a distribution over classes), used directly in the categorical cross-entropy loss.
- Pairs can be drawn randomly, including both same-class and different-class combinations.

15.13.4 Benefits of Augmentation

- **Promotes invariance**: by presenting multiple transformed versions of the same image, the network learns that certain variations (e.g., shift, rotation) are irrelevant.
  - Example: if you always flip horse images, the network may learn a "two-headed horse" template, but if flipping is applied uniformly to all classes, the network can become flip-invariant.
  - Convolutional layers already provide some translation equivariance; augmentation encourages the final representation to be fully invariant.
- **Reduces overfitting**: a larger, more diverse training set (even if synthetic) makes it harder for the network to memorise individual samples.
- **Compensates for class imbalance**: more aggressive augmentation can be applied to underrepresented classes (e.g., adult males) to balance the effective training distribution.
- Note: real, independent new images are always better than augmented copies, but augmentation is a valuable substitute when collection is expensive or impossible.

15.13.5 Pitfalls and Best Practices

- **Hidden artifacts**: geometric transformations (rotation, shearing) may introduce interpolation patterns or border fills that act as telltale signs.
  - If these artifacts appear *only* for a specific class (e.g., only adult male patches are heavily rotated), the network can learn to rely on the artifact rather than the actual content.
  - **Remedy**: apply the same augmentation to *all* classes and to background patches, so that the artifact becomes class-agnostic and the network learns to ignore it.
- Networks are "lazy" optimisers; they will exploit any shortcut (spurious correlation) to minimise the loss. Careful validation is necessary to ensure the model generalises for the right reasons.

15.13.6 Implementation in Deep Learning Frameworks

- **PyTorch**: augmentations are typically applied inside the `DataLoader` via `torchvision.transforms`.
  - Images are transformed on-the-fly every time they are loaded, so the disk footprint does not increase.
  - Over many epochs, the network sees a nearly infinite stream of slightly different versions of each original image, effectively never seeing the exact same input twice.
- **TensorFlow / Keras**: augmentation can be added as a preprocessing layer right after the input, achieving the same on-the-fly behaviour.
- This dynamic augmentation is a standard component of modern training pipelines.

15.13.7 Limitations

- Augmentation does **not** mathematically guarantee invariance; it only *encourages* the network to become robust to the transformations seen during training.
- If a transformation is not included in the augmentation set, the network may still fail when that variation occurs at test time.
- Nevertheless, data augmentation remains one of the most effective regularisation tools when labelled data is scarce.

15.14 Test-Time Augmentation (TTA)

- While training-time augmentation promotes invariance, **test-time augmentation** (TTA) makes predictions more robust by applying augmentations to each test image.
- For a single test sample, generate N augmented versions (e.g., rotations, flips, shifts).
- Run each augmented version through the network to obtain N predictions (posterior probabilities or logits).
- **Aggregate** the predictions:
  - Average the predicted probabilities (soft voting).
  - Majority voting on the hard labels.
  - Any ensemble combination scheme.
- Intuition: even if the network is not perfectly invariant, a difficult orientation may cause a single wrong prediction; averaging over multiple orientations often yields the correct result.
- TTA can be seen as an **ensemble of the same model** with perturbed inputs, improving robustness without training additional networks.

15.15 Designing Augmentations: Label-Specific vs. Label-Agnostic

- Some augmentations are **label-specific**: applied only to samples of certain classes to combat imbalance (e.g., heavily rotating rare "adult male" sea lions).
  - Risk: class-specific artifacts may be learned as discriminative features.
  - Mitigation: ensure any artifact also appears in other classes / background.
- These label-specific transforms **cannot be used at test time** because the label is unknown.
- **Label-agnostic** augmentations (e.g., horizontal flip, mild rotation, brightness changes) are applied to all training samples and can safely be reused in TTA.
- In practice, maintain two sets of transforms:
  1. *Training only*: label-specific, heavily tailored to the domain.
  2. *Training + TTA*: generic, label-preserving for all images.
- Real-world example (wafer defect classification): experts designed defect-specific augmentations (e.g., rotating defect patterns around the border) to generate realistic rare defect samples, used only during training.

15.16 Transfer Learning

15.16.1 Motivation

- Training large CNNs from scratch requires massive datasets (e.g., ImageNet with 1M+ images).
- In real projects, annotated data is often scarce (a few hundred images).
- **Key idea**: reuse a network pretrained on a large, generic dataset (ImageNet) as a feature extractor for a new task.
- The convolutional backbone learns **general visual features** (edges, textures, parts) that transfer well across tasks, even if the target classes are absent from ImageNet.
- The final classifier is **task-specific** (trained on 1000 ImageNet classes) and must be replaced.

15.16.2 Procedure

1. Select a pretrained network (e.g., ResNet, VGG) trained on ImageNet.
2. Remove its classification head (the fully-connected layers).
3. Attach a new classifier designed for the target task (e.g., a linear layer with 6 outputs for sea lion patch classification).
   - Keep the new head small if training data is very limited (e.g., just a linear softmax).
4. **Freeze** the feature extraction layers: set `requires_grad = False` for all parameters in the backbone.
5. Train only the new classifier on the target dataset.
   - The pretrained backbone remains unchanged; it acts as a fixed feature extractor.
   - This is **transfer learning** — the features are transferred, not adapted.

15.16.3 Why It Works

- The ImageNet-trained backbone has learned to extract highly discriminative features for thousands of categories, covering a wide variety of shapes, textures, and patterns.
- These features are **generic enough** to describe new objects (sea lions, defects) because they capture low-level and mid-level visual primitives.
- Training only a small classifier on top of such rich representations drastically reduces overfitting when data is limited.

15.17 Fine-Tuning

15.17.1 When to Fine-Tune

- If the target dataset is medium-sized or the domain differs significantly from ImageNet (e.g., medical images, thermal images), the pretrained features may be suboptimal.
- **Fine-tuning** adapts some (or all) of the pretrained layers to the new task by continuing backpropagation through the entire network.

15.17.2 Best Practice: Two-Stage Training

1. **Stage 1 — Transfer learning**: freeze the backbone, train only the new classifier until convergence.
   - This provides a reasonable starting point for the classification head, avoiding large, destructive gradients from a randomly initialised head flowing into the backbone.
2. **Stage 2 — Fine-tuning**: unfreeze the backbone (or a subset of its layers), then train the entire network with a *very small learning rate*.
   - Small learning rate prevents catastrophic forgetting of the valuable pretrained weights.
   - Early layers (capturing generic edges) can be kept frozen; deeper layers (more task-specific) are often unfrozen.

15.17.3 Risks

- If the backbone is unfrozen too early (before the new head is trained), random gradients from the head can destroy the carefully learned features, leading to poor performance.
- Overfitting remains a concern, especially if the target dataset is small; aggressive regularisation (dropout, weight decay, data augmentation) is essential.

15.18 Pretrained Models in PyTorch

- `torchvision.models` provides many classic architectures (AlexNet, VGG, ResNet, etc.) with convenient access to weights.
- Three common ways to load a model:
  1. `weights=None` — random initialisation; useful for training from scratch on ImageNet or a similar large dataset.
  2. `weights='IMAGENET1K_V1'` — loads the fully pretrained weights, including the classifier (suitable for inference on ImageNet classes).
  3. `weights='IMAGENET1K_V1'` with the classifier replaced — for transfer learning / fine-tuning, you can access only the backbone (e.g., `model.features` in VGG, `list(model.children())[:-1]` for others) and discard the head, saving memory and parameters.

- The choice depends on the use case: transfer learning, fine-tuning, or pure feature extraction.

Chapter 16

Famous CNN Architectures

16.1 Historical Perspective on CNN Architectures

16.1.1 LeNet-5 (1998) – The First Convolutional Neural Network

- **Paper**: *Gradient-Based Learning Applied to Document Recognition* (LeCun, Bottou, Bengio, Haffner, 1998).
- **Task**: Handwritten digit recognition (10 classes, 0–9) from 32×32 grayscale images (often shown as 28×28 after cropping; boundaries are dark background).
- **Architecture**:
  1. `Conv2D`: 6 filters of size 5×5, valid padding $\rightarrow$ output $6 \times 28 \times 28$.
  2. `AvgPool2D`: 2×2, stride $2 \rightarrow 6 \times 14 \times 14$.
  3. `Conv2D`: 16 filters of size 5×5, valid padding $\rightarrow 16 \times 10 \times 10$.
  4. `AvgPool2D`: 2×2, stride $2 \rightarrow 16 \times 5 \times 5$.
  5. `Flatten` $\rightarrow$ vector of length $16 \cdot 5 \cdot 5 = 400$.
  6. `Dense`: 120 neurons.
  7. `Dense`: 84 neurons.
  8. `Dense (output)`: 10 neurons (digit classes).
- **Activation**: tanh (standard at the time), not ReLU.
- **Parameter count** (illustrative):
  - Conv1: $5 \times 5 \times 1 \times 6 + 6 = 156$.
  - Conv2: $5 \times 5 \times 6 \times 16 + 16 = 2\,416$.
  - FC1 (400$\rightarrow$120): $400 \cdot 120 + 120 = 48\,120$.
  - FC2 (120$\rightarrow$84): $120 \cdot 84 + 84 = 10\,164$.
  - Output (84$\rightarrow$10): $84 \cdot 10 + 10 = 850$.
  - Total $\approx 60\,000$ parameters.
- **Key observation**: The vast majority of parameters reside in the fully-connected layers (the classifier), while the convolutional feature extractor contributes only a few hundred. This reflects the limited depth and the design philosophy of the time.

16.1.2 Impact of Padding on Parameter Count

- LeNet uses **valid** padding (no zero-padding), so spatial dimensions shrink after each convolution.
- If **same** padding were used (output size equals input size):
  - After Conv2 + Pooling, the feature map would be $16 \times 14 \times 14$ (instead of $16 \times 5 \times 5$).
  - The flattened vector would have $16 \cdot 14 \cdot 14 = 3\,136$ dimensions instead of 400.
  - The first dense layer ($3\,136 \to 120$) would contain $3\,136 \cdot 120 + 120 \approx 376\,000$ parameters – an order of magnitude larger.
- The choice of valid padding was deliberate: it kept the fully-connected part manageable and exploited the fact that the digit boundaries contained only dark, uninformative pixels.
- **Lesson**: Architectural details like padding can drastically affect model size and feasibility, especially when the final classifier is a dense MLP.

16.1.3 CNN vs. MLP for Image Input

- Flattening the input image directly ($32 \times 32 = 1024$ pixels) and using an MLP with a hidden layer of size 84:
  - Input-to-hidden: $1024 \cdot 84 + 84 \approx 86\,000$ parameters (already more than the entire LeNet).
  - If the image were RGB (3 channels), the input becomes $32 \times 32 \times 3 = 3072$, making the first dense layer about $3072 \cdot 84 \approx 258\,000$ parameters.
- For a CNN, adding colour channels only increases the filter depth:
  - Conv1 filter becomes $5 \times 5 \times 3$ instead of $5 \times 5 \times 1$; parameters grow from 156 to $5 \times 5 \times 3 \times 6 + 6 = 456$ – a negligible increase.
- Thus, CNNs scale gracefully with input channels, while MLPs explode in parameters.

16.2 AlexNet (2012) – The Deep Learning Breakthrough

- **Architecture**: A larger, deeper LeNet-style network, trained on **ImageNet** (1.2M training images, 1000 classes, 224×224 RGB inputs).
- Much more convolutional layers (5), max pooling, and a large fully-connected head.
- Won the ILSVRC 2012 by a huge margin, marking the start of the deep learning era in computer vision.
- **Authors**: Alex Krizhevsky, Ilya Sutskever, Geoffrey Hinton.
- The three Turing Award "godfathers" of deep learning:
  - **Yann LeCun** (LeNet, 1998) – later at Meta (FAIR).
  - **Geoffrey Hinton** (AlexNet, 2012) – later at Google; awarded **Turing Award 2018** (with LeCun and Bengio) and **Nobel Prize in Physics 2024** for contributions to artificial neural networks.
  - **Yoshua Bengio** (LeNet co-author, 1998) – deep learning theory and generative models.
- The progression from LeNet to AlexNet illustrates the scaling power of CNNs: given more layers, more data, and GPU compute, learned features become rich enough to tackle a thousand-class natural image problem.

16.3 Key Takeaways on CNN Network Design

- Convolutional layers are **parameter-efficient** feature extractors; the bulk of parameters often lies in the final dense layers (classifier).
- Design choices (padding, stride, pooling) control the size of the feature vector and therefore the complexity of the classifier.
- Valid padding can be strategically used when boundary content is irrelevant, significantly reducing the number of parameters in the classifier.
- CNNs handle multi-channel inputs gracefully (only the filter depth grows), in stark contrast to MLPs.
- Understanding these trade-offs is essential for interpreting model summaries and for exam exercises on parameter computation and output shape derivation.

16.4 AlexNet (2012) – Details

16.4.1 Architecture and Scale

- Trained on ImageNet: over 1 million training images, 1000 classes, $224 \times 224 \times 3$ inputs.
- Deeper than LeNet: 5 convolutional layers + max-pooling + 3 fully-connected layers.
- Parameter breakdown:
  - Convolutional part: ≈ 3.7 million parameters (filters).
  - Fully-connected part: $\approx 94\%$ of total parameters, mostly in the first dense layer.
  - Total parameters: ≈ 60 million – 1000 times more than LeNet.
- The vast majority of parameters are still in the classifier, even though the feature extractor is much larger.

16.4.2 Novel Design Choices

- **ReLU activation**: faster training than tanh or sigmoid.
- **Dropout**: applied to the dense layers to reduce overfitting.
- **Max pooling**: adds non-linearity and stronger downsampling compared to average pooling.
- **Large first-layer filters**: 11×11 with stride 4, drastically reducing spatial size from 224×224 to 55×55 (with 48 output channels).

16.4.3 Two-GPU Implementation

- At the time (GTX 580, 1.5 GB memory each), a single GPU could not hold all activations for one image.
- The network was **split across two GPUs**:
  - The lower and upper rows in the architecture diagram correspond to the two GPUs.
  - Convolutions have local interactions; cross-GPU connections only at certain layers (indicated by crossed connections), then merged back.
- This highlights a crucial aspect: even if convolutional layers have few parameters, they generate **large activation maps** that demand substantial memory.

16.4.4 Ensembling

- To further improve accuracy, 7 identical AlexNet models were trained with different random initialisations and mini-batch shuffling.
- Predictions from the 7 models were aggregated (e.g., averaging posteriors or majority voting), yielding a significant drop in classification error.
- This is a common ensemble strategy that exploits the stochasticity of gradient descent to obtain diverse, complementary models.

16.5 VGG Network (2015) – The Power of Small Filters

16.5.1 Philosophy: Go Deeper with Smaller Kernels

- Goal: increase network depth while controlling the parameter count.
- Key observation: a cascade of small 3×3 convolutional layers can replicate the receptive field of a single larger filter but with **fewer parameters and more non-linearities**.
- Example:
  - A single 7×7 conv: receptive field 7×7, 49 parameters (assuming $C_{\text{in}} = C_{\text{out}} = 1$), 1 non-linearity.
  - Three stacked 3×3 convs: receptive field $3 \to 5 \to 7$ (same coverage), $3 \times 9 = 27$ parameters, 3 non-linearities.
- This design enables deeper architectures with **richer representational power** without exploding the parameter budget.

16.5.2 VGG Architecture Pattern

- VGG consists of **blocks** of 2 or 3 consecutive 3×3 convolutions (all with same padding, preserving spatial size), followed by a 2×2 max-pooling layer that halves spatial dimensions.
- The number of channels increases progressively: $64 \to 128 \to 256 \to 512 \to 512$.
- After the final convolutional block, the volume is flattened into a very large feature vector (e.g., $7 \times 7 \times 512 = 25\,088$ dimensions) and passed through fully-connected layers.
- Total parameters: ≈ 138 million, with the majority still in the dense layers (due to the huge flattened vector).

16.5.3 Memory Footprint

- Despite parameter efficiency in the convolutional part, VGG stores **very large activation maps** during forward and backward passes.
- Inference on a standard $224 \times 224 \times 3$ image can require around 100 MB just for activations; training requires roughly double (activations + gradients).
- This makes VGG computationally demanding and memory-hungry, despite being conceptually simple.

16.6 Understanding CNNs via Filter Visualisation

16.6.1 First-Layer Filters as Templates

- A convolutional layer acts as a set of learned **templates**: each filter is correlated with the input, producing high activation where the local patch matches the template.
- Visualising the filters of the first layer (closest to the image) provides insight into the low-level patterns the network relies on.

16.6.2 AlexNet First-Layer Filters

- The filters are $11 \times 11 \times 3$, so they can be displayed as small colour images.
- Two main families emerge:
  1. **Grayscale, high-frequency filters** – edge and texture detectors (similar to Gabor filters or DCT basis functions). The network learns that colour is less important for fine spatial structures.
  2. **Colourful, low-frequency filters** – detect colour blobs or smooth gradients (e.g., orange/blue transitions, green/magenta contrasts).
- This mirrors what we observed earlier: early layers capture simple, local patterns that are then combined in deeper layers to form complex detectors.

16.7 Interpreting CNNs Beyond the First Layer

16.7.1 Limitations of Filter Visualisation in Deeper Layers

- First-layer filters are directly visualisable (3 channels $\rightarrow$ colour image) and can be interpreted as edge/colour detectors.
- In deeper layers, filters have as many channels as the preceding activation volume (e.g., 48, 128, 256). They cannot be displayed as simple images.
- Moreover, they no longer operate on raw pixels but on feature maps produced by earlier layers, making the "template matching" interpretation indirect.
- Therefore, alternative methods are needed to understand what deeper layers compute.

16.7.2 Activation Maximisation via Training Set Patches

- A simple, effective approach: for a chosen neuron (or an entire channel) in a convolutional layer, run the *entire training set* through the network.
- Record which input images (or image patches) cause the **maximum activation** for that neuron.
- Because convolutions are translation equivariant, all neurons in the same channel compute the *same operation* at different spatial positions.
- Hence, the patches that maximally activate different neurons in one channel should be visually similar – they represent the pattern that this channel has learned to detect.
- Examples from VGG/AlexNet:
  - A channel that fires on dark circular regions on a light background – often corresponds to a dog's nose or eye.
  - Another channel activates on yellow text on dark background, or dark text on yellow background.

  - Deeper channels respond to more complex structures: wheels, smiling faces, weightlifting equipment.

- As we go deeper, the receptive field of the neuron grows, so the patches become larger and capture more global patterns.

16.7.3 Convolutional Layers as a Cascaded Non-Linear Filter

- A single convolutional layer is a linear filter applied to a local patch.
- Stacking many convolutional layers (with ReLUs and pooling) creates a **huge non-linear filter** that operates on a large receptive field.
- The whole convolutional backbone can be seen as learning a set of complex, non-linear feature detectors, each corresponding to a channel in the last convolutional layer.
- This is why neurons in the same channel respond consistently to the same visual concept – they share the same composite filter weights.

16.7.4 Gradient Ascent for Input Optimisation

- Instead of searching the training set, we can treat the **input image as a learnable parameter** and optimise it to maximise the activation of a target neuron, while keeping the network weights fixed.
- This uses **gradient ascent** (the gradient of the neuron's activation with respect to the input).
- Visualising the resulting (optimised) input reveals what pattern the neuron is most sensitive to.
- Results:
  - Early layers: optimised inputs look like simple textures and small patterns (edges, colour blobs).
  - Mid-level layers: more structured patterns emerge (textures with larger spatial organisation).
  - Deep layers (near the classifier): recognisable higher-level structures appear (object parts, whole objects).

16.7.5 Network Dreams and Class Visualisation

- Extending the idea to the output layer: maximise the activation of a **class logit** (before softmax).
- Adding a regularisation term (e.g., penalising extreme pixel values) encourages the generated image to look natural.
- The resulting images are often called **"network dreams"** – they show what the network associates with a given class.
- Examples (ImageNet classes):
  - "Billiard table": generated image contains elements of a billiard table.
  - "Black swan", "station wagon", "ground beetle", "Indian cobra" – each image visually suggests the class.
- This technique demonstrates that the network has implicitly learned class-specific visual concepts, even though it was only trained on classification.

16.7.6 Summary: Hierarchical Feature Learning in CNNs

- **Early layers**: low-level features (edges, colours, textures) – small receptive fields, simple patterns.
- **Middle layers**: combinations of edges and textures forming object parts (wheels, eyes, text blocks).
- **Deep layers**: complex, class-specific features (whole objects or characteristic parts) – large receptive fields.
- **Classifier**: combines the presence/absence of these high-level features to make a decision.
- This hierarchy emerges naturally from end-to-end training on a large classification task – no manual feature engineering.

16.8 Network in Networks (2014) – New Layer Types

16.8.1 Motivation

- After VGG, researchers explored more powerful building blocks to increase representational capacity without exploding parameter counts.
- The **Network in Networks** (NiN) paper introduced two key ideas:
  1. The **MLP-Conv** layer (a micro-MLP replacing the linear convolution).
  2. **Global Average Pooling** (GAP) to replace the large fully-connected classifier.

16.8.2 MLP-Conv Layer

- Standard convolution: a linear combination (filter) followed by non-linearity (e.g., ReLU).
- Idea: instead of a simple linear mapping, apply a small **Multi-Layer Perceptron** to each local patch.
  - Flatten the local spatial region (all input channels) into a vector.
  - Feed this vector through one or more hidden fully-connected layers with non-linearities.
  - The output becomes the activation for that spatial position.
- The same MLP weights are **shared across all spatial locations**, preserving translation equivariance.
- In practice, this can be implemented with **1×1 convolutions** after a standard convolution.
- **Fate**: Not widely adopted; stacking multiple conventional conv layers with ReLUs proved to be a simpler and equally effective way to increase non-linearity.

16.8.3 Global Average Pooling (GAP)

Problem: Parameter Explosion in the Classifier

- After the final convolutional layer, activations form a volume $H' \times W' \times C'$.
- Flattening yields a vector of size $H' \cdot W' \cdot C'$ (e.g., 25,088 in VGG).
- A fully-connected layer on this vector contains an enormous number of parameters (e.g., $25,088 \times 4096 \approx$ 100M), dominating the model size and causing severe overfitting.

GAP Operation

- Replace flattening + dense layers with a simple, parameter-free operation:

$$f_k = \frac{1}{H' \cdot W'} \sum_{x=1}^{H'} \sum_{y=1}^{W'} f(x, y, k) \qquad \text{for } k = 1, \ldots, C'$$

- Each channel's spatial map is **averaged** to a single scalar, producing a vector of length C'.
- Interpretation: because all neurons in the same channel perform the identical operation (translation equivariance), they detect the same feature at different positions. Averaging them condenses the presence of that feature across the entire image into one number.
- The resulting vector can be fed directly to a **softmax** if C' is set to the number of classes, completely eliminating fully-connected layers.

Advantages

- **Drastic parameter reduction**: no large dense weight matrices; the classifier becomes extremely lightweight or even non-existent (just softmax).
- **Overfitting prevention**: far fewer parameters, especially when training data is limited.
- **Input size flexibility**: convolutional layers can handle arbitrary spatial dimensions; GAP works for any H', W'. Therefore, at inference time, the network can process images of varying sizes (training still requires fixed-size batches).
- **Interpretability**: each output channel directly corresponds to a class score map (before averaging), making the network more transparent.

GAP as a Matrix Operation

- GAP can be expressed as a linear transformation: flatten the volume to vector $\mathbf{x}$ of size $H'W'C'$, then apply a fixed matrix $\mathbf{W}$ of size $C' \times (H'W'C')$.
- $\mathbf{W}$ has a **block-diagonal, sparse structure**:
  - The row for channel k contains $1/(H'W')$ in the positions corresponding to the k-th channel's spatial block, zeros elsewhere.
  - No learnable parameters; gradients can still flow through this fixed matrix during backpropagation.
- This viewpoint confirms GAP fits seamlessly into the gradient-based training framework.

16.8.4 NiN Architecture Summary

- The original NiN stacked several "MLP-Conv" blocks (effectively conv + 1×1 convs) and ended with a GAP layer followed directly by softmax.
- All learnable parameters reside in the convolutional layers; the network is entirely convolutional up to the final pooling.
- This design philosophy influenced later architectures by showing that heavy fully-connected heads are **not necessary** and that more parameters can be shifted into the feature extraction stage.

16.9 Translation Invariance and Global Average Pooling

16.9.1 The Problem with Flattening

- After convolutional feature extraction, a typical classifier uses `Flatten` → `Dense` layers.
- Flattening produces a vector of size $H' \cdot W' \cdot C'$, where H', W' are the spatial dimensions of the final feature maps.
- The fully-connected layer that follows has a weight matrix of size output_dim × $(H'W'C')$, whose dimensions depend on the spatial size of the input.
- Consequently, the network can only process images of the *same spatial resolution* it was trained on. Changing the input size changes the flattened vector length, breaking the matrix multiplication.

16.9.2 GAP Enables Input-Size Flexibility

- Global Average Pooling removes the dependence on spatial dimensions by averaging each channel to a single value, yielding a fixed-length vector of size C' regardless of H' and W'.
- Therefore, at inference time, a CNN ending with GAP (and no dense layers, or only 1×1-equivalent dense layers) can process **images of arbitrary spatial size**.
- The convolutional layers themselves are fully convolutional and naturally adapt to any input size (they slide filters over the whole input).
- Training still requires fixed-size inputs for batch processing, but once trained, the model can accept varied resolutions.

16.9.3 Translation Invariance: GAP vs. Flattening

- Convolutional layers are *equivariant* to translation: shifting the input shifts the output feature maps identically.
- A fully-connected layer after flattening **breaks** this equivariance: the same feature appearing at a different spatial position corresponds to a different set of input neurons, and the classifier sees an entirely different pattern in the flattened vector.
- By averaging each channel over all spatial positions, GAP imposes **strict translation invariance**: the output vector is unaffected by spatial shifts of the detected features.
- The network becomes invariant to translations *by construction*, without having to learn it from augmented data (although augmentation still helps for other transformations).

16.9.4 Illustrative Experiment: Shifted Digits

- A CNN trained on centered digit images was tested on shifted versions of the same digits.
- **CNN with Flattening + Dense classifier**: accuracy collapses; the model tends to always predict a single class (e.g., 6), because it has learned to rely on the absolute spatial positions of strokes.
- **CNN with GAP**: maintains high accuracy on shifted images, as the spatially averaged features are insensitive to the location of the digit.
- This demonstrates that GAP acts as a **structural regulariser**, inherently encouraging the network to learn position-independent features.

16.10 MobileNets: Lightweight Architectures for Edge Devices

16.10.1 Motivation

- After the trend of building deeper, higher-accuracy networks (VGG, Inception, ResNet), a parallel need arose: deploy deep learning on **resource-constrained devices** (smartphones, Raspberry Pi, IoT sensors).
- Even with GAP removing the large dense layers, standard convolutional layers still require substantial computation and memory (many multiply-add operations per pixel, large activation maps).
- **MobileNets** (2017) introduce **depthwise separable convolutions** that drastically reduce computation and parameter count while preserving competitive accuracy.

16.10.2 Standard Convolution Cost

- Given an input activation map of size $D_F \times D_F \times M$ and an output of $D_F \times D_F \times N$ (assuming same spatial size, D_F is feature map width/height, M input channels, N output channels).
- Each of the N filters has size $D_K \times D_K \times M$, where D_K is the kernel size (e.g., 3).
- Number of multiply-add operations per output pixel: $D_K \cdot D_K \cdot M \cdot N$.
- Total operations for the whole layer: $D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F$.
- This grows quickly with the number of channels and spatial size.

16.10.3 Depthwise Separable Convolution

The operation is factorised into two steps:

1. Depthwise Convolution

- Instead of one $D_K \times D_K \times M$ filter, apply M **separate filters**, each of size $D_K \times D_K \times 1$.
- Each filter operates on a *single input channel*, producing an intermediate map of the same spatial size but still with M channels.
- This performs **spatial filtering only, without mixing channels**.
- Computation: $D_K \cdot D_K \cdot M \cdot D_F \cdot D_F$.

2. Pointwise (1×1) Convolution

- Apply N filters of size $1 \times 1 \times M$ to the depthwise output.
- This **mixes information across channels** without any spatial filtering, producing the final $D_F \times D_F \times N$ output.
- Computation: $M \cdot N \cdot D_F \cdot D_F$.

Computational Savings

$$\begin{aligned}
\text{Standard conv cost} &= D_K^2 \cdot M \cdot N \cdot D_F^2 \\
\text{Depthwise separable cost} &= D_K^2 \cdot M \cdot D_F^2 \;+\; M \cdot N \cdot D_F^2 \\
\text{Reduction ratio} &= \frac{D_K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2}{D_K^2 \cdot M \cdot N \cdot D_F^2} = \frac{1}{N} + \frac{1}{D_K^2}
\end{aligned}$$

- For a typical setting $N = 100$, $D_K = 3$, the reduction is about $\frac{1}{100} + \frac{1}{9} \approx 0.12$ – nearly **10× fewer operations**.
- Parameter count is similarly reduced: filters in depthwise step are $D_K^2 M$, pointwise filters are MN, compared to $D_K^2 MN$ for standard conv.

16.10.4 MobileNet Architecture Features

- Built from blocks of depthwise separable convolutions followed by BatchNorm and ReLU.
- Uses **Global Average Pooling** at the end, followed by a single dense layer (or just softmax) for classification → very few parameters in the head.
- Introduces two hyperparameters to further trade off accuracy vs. efficiency:
  - **Width multiplier** $\alpha \in (0, 1]$: scales the number of channels uniformly (e.g., $\alpha = 0.5$ halves M and N).
  - **Resolution multiplier** ρ: scales the input image spatial resolution (e.g., 224 → 128).
- These allow creating a family of models with different performance/profiles.

16.11 Landscape of CNN Architectures (2012–2017)

- A seminal plot shows the evolution of ImageNet top-1 accuracy vs. computational cost (GFLOPs) and model size (circle diameter).
- **AlexNet**: ∼ 55% top-1 accuracy, moderate cost.
- **VGG**: higher accuracy but 2–3× more operations and parameters (up to 155M).
- **MobileNet**: achieves competitive accuracy with drastically lower computation and model size (a few million parameters).
- **Inception** and **ResNet**: push accuracy further by using sophisticated blocks (Inception modules, residual connections) at the expense of more computation – these will be covered next.
- This landscape highlights two parallel goals:
  1. Maximise accuracy (research, cloud servers) – deeper, wider networks.
  2. Maximise efficiency (mobile, IoT) – lightweight architectures like MobileNet.

16.12 Practical Use: Transfer Learning with MobileNet

- MobileNet, pretrained on ImageNet, is an excellent feature extractor for transfer learning because it is small, fast, and still highly discriminative.
- Procedure:
  1. Load a MobileNet pre-trained on ImageNet.
  2. Remove the original classification head.
  3. Attach a new head appropriate for the target task (e.g., GAP + Dense + Softmax for a small dataset).
  4. Fine-tune (optionally freeze the backbone first).
- This will be explored in the upcoming lab session, together with data augmentation techniques.

Chapter 17

Image Augmentation and Transfer Learning

17.1 Evolution of Convolutional Architectures

17.1.1 From AlexNet to Advanced Blocks

- Working with $64 \times 64 \times 3$ RGB images (PyTorch order: channels, height, width).
- 10-class classification; batch size 128.
- AlexNet (2012) was the breakthrough: convolutional layers $\rightarrow$ pooling $\rightarrow$ activation, followed by fully-connected layers for classification.
- A basic CNN block consists of `Conv2d` (with padding to preserve spatial size), `ReLU`, and optionally `MaxPool2d` for downsampling.
- To build a network, stack several such blocks, then flatten and use linear (dense) layers for classification, ending with softmax.
- Example with one block and 32 filters: flatten yields $32 \times 32 \times 32 = 32\,768$ units; a single linear layer to 10 classes gives $\sim 327\,680$ parameters – the vast majority in the dense head.

17.1.2 Parameter Explosion in the Classifier Head

- Flattening after the convolutional part leads to a very large vector, making the subsequent linear layer extremely heavy.
- Even for a simple 10-class problem, the linear layer alone can contribute hundreds of thousands of parameters.
- This large parameter count increases the risk of overfitting and makes training slower.

17.2 Global Average Pooling (GAP) – Network in Network

17.2.1 Motivation and Operation

- Introduced in *Network in Networks* (2013) to replace the heavy fully-connected classifier.
- Instead of flattening all spatial activations, GAP computes the average of each feature map (channel) across all spatial positions.

- In PyTorch, `nn.AdaptiveAvgPool2d(1)` reduces each $H \times W$ channel to a single scalar; subsequent `flatten` gives a vector of length equal to the number of channels.
- For a 32-channel output, this yields a 32-dimensional vector, and a linear layer to 10 classes has only $32 \cdot 10 + 10 = 330$ parameters – negligible compared to the earlier 327k.

17.2.2 Advantages

- Drastic reduction in the number of parameters in the classification head (orders of magnitude).
- Better regularisation and less overfitting, especially with limited data.
- No trade-off in performance; often improves generalisation.
- Encourages translation invariance: averaging over spatial locations makes the representation insensitive to the exact position of features.
- Enables networks to accept variable-size inputs at inference (conv layers adapt, GAP gives fixed-size output).

17.2.3 Impact on Architecture

- The CNN backbone remains unchanged; only the head is modified.
- Typical flow: convolutional blocks → `AdaptiveAvgPool2d(1)` → `Flatten` → `Linear` → softmax.
- This became a standard design pattern after 2013.

17.3 Inception Block – Multi-Scale Feature Processing

17.3.1 Concept

- Introduced in *GoogLeNet* (2014) with the Inception module.
- Instead of a single sequential path, the Inception block processes the input through **multiple parallel branches**, each capturing features at different scales.
- Branches include:
  - 1×1 convolution – captures pixel-wise information, cheap.
  - 3×3 convolution – captures medium-scale patterns.
  - 5×5 convolution – captures larger spatial context.
  - Max-pooling (usually 3×3, stride 1) – adds a pooling-based path.
- Outputs of all branches are concatenated along the channel dimension, giving a richer multi-scale representation.

17.3.2 Dimensionality Reduction with 1×1 Convolutions

- Before the expensive 3×3 and 5×5 convolutions, a 1×1 convolution is applied to reduce the number of input channels (depth).
- This drastically lowers the computational cost and parameter count of the following large kernel convolutions.
- Example: if the reduction ratio is out_filters/8, the 1×1 conv compresses the channel dimension, then the large kernel expands back to the desired output channels.
- The same 1×1 reduction is applied after the max-pooling branch to adjust channel numbers.

17.3.3 Building an Inception Block in PyTorch

- Use `nn.ModuleList` to store the parallel layers, or define separate `nn.Sequential` branches.
- Typical implementation (for one block, with specified input/output channels):
  - Branch 1: `Conv2d(in_ch, out1, 1)` → `ReLU`
  - Branch 2: `Conv2d(in_ch, red2, 1)` → `ReLU` → `Conv2d(red2, out2, 3, padding=1)` → `ReLU`
  - Branch 3: `Conv2d(in_ch, red3, 1)` → `ReLU` → `Conv2d(red3, out3, 5, padding=2)` → `ReLU`
  - Branch 4: `MaxPool2d(3, stride=1, padding=1)` → `Conv2d(in_ch, out4, 1)` → `ReLU`
- All branches produce tensors of the same spatial size (due to padding and stride=1 in pooling), so they can be concatenated along the channel axis.
- The total output channels = out1 + out2 + out3 + out4.
- Multiple such blocks can be stacked, progressively increasing channels and reducing spatial dimensions via occasional strided convolutions or max-pooling.

17.3.4 Advantages of Multi-Scale Processing

- The network is not forced to rely on a single filter size per layer; it learns which scale is most relevant for the task.
- The combination of local (1x1) and more global (5x5) views improves feature richness.
- 1×1 convolutions inside the blocks add non-linearity (with ReLU) while controlling the parameter budget.
- This design influenced many subsequent architectures (ResNet, DenseNet, etc.).

17.4 Summary

- Starting from the basic `Conv-ReLU-Pool` block, deep CNNs suffered from huge fully-connected heads.
- **Global Average Pooling** replaced the flatten-dense part, drastically cutting parameters and improving generalisation.
- The **Inception module** introduced parallel branches with different kernel sizes, capturing multi-scale features efficiently via 1×1 bottlenecks.
- These innovations paved the way for modern convolutional architectures used in transfer learning, fine-tuning, and challenges.

17.5 Inception Module – Multi-Scale Processing (cont.)

17.5.1 Parallel Paths and Concatenation

- The Inception block consists of several parallel branches, each applying convolutions of different kernel sizes (1×1, 3×3, 5×5) and a pooling branch.
- All branches are configured (with padding and stride) to produce feature maps of the same spatial dimensions, which are then **concatenated along the channel axis**.
- This concatenation allows the subsequent layers to see features extracted at multiple scales simultaneously.

- In PyTorch, the branches can be stored in an `nn.ModuleList`, and the forward pass simply concatenates their outputs.
- The network still ends with global average pooling, flatten, and a linear classifier; the number of parameters remains moderate because GAP avoids a large dense head.

17.6 Batch Normalization

17.6.1 Motivation and Operation

- Introduced in 2015 to accelerate training and act as a regularizer.
- For a mini-batch, it normalizes each channel's activations to zero mean and unit variance, then applies a learnable scale γ and shift β.
- The normalization uses batch statistics during training; during inference, it uses running averages (non-trainable).
- In practice, a batch normalization layer has four parameters per channel: γ, β (trainable), and the running mean and variance (non-trainable).

17.6.2 Placement and Benefits

- Typically placed **between the convolution and the activation** (e.g., `Conv2d` → `BatchNorm2d` → `ReLU`).
- Benefits:
  - Speeds up convergence (allows higher learning rates).
  - Reduces sensitivity to weight initialisation.
  - Provides a slight regularisation effect, reducing the need for dropout.
- In PyTorch, `nn.BatchNorm2d(num_features)`; the summary may not list running statistics, only the trainable parameters.

17.7 Residual Networks (ResNet)

17.7.1 The Vanishing Gradient Problem

- As networks become deeper, gradients may become extremely small during backpropagation, preventing earlier layers from learning.
- This limits the feasible depth of plain convolutional networks.

17.7.2 Residual Block – Skip Connection

- A residual block introduces a **skip connection** (or shortcut) that adds the input of the block directly to its output.
- Formally, if the block computes a function $\mathcal{F}(x)$, the output becomes $\mathcal{F}(x) + x$.
- Gradients can flow directly through the identity path, mitigating vanishing gradients and enabling very deep networks (50, 101, 152 layers).

17.7.3 Implementation Details

- A typical residual block contains two `Conv2d` layers, each followed by `BatchNorm2d`.
- The input x is stored, passed through the convolutions, and then **added** to the output before the final ReLU.
- If the dimensions change (e.g., due to stride or a different number of channels), a 1×1 convolution is used on the skip connection to match shapes (projection shortcut).
- The block can optionally end with `MaxPool2d` for downsampling, or use strided convolutions within the block.

17.7.4 Network Architecture

- Overall structure follows the now-standard pattern: initial convolution, stack of residual blocks (with increasing channels and periodic downsampling), global average pooling, flatten, and a linear classifier with softmax.
- ResNets are parameter-heavy (millions of parameters) but highly accurate; they are often used for transfer learning rather than trained from scratch on small datasets.

17.8 Squeeze-and-Excitation (SE) Block

17.8.1 Channel-wise Attention

- Introduced to model the importance of different channels in a feature map.
- The SE block explicitly re-calibrates channel-wise responses by learning per-channel weights.

17.8.2 Three Steps

1. **Squeeze**: Aggregate spatial information of each channel via global average pooling, obtaining a descriptor vector of size $1\times1\times C$.
2. **Excitation**: Feed the descriptor through a small neural network (two linear layers with a ReLU in between, ending with a sigmoid) to produce a weight vector $\mathbf{s} \in (0,1)^C$.
3. **Scale**: Multiply the original feature map channel-wise by the weights $\mathbf{s}$, amplifying important channels and suppressing less relevant ones.

17.8.3 Integration

- The SE block is placed after the standard `Conv2d` + `BatchNorm2d` + `ReLU` sequence (or inside residual blocks) without changing spatial dimensions.
- It adds only a small number of parameters (the linear layers in the excitation step) but can significantly boost performance.
- In PyTorch, the squeeze step uses `AdaptiveAvgPool2d(1)` followed by flattening to $(batch, C)$; the excitation is implemented as `Linear`$(C, C/r) \rightarrow$ `ReLU` $\rightarrow$ `Linear`$(C/r, C) \rightarrow$ `Sigmoid` (where r is a reduction ratio).
- The scaled feature map is obtained by `input * weights.view(batch, C, 1, 1)`.

17.9 Summary of Architectural Evolution

- From AlexNet (2012) onward, a clear design pattern emerged: a **feature extraction backbone** composed of progressively more sophisticated blocks, followed by **global average pooling**, flatten, and a lightweight **linear classifier** with softmax.
- Key innovations:
  - **GAP** (2013) eliminated heavy fully-connected heads, reducing parameters and overfitting.
  - **Inception** (2014) introduced multi-scale parallel branches within a layer.
  - **Batch Normalization** (2015) stabilised and accelerated training.
  - **Residual connections** (2015) enabled very deep networks by addressing vanishing gradients.
  - **Squeeze-and-Excitation** (2017) added dynamic channel-wise attention with minimal overhead.
- These building blocks are now standard in state-of-the-art CNN architectures and are available in PyTorch's model zoo for transfer learning and fine-tuning.

17.10 MobileNet: Lightweight Architecture with Inverted Residuals

17.10.1 Inverted Residual Bottlenecks

- Standard residual blocks follow a **wide** $\rightarrow$ **narrow** $\rightarrow$ **wide** pattern (bottleneck). MobileNet uses **inverted residuals**: **narrow** $\rightarrow$ **wide** $\rightarrow$ **narrow**.
- The block first expands the number of channels (using a 1×1 convolution), then applies a **depthwise separable convolution** (spatial filtering per channel), and finally projects back to the original channel count with another 1×1 convolution.
- This design is more efficient for mobile devices because it keeps the intermediate high-dimensional feature maps only temporarily, reducing memory and computation.

17.10.2 Depthwise Separable Convolution (Recap)

- Factorises a standard convolution into:
  1. **Depthwise convolution**: a single 3×3 filter per input channel, no channel mixing.
  2. **Pointwise convolution**: 1×1 convolution that mixes channels linearly.
- Drastically reduces the number of parameters and multiply-add operations compared to standard convolution.

17.10.3 Squeeze-and-Excitation in MobileNet

- MobileNet (in its later versions) incorporates **Squeeze-and-Excitation** (SE) blocks inside the inverted residuals.
- The SE block recalibrates channel-wise features by learning per-channel weights, emphasising informative channels and suppressing less useful ones.
- With minimal extra parameters, SE improves representational power without compromising efficiency.

17.10.4 Activation Functions

- MobileNet uses **hard-swish** activation:

$$\text{hard-swish}(x) = x \cdot \frac{\text{ReLU6}(x+3)}{6}$$

- This is a computationally cheaper approximation of the swish function, suitable for low-power devices.
- `ReLU6` clips values at 6, providing numerical stability on fixed-point hardware.

17.10.5 Architecture Summary

- MobileNet consists of a stack of **inverted residual blocks with SE and hard-swish**.
- After feature extraction, global average pooling, a small linear classifier, and softmax provide classification.
- The network achieves competitive accuracy with a fraction of the parameters and FLOPs of ResNet or VGG, making it suitable for IoT and mobile deployment.

17.11 Baseline Training without Augmentation: Animal vs. Object Classification

17.11.1 Dataset

- Binary classification: distinguish images of animals (label 1) from objects/items (label 0).
- Images are 224×224 RGB, resized for training.
- Training set: 180 samples; validation and test sets: 30 samples each.
- Channels are rearranged to PyTorch's (C, H, W) format.

17.11.2 Model: Simple CNN (AlexNet style)

- A sequence of convolutional blocks (Conv2d $\rightarrow$ ReLU $\rightarrow$ MaxPool2d) followed by flatten and linear classifier.
- Uses several stacked conv layers with increasing filters.
- Large number of parameters concentrated in the fully-connected head (due to flattening).
- Softmax activation at the output for two classes.

17.11.3 Training and Overfitting

- Training for up to 1000 epochs with early stopping (patience 120).
- Observed behaviour:
  - Training loss decreases and can reach very low values.
  - Validation loss starts increasing after a few epochs — clear **overfitting**.
  - Training F1 score nearly perfect, but validation F1 stagnates at a low level (around 0.33–0.55).
- The extremely small training set (180 images) leads to severe overfitting, motivating the need for data augmentation.

17.12 Image Augmentation

17.12.1 Motivation

- When the training set is limited, artificial expansion via **label-preserving transformations** can significantly improve generalisation.
- Two broad families of augmentations:
  1. **Geometric**: flips, rotations, translations, scaling, shearing.
  2. **Photometric**: brightness, contrast, noise, colour jitter.

17.12.2 PyTorch Implementation

- `torchvision.transforms.Compose` allows chaining multiple transformations.
- Each transformation is applied with a given probability, creating a diverse set of augmented images on-the-fly during training.
- Example: a simple augmentation pipeline

```
from torchvision import transforms
aug = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.5)
])
```

- The original PIL image is passed through the composed transform; the output is a randomly flipped version.
- Visualisation shows that, depending on the random draw, the image may be flipped horizontally, vertically, or both (or neither).

17.12.3 Geometric Augmentations

- **Random flips**: horizontal and vertical mirroring are common and preserve the semantic content of most natural images.
- **Random affine transformations**: include rotation, translation, scaling, and shearing.
  - `transforms.RandomAffine(degrees, translate, scale, shear)` can randomly rotate within a range, shift, zoom, and shear the image.
  - Care must be taken to avoid excessive distortion that changes the class label or introduces unrealistic artifacts.
- All geometric transforms operate on the PIL image before conversion to tensor and normalisation.
- The combination of multiple transforms produces a virtually infinite supply of slightly different training samples, helping to reduce overfitting.

(Note: The discussion continues with photometric transformations and transfer learning in subsequent parts of the session.)

17.13 Geometric Augmentations (cont.)

17.13.1 Random Affine Transformations

- `transforms.RandomAffine(degrees, translate, scale, shear)` performs random rotation, translation, scaling, and shearing.
- **Translation**: shift the image by up to a fraction of its size (e.g., `translate=(0.2, 0.2)`).
  - Teaches the network to be invariant to the position of objects.
  - Pixels shifted outside the frame are filled with a constant value (default 0) or a specified fill (e.g., dataset mean).
- **Rotation**: randomly rotate by up to `degrees` (e.g., 72°). Encourages rotation invariance.
- **Scaling**: random zoom in/out (e.g., `scale=(0.8, 1.2)`). Simulates changes in object size/distance.
- **Fill color**: can be set to a constant, the mean pixel value, or a computed background, so the network learns to ignore padded regions.

17.13.2 Composing Multiple Geometric Transforms

- `transforms.Compose` allows chaining flips, affine transforms, etc., so every training sample undergoes a different combination.
- Example: horizontal flip + vertical flip + random affine (rotation, translation, scaling).
- At each epoch, the same base image is seen under a new, slightly modified appearance, effectively expanding the dataset and reducing overfitting.

17.14 Photometric Augmentations

17.14.1 Color Jitter

- Modifies the brightness, contrast, saturation, and hue of an image without changing its geometry.
- `transforms.ColorJitter(brightness=0.5, contrast=0.5, saturation=0.5, hue=0.2)` randomly perturbs each property within the given range.
- Brightness change: add/subtract a constant to all pixels.
- Contrast: scale pixel values around the mean.
- Saturation: adjust the intensity of colours.
- Helps the network become robust to lighting and colour variations in real-world images.

17.14.2 Composing Geometric and Photometric Augmentations

- A typical training pipeline combines both types:

```
transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomAffine(...),
    transforms.ColorJitter(...),
    transforms.ToTensor(),
    transforms.Normalize(mean, std)
])
```

- This creates a diverse set of training samples, covering appearance changes that do not alter the semantic label.

17.15 Important Considerations for Augmentation

17.15.1 Label Preservation

- Every augmentation must preserve the ground-truth label.
- Counterexample: rotating an analog clock changes the time it shows; horizontal flipping of text may reverse letters.
- For tasks like fine-grained classification (e.g., flower species), excessive colour change or severe geometric distortion might alter the discriminative features.
- Always verify that the applied transformations make sense for the specific problem domain.

17.15.2 Advanced Augmentations: Random Erasing and Cutout

- **Random Erasing**: randomly selects a rectangular region in the image and replaces it with random noise or a constant value (e.g., mean pixel).
  - Simulates occlusion, forcing the network to use the whole object rather than a single distinctive part.
  - Must be applied with care: erasing an eye may change a "cat" into a "no cat" if the dataset is too small.
  - Can be applied multiple times (multiple cutouts) for stronger regularisation.
- **Cutout**: a variant that replaces regions with zeros or the mean, often used on CIFAR-like datasets.
- Adding noise after erasing helps prevent the network from relying on the black/inpainted patches as a cue.

17.16 Implementing Augmented Datasets in PyTorch

17.16.1 Custom Dataset with Transforms

- Create a dataset class (e.g., `AugmentedDataset`) that accepts a transform pipeline.
- In `__getitem__`, load the PIL image, apply the transform (which includes `ToTensor` and normalisation), and return the transformed tensor and label.
- During training, pass the composition of all augmentations; during validation/testing, use only `ToTensor` and normalisation (no geometric/photometric jitter).
- DataLoader is built identically for both training and validation sets, but the underlying dataset's transforms differ.

17.16.2 Training with Augmentation

- The model architecture remains unchanged; augmentation only affects the input pipeline.
- Training loss and F1 curves show slower initial convergence on the training set (because the network sees a new variation each epoch) but better generalisation:
  - Validation F1 continues to improve and does not plateau as quickly.
  - Overfitting is significantly reduced compared to the baseline without augmentation.

- TensorBoard comparison confirms that the augmented model achieves higher validation F1 while the non-augmented model overfits quickly.

17.17 Transfer Learning and Fine-Tuning with MobileNet

17.17.1 The Oxford Pets Dataset

- A fine-grained classification dataset containing 37 categories of cats and dogs.
- Images are resized to 128×128 (or 224×224 for MobileNet compatibility).
- Approximately 100 images per class, balanced.
- Challenging due to subtle inter-class differences and a relatively small number of samples.

17.17.2 Preprocessing for Pretrained Models

- Pretrained models (e.g., MobileNetV3) expect input normalized with specific ImageNet statistics:

$$\text{mean} = [0.485, 0.456, 0.406], \quad \text{std} = [0.229, 0.224, 0.225]$$

- Resize images to the required input size, then apply the same normalisation used during the original training.
- Augmentation (flips, rotations, etc.) is applied on the training set before normalisation to increase diversity.

17.17.3 Three Training Strategies

1. **From Scratch**: Initialise MobileNet with random weights and train the entire network on the Oxford Pets dataset.
   - Suitable only when large amounts of data are available; with small datasets, severe overfitting occurs.
   - Serves as a baseline.
2. **Transfer Learning**: Load MobileNet pretrained on ImageNet, **freeze** all its layers, replace the classifier head with a new one (e.g., linear layer with 37 outputs), and train *only* the new head.
   - Leverages generic features already learned; the small dataset only needs to adapt the final decision boundaries.
   - Freezing prevents the pretrained weights from being destroyed by large gradients from the randomly initialised head.
   - Very fast training and strong performance with limited data.
3. **Fine-Tuning**: After training the new head (transfer learning), **unfreeze** some of the last layers of the backbone and continue training with a very low learning rate.
   - Allows the feature extractor to adapt its higher-level representations to the specifics of the new dataset (e.g., fine-grained pet breeds).
   - Typically unfreeze only the deeper convolutional blocks; early layers (edges, textures) remain frozen.
   - Careful learning rate scheduling is essential to avoid catastrophic forgetting.

17.17.4 Implementation Outline in PyTorch

- Load MobileNetV3-Small (or Large) using `torchvision.models.mobilenet_v3_small(weights='IMAGENET1K_V1')`.
- Replace the classifier:
  - Access the backbone via `model.features` (or `model.children()`).
  - Create a new `nn.Sequential` with `AdaptiveAvgPool2d(1)`, `Flatten`, `Linear`, and `Softmax`.
- For transfer learning: set `param.requires_grad = False` for all parameters in the backbone, `True` only for the new head.
- For fine-tuning: after training the head, unfreeze some of the last layers (e.g., `model.features[-1].requires_grad = True`) and reduce the learning rate (e.g., 10× smaller than the head lr).
- Training loops remain identical; the data loader supplies augmented training samples and non-augmented validation/test samples.

17.18 Transfer Learning and Fine-Tuning with MobileNet (cont.)

17.18.1 Training Strategies

- **Backbone**: synonym for the feature extraction part of a CNN (convolutional layers), before the classification head.
- Three approaches compared on the Oxford Pets dataset (37 classes, small training set):
  1. **From Scratch**:
     - Load MobileNetV3-Small with **random** (blank) weights – no pretrained knowledge.
     - Replace the original classifier with a custom head (e.g., Linear → HardSwish → Dropout → Linear).
     - Train all parameters (over 1 million) on the target dataset.
     - Result: heavy overfitting, validation accuracy ≈ 30%. Even with augmentation, the dataset is too small for such a large model.
  2. **Transfer Learning**:
     - Load MobileNetV3-Small **pretrained on ImageNet** (`weights='IMAGENET1K_V1'`).
     - **Freeze** the entire backbone: set `requires_grad = False` for all its parameters.
     - Replace the classifier head with a new one (randomly initialised).
     - Train *only* the new head; the backbone acts as a fixed feature extractor.
     - Result: validation accuracy rises to ≈ 52% in just 50 epochs. The heavy lifting of feature extraction is already done; only the decision boundaries need learning.
  3. **Fine-Tuning**:
     - Start from the transfer learning model (pretrained backbone + trained new head).
     - **Unfreeze** the last N layers of the backbone (e.g., $N = 20$). Early layers (edges, textures) remain frozen; deeper layers adapt to the specifics of the new data.
     - Set a **lower learning rate** (e.g., 10× smaller than the head's rate) to avoid destroying the valuable pretrained weights.
     - Train the unfrozen backbone layers together with the classification head.
     - Result: validation accuracy improves further to ≈ 64%, as the feature extractor now tailors its high-level representations to the pet breeds.

17.18.2 Implementation Details

- The backbone is accessed via `model.features` in MobileNetV3; the head is replaced by assigning a new `model.classifier`.
- Freezing: iterate over `model.features.parameters()` and set `param.requires_grad = False`.
- Unfreezing selected layers: for the last k modules in `model.features`, set `requires_grad = True`.
- Optimizer: only parameters with `requires_grad = True` are passed to the optimizer.
- Training loops remain identical; the data loader applies augmentation only to the training set.

17.18.3 Performance Comparison

- Training from scratch converges slowly and severely overfits ($\sim$30% accuracy).
- Transfer learning (frozen backbone) reaches $\sim$52% accuracy quickly, with much less overfitting.
- Fine-tuning pushes accuracy to $\sim$64%, showing that a moderate adaptation of the pretrained features yields the best trade-off for a small target dataset.
- TensorBoard curves confirm: the fine-tuned model starts from the transfer learning baseline and continues to improve, while the from-scratch model plateaus at a low level.

17.19 Image Retrieval in the Latent Space

17.19.1 Motivation and Concept

- A CNN trained for classification (e.g., on ImageNet) learns a semantically meaningful **latent space**.
- In this space, images of the same class or similar content are mapped close together, regardless of low-level pixel differences.
- This property can be exploited for **content-based image retrieval**: given a query image, find the most similar images in a database by comparing their feature vectors (embeddings) rather than raw pixels.

17.19.2 Using a Pretrained Network as a Feature Extractor

- Remove the classification head, keeping only the convolutional backbone (up to the global average pooling layer).
- The output of this truncated network is a compact feature vector (the latent representation).
- For all images in the database, compute and store their feature vectors.
- For a query image, compute its feature vector, then find the nearest neighbours in the database using Euclidean (or cosine) distance.

17.19.3 Example: Impact of Feature Quality

- Using a poorly trained, shallow CNN as the feature extractor leads to poor retrieval: for a query dog, the top results may include monkeys, manholes, and unrelated objects — the latent space does not reflect semantics.
- Using a **pretrained MobileNetV3** (even without fine-tuning) yields meaningful neighbours: the top results for a dog query are other dogs, perhaps a bear (visually similar), but all are semantically coherent.
- This demonstrates that the features learned on a large, diverse dataset (ImageNet) capture general visual concepts, making the latent space suitable for retrieval tasks without any task-specific training.

17.19.4 Workflow

1. Build an `EmbeddingModel` by taking the pretrained backbone (without the classifier).
2. Pre-compute and store the feature vectors for the entire image collection.
3. For a query image, obtain its feature vector, compute distances to all stored vectors, and sort them.
4. Retrieve the top-K images with the smallest distances as the most similar.
5. This approach is the foundation of reverse image search engines and is widely used in industrial applications (e.g., defect detection, product recommendation).

Chapter 18

WSL (CAM), Explainability, Other Architectures

18.1 Data Pre-processing for Neural Networks

18.1.1 Why Normalise Input Data

- Gradient-based optimisation (e.g., SGD, Adam) works best when data are centered around the origin and have comparable scales across dimensions.
- Pre-processing prevents numerical instability and speeds up convergence.

18.1.2 Common Normalisation Techniques

- **Zero-centering**: subtract the mean vector $\boldsymbol{\mu}$ of the training set so that the data cloud is centered at the origin.
- **Z-score normalisation**: subtract the mean and divide by the standard deviation along each feature,

$$\widehat{x}_i = \frac{x_i - \mu_i}{\sigma_i},$$

  yielding zero mean and unit variance per feature. Correlations between features are preserved.
- **PCA whitening**: rotate the data to a basis where features are uncorrelated and, optionally, scale to unit variance – the covariance matrix becomes the identity.

18.1.3 Pre-processing for Image Data

- **AlexNet** (ImageNet): subtract the mean image (computed over the whole training set), i.e. a single average RGB value per pixel position.
- **ResNet** etc.: apply per-channel Z-score normalisation – three mean values and three standard deviations for R, G, B.
- **Min-max rescaling** (to $[0, 1]$): requires no dataset-level parameters, therefore can be applied without a training set.

18.1.4 Crucial Rules

- All pre-processing parameters (means, standard deviations) must be **estimated only from the training set** and then applied identically to validation and test data.

- When using a **pre-trained model** (e.g., for transfer learning), always apply the exact same pre-processing that was used during the original training – otherwise the input distribution seen by the first layer changes and the weights may not work as expected.

- PyTorch provides the pre-processing parameters together with the model (e.g., the normalisation constants for ImageNet-trained models).

18.2 Batch Normalisation

18.2.1 Motivation

- During training, the distribution of activations inside a deep network can shift (internal covariate shift), slowing down learning.

- **Batch Normalisation** (BN) normalises activations within a mini-batch, making training faster and more stable.

- BN also acts as a mild regulariser, reducing the need for dropout or careful weight initialisation.

18.2.2 Operation

For a mini-batch of m samples and a fixed channel k (spatial dimensions omitted for simplicity), let the activations be $x^{(1)}, \ldots, x^{(m)}$.

1. Compute the **batch mean** and **batch variance**:

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x^{(i)}, \qquad \sigma^2_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)} - \mu_{\mathcal{B}})^2.$$

2. **Standardise**:

$$\widehat{x}^{(i)} = \frac{x^{(i)} - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}},$$

   where ϵ is a small constant for numerical stability.

3. **Scale and shift** with learnable parameters γ_k and β_k (one pair per channel):

$$y^{(i)} = \gamma_k \, \widehat{x}^{(i)} + \beta_k.$$

The result is a channel-wise normalised activation distribution that the network can freely shift and scale.

18.2.3 Parameters

- **Non-trainable** (computed from data): per-channel batch mean $\mu_{\mathcal{B}}$ and variance $\sigma^2_{\mathcal{B}}$.

- **Trainable** (optimised by gradient descent): per-channel γ_k (scale) and β_k (shift).

- Total learnable parameters of a BN layer: $2 \times$ (number of channels).

- The non-trainable parameters are not updated by the loss; they are derived from the current batch during training.

18.2.4 Training vs. Inference

- **During training**: BN uses the mean and variance of the current mini-batch.
- **During inference**: the model cannot rely on batch statistics. Instead, it uses **running averages** of the mean and variance collected over the training set (often via exponential moving averages). These become fixed parameters of the trained model.
- Once the running statistics are fixed, a BN layer becomes a deterministic linear transformation per channel:

$$y = \frac{\gamma}{\sqrt{\mathrm{Var}[x] + \epsilon}}\, x + \left(\beta - \frac{\gamma\, \mathbb{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} \right).$$

18.2.5 Fusing Batch Normalisation with a Preceding Convolution

- After training, the BN transformation is a simple channel-wise affine map.
- It can be **absorbed** into the weights and biases of the immediately preceding convolutional layer (or fully-connected layer).
- This reduces the number of operations at inference time without changing the network's output, a common optimisation for deployment.

18.3 Outlook

- Batch Normalisation is now a standard component of modern CNN architectures (ResNet, Inception, MobileNet, etc.).
- The next topics will address weakly supervised localisation and a deeper look into ResNet and Inception blocks, completing the study of basic visual recognition models.

18.4 Multi-label Classification

18.4.1 Limitation of Single-Label Assumption

- Many real images contain **multiple objects** (e.g., a cat and a dog together).
- A standard classifier with a softmax output forces a single label per image (the probabilities sum to one), making it impossible to express that two classes are present simultaneously.

18.4.2 Architecture Modification

- Replace the final **softmax** activation with a **sigmoid** activation applied independently to each output neuron.
- Each output neuron now produces a value in $[0, 1]$ that can be interpreted as the probability of the corresponding class being present, *independently* of the others.
- The target vector is encoded as a **binary vector** (e.g., $[1, 1, 0, \dots]$ for an image containing both cat and dog), not a one-hot vector.
- The loss function becomes **binary cross-entropy** (instead of categorical cross-entropy).
- This approach allows the network to predict any subset of the classes, including the empty set ("no label").

18.5 Weakly Supervised Localisation with Class Activation Maps

18.5.1 Can a Classification Network Localise Objects?

- A CNN trained **only with image-level labels** (e.g., "brushing teeth", "cutting trees") can still identify *where* the discriminative object/action occurs in the image.
- This is possible even though the network never saw bounding-box annotations during training.
- The technique relies on a specific architecture: **Global Average Pooling (GAP) followed by a single dense layer**, and produces a **Class Activation Map** (CAM).

18.5.2 Required Architecture

- **Backbone**: standard convolutional feature extractor ending with a convolutional layer that produces a volume of size $H \times W \times C$.
- **GAP**: reduces each of the C feature maps to a single scalar,

$$F_k = \frac{1}{HW} \sum_{x,y} f_k(x, y), \qquad k = 1, \ldots, C.$$

- **Classifier**: a single dense (linear) layer that maps the C-dimensional vector $(F_1, \ldots, F_C)$ to L class scores (logits). No hidden layers; only one weight matrix $\mathbf{W} \in \mathbb{R}^{C \times L}$ (plus biases, but bias is often ignored for CAM computation).
- Optionally, a softmax is applied to obtain class probabilities, but the CAM is computed from the logits.

18.5.3 Class Activation Map Derivation

For a given class c, the score (logit) before softmax is

$$S_c = \sum_{k=1}^{C} w_{k,c}\, F_k = \sum_{k=1}^{C} w_{k,c} \left(\frac{1}{HW} \sum_{x,y} f_k(x, y) \right) = \frac{1}{HW} \sum_{x,y} \underbrace{\left(\sum_{k=1}^{C} w_{k,c}\, f_k(x, y) \right)}_{M_c(x,y)}.$$

Thus S_c is the spatial average of the **Class Activation Map** $M_c(x, y)$.

- $M_c(x, y)$ is a 2D map (size $H \times W$) that indicates the contribution of each spatial location to the score of class c.
- The map is obtained entirely from quantities already present in the network: the feature maps f_k and the learned classifier weights $w_{k,c}$.
- It requires **no additional training** and can be computed for any image after the classification network has been trained.

18.5.4 Visualising and Using the CAM

- Upsample $M_c(x, y)$ (e.g., via bilinear interpolation) to the size of the input image, yielding a **heatmap** that highlights the regions most relevant for the predicted class.
- Examples from human activity recognition:
  - "Brushing teeth": the heatmap focuses on the **toothbrush**, not on the face or the human.
  - "Cutting trees": the heatmap highlights the **saw**, with little activation on the person's face.

- This demonstrates that the network learns to rely on **discriminative elements** (the toothbrush, the saw) rather than on the co-occurring human, because only those provide distinctive information for the task.
- After obtaining the heatmap, a simple bounding box can be produced (e.g., by thresholding the map and fitting a rectangle), yielding **localisation without any bounding-box supervision**.

18.5.5 Why This Works

- Convolutional layers preserve spatial information via translation equivariance; the feature maps $f_k(x, y)$ still contain location information.
- Each channel k of the last convolutional layer tends to act as a **semantic detector** (e.g., "dog nose", "yellow text on dark background"), as shown by the maximum activation experiments from previous lectures.
- GAP summarises *how much* of that semantic feature is present anywhere in the image.
- The weights $w_{k,c}$ learned by the linear classifier encode *how important* each semantic feature is for class c.
- The CAM $M_c(x, y) = \sum_k w_{k,c} f_k(x, y)$ therefore re-projects the classifier's decision back to the spatial domain, revealing which locations contain the features that drove the classification.

18.5.6 Practical Implications

- CAMs provide a simple, interpretable visualisation of what a CNN "looks at" when making a prediction, without extra training or architectural changes (provided the network already uses GAP + a single linear layer).
- The method can be extended to other global pooling strategies and forms the basis of later techniques (e.g., Grad-CAM) that remove the architectural constraint of a single dense layer.
- It exemplifies a broader principle: **networks trained for one task (classification) often learn representations that support other tasks (localisation)**, a key motivation for transfer learning and multi-task learning.

18.6 Class Activation Maps (CAM) for Weakly Supervised Localization

18.6.1 From Global Average Pooling to the Final Score

- After the convolutional backbone and GAP, the feature vector is $\mathbf{F} = (F_1, \ldots, F_C)$ where

$$F_k = \frac{1}{HW} \sum_{x,y} f_k(x, y).$$

- A single dense (linear) layer produces the class score (logit) for class c:

$$S_c = \sum_{k=1}^{C} w_{k,c} F_k + b_c \quad \text{(bias often omitted in CAM computation).}$$

- Because each F_k is a spatial average, this can be rewritten as

$$S_c = \frac{1}{HW} \sum_{x,y} \underbrace{\left(\sum_{k=1}^{C} w_{k,c} f_k(x, y) \right)}_{M_c(x,y)}.$$

- $M_c(x, y)$ is the **Class Activation Map** for class c – a 2D map (size $H \times W$) whose spatial average equals the class score.

18.6.2 Intuitive Interpretation

- Each channel k of the final convolutional layer acts as a **semantic detector** (e.g., "dog nose", "human face", "text on yellow background"), activating strongly where that specific pattern appears.
- The weight $w_{k,c}$ encodes the **relevance** of the k-th semantic feature for class c.
  - If the feature k is highly discriminative for class c, $w_{k,c}$ will be large (positive); if it is irrelevant or characteristic of another class, the weight will be small or negative.
- The CAM $M_c(x, y) = \sum_k w_{k,c} f_k(x, y)$ is therefore a **weighted combination of the feature maps**: it highlights regions that contain features important for class c and suppresses regions whose features are irrelevant or contradictory.
- Even though the network was trained only with image-level labels, the spatial information is preserved up to the final convolutional layer and can be recovered via this linear combination.

18.6.3 Visualising the CAM

- For a given input image and a target class c, compute $M_c(x, y)$.
- The spatial resolution of M_c is $H \times W$ (much smaller than the input image, e.g., 7×7 or 14×14). It is upsampled (e.g., bilinear interpolation) to the original image size to produce a **heatmap**.
- Examples:
  - **Australian Terrier**: the heatmap focuses on the dog's face, tail, and hand-interaction, ignoring the human body and background.
  - **Class "man"** for the same image: the heatmap shifts to the human's face and torso, because different weights (from the "man" output neuron) are used.
- This illustrates that CAMs are **class-specific**: the same feature maps can be combined differently to explain different classes.

18.6.4 Two Major Uses

1. **Explainability**: understand which parts of the image the network relies on for its decision (e.g., the face of the terrier, the saw in "cutting trees").
2. **Weakly Supervised Localization**: even though the network was not trained with bounding-box annotations, a rough object localisation can be obtained by thresholding the CAM and fitting a bounding box.

18.6.5 Limitation of CAM

- The derivation **requires a single linear layer** after the GAP. If multiple fully-connected layers (or other complex heads) are used, the direct back-projection $\sum w_{k,c} f_k$ is no longer possible.
- Networks like AlexNet and VGG, which have large multi-layer classifiers, cannot directly use this method.
- This architectural constraint may force a trade-off between classification performance and interpretability/localisation.
- The heatmap resolution is limited by the spatial size of the final convolutional layer; upsampling only partially recovers fine details.

18.6.6 GAP vs. Global Max Pooling

- CAMs can also be defined with global max pooling (GMP), but GAP tends to produce smoother, more diffuse maps that capture the whole extent of the relevant object.
- GMP highlights only the single most activated location per channel, which can be sparser and less complete for localisation.

18.7 Weakly Supervised Localization from CAM

18.7.1 Problem Setting

- **Goal**: localise objects with bounding boxes without any bounding-box annotations during training.
- **Given**: a network trained *only* for image classification (image-level labels).
- **Approach**: leverage the class activation maps produced by that network.

18.7.2 Procedure

1. Train a classifier (backbone + GAP + single dense layer) for the desired classification task.
2. For a test image and a chosen class c (usually the predicted class), compute the CAM $M_c(x, y)$.
3. Threshold the (upsampled) heatmap: for example, set to zero all values below 20% of the maximum activation.
4. The remaining positive region forms a binary mask; draw the tightest bounding box that encloses this mask.

18.7.3 Example

- A network trained on general object categories; for an image containing a grasshopper, the CAM for class "grasshopper" highlights the insect.
- The ground-truth bounding box (unseen during training) is shown in green; the bounding box derived from the thresholded CAM is shown in red – they are well aligned, demonstrating that the classifier implicitly learns to localise.

18.8 Going Deeper: Parallel Convolutional Architectures

18.8.1 Motivation for Parallel Paths

- The trend after AlexNet and VGG was to build **deeper** networks to learn richer, more discriminative features.
- However, simply stacking more layers poses two challenges:
  1. **Training deeper networks** becomes difficult due to vanishing gradients and optimization issues.
  2. **Object scale variation**: a distinctive visual feature (e.g., a cat's nose) may appear at very different sizes in different images (e.g., 40×40 pixels vs. 100×100 pixels). A fixed receptive field may fail to capture the feature at all scales.
- To address both, the **Inception module** (introduced in *GoogLeNet* / Inception v1, 2015) processes the input through multiple parallel convolutional branches with different filter sizes.

18.8.2 Inception Module: Multi-Scale Processing in One Block

- Instead of a single sequential stream, the block applies **several convolutional (and pooling) operations in parallel** to the same input activation volume.
- The branches are:
  - 1×1 convolution – captures very fine, pixel-wise patterns.
  - 3×3 convolution – captures medium-scale structures.
  - 5×5 convolution – captures larger spatial context.
  - 3×3 max pooling (with stride 1 and padding to preserve spatial size) – provides a pooling-based feature map that adds further non-linearity.
- **Padding**: each convolutional branch uses `same` padding so that all output feature maps have the **same spatial resolution** as the input.
- The outputs are then **concatenated along the depth (channel) dimension**, producing a single output volume with the original spatial size but a larger number of channels.

18.8.3 Concatenation and Dimensionality Example

- Input: $28 \times 28 \times 256$.
- Example branch configurations (number of filters per branch):
  - 1×1 conv: 64 filters $\rightarrow 28 \times 28 \times 64$
  - 3×3 conv: 128 filters $\rightarrow 28 \times 28 \times 128$
  - 5×5 conv: 32 filters $\rightarrow 28 \times 28 \times 32$
  - Max-pool + 1×1 conv (to adjust channels): e.g., 32 filters $\rightarrow 28 \times 28 \times 32$
- Concatenation yields output size $28 \times 28 \times (64 + 128 + 32 + 32) = 28 \times 28 \times 256$ (or any desired total).
- By concatenating features extracted at different scales, the subsequent layers can simultaneously see fine details and broader context.

18.8.4 Why Parallel Convolutions Help

- The network does not need to commit to a single filter size; it can learn which scale is most relevant for each feature and location.
- The 1×1 convolutions (often used as bottlenecks before the expensive 3×3 and 5×5 branches, as discussed in the lab) reduce computational cost and introduce additional non-linearities.
- This design enables **going deeper** with more layers while keeping the parameter budget under control and improving scale invariance.
- Inception networks (GoogLeNet, Inception v2, v3, etc.) became highly successful, demonstrating the power of multi-scale parallel processing inside a single layer.

18.9 Inception Networks: Multi-Scale Processing with Bottlenecks

18.9.1 The Need for Efficient Multi-Scale Blocks

- The Inception module processes the same input through parallel branches with different filter sizes (1×1, 3×3, 5×5) and a pooling branch, then concatenates the outputs.

- This provides features at multiple receptive field scales simultaneously, addressing the problem of object scale variation.
- However, the direct concatenation of high-dimensional feature maps leads to an explosion in the number of operations and parameters, especially when stacked deeply.
- For example, a single naive Inception block can require hundreds of millions of operations, making very deep networks computationally infeasible.

18.9.2 Reducing Computation with 1×1 Bottlenecks

- The solution is to insert a 1×1 **convolutional layer** before the expensive 3×3 and 5×5 convolutions.
- A 1×1 convolution acts as a **bottleneck**: it reduces the number of channels (depth) while preserving spatial dimensions.
  - Example: an input of $56 \times 56 \times 64$ can be compressed to $56 \times 56 \times 32$ by using only 32 1×1 filters.
  - The subsequent 3×3 or 5×5 convolution then operates on this lower-depth volume, drastically reducing the number of multiplications and parameters.
- After the max-pooling branch, a 1×1 convolution is also added, not primarily for compression, but to introduce learnable processing along that path.
- The overall Inception module thus becomes computationally efficient while retaining multi-scale feature extraction.

18.10 GoogLeNet (Inception v1) Architecture

- GoogLeNet consists of 27 layers: initial standard convolutional layers, followed by **9 Inception modules**, and finally a Global Average Pooling (GAP) layer, a single linear classifier, and softmax.
- Total parameters: only **5 million** – about 20–25 times fewer than VGG's 138M.
- The vast majority of parameters are now spent in the feature extraction (convolutional) part, not in a huge fully-connected head.
- A simple linear classifier on the GAP features is sufficient because the Inception modules learn extremely rich, discriminative features.

18.10.1 Auxiliary Classifiers for Gradient Flow

- To combat vanishing gradients in such a deep network, two auxiliary classification heads were attached at intermediate layers during training.
- These heads take activations from earlier stages, flatten them, and apply a small classifier; their loss is combined with the main loss.
- This injects additional gradient signals into the lower layers, improving training stability.
- At inference time, the auxiliary heads are removed.

18.11 The Problem of Very Deep Networks

- Intuitively, a deeper network should be at least as good as a shallower one: the extra layers could simply learn the identity mapping and pass information through.
- However, experiments showed that a 56-layer network performed worse than a 20-layer network on **both training and test sets**.
- This is not overfitting (where training error would be lower but test error higher); it is an **optimization difficulty** – the deeper network fails to learn a good solution even on the training data.
- The core issue: learning the identity mapping via stacked non-linear layers is surprisingly difficult for stochastic gradient descent.

18.12 Residual Networks (ResNet)

18.12.1 Skip Connections and Residual Learning

- To make identity mappings easily learnable, ResNet introduces **skip connections** (or shortcuts) that add the input of a block directly to its output.
- A standard block computes some transformation $\mathcal{F}(x)$; the residual block outputs $\mathcal{H}(x) = \mathcal{F}(x) + x$.
- If the optimal mapping for that block were the identity, the network can simply drive the weights of $\mathcal{F}$ to zero, which is easier than learning $\mathcal{F}(x) = x$ directly.
- The layers thus learn a **residual function** $\mathcal{F}(x) = \mathcal{H}(x) - x$, giving the network its name.

18.12.2 Block Design and Practical Considerations

- A typical residual block contains two or three convolutional layers with Batch Normalization and ReLU.
- The skip connection requires the input and output to have the **same dimensions** (spatial size and number of channels).
- When dimensions change (e.g., after downsampling or channel doubling), a 1×1 convolution is used in the skip connection to project the input to the correct shape (projection shortcut).
- Stacks of residual blocks allow training networks with **152 layers** (and even 1000+ layers on CIFAR-10) without degradation.
- ResNet achieved a dramatic reduction in classification error, surpassing human-level performance on ImageNet.

18.12.3 Impact and Variants

- The ResNet principle enabled the deep learning community to scale network depth to hundreds of layers, leading to significant improvements in accuracy.
- Subsequent architectures combined residual learning with Inception modules (Inception-ResNet) or widened the residual blocks (Wide ResNet), further pushing performance.
- The idea of skip connections has become a fundamental component in modern network design, including beyond vision (e.g., transformers).

18.13 Concluding Remarks and Outlook

- The evolution from AlexNet to ResNet illustrates a clear trajectory: deeper networks, more sophisticated blocks (Inception, residual), and a shift of parameters from the classifier head to the feature extractor.
- Key innovations – GAP, 1×1 bottlenecks, batch normalization, skip connections – all serve to make very deep networks trainable and efficient.
- The next lectures will move beyond image classification to **higher-level visual recognition tasks**: semantic segmentation and object detection, where these CNN backbones are used as the core feature extractors.

Chapter 19

Semantic Segmentation, Fully Convolutional CNN

19.1 Introduction to Image Segmentation

19.1.1 From Unsupervised Clustering to Semantic Segmentation

- **Traditional image segmentation**: partition an image into disjoint regions based on low-level similarity (e.g., colour). This was an unsupervised clustering problem.
- **Modern semantic segmentation** (deep learning era): partition the image into regions that correspond to *semantic categories* (e.g., person, car, road). Each pixel is assigned a class label, including a "background" class.
- The task can be seen as **pixel-wise classification**: the output is a segmentation map S of the same spatial dimensions as the input image, where each pixel value belongs to a predefined set of labels Λ.

19.1.2 Distinction from Instance Segmentation

- Semantic segmentation does **not** separate different instances of the same class; all pixels labelled "person" form one merged region, regardless of how many individuals are present.
- Counting individuals or distinguishing separate objects of the same class requires **instance segmentation**, which is a different problem (often addressed in object detection pipelines).
- The lack of instance separation limits the use of semantic segmentation for tasks like counting people in a crowd, but it is perfectly suited for measuring areas covered by certain classes (e.g., crop types, roads, tumours).

19.2 Supervised Learning Setup for Segmentation

19.2.1 Training Data Requirements

- A training set for semantic segmentation consists of pairs (input image, segmentation ground truth).
- The ground truth is a dense label map, typically created by manually delineating object boundaries with polygons or pixel-level painting.
- Creating such annotations is **extremely time-consuming**; however, large public datasets exist for natural images.

19.2.2 The COCO Dataset (2014)

- **Common Objects in Context** (COCO), released by Microsoft.
- Over 200 000 images annotated for multiple tasks: semantic segmentation, instance segmentation, and human keypoints.
- Covers 80 object categories; annotations include separate instance masks, which can be merged to obtain semantic segmentation labels.
- The dataset can be queried by object categories to retrieve relevant images, facilitating training and evaluation.

19.3 Utility of Semantic Segmentation

- While instance-aware tasks dominate autonomous driving, semantic segmentation is critical in domains where the goal is to **measure areas** rather than count instances:
  - **Remote sensing**: crop type mapping, urban area monitoring.
  - **Medical imaging**: tumour volume estimation, organ delineation.
- In these cases, the exact separation of individual instances is unnecessary; the class-specific coverage area is the primary interest.

19.4 A Naive Baseline: Patch-Based Classification

19.4.1 Principle

- Before dedicated segmentation architectures, a straightforward approach is to reuse a standard CNN classifier.
- From the training images, **crop small patches** (e.g., 90×90 pixels). For each patch, define its label as the *label of the central pixel* (obtained from the ground-truth segmentation map).
- Train a classifier to predict the central-pixel class from the surrounding image patch.
- At inference, slide the patch over the whole image, classify the centre pixel at each position, and assemble the segmentation map.

19.4.2 Why Central-Pixel Labelling?

- If the label were assigned based on the **majority** of pixels in the patch, fine structures (e.g., a thin stem) would be overwhelmed by the background and lost.
- Labelling by the central pixel forces the network to focus on the local semantics at the exact prediction location, preserving details and accurate boundaries.

19.4.3 Limitations

- **Redundant computation**: neighbouring patches overlap heavily; the same pixels are processed many times, making inference extremely slow.
- **Fixed receptive field**: the patch size determines the context available for classification; if chosen too small, the network lacks global information; if too large, fine details may be missed and computational cost grows.
- This approach does not exploit the fact that a single forward pass through a fully convolutional network could produce a whole segmentation map at once, which motivates the development of dedicated architectures.

19.5 Semantic Segmentation: Definition and Motivation

19.5.1 From Unsupervised Clustering to Pixel-Wise Classification

- **Traditional image segmentation** (pre-deep learning): partition an image into disjoint regions based on low-level similarity (e.g. colour). This is an unsupervised clustering problem, with no semantic labels.

- **Modern semantic segmentation** (deep learning era): assign a *semantic class label* to every pixel. The output is a segmentation map S of the same spatial dimensions as the input image, where $S(r, c) \in \Lambda$ and Λ is the set of possible classes (always including a "background" class).

- The task can be seen as **pixel-wise classification**: unlike image-level classification, the prediction is dense, covering the whole image.

19.5.2 Distinction from Instance Segmentation and Object Detection

- Semantic segmentation does **not** separate different instances of the same class. All pixels labelled "person" are merged into one region, making it impossible to count individuals.

- **Instance segmentation** (and object detection) provide individual instance boundaries or bounding boxes; semantic segmentation only provides a *per-class* spatial coverage.

- **Utility of semantic segmentation**: ideal for tasks where the goal is to measure the area covered by certain classes, e.g. crop mapping, tumour volume estimation, road coverage in remote sensing.

19.6 Supervised Learning Setup for Segmentation

19.6.1 Training Data and Annotation

- Training a segmentation model requires pairs of (input image, ground-truth segmentation map).

- Annotating segmentation maps is extremely time-consuming: every pixel must be assigned a label, often via polygon delineation or pixel-level colouring.

- Nevertheless, large public datasets exist; COCO (Common Objects in Context, 2014) provides over 200,000 images annotated for semantic segmentation, instance segmentation, and human keypoints across 80 categories.

19.7 A Naive Baseline: Patch-Based Classification

19.7.1 Central-Pixel Labelling

- Before dedicated segmentation architectures, a straightforward method is to reuse a standard CNN classifier on image patches.

- For each training image, crop small patches (e.g. 90×90 pixels). Label each patch with the class of its **central pixel**, taken from the ground-truth segmentation.

- Reason: if the patch were labelled by the majority class, fine structures (e.g. a thin stem) would be overwhelmed by the surrounding background and lost.

19.7.2 Severe Limitations

- **Massive redundant computation**: neighbouring patches overlap heavily; the same pixels are processed many times, making inference extremely slow.
- **Fixed receptive field**: the patch size determines the context available for classification – too small misses global information, too large loses fine detail and increases cost.
- **Explosion of training data**: storing all cropped patches and their labels multiplies the training set size, becoming impractical for large images.
- This approach is not competitive and motivated the development of fully convolutional architectures.

19.8 Fully Convolutional Networks (FCNs) for Segmentation

19.8.1 Convolution-Only Architecture

- A natural extension of CNN classifiers is to **remove all fully-connected layers** (and flattening/global pooling) and keep only convolutional layers.
- With `same` padding and stride 1, the spatial dimensions are preserved throughout the network.
- The final convolutional layer has L filters (one per class), producing an output volume of size $H \times W \times L$.
- Apply a **softmax depth-wise** (i.e. along the channel axis at each pixel) to obtain a probability distribution over the L classes for each pixel.
- The predicted segmentation is obtained by taking the **argmax** across the channel dimension.

19.8.2 Limitations

- **Slow growth of receptive field**: without pooling or strided convolutions, the receptive field expands very slowly with depth, making it hard to capture high-level semantic content.
- **Huge computational and memory cost**: activations remain at full image resolution throughout the network, requiring an enormous number of operations and parameters.
- This motivates architectures that temporarily reduce spatial resolution for semantic extraction, then restore it for precise localisation.

19.9 Encoder-Decoder Architecture

19.9.1 Conflicting Goals

- Semantic segmentation faces two opposing requirements:
  1. **Semantic extraction** (the "what"): requires a large receptive field, achieved by reducing spatial resolution and increasing channels.
  2. **Accurate localisation** (the "where"): requires fine spatial detail, best preserved at high resolution.
- The solution is an **encoder-decoder** structure:
  - **Encoder** (contracting path): progressively downsamples the feature maps, extracting high-level semantic features (like a standard CNN classifier's backbone).
  - **Decoder** (expanding path): progressively upsamples the features back to the original image size, recovering spatial detail.

19.9.2 The Spatial Bottleneck

- At the bottleneck, the spatial resolution is at its minimum, and each channel corresponds to a different high-level semantic detector (e.g. “smiling face”, “dog nose”).
- The decoder must map these compact, semantically rich features to a full-resolution segmentation map.

19.10 Upsampling Strategies for the Decoder

19.10.1 Non-Learnable Upsampling

- **Nearest-neighbour upsampling**: each value is simply replicated to fill a larger block (e.g. $2 \to 4$ by repeating every entry).
- **Bed of nails**: the enlarged feature map is filled with zeros everywhere except at fixed positions (e.g. top-left of each block), where the original value is placed.
- **Max unpooling**: during max pooling in the encoder, the location of the maximum is stored. In the decoder, the value is placed back into that exact position, and zeros are placed elsewhere.
- All these methods operate **channel-wise** and have **no trainable parameters**. They simply produce a larger, sparse version of the input feature map without learning how to best reconstruct details.

19.10.2 Learnable Upsampling with Convolutions

- A common technique: after a non-trainable upsampling step (e.g. nearest-neighbour or bed of nails), add one or more **convolutional layers**.
- This makes the overall upsampling block **learnable**: the network can learn how to combine and refine the upsampled features.
- Typically, each upsampling block inverts a downsampling block: it first increases the spatial size (e.g. by factor 2), then applies convolutional layers (with possible batch normalisation and activation) that usually **reduce** the number of channels.

19.10.3 Transposed Convolution (Learnable Upsampling)

- Transposed convolution (often called **deconvolution**) provides a fully learnable upsampling operation.
- **Principle**: the learned filter acts as a “building block”.
  - A low-resolution activation map (e.g. 2×2) is multiplied by a filter of size e.g. 3×3.
  - For each activation value, the filter (scaled by that value) is placed at the corresponding output position.
  - Where the filter windows overlap, the values are **summed**.
- In a 1D illustration:
  - Input: $[i_1, i_2]$; filter: $[f_1, f_2, f_3]$; stride 1.
  - i_1 contributes $[i_1 f_1, i_1 f_2, i_1 f_3]$.
  - i_2 contributes $[i_2 f_1, i_2 f_2, i_2 f_3]$, shifted by stride.
  - Overlap yields output length > 2; overlapping values are added.
- The filter parameters are learned via backpropagation, allowing the network to learn an optimal upsampling kernel for each channel.
- Transposed convolution effectively performs a **learnable mapping from a low-resolution feature space to a high-resolution output**, mixing information across channels in the same way a standard convolution does, but in the opposite direction.

19.11 Transposed Convolution Details

19.11.1 Filter Dimensions

- In a transposed convolution, the filter must have the same depth as the number of input channels (like standard convolution), and the number of filters determines the number of output channels.
- Using a stride > 1 in the decoder expands spatial dimensions (the opposite of standard convolution in the encoder).

19.11.2 Terminology

- Transposed convolution is also known as:
  - **Fractionally strided convolution** (the filter moves by fractional pixel steps to produce a larger output)
  - **Backward strided convolution**
  - **Deconvolution** (misleading, since it is not the mathematical inverse of convolution)
- The preferred term is **transposed convolution**.

19.11.3 Learnable Upsampling Blocks

- A learnable upsampling block can be built either by:
  1. A non-trainable upsampling (nearest neighbour, bed of nails) followed by one or more standard convolutions.
  2. A transposed convolution directly.
- All segmentation architectures contain such learnable upsampling modules.
- After the final upsampling layer, the volume has L channels (one per class). A softmax along the channel dimension produces per-pixel class probabilities; the argmax yields the segmentation map.

19.12 Training a Segmentation Network

19.12.1 Loss Function

- The loss is computed **pixel-wise** using the categorical cross-entropy between the predicted probability vector and the one-hot encoded ground-truth label at each pixel.
- Formally, for a pixel at position j:

$$\mathcal{L}_j = -\sum_c y_{j,c} \log p_{j,c}$$

  where $y_{j,c}$ is 1 if pixel j belongs to class c, and $p_{j,c}$ is the softmax output.
- The total loss for an image is the **sum (or average) over all pixels**.
- Because an image provides a very large number of pixel samples, a single image already constitutes a large batch. Many segmentation networks (e.g. U-Net) can be trained with batch size 1.

19.12.2 Extension to Per-Pixel Regression

- If the task is a dense regression (e.g. predicting a depth or height map), the categorical cross-entropy is replaced by a mean squared error loss, again summed over all pixels.

19.13 U-Net: The Classic Encoder–Decoder Architecture

19.13.1 Overview

- U-Net is a **fully convolutional** encoder–decoder network designed for biomedical image segmentation (e.g. cell boundaries in electron microscopy).
- **Encoder (contracting path)**: a series of 3×3 convolutions (valid padding, no zero-padding) followed by ReLU, and 2×2 max pooling with stride 2.
- **Decoder (expanding path)**: transposed convolution (2×2, stride 2) to double the spatial size and halve the number of channels, concatenation with the corresponding feature map from the encoder, and then 3×3 convolutions + ReLU.
- **Bottleneck**: at the bottom, the spatial resolution is minimal (32×32) but with a high number of channels (1024), representing the most abstract semantic features.
- **Output**: a 1×1 convolution reduces the depth to the number of classes ($L = 2$ for cell/background), followed by softmax.
- Input and output sizes differ: the output is slightly smaller than the input because valid convolutions progressively trim borders; pixels without full context are not classified. (Tiling can be used for full-resolution prediction.)

19.13.2 Skip Connections

- At each upsampling step, the feature map from the corresponding level in the encoder is **cropped** (to match the spatial size of the upsampled map) and **concatenated** channel-wise.
- This fuses high-resolution information (precise localisation) from the encoder with the semantically rich features from the decoder.
- The concatenated volume is then processed by convolutions, which learn how to combine the two information streams.

19.13.3 Why Valid Padding?

- U-Net uses valid convolutions throughout, which gradually shrink the feature maps. This ensures that at each skip connection the encoder map exactly covers the central region that corresponds to the upsampled decoder map after cropping.
- As a result, only pixels whose full receptive field lies within the input image are predicted; border pixels with insufficient context are ignored in the output, which is acceptable for biomedical tasks where images can be tiled.

19.13.4 Training Details

- Binary cross-entropy loss (for two-class segmentation) summed over all pixels.
- Batch size = 1: a single large image provides enough pixel-level training signals.
- Training time: around 10 hours on a GPU of that era.
- Data augmentation is essential due to limited annotated biomedical images.

19.14 Data Augmentation for Segmentation

19.14.1 Transformations Used

- Shift and rotation augmentations; rotation is particularly useful because fully convolutional networks are already translation equivariant, so shifting adds little benefit.
- **Elastic deformations**: simulate realistic tissue deformations, a crucial augmentation for biological images.
- Augmentations are applied to both the input image and the corresponding segmentation mask simultaneously (the same transformation must be applied to preserve spatial alignment).
- Augmentation promotes invariance to the applied transformations and significantly increases the effective training set size, preventing overfitting on small datasets.

19.15 Transposed Convolution Details

19.15.1 Filter Dimensions and Stride

- The filter depth must equal the number of input channels, as in a standard convolution.
- The number of filters determines the number of output channels.
- Using a stride larger than one in the decoder expands the spatial dimensions (opposite to the downsampling effect in the encoder).

19.15.2 Terminology

Transposed convolution is known under several names:

- **Fractionally strided convolution** (the kernel slides by fractional pixel steps to produce a larger output).
- **Backward strided convolution**.
- **Deconvolution** – misleading, as it is not the mathematical inverse of convolution.

The preferred term is **transposed convolution**.

19.15.3 Learnable Upsampling Blocks

- A learnable upsampling block can be built in two ways:
  1. Non-trainable upsampling (nearest neighbour, bed of nails) followed by one or more standard convolutions.
  2. A transposed convolution directly.
- All modern segmentation architectures employ such learnable upsampling modules.
- The final layer produces a volume with L channels (one per class). A softmax along the channel dimension gives per-pixel class probabilities, and the argmax yields the segmentation map.

19.16 Training a Segmentation Network

19.16.1 Pixel-Wise Loss Function

- The loss is computed independently for each pixel using the categorical cross-entropy between the predicted probability vector and the one-hot encoded ground-truth label.
- For pixel j:

$$\mathcal{L}_j = -\sum_c y_{j,c} \log p_{j,c},$$

  where $y_{j,c} \in \{0, 1\}$ indicates whether pixel j belongs to class c, and $p_{j,c}$ is the softmax output.
- The total loss for an image is the sum (or average) over all pixels.
- Because a single image provides a large number of pixel-level training samples, segmentation networks can be trained with batch size 1 (the image itself acts as a large batch).

19.16.2 Extension to Dense Regression

- If the task is a per-pixel regression (e.g. depth or height map prediction), categorical cross-entropy is replaced by a mean squared error loss, again aggregated over all spatial locations.

19.17 U-Net Architecture

19.17.1 Overview

- U-Net is a **fully convolutional** encoder–decoder network designed for biomedical image segmentation.
- **Encoder (contracting path)**: blocks of 3×3 convolutions (valid padding, no zero-padding) + ReLU, followed by 2×2 max pooling (stride 2) for downsampling. Each block doubles the number of channels.
- **Decoder (expanding path)**: transposed convolution (2×2, stride 2) doubles spatial size and halves channels; the result is concatenated with a cropped feature map from the encoder; then 3×3 convolutions + ReLU process the concatenation.
- **Bottleneck**: at the lowest resolution (32×32), the volume has many channels (1024) capturing high-level semantics.
- **Output**: a 1×1 convolution reduces the depth to the number of classes ($L = 2$ for cell/background), followed by softmax.
- The output spatial size is slightly smaller than the input due to valid convolutions; border pixels without full context are not classified. (Tiling can be used for full-resolution prediction.)

19.17.2 Skip Connections

- Feature maps from the encoder are **cropped** to match the spatial size of the corresponding upsampled decoder map and then **concatenated** channel-wise.
- This fuses high-resolution localisation information from the encoder with the semantically rich features from the decoder.
- Subsequent convolutions learn how to optimally combine these two streams.

19.17.3 Why Valid Padding?

- U-Net uses only valid convolutions, causing the spatial dimensions to shrink gradually.
- This ensures that at each skip connection the cropped encoder map exactly covers the region that corresponds to the upsampled decoder map.
- As a result, only pixels whose full receptive field lies within the input image are predicted; border pixels are discarded. This is acceptable in many biomedical tasks where images can be processed in overlapping tiles.

19.17.4 Training Details

- **Loss**: binary cross-entropy (for two-class segmentation) summed over all pixels.
- **Batch size**: 1 (a single image supplies enough pixel-wise supervision).
- Training time on a mid-range GPU: approximately 10 hours.
- Strong data augmentation is essential because annotated biomedical images are scarce.

19.18 Data Augmentation for Segmentation

19.18.1 Elastic Deformations

- A grid is overlaid on the image, the grid points are randomly displaced, and the image is warped via bilinear interpolation following the deformed grid.
- This simulates realistic tissue deformations, highly relevant for biological images.
- As always, the chosen augmentations must be **realistic** and **consistent** with the nature of the problem.

19.18.2 Augmenting the Segmentation Mask

- The exact same geometric transformation must be applied to the ground-truth segmentation mask so that spatial alignment is preserved.

19.18.3 Separation of Touching Objects

- In cell segmentation, it is crucial that neighbouring cells remain separated by a thin boundary of background pixels, otherwise they merge into a single connected component.
- To enforce this, the authors of U-Net introduced a **weighted loss** that emphasises the narrow regions between touching cells.

19.19 Weighted Loss Functions in U-Net

19.19.1 Motivation

- A plain pixel-wise cross-entropy loss may miss fine separating boundaries, causing different cells to be merged.
- A weighted loss gives higher importance to:
  1. Rare (underrepresented) classes.
  2. Pixels near the borders between two touching objects.

19.19.2 Class-Specific Weights (w_c)

- Each class c is assigned a weight inversely proportional to its frequency, compensating for class imbalance (similar to handling credit card fraud or other skewed datasets).

19.19.3 Boundary Weight Map

- For each pixel, let d_1 be the distance to the nearest cell border, and d_2 the distance to the second nearest cell border.
- The weight map term is designed to be large where d_1 and d_2 are both small (i.e. the narrow gap between two cells), and small inside a cell or in the background.
- This forces the network to pay special attention to the critical separation regions.
- The full per-pixel weight is w_c multiplied by the boundary weight term; weights are pre-computed once from the training ground-truth masks.

19.20 Case Study: Kidney Biopsy Segmentation

19.20.1 Problem and Annotation Scarcity

- Goal: segment the **interstitial region** (the tissue between tubules) in kidney biopsies; its area is a biomarker for disease progression.
- Annotating such images requires a medical doctor and is extremely time-consuming (several hours per image).
- To reduce annotation effort, **scribble annotations** (sparse lines) can be used instead of full pixel-wise masks – a few minutes per slide.

19.20.2 Domain Shift and Scribble Annotations

- The appearance of the interstitial tissue varies across patients (healthy vs. chronic kidney disease), introducing a domain shift.
- Annotated healthy images do not directly generalise well to diseased patients.
- A practical strategy: train a general model on whatever (possibly dense) annotations are available, then **fine-tune** the model on a few scribble annotations from the target patient.

19.20.3 Fine-Tuning for Patient Adaptation

- Fine-tuning on the patient's own sparse scribbles adapts the network to the specific image characteristics, achieving good segmentation despite limited and sparse labels.
- This approach tackles both data scarcity and domain shift simultaneously.

19.21 From Dense to Convolutional Layers: Revisiting the Connection

19.21.1 Feeding Larger Images to a CNN

- A standard CNN classifier trained on $N \times N$ images contains convolutional layers followed by a flattening and fully connected layers.

- If a larger image is presented at test time, the convolutional layers can still process it (they are fully convolutional), producing larger feature maps.
- However, the flattening step expects a fixed vector length, so the mismatch breaks the network.
- Replacing flattening with global average pooling (GAP) removes this restriction, as already discussed.

19.21.2 Simplifying to a 1×1 Activation and Dense Layer

- To analyse the relationship between dense and convolutional layers, assume the convolutional backbone outputs a $1 \times 1 \times C$ activation volume (a C-dimensional vector), followed by a single dense layer with L outputs.
- This configuration is exactly a classifier trained on small image patches.

19.21.3 Linear Combination View

- The output of the first neuron, before softmax, is the familiar linear combination:

$$S_1 = \sum_{j=1}^{C} W_{1j}\, X_j + b_1,$$

  where X_j is the j-th channel of the 1×1 feature map, W_{1j} are the learned weights, and b_1 is the bias.
- This formulation is identical to that of a convolutional layer with 1×1 filters; it will be used to show how a fully connected layer can be **reinterpreted** as a convolutional layer, enabling dense predictions from a classification network.

19.22 Reinterpreting Dense Layers as Convolutional Layers

19.22.1 The Linear Combination View

- For a classifier with a final $1 \times 1 \times C$ activation (after a convolutional backbone) followed by a single dense layer with L outputs, the score for neuron 1 is the familiar linear combination:

$$S_1 = \sum_{j=1}^{C} W_{1j}\, X_j + b_1,$$

  where X_j is the j-th channel of the 1×1 feature map, W_{1j} are the learned weights, and b_1 the bias.
- This is exactly the **inner product** between the weight vector $\mathbf{W}_1$ (size C) and the feature vector $\mathbf{X}$.

19.22.2 Equivalence to 1×1 Convolution

- The same operation can be expressed as a **correlation** of the weight vector $\mathbf{W}_1$ with the $1 \times 1 \times C$ activation, evaluated at position $(0, 0)$ (valid convolution):

$$S_1 = (\mathbf{W}_1 \star \mathbf{X})(0, 0) + b_1.$$

- Thus one output neuron of a dense layer is equivalent to a $1 \times 1 \times C$ **filter** in a convolutional layer.
- The entire dense layer with L output neurons maps to a convolutional layer with L **filters of size** $1 \times 1 \times C$.

19.22.3 Extending to Multi-Layer Perceptrons

- Any stack of fully connected layers can be recursively converted: each dense layer becomes a 1×1 convolution with the appropriate number of input and output channels.
- The resulting network is **fully convolutional** – it contains only convolutions (and non-linearities, pooling, etc.), no flattening or dense layers.

19.22.4 Handling Larger Activation Maps (Flattening and GAP)

- If the classifier uses **flattening** (feature map $H \times W \times C \to$ vector of length HWC), the first dense layer has HWC input weights per neuron.
- These weights can be reshaped into a filter of size $H \times W \times C$ (one filter per output neuron). Applying this filter with **valid** convolution (no padding, stride 1) to the same $H \times W \times C$ volume yields a single scalar per filter – exactly the original dense layer output.
- If the classifier uses **Global Average Pooling** (GAP), averaging each channel can be seen as a convolution with C filters of size $H \times W$, where filter k has all ones in channel k and zeros elsewhere (valid convolution, producing $1 \times 1 \times C$). This is also convertible.
- Thus, **any CNN classifier can be transformed into a fully convolutional network** that produces the same output for the original input size, but can also process larger images.

19.23 Fully Convolutional Conversion for Dense Prediction

19.23.1 From Classifier to Segmentation Heatmaps

- Once a trained classifier is converted to a fully convolutional network, feeding a larger input image yields a **spatial output map** instead of a single vector.
- Each spatial position in the output corresponds to the classification result for the corresponding **receptive field** in the input.
- Example: converting a cat/dog classifier produces a heatmap where the "tabby cat" channel shows high activations over the cat region, and the "dog" channel fires on the dog region.
- This gives a **coarse segmentation** or localisation map directly from a network trained only with image-level labels.

19.23.2 Implementation: Weight Transfer

- To convert a trained model, the weights from each dense layer are extracted and reshaped into convolutional filters:
  - Dense layer with C_{in} inputs and C_{out} outputs $\to$ C_{out} filters of size $1 \times 1 \times C_{\text{in}}$ (if preceded by 1×1 feature map, or larger spatial size if flattening was used, with valid padding).
- This weight copying is exact; the converted network mathematically computes the same function on the original input size.

19.23.3 Shift-and-Stitch for Higher Resolution

- The output heatmap is typically at a much lower resolution than the input due to downsampling in the backbone (e.g. factor 32).
- **Shift-and-stitch**: one can process multiple shifted versions of the input and interleave the results to obtain a higher-resolution output, at the cost of multiple forward passes.

- A more efficient trick: expand the convolutional filters by inserting zeros (à la "à trous" algorithm), which increases the output stride without re-training, yielding denser predictions in a single pass.
- Nevertheless, the resulting maps remain **coarse** and lack the fine detail of a properly trained segmentation network with learnable upsampling.

19.23.4 When to Use This Approach

- If dense pixel-level annotations are available, a dedicated architecture like **U-Net** (with trainable upsampling layers) is far superior.
- However, when only **classification labels** (image-level or patch-level) are available, converting a classifier to a fully convolutional network provides a form of **weakly supervised localisation** or coarse segmentation.
- Example: the Steller sea lion counting problem – train a patch classifier on 90×90 patches, convert it to fully convolutional, and slide it over large drone images to localise different animal classes.

19.24 Closing the Loop: CNN $\leftrightarrow$ Dense Layers

19.24.1 Duality of Interpretations

- Earlier in the course, a convolutional layer was shown to be a **sparse fully-connected layer with weight sharing**.
- Now the reverse holds: a fully-connected (dense) layer can be seen as a **convolutional layer with 1×1 filters** (or larger filters if flattening was used).
- Therefore, the whole network can be viewed either as an MLP with structured weight matrices, or as a fully convolutional architecture. Both perspectives are mathematically equivalent.

19.24.2 Importance of This Understanding

- This duality is not just theoretical; it enables practical techniques like fully convolutional conversion for dense prediction, efficient implementation tricks, and deeper insight into architectural design.
- Mastering this connection distinguishes a deep learning engineer who truly understands CNNs from a user who merely applies existing code.

19.25 Outlook

- The next lecture will cover **object detection**, which is a less dense topic than segmentation, possibly allowing a brief recap of any remaining points.
- The journey from basic classification to segmentation and detection completes the core visual recognition problems addressed by CNNs.

Chapter 20

Famous Architectures, Object Localisation and Class Activation Maps

20.1 Advanced Architectures based on ResNet

20.1.1 ResNet Recap

- **Core idea**: Deeper networks can learn more complex functions, but naïvely stacking layers leads to difficulty in training (vanishing gradients, degradation). ResNet introduces **skip connections** (identity shortcuts) that allow the network to learn residual mappings.
- If a layer is not needed, the identity mapping can be learned easily by driving the residual function $\mathcal{F}(x)$ to zero; the output becomes $\mathcal{F}(x) + x \approx x$.
- **Basic block** (two-layer variant):
  - `Conv2d` (3×3) → BatchNorm → ReLU
  - `Conv2d` (3×3) → BatchNorm
  - If input and output channels differ (e.g., after channel doubling or spatial down-sampling), a **projection shortcut** (1×1 convolution) is applied to the input before adding.
  - Final ReLU after addition.
  - Max pooling at the end of the block for spatial reduction (some variants use stride in the convolution).
- **Full model**:
  - A sequence of such blocks; the number of channels is doubled every time a new block begins.
  - After the convolutional backbone: **Global Average Pooling**, flatten, and a dense (linear) layer for classification.

20.1.2 Wide ResNet

- **Motivation**: Instead of going very deep, increase the *width* (number of filters per layer). Wider layers are more amenable to parallelization on GPUs, leading to faster training even with fewer layers.
- **Widening factor** k: typically $4 \leq k \leq 12$. The number of filters in the bottleneck is multiplied by k.
- **Block design** (two 3×3 convolutions, no 1×1 bottleneck in this variant):

  - `Conv2d`(in_channels → `out_filters` $\times k$) → BatchNorm → ReLU → Dropout (optional)
  - `Conv2d`(`out_filters` $\times k$ → `out_filters`) → BatchNorm
  - Projection shortcut if needed; addition; ReLU; max pooling.

- The rest of the model (stack of blocks, GAP, linear classifier) is identical to ResNet.
- The number of parameters increases with k, but the depth can be reduced, often yielding similar accuracy with better computational efficiency.

20.1.3 ResNeXt

- **Motivation**: Introduce **multiple parallel paths ("experts")** inside a residual block, each learning different features, and then aggregate them. This is inspired by the "split-transform-merge" strategy also found in Inception modules.
- **Cardinality**: the number of parallel paths (analogous to the number of groups in grouped convolution).
- **Equivalent implementations**:
  1. Separate branches with identical topology, outputs summed.
  2. Branches whose outputs are concatenated, then passed through a 1×1 convolution.
  3. **Grouped convolution** (most efficient): a single convolution where input channels and filters are divided into groups; each group operates on a disjoint subset of channels and the results are concatenated (or summed, depending on the formulation).
- **Grouped convolution in PyTorch**: `nn.Conv2d(..., groups=cardinality)` splits input channels and output filters into `groups` independent convolutions. The output channels are concatenated automatically.
- **ResNeXt block** (bottleneck style):
  - `Conv2d`(in_ch → D, kernel=1) → BN → ReLU
  - `Conv2d`(D → D, kernel=3, groups=cardinality) → BN → ReLU
  - `Conv2d`(D → out_filters, kernel=1) → BN
  - Projection shortcut (if needed); addition; ReLU; max pooling.
- The rest of the architecture remains the same sequence of blocks, GAP, linear classifier.
- ResNeXt achieves state-of-the-art performance by controlling both depth and the number of parallel transformations (cardinality) without dramatically increasing parameters.

20.1.4 Common Structure and Implementation Notes

- All these models follow the pattern: **stack of residual blocks** (with increasing channels and occasional down-sampling), **Global Average Pooling**, **flatten**, and a **linear classifier** with softmax.
- In practice, pre-implemented versions are available in `torchvision.models`; the code shown is for educational understanding of the architectural principles.
- The "graph" visualizations of the models show the skip connections and the data flow clearly, reflecting the block structure described above.

20.2 DenseNet

20.2.1 Core Idea

- **Dense connectivity**: Each layer receives as input the concatenation of the feature maps from *all preceding layers* inside the same block.
- Let $X_0, X_1, \ldots, X_{\ell-1}$ be the outputs of previous layers. The ℓ-th layer H_ℓ computes

$$X_\ell = H_\ell\big([X_0, X_1, \ldots, X_{\ell-1}]\big),$$

  where $[\cdot]$ denotes channel-wise concatenation.
- The classifier thus sees both high-level and low-level features, which empirically improves performance.

20.2.2 Composite Function and Growth Rate

- The layers inside a dense block use a **BN-ReLU-Conv** order (batch normalization, ReLU, then convolution) – the opposite of the conventional Conv-BN-ReLU. The authors found this ordering beneficial in practice.
- Each layer produces a fixed number k of new feature maps, called the **growth rate**. Typically $k = 12$ or $k = 32$.
- After ℓ layers, the number of input channels to layer $\ell + 1$ is $k_0 + \ell \cdot k$, where k_0 is the number of input channels to the block.

20.2.3 Block Structure and Implementation

- A dense block consists of a sequence of *units*. Each unit = BN $\rightarrow$ ReLU $\rightarrow$ Conv (3×3, output channels = growth rate k).
- Implementation: maintain a tensor `x` that starts as the block input. For each unit, compute `out = unit(x)`; then update `x = torch.cat([x, out], dim=1)`.
- At the end of the block, a **1×1 convolution** projects the concatenated features back to a desired number of output channels, followed by a max-pooling layer for down-sampling.

20.2.4 Full Model

- Initial convolutional layer (e.g., stride 2, with batch norm and ReLU).
- A sequence of dense blocks, separated by down-sampling layers (the 1×1 conv + pooling inside each block).
- After the last block: **Global Average Pooling**, flatten, and a linear classifier.
- The forward pass is identical in structure to ResNet-like models, only the internal block computation differs.

20.2.5 Visualization (Graph)

- In the computational graph, a dense block shows the progressive concatenation: input (32 channels) $\rightarrow$ first layer adds 16 $\rightarrow$ concatenated 48 $\rightarrow$ second layer adds 16 $\rightarrow$ 64, etc.
- This visual feedback makes the dense connectivity pattern immediately apparent.

20.3 EfficientNet

20.3.1 Motivation: Balancing Network Dimensions

- Conventional CNN design independently tunes **depth** (L), **width** (number of channels, C), and **input resolution** (H, W).
- These three dimensions are interdependent: larger images benefit from deeper and wider networks to capture more complex patterns.
- **EfficientNet** proposes a principled *compound scaling* method that jointly scales all three dimensions with a single compound coefficient ϕ.

20.3.2 Compound Scaling Formulation

- Define a **baseline** EfficientNet-B0 with optimal depth d, width w, and resolution r found via neural architecture search under computational constraints.
- To scale up the network, all dimensions are increased by a constant factor:

$$\text{depth} = \alpha^\phi, \quad \text{width} = \beta^\phi, \quad \text{resolution} = \gamma^\phi,$$

  with $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ and $\alpha, \beta, \gamma \geq 1$ determined by a small grid search.
- The constraint $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ arises because FLOPs roughly double when depth, width^2, or resolution^2 are doubled. Keeping the product ≈ 2 ensures the total FLOPs increase by 2^ϕ when scaling from the baseline.
- For instance, the authors found $\alpha = 1.2,\ \beta = 1.1,\ \gamma = 1.15$ to work well.

20.3.3 Neural Architecture Search (NAS) for Baseline

- EfficientNet-B0 itself is obtained via **Neural Architecture Search**, an automated hyperparameter tuning method that shares weights among candidate architectures to efficiently explore the design space.
- The search maximizes accuracy subject to memory and FLOP limits, yielding a highly efficient base network.
- Different from naive grid search, NAS reuses weights across similar architectures, drastically reducing the number of training runs.

20.3.4 MBConv Block (Mobile Inverted Bottleneck with Squeeze-Excitation)

- The building block of EfficientNet is the **MBConv** block, inherited from MobileNetV2 and augmented with **Squeeze-and-Excitation** (SE).
- Structure of an MBConv block:
  1. **Expansion**: 1×1 convolution that increases the number of channels by an expansion factor e (only if $e > 1$).
  2. **Depthwise convolution**: $k \times k$ grouped convolution (groups = number of input channels) applied channel-wise; efficient for spatial filtering.
  3. **Squeeze-and-Excitation** (SE):
     - Global Average Pooling to obtain a channel descriptor.
     - Two fully-connected (or 1×1 conv) layers with a Swish activation and a sigmoid to produce per-channel weights.

   - Multiply the original feature maps by these weights to emphasise important channels.
4. **Projection**: 1×1 convolution to project back to the desired number of output channels.
5. A residual connection is added if the input and output shapes match and stride is 1.

- The SE module acts as a lightweight channel-wise attention mechanism that boosts representational power with minimal extra parameters.

20.3.5 Overall Architecture

- EfficientNet stacks multiple MBConv blocks with increasing channels and decreasing spatial resolution (via strided convolutions/max pooling in some blocks).
- The final layers are a 1×1 convolution, Global Average Pooling, and a linear classifier.
- This design achieves state-of-the-art accuracy while being substantially smaller and faster than previous convolutional networks, illustrating the power of compound scaling.

20.4 Object Localization

20.4.1 From Classification to Localization

- **Classification**: given an image containing a single object, assign it to one of a fixed set of categories.
- **Localization**: still assumes a *single object* per image, but the goal is to predict a **bounding box** that tightly encloses the object.
- A bounding box is defined by four real numbers:

$$(x_{\min}, y_{\min}, x_{\max}, y_{\max}) \quad \text{or} \quad (x, y, w, h),$$

  where (x, y) is the top-left corner and w, h are width and height.
- When multiple objects are present, the task becomes **object detection** (predicting a set of bounding boxes, covered later).

20.4.2 Regression Formulation

- The input is an RGB image $X \in \mathbb{R}^{H \times W \times 3}$.
- The output is a vector $\hat{\mathbf{b}} = (\hat{x}, \hat{y}, \hat{w}, \hat{h}) \in \mathbb{R}^4$.
- The model architecture is identical to a classifier up to the last layer:
  - A convolutional backbone (feature extractor) – the same ones used for classification.
  - A **regression head** made of fully-connected layers (e.g., linear + ReLU + linear) that outputs exactly 4 numbers.
  - **No activation** (no softmax/sigmoid) on the final output, because the coordinates are unconstrained real numbers.
- **Loss function**: Mean Squared Error (L2 loss) between the predicted and ground-truth bounding box vectors:

$$\mathcal{L} = \frac{1}{4} \sum_{i=1}^{4} (\hat{b}_i - b_i)^2.$$

20.4.3 Coordinate Normalization

- Bounding box coordinates are usually normalised to the interval $[0, 1]$ by dividing by the image width and height.
- This stabilises training (values stay in the same range as normalised pixel intensities) and makes the model independent of the actual image size.
- A prediction $x < 0$ or $y < 0$ means the object starts outside the image (truncated at the top/left); $x + w > 1$ or $y + h > 1$ means it exceeds the bottom/right boundary.

20.5 Data Preparation for Localization

20.5.1 Dataset and Annotations

- Example: a dataset of cat and dog images. Each image comes with a CSV file containing:
  - filename, original dimensions, class label (cat=0, dog=1), and bounding box coordinates as $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$.
- The coordinates are converted to the normalised (x, y, w, h) format.

20.5.2 Image Loading and Preprocessing

- Images are loaded with OpenCV, which reads them in **BGR** order; they are converted to RGB.
- After loading, images are cropped (centre crop) and resized to a fixed size (e.g., 224×224).
- Pixel values are normalised to $[0, 1]$ using ImageNet mean and standard deviation (if a pretrained backbone is used).

20.5.3 Dataset Class

- A single `Dataset` class can serve both classification and regression tasks:
  - For classification, it returns $(\text{image}, \text{label})$.
  - For localization, it returns $(\text{image}, \text{bounding_box})$.
- The class applies data augmentations (geometric and photometric) and converts the arrays to PyTorch tensors.

20.6 Combined Classification and Localization

20.6.1 Two Independent Networks (Naive Approach)

- Train **two separate models**:
  - A classifier $f_{\theta_c} : X \to \text{class}$.
  - A regressor $f_{\theta_r} : X \to \text{bbox}$.
- **Advantages**:
  - Flexibility: the classifier can be trained on a larger classification-only dataset; the localizer on a (usually smaller) dataset with bounding boxes.
  - Simplicity in implementation and training loops.
- **Disadvantages**:

  - Inefficiency: two separate feature extraction backbones, doubling compute and memory.
  - Lack of semantic coherence: the classifier may be correct while the localizer predicts a box on a completely wrong region, because the two networks do not share any information.

20.6.2 Multi-Task Learning Approach (Preview)

- A more efficient and coherent solution is to share the feature extraction backbone and attach **two heads**:
  - A classification head (producing class probabilities).
  - A regression head (producing bounding-box coordinates).
- The whole network is trained jointly with a combined loss. This will be covered in detail in a later lecture.

20.7 Bounding Boxes and Data Augmentation

- When applying **geometric transformations** (e.g., random flip, rotation, translation, scaling) to the input image, the ground-truth bounding box must be transformed accordingly.
- If an image is flipped horizontally, the x coordinate of the box must be mirrored. If the image is cropped or translated, the box must be shifted.
- Failure to synchronise the bounding box with the augmented image leads to noisy, incorrect labels and severely degrades training.
- In PyTorch, custom augmentation pipelines for detection often use libraries like `torchvision.tv_tensors` or `albumentations` that can handle bounding boxes together with images.

20.8 Weakly Supervised Localisation via Class Activation Maps

20.8.1 From CAM to Bounding Boxes

- If a dataset has **only class labels** (no bounding-box annotations), it is still possible to obtain rough object locations using **Class Activation Maps** (CAMs).
- The CAM for the predicted class highlights the discriminative image regions. By thresholding the heatmap (e.g., retaining the top 20% of the maximum activation), a binary mask of the object can be obtained.
- A bounding box is then fitted around the largest connected component of this binary mask.
- This provides a form of **weakly supervised localisation** – the network was never explicitly trained to output coordinates, yet it can localise objects.

20.8.2 Dataset with Both Cats and Dogs

- A specific test dataset containing images with **both a cat and a dog** is used to evaluate CAM-based localisation.
- The same pretrained classifier (or a model fine-tuned for two classes) is converted to a fully convolutional network, and CAMs for the two classes are computed independently to localise each animal.
- This illustrates how classification networks can be repurposed for localisation when annotations are scarce.

20.9 Squeeze-and-Excitation Block (within EfficientNet's MBConv)

20.9.1 Function of the SE Block

- The Squeeze-and-Excitation (SE) module computes a channel-wise importance vector that is multiplied element-wise with the feature map.
- Steps:
  1. **Squeeze**: Global Average Pooling to obtain a descriptor of size $1 \times 1 \times C$.
  2. **Excitation**: Two 1×1 convolutions (or linear layers) with a Swish activation and a sigmoid produce per-channel weights $\in (0, 1)$.
  3. **Scale**: Multiply the original feature maps channel-wise by the weights to emphasise important channels.

20.9.2 MBConv Block Structure

- The MBConv block (used in EfficientNet) combines:
  - **Expansion**: 1×1 convolution (if expansion factor > 1).
  - **Depthwise convolution**: channel-wise spatial filtering (grouped convolution with groups = input channels).
  - **SE module**: as described above.
  - **Projection**: 1×1 convolution back to desired output channels.
  - **Residual connection**: added if input and output shapes match and stride = 1.
  - **Max pooling** (or stride in depthwise conv) for down-sampling.
- In the forward pass:
  - Expand $\rightarrow$ depthwise conv $\rightarrow$ SE (multiply) $\rightarrow$ project $\rightarrow$ add skip connection $\rightarrow$ (optional max pool).

20.10 Object Localization

20.10.1 From Classification to Bounding Box Regression

- **Classification**: input image $\rightarrow$ single class label, assuming one main object.
- **Localization**: same single-object assumption, but output is a **bounding box** tightly enclosing the object.
- A bounding box is defined by four real numbers:
  - `xywh` format: (x, y, w, h) – top-left corner, width, height.
  - `xyxy` format: $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$.
- The task is a **regression** problem: predict four continuous coordinates.

20.10.2 Loss Function and Output Layer

- Model architecture: same convolutional backbone as classification, followed by a regression head (e.g. Linear → ReLU → Linear).
- The final layer has **4 output neurons** with **no activation function** (unconstrained real numbers).
- **Loss**: Mean Squared Error (MSE) between predicted $\hat{\mathbf{b}}$ and ground-truth $\mathbf{b}$:

$$\mathcal{L} = \frac{1}{4}\sum_{i=1}^{4}(\hat{b}_i - b_i)^2.$$

- Coordinates are usually normalised to $[0, 1]$ by dividing by image width and height. This stabilises training and allows predictions < 0 or > 1 for truncated objects.

20.11 Data Preparation for Localization

20.11.1 Dataset Structure and Preprocessing

- Example: a CSV file containing per image: filename, original dimensions, class label, and bounding box as $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$.
- Images are loaded with OpenCV (BGR → RGB), centre-cropped and resized to a fixed size (e.g. 224×224).
- Bounding box coordinates are normalised by dividing x and y by the image width/height; w and h similarly, producing values in $[0, 1]$.

20.11.2 Dataset Class for Both Tasks

- A single `Dataset` class supports classification (returns image + label) or localization (returns image + bounding box) by switching based on whether labels or bounding boxes are provided at initialisation.
- The class applies augmentations (see below) and converts to PyTorch tensors.
- Image pixel values are normalised using ImageNet statistics (if using a pretrained backbone).

20.11.3 Bounding Boxes and Data Augmentation

- **Geometric augmentations** (flip, translation, rotation, scaling) must be applied **jointly** to both image and bounding box.
- In PyTorch, `torchvision.tv_tensors` provides a wrapper for bounding boxes (e.g. `BoundingBoxes`) that tracks the box format (`xyxy` or `xywh`) and canvas size. The augmentation pipeline transforms both image and targets correctly.
- During loading, coordinates are temporarily converted back to absolute pixel values for the augmentation, then re-normalised afterward.
- Example: a horizontal flip mirrors the x coordinates; translation shifts the box by the same amount.
- Visual inspection confirms that the bounding box remains correctly placed after augmentation (though the dataset itself may contain noisy ground-truth boxes).

20.12 Model Architecture for Localization

20.12.1 Transfer Learning with EfficientNet

- **Backbone**: EfficientNet-B0 pretrained on ImageNet (`weights='IMAGENET1K_V1'`).
- **Freezing**: set `requires_grad = False` for all backbone parameters to keep pretrained features fixed.
- **Regression head**: replaces the original 1000-class classifier with:

$$\texttt{Dropout} \rightarrow \texttt{Linear}(\text{num_features}, 4)$$

- The forward method simply passes the image through the backbone (which now includes the new head) and returns the four bounding-box coordinates.

20.12.2 Interpretation of the Final Layer

- A `Linear` layer with 4 outputs and no activation function unambiguously indicates a regression problem – in this case, bounding-box localisation.
- This is a key point for exam questions: the structure of the last layer reveals the task being solved.

20.13 Training and Evaluation Loop

20.13.1 Batch Normalisation and Dropout Modes

- `model.train()` enables dropout and updates batch normalisation statistics (running mean/variance) during training.
- `model.eval()` freezes batch normalisation (uses accumulated running statistics) and disables dropout; essential for consistent validation.

20.13.2 Training Step

- Receive image and ground-truth bounding box from the data loader.
- Forward pass: $\hat{\mathbf{b}} = \text{model}(x)$.
- Compute MSE loss: $\mathcal{L} = \frac{1}{4}\sum_i (\hat{b}_i - b_i)^2$.
- Backward pass, gradient scaling (if using AMP), optimizer step.

20.13.3 Validation and Early Stopping

- Validation loop runs under `torch.no_grad()` and `model.eval()`.
- In addition to the validation loss, the **Spearman rank correlation** between predicted and ground-truth coordinates is computed.
- **Early stopping** is based on the Spearman correlation (not the training MSE), to avoid biasing towards the optimised loss and to assess monotonic ranking quality of the predicted boxes.
- Spearman correlation measures the rank-based agreement; a value near 1 indicates that the order of predictions matches the order of ground truths, providing a complementary view to the L2 loss.

20.14 Combined Classification and Localization

20.14.1 Two Independent Networks

- A naive solution: train a separate classifier and a separate regressor, each with its own backbone.
- **Advantages**: flexibility – the classifier can be trained on a large classification-only dataset, the localiser on the (usually smaller) set with boxes.
- **Disadvantages**: inefficient (two feature extractors), and no semantic consistency – the class may be correctly predicted while the box is placed on an unrelated region because the two branches do not share information.

20.14.2 Multi-Task Learning (Preview)

- A better approach: share the convolutional backbone and attach two heads (classification and regression), training them jointly with a combined loss. This will be covered in detail in a later lecture.

20.15 Weakly Supervised Localisation via Class Activation Maps

- When **no bounding-box annotations** are available, a classifier can still be used to roughly localise objects.
- **Class Activation Map (CAM)** for the predicted class highlights the discriminative regions.
- Thresholding the CAM (e.g. keep values above 20% of the maximum) yields a binary mask; a bounding box can be fitted around its largest connected component.
- This provides a form of **weakly supervised localisation** – no explicit box labels needed.
- In the exercise session, this will be demonstrated on images containing both cats and dogs: a single classifier produces heatmaps for both classes, localising each animal independently.

20.16 Fine-Tuning the Classifier

20.16.1 Unfreezing Selected Layers

- After initial transfer learning (frozen backbone), further improvement can be obtained by **fine-tuning**: unfreezing some of the last layers of the backbone.
- Typically, only the deepest blocks are unfrozen (e.g., the last three blocks), while early layers remain fixed.
- Setting `requires_grad = True` for these layers makes them trainable; the number of trainable parameters increases significantly (e.g., to nearly 80% of the total).

20.16.2 Training Configuration

- **Learning rate**: a lower learning rate is essential during fine-tuning (e.g., 10^{-4} vs. 10^{-3} for the head) to avoid destroying the pretrained features.
- The loss function and optimizer remain the same as in the transfer learning stage (cross-entropy for classification, Adam).
- Early stopping patience is kept small (e.g., 5 epochs) because the model converges quickly.

20.16.3 Results

- Fine-tuning on the cat/dog dataset reaches nearly 100% accuracy, with only a single misclassified sample (a cat mislabeled as dog).
- The improvement over pure transfer learning demonstrates the benefit of adapting high-level features to the specific dataset.

20.17 Weakly Supervised Localization via Class Activation Maps (CAM)

20.17.1 Motivation

- When only a classifier is available (no bounding-box regressor), we can still localize objects using **Class Activation Maps**.
- CAM highlights the image regions that most contribute to the classification score for a given class.

20.17.2 CAM Computation

- Require a network with **Global Average Pooling (GAP)** followed by a single linear layer (no further dense layers).
- Let $f_k(x, y)$ be the activation of channel k at spatial location (x, y) in the last convolutional layer (before GAP), and $w_{k,c}$ the weight connecting channel k to output class c.
- The class activation map for class c is:

$$\mathrm{CAM}_c(x, y) = \sum_k w_{k,c}\, f_k(x, y).$$

- In PyTorch, a **forward hook** is registered on the target layer (last conv before GAP) to capture the activations.
- The weights $w_{k,c}$ are extracted from the classifier's linear layer.
- The weighted sum yields a low-resolution heatmap (same spatial size as the feature map). It is then upsampled (e.g., bilinear interpolation) to the original image size.

20.17.3 Extracting Bounding Boxes from CAM

- **Threshold** the upsampled CAM (e.g., keep values above 0.2 of the maximum) to obtain a binary mask.
- Use `cv2.findContours` to detect the connected component, and fit a tight bounding box around it.
- The resulting box is coarse but provides a **weakly supervised localization** – no bounding-box labels were used in training.
- Example on the cat/dog dataset: the CAM focuses on the head/face of the animal; the extracted box is reasonable but less precise than a dedicated regressor.

20.18 Grad-CAM: Generalization of CAM

20.18.1 Limitations of CAM

- CAM requires a specific architecture (GAP + single linear layer). Networks with multiple dense layers, or without GAP, cannot directly produce CAM.

20.18.2 Grad-CAM Principle

- **Grad-CAM** uses the gradient of the class score with respect to the feature map activations.
- For a given class c and a chosen convolutional layer, compute the gradients $\frac{\partial y_c}{\partial f_k(x,y)}$ via backpropagation.
- The importance weight for channel k is obtained by **global average pooling** of these gradients:

$$\alpha_k^c = \frac{1}{HW} \sum_{x,y} \frac{\partial y_c}{\partial f_k(x,y)}.$$

- The Grad-CAM heatmap is then:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k^c \, f_k \right).$$

- The ReLU keeps only positive contributions (features that increase the class score).

20.18.3 Advantages over CAM

- Works with **any** convolutional network, regardless of the classification head (multiple dense layers, etc.).
- Can be applied to **any intermediate convolutional layer**, not just the last one.
- Visualizations at different depths show that deeper layers capture more semantic, object-level concepts, while earlier layers highlight finer textures or edges.
- Example: a dog is well localized using the last layer, but poorly localized using the fifth-last layer, illustrating the hierarchical nature of learned features.

20.18.4 Implementation Notes

- Requires registering a **backward hook** on the target layer to capture gradients.
- Computed similarly to CAM but using gradient-derived weights instead of classifier weights.

20.19 Handling Multiple Objects with CAM

- When an image contains **multiple objects** (e.g., a cat and a dog), a single classifier predicts one label, but CAM can be computed for *each class* separately by using the corresponding classifier weights.
- For the cat class, the CAM highlights the cat region; for the dog class, it highlights the dog region.
- This provides a way to localize multiple distinct objects even though the network was trained only for single-label classification.
- The localization is not perfect, but it demonstrates that class-specific information is spatially preserved in the feature maps.

Chapter 21

Fully Convolutional Neural Networks and Semantic Segmentation

21.1 Baseline CNN for Sea Lion Patch Classification

21.1.1 Dataset and Preprocessing

- **Task**: classify 96×96 RGB aerial images of sea lions into six classes (adult male, adult female, sub-adult male, juvenile, pup, background).
- Training set: ∼3 000 samples; validation and test: ∼1 200 each.
- Images are converted to tensors with channels first, augmented with random horizontal flips and colour jitter (no geometric distortions that alter object labels).
- Data loaders: batch size 64, shuffling for training.

21.1.2 Standard CNN Architecture

- **Feature extractor**: stack of `Conv2d` (with padding) + ReLU + `MaxPool2d` layers.
- Spatial size progression: $96 \to 48 \to 24 \to 12$; channel depth: $3 \to 32 \to 64 \to 128 \to 256$.
- After the last convolutional layer: feature map size $256 \times 12 \times 12$.
- **Classifier head**:
  - `Flatten` → vector of length $256 \cdot 12 \cdot 12 = 36\,864$.
  - `Dropout` → `Linear`(36 864, 256) → `ReLU` → `Dropout` → `Linear`(256, 6).
- Total parameters: ∼9.4 million in the first dense layer alone ($36\,864 \times 256 + 256$), plus other layers.

21.1.3 Training and Performance

- Training for up to 1 000 epochs with early stopping (patience on validation accuracy).
- Overfitting appears after a few epochs; best model achieves ∼60% accuracy on the balanced 6-class validation set.
- Confusion matrix shows some confusion among similar classes (e.g., juvenile vs. adult female), but performance is reasonable given the low resolution and hard task.

21.2 Motivation for Fully Convolutional Conversion

- The standard classifier processes fixed-size input patches and outputs a single label vector.
- For dense prediction (semantic segmentation) or efficient sliding-window evaluation on larger images, we want a **fully convolutional** architecture that takes an input of arbitrary size and produces a spatial output map.
- Key observation: a fully-connected (dense) layer can be **reinterpreted** as a convolutional layer with a kernel size equal to the spatial dimensions of its input.

21.3 Converting Dense Layers to Convolutional Layers

21.3.1 The Core Principle

- A fully-connected layer with input vector $\mathbf{x} \in \mathbb{R}^{C_{\text{in}} \cdot H_{\text{in}} \cdot W_{\text{in}}}$ and output $\mathbf{y} \in \mathbb{R}^{C_{\text{out}}}$ performs:

$$y_j = \sum_{i=1}^{C_{\text{in}} H_{\text{in}} W_{\text{in}}} W_{ji} x_i + b_j.$$

- If $\mathbf{x}$ is originally a 3D tensor of shape $(C_{\text{in}}, H_{\text{in}}, W_{\text{in}})$ that was flattened, we can **reshape** each row of the weight matrix into a 3D filter of exactly the same spatial and channel dimensions: $(C_{\text{in}}, H_{\text{in}}, W_{\text{in}})$.
- Convolving this filter with the original 3D input, using **valid** padding (no zero-padding) and stride 1, produces a single output pixel. The result is mathematically identical to the dense layer's output.
- Therefore, a dense layer with C_{out} output neurons is equivalent to a convolutional layer with C_{out} filters of spatial size $H_{\text{in}} \times W_{\text{in}}$ and depth C_{in}.

21.3.2 Applying to Our Network

- The feature map before flattening has dimensions $256 \times 12 \times 12$.
- The first dense layer ($36\,864 \to 256$) can be replaced by:
  - A `Conv2d` layer with 256 filters of size 12×12, each of depth 256, with **valid** padding (i.e. `padding=0`).
  - Input volume $256 \times 12 \times 12$, convolved with 256 such filters $\to$ output volume $256 \times 1 \times 1$.
- This 1×1 output can be flattened (or directly viewed as a vector) and passed to a final 1×1 convolution that maps 256 to 6 classes (equivalent to a dense layer), yielding a $6 \times 1 \times 1$ tensor.
- The whole network becomes **fully convolutional**: composed exclusively of convolutional layers (plus pooling and activations). It can now accept images larger than 96×96 and produce a correspondingly larger output map, enabling dense (pixel-wise) predictions.

21.3.3 Parameter Count Remains Identical

- The number of parameters does not change: the $12 \times 12 \times 256$ filter has $12 \cdot 12 \cdot 256 = 36\,864$ weights per output channel, exactly the same as the dense layer's weight row, plus one bias per channel.
- The second dense layer ($256 \to 6$) is replaced by 6 filters of size $1 \times 1 \times 256$, also with identical parameter count.
- The advantage is **not** parameter reduction, but the ability to process arbitrary input sizes and produce spatially structured outputs.

21.4 From Classification to Dense Prediction

- With the fully convolutional conversion, feeding an image larger than the training patch size yields an output tensor of dimensions $(6, H', W')$ where H', W' depend on the input size and the downsampling factors of the network.
- Each spatial location (i, j) in the output corresponds to the classification result for a patch centred at the corresponding receptive field in the input.
- This turns the classifier into a **sliding-window detector** evaluated in a single forward pass, drastically improving efficiency over explicitly looping over patches.
- The same principle is the foundation for fully convolutional semantic segmentation networks (e.g., U-Net), which add a decoder to upsample the output back to the original image resolution.

21.5 Summary

- A standard CNN classifier with dense layers can be transformed into a fully convolutional network by reshaping the weight matrices into convolution kernels of matching spatial dimensions.
- This conversion preserves the learned function exactly while making the network applicable to variable-sized inputs.
- The technique is a fundamental step towards building architectures for dense prediction tasks such as semantic segmentation.

21.6 Fully Convolutional Conversion of the Sea Lion Classifier

21.6.1 Reinterpreting Dense Layers as Large-Kernel Convolutions

- A fully-connected (dense) layer with input vector $\mathbf{x} \in \mathbb{R}^N$ and output $\mathbf{y} \in \mathbb{R}^M$ can be viewed as a convolution with M filters each of size equal to the *unflattened* input volume.
- For the sea lion classifier:
  - The feature extractor outputs a volume of shape $(256, 12, 12)$.
  - The first dense layer $(36\,864 \to 256)$ is equivalent to a `Conv2d` with:
    - 256 filters,
    - kernel size 12×12,
    - depth 256,
    - **valid** padding (i.e. no zero-padding, stride 1).
  - Convolving this $12 \times 12 \times 256$ filter over the same-sized input volume yields a 1×1 output per filter, exactly matching the dense layer's output.
- The second dense layer $(256 \to 6)$ becomes a `Conv2d` with 6 filters of kernel size 1×1 and depth 256, mapping the $256 \times 1 \times 1$ volume to a $6 \times 1 \times 1$ classification output.
- The total number of learnable parameters remains **identical**:

$$\text{Conv equivalent: } 256 \times (12 \times 12 \times 256 + 1) = 256 \times (36\,864 + 1) \approx 9.44\text{M},$$

  exactly as in the original dense layer.

21.6.2 Advantage: Processing Larger Images

- Once the network is fully convolutional, it is no longer constrained to the fixed 96×96 input size.
- Feeding an image larger than 96×96 causes the convolutional layers to produce proportionally larger feature maps.
- The same 12×12 filters now **slide** over the larger final feature map, producing a 2D output map (heatmap) instead of a single vector.
- Each spatial position in the output corresponds to the classification score for a patch centered at the corresponding receptive field in the input image.
- **Constraint**: the input must be at least 96×96 because max-pooling layers reduce spatial size; an input smaller than 96×96 would lead to a final feature map smaller than 12×12, breaking the 12×12 convolution. Larger sizes are unrestricted.

21.7 Weight Transfer from Trained Dense Layers

21.7.1 Reshaping Weights

- The learned weights of the original dense layers are **reshaped** into convolutional filters without retraining.
- Procedure:
  - For the first dense layer (weight matrix $\mathbf{W} \in \mathbb{R}^{256 \times 36\,864}$):
    - Reshape each row (size 36 864) into a 3D filter of shape $(256, 12, 12)$.
    - The biases remain as a vector of size 256.
  - For the second dense layer ($\mathbf{W} \in \mathbb{R}^{6 \times 256}$):
    - Reshape each row into a filter of shape $(256, 1, 1)$.
    - Biases are a vector of size 6.
- In PyTorch, this is implemented by extracting the state dict of the trained classifier, cloning the tensors, and assigning the reshaped weights to the corresponding convolutional layers of the new fully-convolutional model.
- The use of `.clone()` prevents inadvertent sharing of memory, ensuring the original model remains unmodified.

21.7.2 Verification of Exact Equivalence

- To verify that the weight transfer is mathematically exact, a test image is passed through both the original dense CNN and the newly constructed fully-convolutional network.
- The dense network produces a vector of class scores (logits); the convolutional network produces a $6 \times 1 \times 1$ tensor.
- After squeezing the spatial dimensions and applying softmax, the two outputs are compared.
- The maximum absolute difference between the two probability vectors is zero (within numerical precision), confirming that the networks compute the identical function on 96×96 inputs.

21.8 Dense Prediction on Larger Images

21.8.1 Generating Heatmaps

- The converted fully-convolutional network is applied to a large aerial image ($3\,000 \times 5\,000$ pixels) containing many sea lions.
- The output is a 3D tensor of shape $(6, H', W')$ where H', W' are determined by the input size and the downsampling factors of the network.
- Each spatial location (i, j) holds a 6-dimensional vector of class scores. Applying softmax along the channel dimension yields a per-pixel probability distribution over the six classes.
- The resulting heatmaps for each class show regions where the network detects the corresponding sea lion type or background.
- This enables **efficient whole-image analysis** in a single forward pass, circumventing the need to explicitly crop and classify every patch – a critical advantage for large-scale remote sensing applications.

21.8.2 Application Context

- The technique turns a patch-based classifier into a **sliding-window detector** that runs at a fraction of the computational cost.
- It forms the foundation for fully convolutional semantic segmentation architectures (e.g., U-Net) that combine such a convolutionalized classifier with a learnable upsampling path.

21.9 Dense Prediction on Large Sea Lion Images

21.9.1 Generating Class Heatmaps

- The fully-convolutional network processes a large aerial image (e.g. $3\,000 \times 5\,000$ pixels) in a single forward pass.
- Output shape is $(6, H', W')$ where H', W' are smaller than the input due to max-pooling; here 457×691.
- Each spatial location holds a 6-dimensional class-score vector; applying softmax along the channel axis yields per-pixel class probabilities.
- The resulting low-resolution heatmaps are upsampled (e.g. bilinear interpolation) to the original image size for overlay.

21.9.2 Per-Class Heatmap Visualisation

- **Adult females**: bright spots correspond to actual sea lion locations.
- **Adult males**: some bright spots visible, though the network's accuracy is limited.
- **Pups**: high-intensity regions appear where pups are present; lower intensity elsewhere.
- **Juveniles**: challenging class – the heatmap may not align perfectly with ground truth due to low resolution and inter-class similarity.
- Despite being trained as a patch classifier, the fully-convolutional network effectively produces a rough **semantic segmentation** of the entire scene, assigning a class probability to every pixel.

21.10 Semantic Segmentation

21.10.1 Definition

- Semantic segmentation is the task of assigning a class label to *every pixel* in an image.
- It extends the dense prediction capability just demonstrated, but now the goal is to obtain pixel-accurate class maps, not coarse heatmaps.

21.10.2 Cityscapes Dataset (Downsampled)

- **Full Cityscapes**: 29 fine-grained classes for autonomous driving scenes.
- For the exercise, images are downsampled to 64×128 pixels and classes are reduced to **8 merged categories**:
  - *Static ground* (background, sidewalk, parking, etc.)
  - *Road* (road, lane markings)
  - *Infrastructure* (building, wall, fence, bridge, tunnel, etc.)
  - *Vegetation* (trees, grass)
  - *Sky*
  - *Vehicle* (car, truck, bus, etc.)
  - ...
- Training set: $\sim$2 000 images; validation and test: $\sim$300 each.

21.10.3 Data Handling and Augmentation

- Images and corresponding label maps are loaded, resized, and normalised.
- **Crucial**: geometric augmentations (e.g. random flip) must be applied *identically* to both image and segmentation mask. Otherwise, the mask would no longer correspond to the transformed image.
- The augmented dataset uses `RandomHorizontalFlip` and colour jitter, applied consistently to the pair.

21.10.4 Visualisation

- Sample images and their ground-truth segmentation masks are displayed with a fixed colour mapping (e.g. road in grey, vehicles in yellow, vegetation in green, sky in blue).
- The low resolution (64×128) makes fine details difficult, but the main object categories are still discernible.

21.11 UNet Architecture for Semantic Segmentation

21.11.1 Motivation

- A simple fully-convolutional classifier gives coarse heatmaps; to obtain pixel-accurate segmentation, an **encoder-decoder** architecture with skip connections is employed.
- **UNet** is a classic fully-convolutional network for biomedical and general segmentation tasks.

21.11.2 Encoder (Contracting Path)

- Repeated application of two 3×3 convolutions (each followed by BatchNorm and ReLU) and a 2×2 max-pooling with stride 2.
- At each down-sampling step, spatial dimensions halve and the number of feature channels doubles.
- The bottom of the "U" is the **bottleneck**: the lowest spatial resolution but highest number of channels – representing the most abstract, semantic features.

21.11.3 Decoder (Expanding Path)

- Each step consists of:
  1. **Upsampling** (nearest-neighbour, factor 2) to double spatial size.
  2. A 2×2 "up-convolution" (or more commonly, upsampling followed by a standard convolution) that halves the number of channels.
  3. **Concatenation** with the cropped feature map from the corresponding encoder level (skip connection).
  4. Two 3×3 convolutions (BatchNorm, ReLU) to process the combined features.
- Upsampling + convolution is preferred over transposed convolution in modern implementations for better quality and simpler training.

21.11.4 Skip Connections

- The concatenation of high-resolution encoder features with the upsampled decoder features provides fine spatial detail that would otherwise be lost in the bottleneck.
- This enables the network to produce sharp, pixel-accurate segmentation boundaries despite the down-sampling/up-sampl process.

21.11.5 Final Layer

- After the last decoder block, a 1×1 convolution maps the feature volume to the desired number of classes (8).
- No softmax is applied in the forward pass; it is incorporated in the loss function (e.g. cross-entropy) during training. The argmax over channels gives the final segmentation map.

21.11.6 Implementation Approach

- A `UNetBlock` is defined as a sequence of convolutional, batch-normalisation, and ReLU layers.
- The full UNet is built by stacking such blocks in the down-path, with pooling between them, and then symmetrically in the up-path, with upsampling and concatenation.
- This modular approach mirrors standard practice and facilitates experimentation with different depths and channel configurations.

21.12 U-Net Implementation for Semantic Segmentation

21.12.1 Architecture Construction

- U-Net is built with a symmetric **encoder (contracting path)** and **decoder (expanding path)**.

- **Encoder**: repeated application of `UNetBlock` (two 3×3 convolutions + BatchNorm + ReLU) followed by 2×2 max-pooling. Spatial dimensions halve and channels double at each step.
- **Bottleneck**: the lowest spatial resolution (e.g. 16×32) but with the highest number of channels – the compact semantic representation.
- **Decoder**: at each step, **nearest-neighbour upsampling** (factor 2) followed by a `UNetBlock` and **concatenation** with the corresponding cropped encoder feature map (skip connection).
- The skip connections combine high-resolution spatial information from the encoder with the upsampled semantic features, enabling precise localisation.
- **Output layer**: a final 1×1 convolution that maps the feature volume to the number of classes (8), producing an output of shape $(8, H, W)$.
- The forward pass sequentially applies down-blocks, bottleneck, up-blocks with concatenation, and the output convolution.

21.12.2 Network Dimensions

- Input: RGB images of size 64×128.
- Channel progression: $3 \to 32 \to 64 \to 128 \to 256$ (bottleneck), then decreasing symmetrically back to 8 output channels.
- The final output is a volume of $(8, 64, 128)$ – one heatmap per class.
- The predicted segmentation map is obtained by taking the **argmax** over the channel dimension.

21.13 Training the U-Net

21.13.1 Loss and Metrics

- **Loss**: pixel-wise cross-entropy, which treats each pixel as an independent classification problem.
- **Evaluation metric**: **Intersection over Union (IoU, Jaccard index)**.

$$\text{IoU} = \frac{|\text{prediction} \cap \text{ground truth}|}{|\text{prediction} \cup \text{ground truth}|}$$

- IoU is computed per class and then averaged (mean IoU). Background is often excluded or weighted to handle class imbalance.
- IoU provides a strict measure of spatial overlap; a perfect prediction yields IoU = 1.

21.13.2 Training Loop and Visualisation

- At the end of each epoch, the loss, accuracy, and mean IoU are plotted. Additionally, the predicted segmentation for a fixed validation image is visualised to monitor qualitative progress.
- Early in training, predictions are coarse (e.g. much of the scene classified as road). As training progresses, buildings, sky, cars, and vegetation become correctly segmented.
- Example progression: validation mIoU starts at $\sim$ 0.48, rises to 0.71 after a few epochs, reaches 0.82–0.84 after full training.

21.13.3 Training Behaviour and Overfitting

- Training from scratch (no pretrained backbone) on the downsampled Cityscapes subset.
- The loss and pixel accuracy improve for a number of epochs, then the model begins to overfit (validation loss increases while training loss continues to drop).
- Early stopping is employed; the best model (highest validation mIoU) is restored.
- Best validation mIoU ≈ 0.84; test mIoU ≈ 0.83, demonstrating good generalisation.

21.14 Test Evaluation and Qualitative Results

21.14.1 Performance on Test Set

- The saved best model is evaluated on the test set, yielding mIoU consistent with validation.
- The network performs well on large, distinct classes (road, sky, vegetation, cars) but struggles with very thin structures (e.g. guard rails, which are often 1–2 pixels wide). This is an inherent difficulty of low-resolution segmentation.

21.14.2 Visualising Predictions

- Side-by-side comparisons of input image, ground-truth mask, and predicted mask show accurate delineation of roads, buildings, cars, and sky.
- Mistakes occur mainly on small or rare classes, as expected.

21.15 Per-Class Heatmap Visualisation

21.15.1 Before the Argmax

- The final convolutional layer produces an 8-channel volume; each channel is a heatmap for one class, giving the score (logit) before the softmax/argmax.
- Visualising individual channels shows where the network "sees" each class.
  - The **background** heatmap fires strongly on the car and weakly on distant regions.
  - The **sky** heatmap is bright in the upper part of the image, suppressing other classes.
  - The **road** heatmap highlights the ground plane.
  - The **building** heatmap responds to vertical structures.
- The final prediction is the class with the **highest** score at each pixel; this visualisation explains the network's decision.

21.16 Summary

- U-Net provides an end-to-end trainable, fully convolutional architecture for pixel-wise semantic segmentation.
- The encoder extracts semantic features; the decoder with skip connections recovers fine spatial details.
- Even when trained from scratch on a small dataset, it can achieve respectable IoU scores, though pretrained backbones and data augmentation further improve performance.
- The combination of cross-entropy loss and IoU evaluation is standard for segmentation tasks; class imbalance must be carefully managed.

Chapter 22

Object Detection

22.1 From Localization to Object Detection

22.1.1 Recap of Single-Object Localization

- **Classification**: input image → a single class label from a predefined set.
- **Localization**: still a single object per image, but the goal is to predict a bounding box tightly enclosing the object.
- A bounding box is defined by **four real numbers**:

$$(x_{\min}, y_{\min}, w, h) \quad \text{or} \quad (x_{\min}, y_{\min}, x_{\max}, y_{\max}).$$

- **Architecture**: a convolutional backbone extracts features, followed by a regression head that outputs exactly 4 neurons. The final layer uses a linear activation (no non-linearity), making it a standard regression problem.
- **Loss**: mean squared error (MSE) between the predicted and ground-truth bounding-box coordinates (or mean absolute error).
- **Limitation**: if the image contains two objects (e.g., two persons), the network still outputs a single bounding box; it cannot represent multiple instances.

22.1.2 Extending Regression to Multiple Landmarks

- The same regression framework can be applied to predict **multiple landmarks**, e.g., facial keypoints or human body joints (human pose estimation).
- Instead of 4 outputs, the network predicts $2 \times K$ numbers for K landmarks (each with x and y coordinates).
- This is still a single-output regression problem; the network always returns the same number of coordinates, even if some body parts are occluded or outside the image.
- **Limitation**: a single skeleton is predicted, regardless of how many persons are present; it cannot separate multiple individuals.
- Multi-person pose estimation (e.g., OpenPose) requires a fundamentally different approach that can output a variable number of skeletons, which leads to detection-style methods.

22.1.3 Multi-Task Learning: Classification + Localization

- For a single object per image, we often want **both** its class label and its bounding box.
- A natural solution is **multi-task learning**: share the feature extractor (backbone) and attach two parallel heads to the same latent representation.
  - **Classification head**: fully-connected layers ending with a softmax (output dimension = number of classes).
  - **Regression head**: fully-connected layers ending with 4 linear neurons (bounding box coordinates).
- The network now has two outputs: class probabilities and bounding-box coordinates.

Combined Loss Function

- The training loss must account for both tasks. The two individual losses are combined by a weighted sum:
$$\mathcal{L} = \alpha\,\mathcal{L}_{\text{class}} \;+\; (1-\alpha)\,\mathcal{L}_{\text{reg}},$$
where $\mathcal{L}_{\text{class}}$ is typically categorical cross-entropy and $\mathcal{L}_{\text{reg}}$ is a regression loss (e.g., MSE).
- The hyperparameter α balances the two terms.
- **Crucial**: The two losses must have comparable magnitudes. If the regression loss is computed on raw pixel coordinates (e.g., 0–200) while the classification loss is a cross-entropy (typically < 1), the regression term will dominate the gradient.
- Normalising the bounding-box coordinates to $[0, 1]$ (by dividing by image width/height) helps keep the scales consistent. Other balancing strategies include weighting the losses or scaling one of them.

22.1.4 Towards Object Detection: Multiple Instances per Image

- In realistic images, **multiple objects** of possibly different classes appear simultaneously.
- The desired output is no longer a fixed-size vector, but a **set of tuples** $\{(c_i, b_i)\}_{i=1}^{N}$, where c_i is the class label and b_i the bounding-box coordinates. The number N depends on the image content.
- This breaks the standard supervised learning assumption of one-to-one mapping between input and output.
- **Training data** must provide multiple annotations per image: each image is accompanied by a list of bounding boxes with associated class labels.
- The architectural challenge is to design a network that can propose a **variable number of detections** and be trained end-to-end with such list-valued targets.
- Early approaches (e.g., sliding-window detectors, region-proposal networks) decompose the problem into two stages: (1) propose candidate regions that may contain an object; (2) classify and refine each region.
- Modern single-stage detectors (e.g., YOLO, SSD) directly predict a fixed number of candidate boxes and associated class scores, then filter them; two-stage detectors (e.g., Faster R-CNN) use a region proposal network followed by a per-region classifier.
- These methods constitute the core topic of **object detection**, which will be detailed in the following.

22.2 From Single-Object Localization to Object Detection

22.2.1 Recap: Classification vs. Localization

- **Classification**: assigns a single class label to an input image.
- **Localization**: predicts a bounding box around the *single* main object in the image.
- A bounding box is parameterized by four real numbers: $(x_{\min}, y_{\min}, w, h)$ (top-left corner, width, height) or equivalently $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$.
- The network architecture terminates with four output neurons and a linear activation (no non-linearity), making it a standard regression problem trained with Mean Squared Error (MSE).
- Limitation: if multiple objects are present, the network can only output a single box.

22.2.2 Extending Regression to Multiple Landmarks

- The same regression framework can predict **multiple landmarks** (e.g., facial keypoints or human body joints).
- For K landmarks, the network outputs $2K$ coordinates. This is still a fixed-size output, always returning a complete skeleton even when some joints are occluded.
- Predicting skeletons for *multiple persons* requires handling a variable number of outputs, a problem that motivates detection-style architectures.

22.2.3 Multi-Task Learning: Joint Classification and Localization

- When both the class label and the bounding box are needed for a single object, a **multi-task network** can be used.
- A shared convolutional backbone produces a latent representation, from which two heads branch:
  - **Classification head**: fully-connected layers + softmax (C output neurons for C classes).
  - **Regression head**: fully-connected layers with 4 linear output neurons (bounding box coordinates).
- **Combined loss**: the two tasks contribute to a single scalar loss via a weighted sum,

$$\mathcal{L} = \alpha\, \mathcal{L}_{\text{class}} + (1 - \alpha)\, \mathcal{L}_{\text{reg}},$$

  where $\mathcal{L}_{\text{class}}$ is categorical cross-entropy and $\mathcal{L}_{\text{reg}}$ is MSE.
- The balance coefficient α and proper scaling of the losses (e.g., normalising box coordinates to $[0, 1]$) are crucial to prevent one task from dominating gradient updates.

22.2.4 The Object Detection Problem: Multiple Instances per Image

- In real images, multiple objects of different classes appear together. The desired output is a **set of tuples** $\{(c_i, b_i)\}_{i=1}^{N}$, where N varies with image content.
- This breaks the standard supervised learning assumption of a fixed-size output; the network must be able to propose a variable number of detections.
- Training data therefore contains per-image *lists* of ground-truth bounding boxes with class labels.
- A naive sliding-window classification approach (evaluate a classifier at every position and scale) is extremely slow and requires a background class.

22.3 Evaluation Metric: Intersection over Union (IoU)

- To measure the quality of a predicted bounding box, the **Intersection over Union (IoU)** is used:

$$\text{IoU} = \frac{|\text{predicted box} \cap \text{ground truth box}|}{|\text{predicted box} \cup \text{ground truth box}|}.$$

- IoU ranges from 0 (no overlap) to 1 (perfect match). A detection is typically considered correct if IoU exceeds a threshold (e.g., 0.5).
- IoU is a key component both for performance assessment (mean Average Precision, mAP) and as a criterion in detection losses.

22.4 Early Deep Learning Object Detectors

22.4.1 Traditional Region Proposal Algorithms

- Before fully convolutional solutions, handcrafted **region proposal methods** (e.g., Selective Search) extracted around 2000 candidate bounding boxes per image based on low-level cues (edges, textures, blobs).
- These algorithms achieved high *recall* (most objects were covered) but very low *precision* (many false positives). They provided the initial set of regions for classification.

22.4.2 R-CNN: Region-based Convolutional Neural Network

- **Pipeline**: (1) extract ~ 2000 region proposals; (2) warp each region to a fixed size; (3) process each through a CNN (AlexNet) to obtain a latent vector; (4) classify the region with a set of SVMs (one per class + background); (5) refine the bounding box with a linear regression layer.
- **Limitations**:
  - Extremely slow at inference (processing each proposal independently) and training (feature vectors for 2000 proposals per image had to be stored on disk).
  - Not end-to-end trainable: CNN, SVMs, and bounding-box regressor were trained separately.
  - The region proposal step was a fixed, non-learnable algorithm.

22.4.3 Fast R-CNN: Shared Feature Maps and RoI Pooling

- **Key idea**: avoid redundant computation by running the CNN *once* on the whole image, then projecting region proposals onto the resulting feature map.
- **RoI Pooling layer**: for each proposed region (of arbitrary size and aspect ratio), divides it into a fixed $H \times W$ grid (e.g., 7×7) and max-pools each cell, producing a fixed-size feature vector regardless of the region's shape.
- This fixed-size vector is fed to a small fully-connected network that branches into:
  - A **classification head** (softmax over $C + 1$ classes, including background).
  - A **bounding-box regression head** (adjustments to the proposal coordinates).
- **Multi-task loss**: jointly optimised classification and regression losses, enabling end-to-end training of the whole pipeline (except the external region proposal step).
- **Speed**: inference time dropped from ~ 47 seconds (R-CNN) to ~ 2 seconds per image, but the external region proposal step remained a bottleneck.

22.4.4 Faster R-CNN: Learnable Region Proposals

- The remaining non-learnable component — the region proposal algorithm — is replaced by a **Region Proposal Network (RPN)**.
- The RPN is a small fully-convolutional network that operates on the *same* feature map produced by the backbone. It slides a small window over the feature map and, at each location, simultaneously predicts:
  - **Objectness score**: whether an object is present.
  - **Bounding-box refinements**: for a set of predefined anchor boxes of various scales and aspect ratios.
- The RPN is **trainable** and shares computations with the detection network, making the entire system differentiable and trainable end-to-end.
- This architecture achieves near real-time inference while maintaining high accuracy, and forms the foundation for many modern object detectors.

22.5 Region Proposal Network (RPN) – Making Proposals Learnable

22.5.1 Motivation

- In Fast R-CNN, the **region proposal step** (e.g., Selective Search) is external, handcrafted, and cannot be optimized jointly with the detector. It also produces too many proposals (≈2000) and is agnostic to the object classes of interest.
- **Faster R-CNN** replaces this step with a **Region Proposal Network (RPN)**, a small fully convolutional network that operates on the same feature maps as the detection head, making the entire pipeline **end-to-end trainable**.
- The RPN learns to propose regions that are likely to contain objects of the target classes, producing **fewer and more precise proposals** (e.g., 300 instead of 2000).

22.5.2 RPN Operation

- The RPN takes as input the convolutional feature map (the latent representation) produced by the backbone.
- A small sliding window (e.g., 3×3 convolution) is applied over the feature map, followed by two sibling 1×1 convolutional layers:
  - **Objectness classification**: for each spatial location and for each of k predefined **anchor boxes**, outputs two scores (object / not-object).
  - **Bounding-box regression**: outputs $4k$ coordinates that refine the anchor box to better fit the object.
- Thus the RPN performs a **multi-task learning** problem at every sliding position: estimating whether an object exists *inside* each anchor, and correcting the anchor's position.

22.5.3 Anchor Boxes

- Anchors are a set of **fixed reference bounding boxes** with different scales and aspect ratios (e.g., 1:1, 1:2, 2:1).
- At each spatial location of the feature map, the RPN predicts objectness and regression corrections for *every* anchor (i.e., for k anchors, $2k$ objectness scores and $4k$ regression values).
- The architecture is independent of the exact anchor shapes; changing the anchor set only changes the interpretation of the outputs, not the network structure.
- Anchors allow the network to handle objects of various sizes and aspect ratios without needing to run multiple scales of the input image.

22.5.4 Training the RPN

- A ground-truth label for each anchor is generated by comparing it with the annotated bounding boxes in the training image. An anchor is considered **positive** if it has high IoU with any ground-truth box, or is the best match; **negative** if its IoU is low (e.g., < 0.3).
- The RPN is trained with a combined loss:

$$\mathcal{L}_{\text{RPN}} = \mathcal{L}_{\text{cls}}(\text{objectness}) + \lambda\, \mathcal{L}_{\text{reg}}(\text{box refinements}),$$

  where the regression loss is applied only to positive anchors.
- After training, the top-scoring proposals (e.g., 300) are kept after applying **non-maximum suppression (NMS)** to remove highly overlapping boxes.

22.6 Faster R-CNN: Unified Object Detection

22.6.1 Architecture Overview

- Faster R-CNN consists of three main components:
  1. **Backbone**: a CNN (e.g., VGG, ResNet) that computes a shared feature map for the whole image.
  2. **Region Proposal Network (RPN)**: proposes candidate object regions from the feature map.
  3. **Detection head**: for each proposed region, applies RoI Pooling to extract a fixed-size feature vector, then passes it through fully-connected layers to obtain:
     - Class probabilities over C object classes + background.
     - Bounding-box regression offsets for *each of the C classes* (i.e., $4C$ values; no regression for background).

22.6.2 Training Strategy

Faster R-CNN is typically trained in a staged, alternating fashion to avoid destructive interference between the RPN and the detection head:

1. **Stage 1**: Train the RPN alone (backbone frozen or initialised with pretrained weights). Loss based on objectness and box regression for anchors.
2. **Stage 2**: Freeze the RPN and train the detection head using the proposals generated by the RPN. The backbone may remain frozen or be fine-tuned.
3. **Stage 3 (optional)**: Freeze the backbone and detection head, then fine-tune only the RPN's layers to further refine proposals.
4. **Stage 4**: Keep the RPN fixed and fine-tune the detection head (or vice versa) to achieve the best overall performance.

This staged training allows each sub-network to converge before the other is modified, ensuring stability.

22.6.3 Speed and Accuracy

- R-CNN: $\sim$ 49 seconds per image.
- Fast R-CNN: $\sim$ 2.3 seconds.
- Faster R-CNN: $\sim$ 0.2 seconds (5 FPS) on GPU, enabling near real-time detection while maintaining state-of-the-art accuracy.

22.7 Single-Shot Detectors: YOLO

22.7.1 Motivation and Philosophy

- Two-stage detectors (Faster R-CNN) achieve high accuracy but are inherently sequential (propose, then classify). For real-time applications, a **single-shot** approach is desirable.
- **YOLO (You Only Look Once)** re-frames object detection as a **single regression problem**: directly from image pixels to bounding box coordinates and class probabilities in one forward pass, without a separate proposal stage.

22.7.2 Core Idea

- The input image is divided into an $S \times S$ grid (e.g., 7×7).
- Each grid cell is responsible for predicting:
  - **B** bounding boxes (each with coordinates and a confidence score that an object is present).
  - **C** class probabilities (conditioned on the cell containing an object).
- At training time, a grid cell is assigned to detect the object whose center falls inside that cell.
- The predictions are compared against ground-truth boxes, and a combined loss (classification + localization + confidence) is optimised end-to-end.

22.7.3 Comparison with Anchor-Based Methods

- YOLO also uses **anchors** (pre-defined shapes) to stabilise bounding box prediction. However, unlike Faster R-CNN, it predicts everything in a single evaluation of the network, making it extremely fast (real-time, ¿30 FPS) at the cost of slightly lower accuracy on small objects.
- Subsequent versions (YOLOv2, v3, ...) have continuously improved the architecture, balancing speed and accuracy.

22.7.4 Impact

- YOLO and similar single-shot detectors (e.g., SSD) established a new paradigm that trades a small accuracy drop for a dramatic speed increase, enabling applications like real-time video analysis, autonomous driving, and robotics.

22.8 Summary: Evolution of Object Detection

- **R-CNN**: proposal generation + independent CNN classification $\rightarrow$ very slow, not end-to-end.
- **Fast R-CNN**: shared convolutional features + RoI Pooling $\rightarrow$ end-to-end except for external proposals, much faster.
- **Faster R-CNN**: integrated RPN $\rightarrow$ fully end-to-end trainable, high accuracy, near real-time.

- **YOLO / SSD**: single-shot regression from pixels to boxes → real-time, simpler pipeline, slightly less accurate for small objects.
- All these architectures build upon the fundamental CNN components (convolutions, pooling, fully-connected layers) learned throughout this course, demonstrating the power of deep learning for high-level visual recognition.

22.9 YOLO – Single-Shot Object Detection

22.9.1 Core Idea

- YOLO (You Only Look Once) formulates object detection as a **single regression problem**, directly mapping from image pixels to bounding box coordinates and class probabilities in one forward pass.
- The input image is divided into an $S \times S$ grid (e.g., 7×7).
- Each grid cell predicts:
  - **B** bounding boxes (each with x, y, w, h and a confidence score that an object is present).
  - **C** class probabilities (conditioned on the cell containing an object).
- The final output is a large tensor of shape $S \times S \times (B \cdot 5 + C)$.

22.9.2 Anchors in YOLO

- YOLO also uses **anchor boxes** (predefined shapes) to stabilise bounding box predictions, similar to the region proposal network in Faster R-CNN.
- For each anchor, the network predicts an offset to refine its position and size, plus a class score.
- The training loss is designed such that each grid cell and anchor is assigned responsibility for detecting an object if the object's centre falls into that cell and the anchor matches the ground truth.
- Anchors allow the network to handle objects of different scales and aspect ratios without processing the image multiple times.

22.9.3 Training and Inference

- The entire network is trained **end-to-end** with a combined loss covering classification, localization, and confidence.
- At inference, a single forward pass yields a large number of candidate boxes. **Non-maximum suppression (NMS)** removes highly overlapping low-confidence boxes, keeping only the best detections.
- Single-shot detectors (YOLO, SSD) are significantly **faster** than two-stage detectors (Faster R-CNN), enabling real-time applications, though they may sacrifice some accuracy on small objects.

22.10 Instance Segmentation – Mask R-CNN

22.10.1 Motivation

- Object detection provides bounding boxes and class labels. **Instance segmentation** extends this by also predicting a pixel-wise segmentation mask *inside* each detected bounding box, separating individual objects even if they belong to the same class (e.g., dog1, dog2).
- This requires outputting a **variable number of masks**, one per detected instance.

22.10.2 Mask R-CNN

- Mask R-CNN builds upon Faster R-CNN by adding a third branch to the detection head: in parallel to the classification and bounding-box regression branches, a **small fully convolutional network** predicts a binary mask for each region proposal.
- RoI Pooling is replaced with **RoI Align**, which uses bilinear interpolation to avoid the coarse spatial quantization of RoI Pooling, preserving fine spatial details crucial for accurate masks.
- The network is trained end-to-end with a multi-task loss that combines classification, bounding-box regression, and mask prediction (per-pixel binary cross-entropy).
- Results: extremely accurate instance-level segmentation, even for small or overlapping objects (e.g., crowds, distant persons, objects on a shelf).

22.10.3 Human Pose Estimation with Mask R-CNN

- Mask R-CNN can be extended to predict human **skeletons** (keypoints) by adding a keypoint branch, outputting joint locations for each detected person.
- This enables simultaneous detection, instance segmentation, and pose estimation of multiple people in a single network.

22.11 Feature Pyramid Networks (FPN)

22.11.1 Motivation

- Objects in images appear at vastly different scales. A single feature map at one resolution may be inadequate for detecting both small and large objects.
- Feature Pyramid Networks **fuse semantically strong deep features** (high-level, low-resolution) with **spatially precise shallow features** (low-level, high-resolution) through a top-down pathway and lateral skip connections.

22.11.2 Architecture

- The backbone (e.g., ResNet) produces a hierarchy of feature maps at different scales (bottom-up pathway).
- A top-down pathway upsamples the deep feature maps and merges them with the corresponding bottom-up maps via element-wise addition (lateral connections).
- Predictions (objectness, bounding boxes, or masks) are made **independently at each level** of the pyramid, allowing the network to detect objects at all scales.
- FPN can be used as the feature extractor for RPN (Faster R-CNN) or directly for single-shot detectors, significantly improving detection of small objects.

22.12 Advanced Research Directions

22.12.1 3D Object Detection and Autonomous Driving

- Extending 2D bounding boxes to **3D bounding boxes** requires depth information, often obtained from LiDAR sensors.
- Combining image data and LiDAR point clouds (which are irregular, not on a grid) is a challenging research problem, addressed in the advanced deep learning course.

- Applications include autonomous vehicles, where precise 3D localization of surrounding objects is critical.

22.12.2 Weakly Supervised and Self-Supervised Detection

- Training detectors with limited or no bounding-box annotations: using **Class Activation Maps (CAMs)** or other weak supervision signals to localize objects.
- Example: detecting forbidden items in a waste sorting plant using only image-level labels or weak cues.

22.12.3 Other Active Topics

- **Video object segmentation** and **camouflaged animal detection** in challenging visual conditions.
- **Medical image segmentation** with limited data: generating realistic training samples (e.g., GANs) for 2D/3D segmentation.
- **6-DoF object pose estimation**: estimating the 3D position and orientation of an object from a single image, crucial for robotics and AR.
- **Human pose estimation in the wild**: from drone footage, for sports activity analysis.
- **Non-conventional imaging modalities**: processing raw sensor data (e.g., event cameras, photon-counting sensors) instead of standard RGB images.
- **Federated and contrastive learning**: learning from decentralized data and without labels, covered in the Advanced Deep Learning course.

22.13 Course Summary and Advanced Deep Learning

- This course provided a thorough understanding of CNNs, from basic layers and architectures to advanced visual recognition tasks (classification, localization, segmentation, detection).
- The next step is the **Advanced Deep Learning** course, which covers:
  - **Unsupervised and self-supervised learning** (anomaly detection, contrastive learning).
  - **Transformers** and **multimodal learning** (vision + language, CLIP).
  - **Generative AI** (GANs, diffusion models).
  - **Deep learning beyond images** (3D data, videos, graph neural networks).
  - **Explainability** and other salient topics.
- Alternatively, a computer vision course focusing on mathematical and algorithmic foundations (non-deep) is available in the second semester.

Chapter 23

Multi-Task Learning, Object Detection and Instance Segmentation

23.1 Multi-Task Learning for Joint Classification and Localization

23.1.1 Concept

- **Multi-task learning** trains a single network to perform several tasks simultaneously by sharing a common feature extractor and using task-specific heads.
- Example: predicting both the class label and the bounding box of the main object in an image.
- The convolutional backbone learns features that are useful for both classification and regression; separate heads specialise in each task.
- The total loss is a weighted combination of the individual task losses (e.g., cross-entropy for classification, MSE for regression).

23.1.2 Dataset and Annotations

- The dataset (cats and dogs) provides for each image:
  - The class label (cat or dog).
  - The bounding box coordinates (normalised to $[0, 1]$).
- A custom `Dataset` class returns a tuple `(image, label, bounding_box)` for every sample.

23.1.3 Data Augmentation with Bounding Boxes

- **Geometric augmentations** (e.g., horizontal flip) must be applied **identically** to both the image and the target bounding box; otherwise the box would no longer correspond to the object.
- Implementation steps:
  1. Normalised coordinates are temporarily converted to absolute pixel values.
  2. The image and the target (labels and boxes) are wrapped in `tv_tensors` for `torchvision` compatibility.
  3. Augmentations are applied to the image and the target dictionary simultaneously.
  4. After augmentation, the box coordinates are re-normalised by the new image dimensions.
- A visual check confirms that the bounding box remains correctly placed after flipping.

23.1.4 Data Splitting and Data Loaders

- Training set: 960 images (after 20% validation split), balanced between cats and dogs.
- Validation set: 240 images.
- Data loaders use a batch size of 64, shuffle the training set, and apply random horizontal flip (probability 0.5) as augmentation.
- The `Dataset` is instantiated with *both* labels and bounding boxes, so each batch provides images, class labels, and bounding-box coordinates.

23.2 Model Architecture for Multi-Task Localization

23.2.1 Backbone and Feature Sharing

- A pretrained **EfficientNet-B0** (ImageNet weights) is used as the convolutional backbone.
- The backbone parameters are **frozen** initially (transfer learning); only the new heads are trained.
- The backbone's last feature map is passed through **Global Average Pooling** (GAP) to obtain a feature vector.
- This feature vector is shared between the two tasks.

23.2.2 Task-Specific Heads

- **Classification head**: `Dropout` → `Linear`(feature_dim, num_classes). Output: logits for the two classes.
- **Regression head**: `Dropout` → `Linear`(feature_dim, 4). Output: the four bounding-box coordinates (normalised).
- Both heads receive the *same* flattened feature vector.

23.2.3 Forward Pass

1. Input image → backbone → feature map.
2. Global average pooling → vector of size `num_features`.
3. Flatten (to remove spatial dimensions of size 1).
4. Feed to classification head → class logits.
5. Feed to regression head → bounding-box coordinates.
6. Return both outputs.

23.3 Training Considerations

23.3.1 Loss Function

- The two tasks produce separate losses: e.g., categorical cross-entropy for classification and mean squared error for regression.
- These are combined into a single scalar loss with a balancing weight (not detailed in this snippet, but essential for multi-task learning).

23.3.2 Evaluation Metric

- **Spearman rank correlation** is used to evaluate the quality of the predicted bounding boxes.
- It measures the monotonic relationship between predicted and ground-truth coordinates and is more robust than Pearson correlation for non-linear dependencies.
- Using a different metric for early stopping than the training loss helps avoid overfitting and gives a complementary view of performance.

23.4 Training and Evaluation of the Multi-Task Model

23.4.1 Loss Combination

- The model produces two outputs: class logits $\hat{\mathbf{y}}$ and bounding-box coordinates $\hat{\mathbf{b}}$.
- The total loss is a weighted sum of the two task-specific losses:

$$\mathcal{L} = \lambda_{\text{cls}}\, \mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) \;+\; \lambda_{\text{box}}\, \mathcal{L}_{\text{MSE}}(\hat{\mathbf{b}}, \mathbf{b}),$$

  where $\mathcal{L}_{\text{CE}}$ is categorical cross-entropy and $\mathcal{L}_{\text{MSE}}$ is mean squared error.
- Backpropagation on the total loss updates the shared backbone (through both tasks) and the respective heads (only through their own loss).
- The weights λ_{cls}, λ_{box} can be tuned; equal weights are a natural starting point but may need adjustment if the losses have very different magnitudes.

23.4.2 Training and Validation Loops

- `train_epoch`: model in `train()` mode, iterate over batches containing `(images, labels, boxes)`.
- Forward pass $\rightarrow$ logits and predicted boxes.
- Compute the individual losses and the combined loss; backward pass on the combined loss.
- Validation follows the same structure but under `model.eval()` and `torch.no_grad()`.
- The **Spearman correlation** between predicted and ground-truth boxes is tracked as an additional regression metric.

23.4.3 Training Results

- **Transfer learning** (frozen backbone): both classification loss and regression loss decrease; accuracy reaches $\sim 98\%$, Spearman correlation ~ 0.82.
- **Fine-tuning**: the last three blocks of EfficientNet are unfrozen, learning rate reduced to 10^{-4}. Classification accuracy reaches 99%, Spearman correlation 0.85.
- The total loss may stop improving because the classification loss saturates while the regression loss still decreases. Adjusting the loss weights could allow further improvement on the bounding-box task.
- At inference, the model receives an image and returns both the class probabilities (after softmax) and the bounding-box coordinates in a single forward pass.

23.5 From Localization to Object Detection

23.5.1 The Core Difference

- **Localization** (and the multi-task model above) assumes a **single object** per image. The output is one bounding box and one class label.
- **Object detection** deals with **multiple objects** per image, requiring the output of a *variable number* of bounding boxes and class labels.
- Deep learning detectors typically output a **fixed maximum number** of candidate detections and then apply **post-processing** to filter and remove duplicates:
  - **Confidence thresholding**: keep only boxes with a class confidence above a threshold.
  - **Non-maximum suppression (NMS)**: among overlapping boxes of the same class, keep the one with the highest score and remove the others.

23.6 The Penn-Fudan Pedestrian Dataset

23.6.1 Overview

- A classic small dataset for pedestrian detection and instance segmentation.
- Provides RGB images and corresponding **instance segmentation masks**.
- The masks encode each object with a unique integer ID (pixels belonging to object i have value i, background is 0). This is the **instance segmentation** format, which separates individual instances of the same class.

23.6.2 From Masks to Bounding Boxes

- For each object ID in the mask:
  1. Create a binary mask where pixels equal to that ID are 1, others 0.
  2. Find the **minimum and maximum coordinates** along height and width where the binary mask is 1.
  3. The resulting rectangle $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$ is the bounding box for that instance.
- This process converts an instance segmentation annotation into a set of bounding boxes, suitable for object detection training.

23.6.3 Dataset Class

- The custom `Dataset` stores paths to images and masks; samples are loaded on-the-fly in `__getitem__`.
- Transforms: for simplicity, only **colour augmentations** are applied (no geometric transforms), so the masks do not need to be transformed together with the images (avoiding the complexity of aligning multiple bounding boxes).
- Each sample returns the image tensor and a target dictionary containing:
  - `boxes`: a tensor of shape $(N, 4)$ with the N ground-truth bounding boxes.
  - `labels`: a tensor of shape $(N,)$ with the class labels (all 1 for pedestrian in this dataset).
  - `masks`: (optionally) the binary masks for instance segmentation.

23.7 Training Considerations for Object Detection

- The model must handle a variable number of ground-truth objects per image. This is typically achieved by treating the predictions and targets as **sets** and using a matching loss (e.g., Hungarian matching) or by pre-defining a fixed number of detection slots and assigning ground truths to the best-matching slots.
- The detection loss usually contains both a **classification term** (per detection) and a **bounding-box regression term** (only for detections matched to a ground truth).
- The specific architectures and training procedures (Faster R-CNN, YOLO, etc.) are covered in the following parts of the course.

23.8 Dataset Preparation for Object Detection

23.8.1 On-the-Fly Image Loading

- For large datasets, it is common to load images one by one in `__getitem__` rather than storing the entire dataset in memory.
- The Penn-Fudan pedestrian dataset contains only ~ 170 images, but the same pattern generalises to larger datasets.
- In `__getitem__`: load the image (apply colour augmentations), load the corresponding instance mask, then extract object IDs (ignoring background 0).
- For each object ID, create a binary mask, find the bounding box as the tightest enclosing rectangle, and append to lists.
- Ensure that only valid bounding boxes (with positive area) are kept.
- Convert the lists of boxes, labels (all pedestrians $\rightarrow$ class 1), and masks to tensors.
- The target dictionary follows the COCO format:
  - `boxes`: $(N, 4)$ tensor of N ground-truth bounding boxes.
  - `labels`: $(N,)$ tensor of class labels.
  - `masks`: (N, H, W) binary masks (used only for instance segmentation).
  - `image_id`, `area`, `iscrowd`: auxiliary fields (set `iscrowd` to 0 for clean objects).

23.8.2 Geometric Augmentation via Dataset Concatenation

- Colour jittering (brightness, contrast, saturation) is applied directly in the dataset and does not affect targets.
- **Horizontal flipping** is performed as a separate step: create a flipped copy of every image and its corresponding mask, then recompute the bounding boxes from the flipped masks.
- This results in a second dataset of flipped samples, which is concatenated with the original dataset using `torch.utils.data.ConcatDataset`.
- The training set effectively doubles in size (from 100 to 200 images in this example).
- This approach avoids the complexity of applying geometric transforms with multiple boxes inside the dataset's `__getitem__`, while still ensuring the bounding boxes remain consistent.

23.8.3 Data Loaders and Batch Size Considerations

- Training and validation data loaders use a small batch size (e.g., 2–8) because object detection models are significantly larger than simple classifiers and require more GPU memory.
- **Test data loader uses batch size = 1**. Since each image can contain a different number of objects, stacking tensors of varying sizes would require padding or special collation. Batch size 1 simplifies inference and is standard for evaluation.
- Shuffling is applied only to the training set; validation and test are processed sequentially.

23.9 Object Detection vs. Semantic and Instance Segmentation

- **Semantic segmentation**: assigns a class label to every pixel, but cannot distinguish different instances of the same class (e.g., two overlapping cats are merged into one region).
- **Object detection**: provides a set of bounding boxes, each with a class label, thereby separating individual instances.
- **Instance segmentation**: extends object detection by predicting a pixel-wise mask inside each bounding box, giving the exact shape of each instance.
- The Penn-Fudan dataset provides instance masks; bounding boxes are derived from these masks. This illustrates the close relationship between detection and instance segmentation.

23.10 Evaluation Metrics for Object Detection

23.10.1 Intersection over Union (IoU)

- IoU between a predicted box B_p and a ground-truth box B_{gt} is defined as:

$$\text{IoU}(B_p, B_{gt}) = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}$$

- A predicted box is considered a **true positive** (TP) if it has an IoU with some ground-truth box above a threshold (e.g., 0.5) and is the best match; otherwise it is a **false positive** (FP). Ground-truth boxes not matched by any prediction are **false negatives** (FN).
- There is no concept of "true negatives" in detection; the background class is not explicitly counted.

23.10.2 Precision, Recall, and F1 Score

- **Precision** $= \frac{\text{TP}}{\text{TP+FP}}$ (how many detected boxes are correct).
- **Recall** $= \frac{\text{TP}}{\text{TP+FN}}$ (how many of the real objects were found).
- **F1 score** $= 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision+recall}}$, the harmonic mean.
- These metrics are computed for a fixed confidence threshold on the detections. Varying the threshold yields a precision–recall curve, and the **average precision (AP)** summarises performance across all thresholds.
- For instance segmentation, the same metrics are used, but the IoU is computed on the predicted and ground-truth masks.

23.11 Historical Object Detection Architectures

23.11.1 R-CNN (Region-based CNN)

- **Pipeline**:
  1. Use an external region proposal algorithm (e.g., Selective Search) to generate ~ 2000 candidate regions.
  2. Warp each region to a fixed size and run it independently through a CNN to extract a feature vector.
  3. Classify each region with a set of SVMs (one per class + background).
  4. Refine the bounding box coordinates with a linear regression layer.
- **Drawbacks**:
  - Extremely slow (~ 47 seconds per image) because the CNN is applied to every proposal.
  - Training is multi-stage, not end-to-end.
  - The region proposal step is fixed and not learnable.

23.11.2 Fast R-CNN

- **Key improvement**: the whole image is processed *once* by a CNN to produce a convolutional feature map.
- For each region proposal, **RoI (Region of Interest) Pooling** extracts a fixed-size feature vector from the shared feature map, avoiding redundant computation.
- The fixed-size vector is fed to a fully-connected network that simultaneously predicts:
  - Class probabilities (softmax over $C + 1$ classes, including background).
  - Bounding-box regression offsets.
- **Advantages**:
  - Much faster than R-CNN (~ 2.3 seconds per image, about $20\times$ speedup).
  - End-to-end trainable (except for the external region proposal step).
  - Uses a multi-task loss (classification + regression) in a unified framework.
- The external region proposal algorithm remains a bottleneck and prevents the system from being fully learnable, motivating the development of Faster R-CNN.

23.12 Faster R-CNN: Learnable Region Proposals

23.12.1 Motivation

- Fast R-CNN still relies on an external region proposal algorithm (e.g., Selective Search) that is not trainable and produces many redundant proposals.
- **Faster R-CNN** removes this dependency by introducing a **Region Proposal Network (RPN)** that is jointly trained with the detector.
- The RPN shares the convolutional feature map with the detection head, making the entire pipeline end-to-end trainable.

23.12.2 Region Proposal Network (RPN)

- The RPN takes a convolutional feature map and slides a small window over it.
- At each sliding position, it considers k predefined **anchor boxes** of different scales and aspect ratios.
- For each anchor, two sibling 1×1 convolutional layers predict:
  - **Objectness score** (2 values: object / not object).
  - **Bounding-box regressions** (4 values: offsets from the anchor to the object box).
- Anchors that have high objectness and are refined by the regressions become **region proposals** fed to the detection head.
- Non-maximum suppression (NMS) is applied to keep only the best proposals (typically ~ 300).

23.12.3 Feature Pyramid Network (FPN) Extension

- Objects appear at different scales; a single feature map may miss very small objects.
- **Feature Pyramid Network (FPN)** augments the backbone with a top-down pathway and lateral connections, producing a set of feature maps at multiple resolutions (e.g., 5 levels).
- The RPN is applied independently to each FPN level, enabling the detection of objects at a wide range of sizes.
- Small objects are detected on higher-resolution (early) feature maps, large objects on lower-resolution (deep) feature maps.
- FPN significantly improves detection of small instances with minimal computational overhead.

23.13 Loading a Pre-trained Faster R-CNN from TorchVision

- The model is obtained via `torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)`.
- It uses **ResNet-50** as the backbone and includes a **Feature Pyramid Network**.
- The model has been pre-trained on COCO (80 object classes) and contains ~ 41 million parameters.
- The architecture is complex; it is not implemented from scratch in the exercise but inspected to understand its components.

23.13.1 Model Architecture Breakdown

- **Transform**: normalises images (ImageNet statistics) and resizes them to a fixed range for multi-scale training.
- **Backbone (body)**: ResNet-50 consisting of `layer1` to `layer4`. The last feature map has a stride of 32.
- **Feature Pyramid Network**: takes the feature maps from the backbone's layers, applies a top-down pathway with lateral connections, and outputs 5 feature maps at strides $\{4, 8, 16, 32, 64\}$ (the last one is obtained by a 2×2 max-pooling on the deepest FPN level).
- **Region Proposal Network (RPN)**:
  - **Anchor generator**: defines k anchors per spatial location on each FPN level (commonly $k = 3$ scales $\times 3$ aspect ratios $= 9$).

  - **RPN head**: for each FPN level, applies a 3×3 convolution followed by two sibling 1×1 convolutions to predict objectness scores and bounding-box deltas.

- **ROI Head**: takes the region proposals and, for each proposal, extracts a fixed-size feature vector via RoI Align (or RoI Pooling), then passes it through shared fully-connected layers, and finally two parallel heads:
  - **Classification** (class logits for C object classes + background).
  - **Bounding-box regression** (offsets for each of the C classes).

23.14 Evaluating the Pre-trained Model on Penn-Fudan

- The COCO-pre-trained Faster R-CNN is applied directly to the pedestrian dataset.
- It detects many objects because COCO contains 80 classes (e.g., cars, traffic lights) in addition to pedestrians.
- Quantitative evaluation: **high recall** (it finds most pedestrians) but **low precision** (many false positives) – e.g., 90 true positives, 285 false positives.
- Example images show correct pedestrian detections alongside false positives such as traffic lights, cars, etc.

23.15 Inspecting the Region Proposal Network

23.15.1 Objectness Heatmaps

- For a given input image, the RPN produces, for each FPN level, an **objectness score map** – a heatmap indicating the probability that an object (any class) is present at each location.
- To obtain the heatmaps:
  1. Pass the image through the backbone and FPN to get a list of 5 feature maps.
  2. Apply the RPN head to each feature map.
  3. Extract the objectness scores (e.g., the logits for the "object" class) and upsample them to the input image size for visualisation.
- Visualisation shows:
  - Large objects are detected at deeper (low-resolution) FPN levels.
  - Small objects (e.g., distant pedestrians, traffic lights) are visible only in the higher-resolution levels.
  - This demonstrates the necessity of the multi-scale prediction provided by the FPN.

23.15.2 Proposal Generation

- The RPN also predicts, for each anchor, **bounding-box deltas** (t_x, t_y, t_w, t_h) that transform the fixed anchor into a refined proposal.
- Given an anchor of centre (x_a, y_a), width w_a, height h_a, the predicted proposal coordinates are computed as:

$$x = x_a + t_x w_a, \quad y = y_a + t_y h_a,$$
$$w = w_a \exp(t_w), \quad h = h_a \exp(t_h).$$

- These refined boxes are the region proposals that are passed to the ROI head for final classification and bounding-box regression.

- In the notebook, the proposals can be extracted and visualised to understand which regions the RPN considers promising.

23.16 Non-Maximum Suppression (NMS)

23.16.1 Principle

- After the Region Proposal Network (RPN) or the final detection head, many bounding boxes can overlap heavily around the same object.
- **Non-Maximum Suppression** removes redundant, highly overlapping boxes, keeping only the most confident one.
- Algorithm:
  1. Sort all boxes by their confidence score (descending).
  2. Select the box with the highest score; compute its Intersection over Union (IoU) with all remaining boxes.
  3. Remove boxes whose IoU exceeds a set threshold (e.g., 0.7).
  4. Repeat the process with the next highest-scoring box among the survivors.
- This is applied to RPN proposals and again to the final class-specific detections.

23.16.2 Disabling NMS in Code

- In `torchvision`, NMS behaviour can be controlled by the `nms_thresh` parameter. Setting it to a very high value (e.g., 0.99) practically disables suppression, allowing inspection of all raw proposals.

23.17 Region Proposal Network: From Anchors to Proposals

23.17.1 Anchor Generation and Bounding-Box Decoding

- The RPN places a set of **anchor boxes** (pre-defined sizes and aspect ratios) at each spatial location of the feature map.
- For each anchor, two sibling 1×1 convolutional layers predict:
  - Objectness score (object / not object).
  - **Bounding-box regression deltas** (t_x, t_y, t_w, t_h).
- The actual proposal coordinates are obtained by **decoding** the anchors with the predicted deltas:

$$x = x_a + t_x w_a, \quad y = y_a + t_y h_a,$$
$$w = w_a \exp(t_w), \quad h = h_a \exp(t_h),$$

  where (x_a, y_a, w_a, h_a) are the anchor's centre, width, and height.
- This decoding step yields a dense set of candidate boxes (number of pixels $\times$ k).

23.17.2 NMS in the RPN

- After decoding, thousands of proposals are generated. NMS is applied to reduce them to a manageable number (typically ~ 2000 during training, ~ 300 at inference).
- The surviving proposals are those with high objectness and low mutual overlap.

23.17.3 RPN Proposals vs. Final Detections

- RPN proposals are **coarse**: they roughly enclose objects, but boundaries are not precise.
- The **ROI head** further refines each proposal's coordinates and assigns a specific class label, producing the final, tightly-fitting detections.
- Visualisation of RPN proposals shows many approximate boxes; final detections are much cleaner.

23.18 Fine-Tuning Faster R-CNN on a Custom Dataset

23.18.1 Training Loop with Multi-Task Loss

- The model is pre-trained on COCO (80 classes). For fine-tuning on the Penn-Fudan pedestrian dataset, the number of output classes is changed (e.g., to 2: background + pedestrian).
- The forward pass returns a **dictionary of losses** combining:
  - RPN objectness loss (binary classification: object/not object).
  - RPN bounding-box regression loss (only for positive anchors).
  - ROI head classification loss (multi-class cross-entropy).
  - ROI head bounding-box regression loss (per class).
- The total loss is the **sum** of all four terms. Backpropagation is performed on this combined loss.
- Data loading provides one image and a target dictionary (boxes, labels) per iteration; batch size is typically 1 or very small due to GPU memory constraints.

23.18.2 Optimizer and Learning Rate Scheduler

- Adam optimizer with weight decay (e.g., 10^{-4}) is used.
- A **cosine annealing** learning rate scheduler is employed: it starts with a moderate learning rate ($\sim 10^{-4}$) and smoothly decays it over the epochs, stabilising convergence and reducing overfitting.

23.18.3 Training Progress and Metrics

- Before fine-tuning, the COCO-pre-trained model already has high recall on pedestrians (finds most of them) but very low precision due to many false positives from other COCO classes.
- During fine-tuning:
  - The total loss steadily decreases.
  - **Precision** rises sharply (e.g., from 22% to 91%) as the network learns to suppress irrelevant classes.
  - **Recall** remains high ($\sim$ 97%).
- Only a few epochs (e.g., 5–10) are needed because the dataset is small and the backbone is already very powerful.

23.18.4 Confidence Threshold Tuning

- At inference, a detection is kept only if its class confidence exceeds a user-defined threshold.
- By varying this threshold, one can trade precision for recall:
  - Lower threshold (0.3) $\rightarrow$ higher recall (98.5%), more false positives.
  - Higher threshold (0.9) $\rightarrow$ lower recall (95%), higher precision.
- The optimal threshold depends on the application requirements.

23.19 Instance Segmentation with Mask R-CNN

23.19.1 Architecture Extension

- **Mask R-CNN** builds directly upon Faster R-CNN by adding a third, parallel branch to the ROI head: the **mask head**.
- For each region proposal (after RoI Align, which preserves spatial accuracy via bilinear interpolation), a small fully convolutional network predicts a binary mask of size 28×28 for *each* object class.
- The mask head typically contains several 3×3 convolutions, followed by a 2×2 transposed convolution to upsample from 14×14 to 28×28.
- The predicted mask for the true class is selected during training; at inference, the mask of the highest-scoring class is used.
- The total model size increases only slightly (about 2 million additional parameters) compared to Faster R-CNN.

23.19.2 Fine-Tuning and Results

- The fine-tuning procedure is almost identical to Faster R-CNN: load COCO-pre-trained Mask R-CNN, adapt the classifier and mask predictor to the new number of classes, and train on the pedestrian dataset.
- After fine-tuning, the model outputs for each detected pedestrian:
  - A bounding box.
  - A class label and confidence score.
  - A pixel-wise binary mask (upsampled to the bounding box size and then placed in the full image).
- Visual results show clean instance masks that separate overlapping pedestrians, demonstrating the benefit of the additional mask head.